Microsoft Products Azure Services

Design Microsoft Teams

> 📁 **This guide has been split into multiple pages for better navigation.**
>
> **→ Go to the new guide**

Quick Links

Page	Description
Overview	Architecture, components, clarifying questions
Chat & Messaging	Messaging flow, sync, presence, notifications
Video & Calls	WebRTC, SFU, recording, screen sharing
File Sharing	Upload flow, SAS tokens, CDN
Databases	Cosmos DB, SQL, Redis, Blob, Search
Interview Tips	Trade-offs, how to explain, Q&A

Original Content (kept for reference)

Problem Statement

Design a real-time collaboration platform like Microsoft Teams that supports messaging, video calls, file sharing, and integrations for millions of concurrent users.

Clarifying Questions to Ask

1. **Scale**: How many daily active users? (e.g., 300M DAU)
2. **Features**: Which features to prioritize? (chat, video, file sharing, screen share)
3. **Message retention**: How long to store messages? (compliance requirements)
4. **Availability**: What's the target uptime? (99.99%?)

5. **Latency**: Max acceptable latency for messages? (<100ms?)
6. **Offline support**: Should users see messages when back online?
7. **Enterprise features**: Do we need admin controls, compliance, DLP?
8. **Call size**: Max participants in a call? (1:1, group up to 1000?)
9. **Recording**: Do calls need to be recorded and stored?
10. **PSTN**: Integration with phone networks?

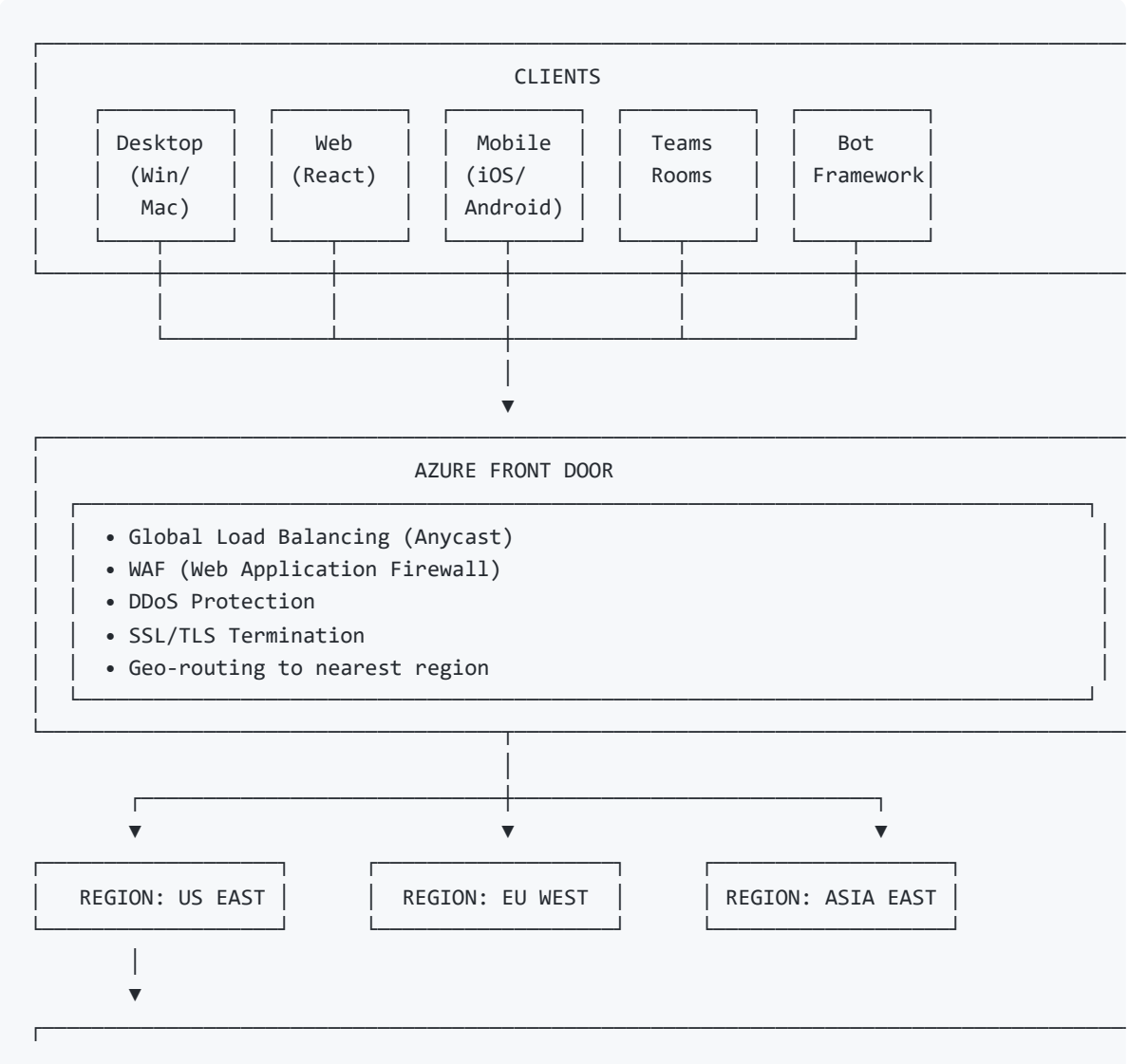
Design Reasoning

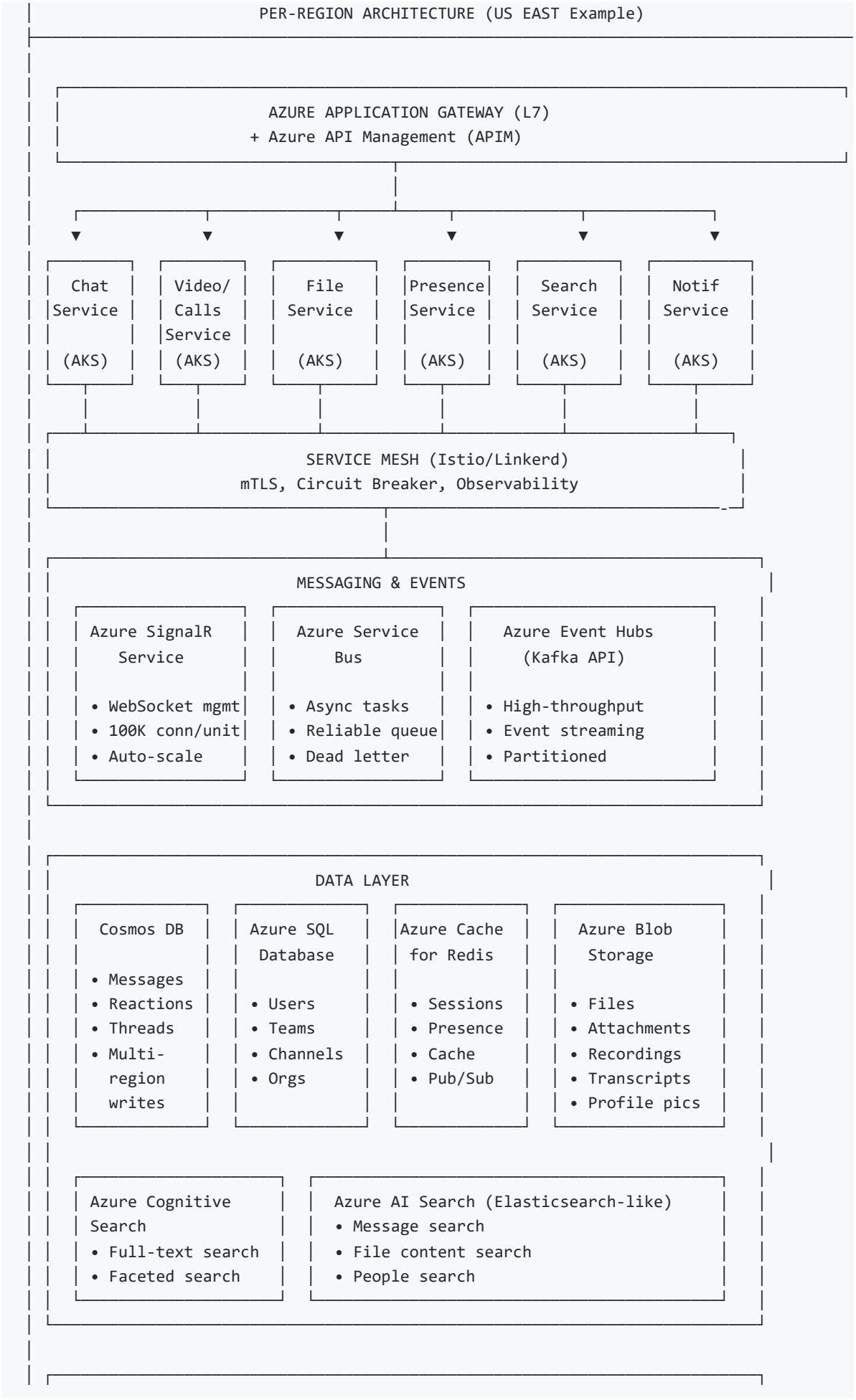
> **Core philosophy**: Build a modular, microservice-based real-time collaboration platform where each feature domain (chat, video, files, presence) scales independently, leveraging Azure-managed services for global low-latency delivery.

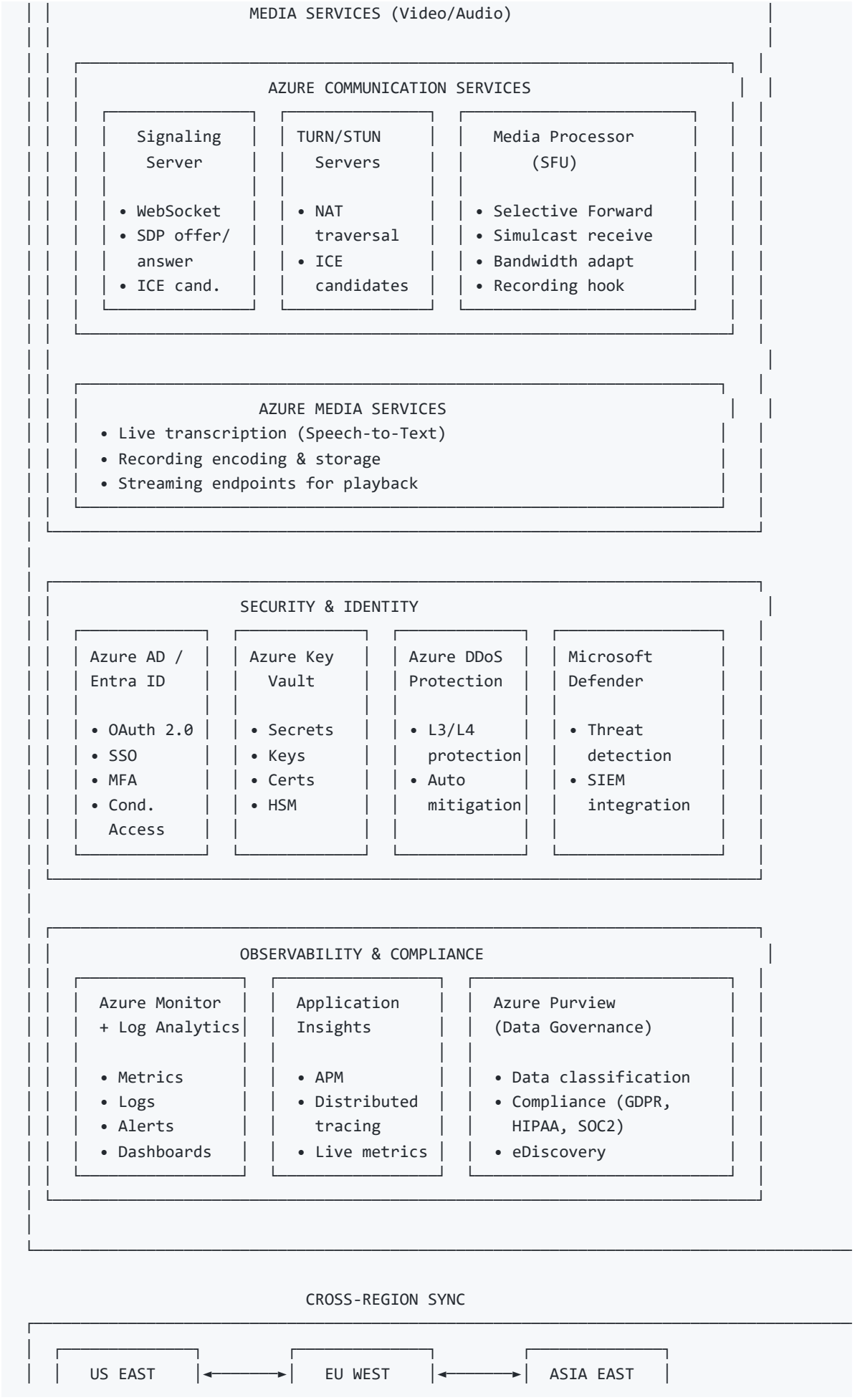
Decision	Choice	Why	Alternative Considered
Real-time transport	Azure SignalR (WebSockets)	Persistent connections needed for sub-100ms message delivery and presence updates to 300M DAU	HTTP long-polling — higher latency, more server overhead at scale
Compute	AKS with per-feature microservices	Chat, video, file, and presence have vastly different scaling profiles and failure domains	Monolith on App Service — cannot scale video independently from chat
Database	Cosmos DB (multi-region writes)	99.99% availability with single-digit-ms reads across global regions for chat history	Azure SQL — lacks native multi-region write support at this scale
Video relay	SFU (Selective Forwarding Unit)	Scales to large meetings by forwarding streams selectively instead of mixing	MCU (Mixing) — CPU-intensive, adds latency for 1000-participant calls
Event streaming	Azure Event Hubs (Kafka API)	Handles millions of events/sec for presence, notifications, and analytics pipelines	Service Bus alone — not designed for high-throughput streaming workloads

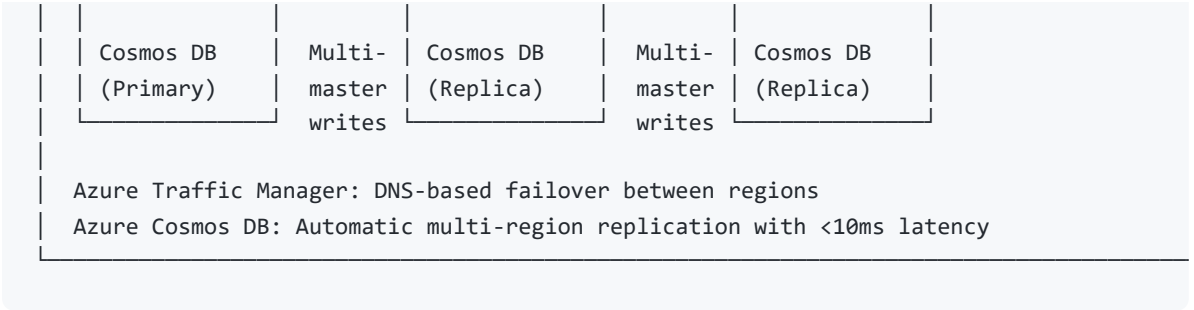
File storage	Azure Blob Storage + CDN	Cost-effective for large files; SAS tokens enable secure direct upload/download	Storing files in database — expensive and slow for large media
Caching	Azure Cache for Redis	Sub-ms presence lookups and session state for millions of concurrent users	In-memory per-pod cache — not shared across pods, causes stale data
Service mesh	Istio/Linkerd	mTLS between services, circuit breakers, and observability without app code changes	Custom middleware — significant development and maintenance burden

Complete Azure Architecture




PER-REGION ARCHITECTURE (US EAST Example)
AZURE APPLICATION GATEWAY (L7)
+ Azure API Management (APIM)
Chat Service (AKS)
Video/ Calls Service (AKS)
File Service (AKS)
Presence Service (AKS)
Search Service (AKS)
Notif Service (AKS)
SERVICE MESH (Istio/Linkerd)
mTLS, Circuit Breaker, Observability
MESSAGING & EVENTS
Azure SignalR Service
• WebSocket mgmt
• 100K conn/unit
• Auto-scale
Azure Service Bus
• Async tasks
• Reliable queue
• Dead letter
Azure Event Hubs (Kafka API)
• High-throughput
• Event streaming
• Partitioned
DATA LAYER
Cosmos DB
• Messages
• Reactions
• Threads
• Multi-region writes
Azure SQL Database
• Users
• Teams
• Channels
• Orgs
Azure Cache for Redis
• Sessions
• Presence
• Cache
• Pub/Sub
Azure Blob Storage
• Files
• Attachments
• Recordings
• Transcripts
• Profile pics
Azure Cognitive Search
• Full-text search
• Faceted search
Azure AI Search (Elasticsearch-like)
• Message search
• File content search
• People search


MEDIA SERVICES (Video/Audio)
AZURE COMMUNICATION SERVICES
Signaling Server
• WebSocket
• SDP offer/ answer
• ICE cand.
TURN/STUN Servers
• NAT traversal
• ICE candidates
Media Processor (SFU)
• Selective Forward
• Simulcast receive
• Bandwidth adapt
• Recording hook
AZURE MEDIA SERVICES
• Live transcription (Speech-to-Text)
• Recording encoding & storage
• Streaming endpoints for playback
SECURITY & IDENTITY
Azure AD / Entra ID
• OAuth 2.0
• SSO
• MFA
• Cond. Access
Azure Key Vault
• Secrets
• Keys
• Certs
• HSM
Azure DDoS Protection
• L3/L4 protection
• Auto mitigation
Microsoft Defender
• Threat detection
• SIEM integration
OBSERVABILITY & COMPLIANCE
Azure Monitor + Log Analytics
• Metrics
• Logs
• Alerts
• Dashboards
Application Insights
• APM
• Distributed tracing
• Live metrics
Azure Purview (Data Governance)
• Data classification
• Compliance (GDPR, HIPAA, SOC2)
• eDiscovery
CROSS-REGION SYNC
US EAST
EU WEST
ASIA EAST



API Design

Chat APIs

Method	Endpoint	Description
`POST`	`/api/v1/messages`	Send a message to a channel or chat
`GET`	`/api/v1/messages?chatId={id}&limit=50&before={cursor}`	Fetch messages with cursor pagination
`PUT`	`/api/v1/messages/{messageId}`	Edit a message
`DELETE`	`/api/v1/messages/{messageId}`	Delete a message
`POST`	`/api/v1/messages/{messageId}/reactions`	Add a reaction

Presence APIs

Method	Endpoint	Description
`PUT`	`/api/v1/presence`	Update user presence status
`GET`	`/api/v1/presence/batch?userIds={ids}`	Get presence for multiple users

Call/Meeting APIs

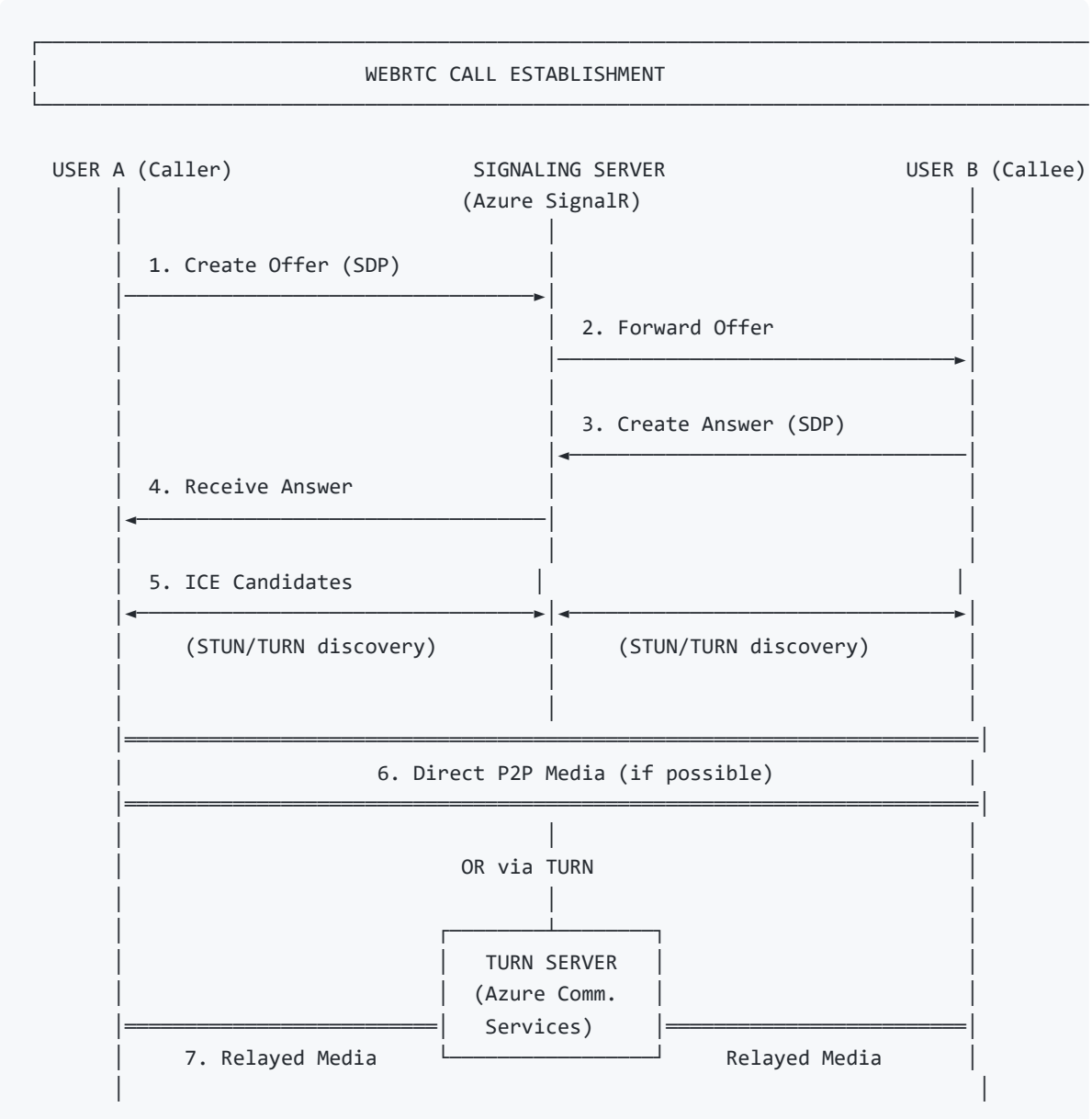
Method	Endpoint	Description
`POST`	`/api/v1/calls`	Initiate a call
`POST`	`/api/v1/calls/{callId}/join`	Join an existing call
`POST`	`/api/v1/calls/{callId}/leave`	Leave a call
`POST`	`/api/v1/meetings`	Schedule a meeting

WebSocket Events

Event	Direction	Payload
`message.new`	Server → Client	`{ chatId, message, sender, timestamp }`
`message.typing`	Client → Server	`{ chatId, userId }`
`presence.changed`	Server → Client	`{ userId, status }`
`call.incoming`	Server → Client	`{ callId, caller, type }`

Video/Calls Deep Dive

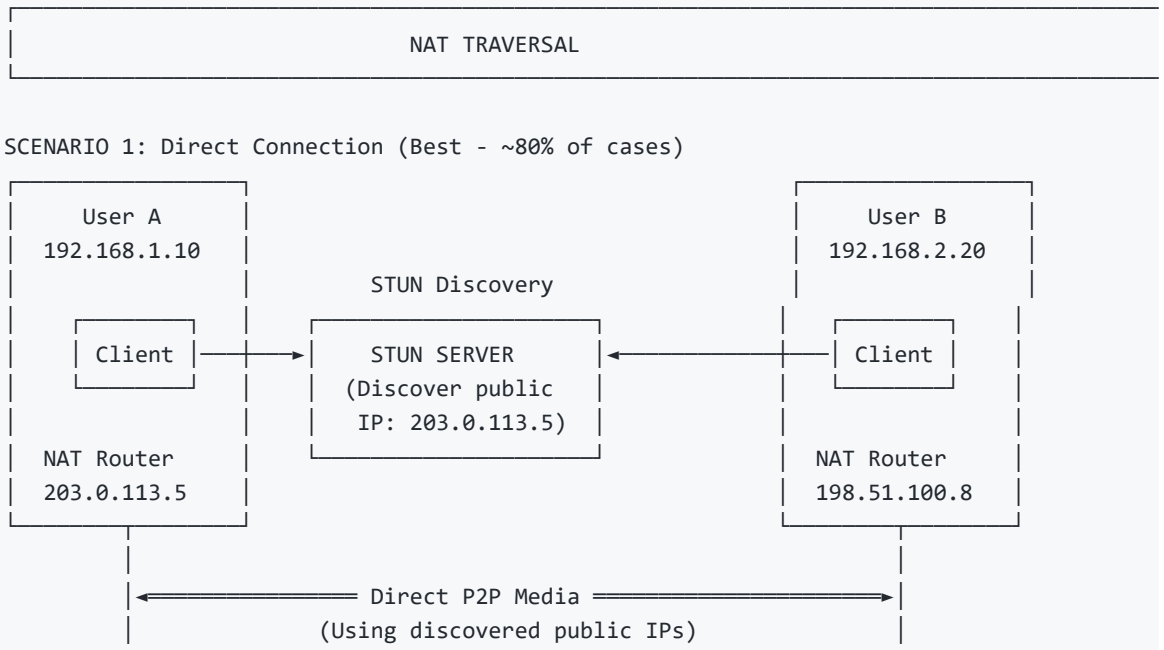
WebRTC Call Flow



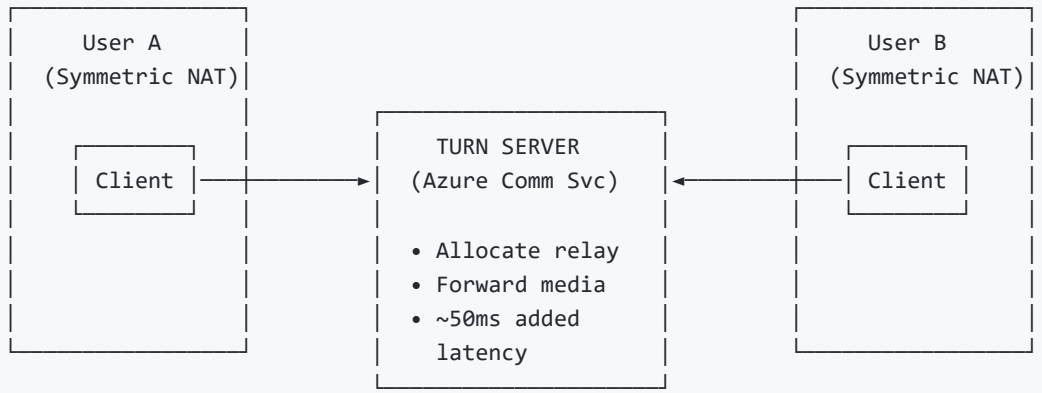
SDP (Session Description Protocol) contains:

- Media types (audio, video, screen share)
- Codecs supported (VP8, VP9, H.264, Opus)
- ICE candidates (connection paths)
- DTLS fingerprints (encryption)

STUN/TURN NAT Traversal



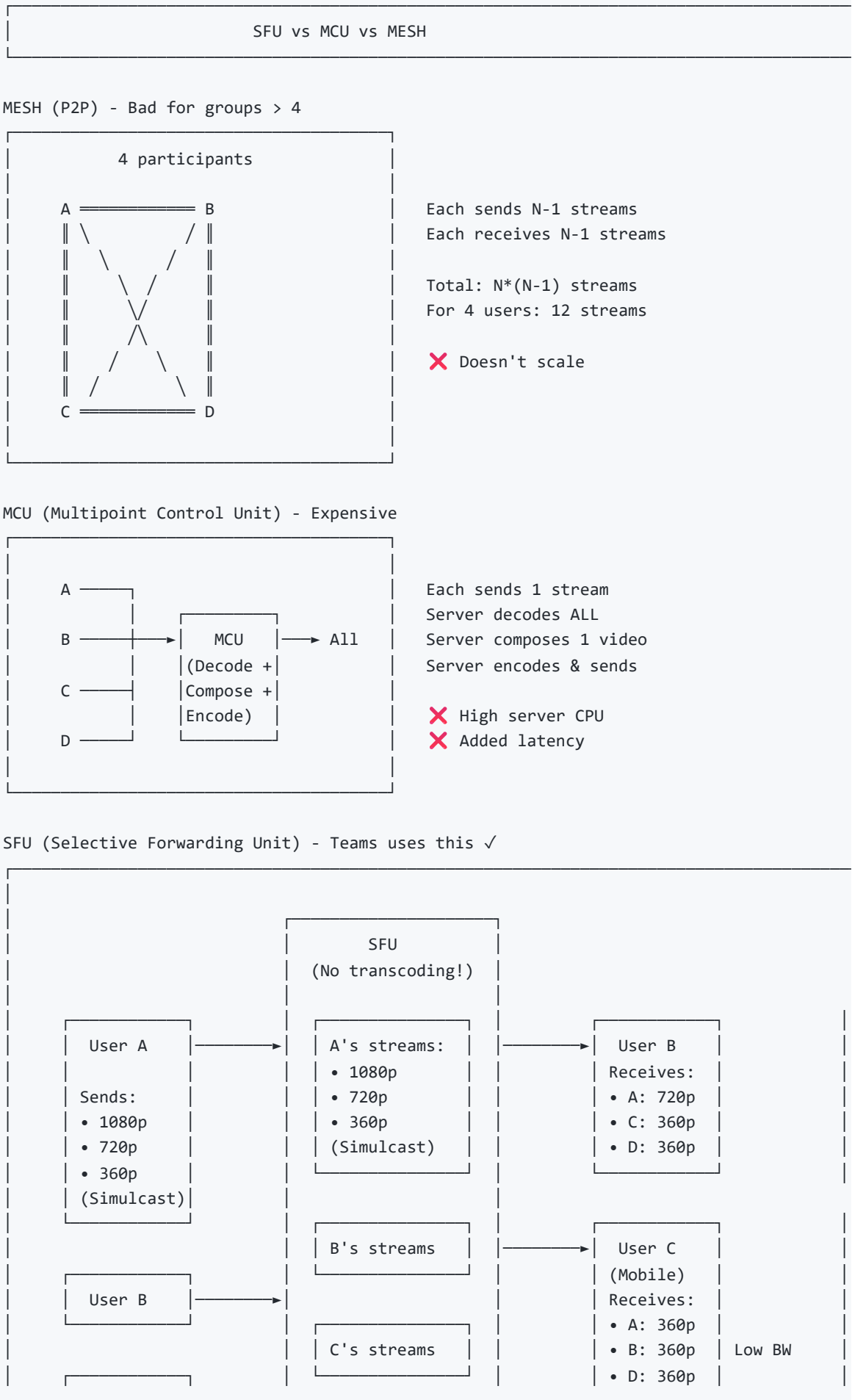
SCENARIO 2: TURN Relay (Fallback - ~20% of cases)

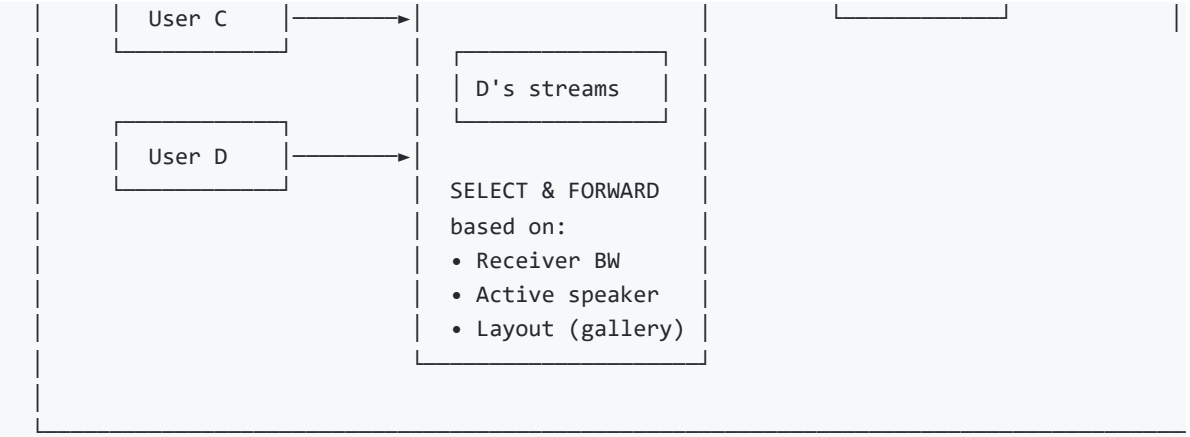


ICE (Interactive Connectivity Establishment):

1. Gather candidates (host, server-reflexive, relay)
2. Exchange via signaling
3. Connectivity checks (STUN binding)
4. Select best working pair

SFU Architecture (Selective Forwarding Unit)

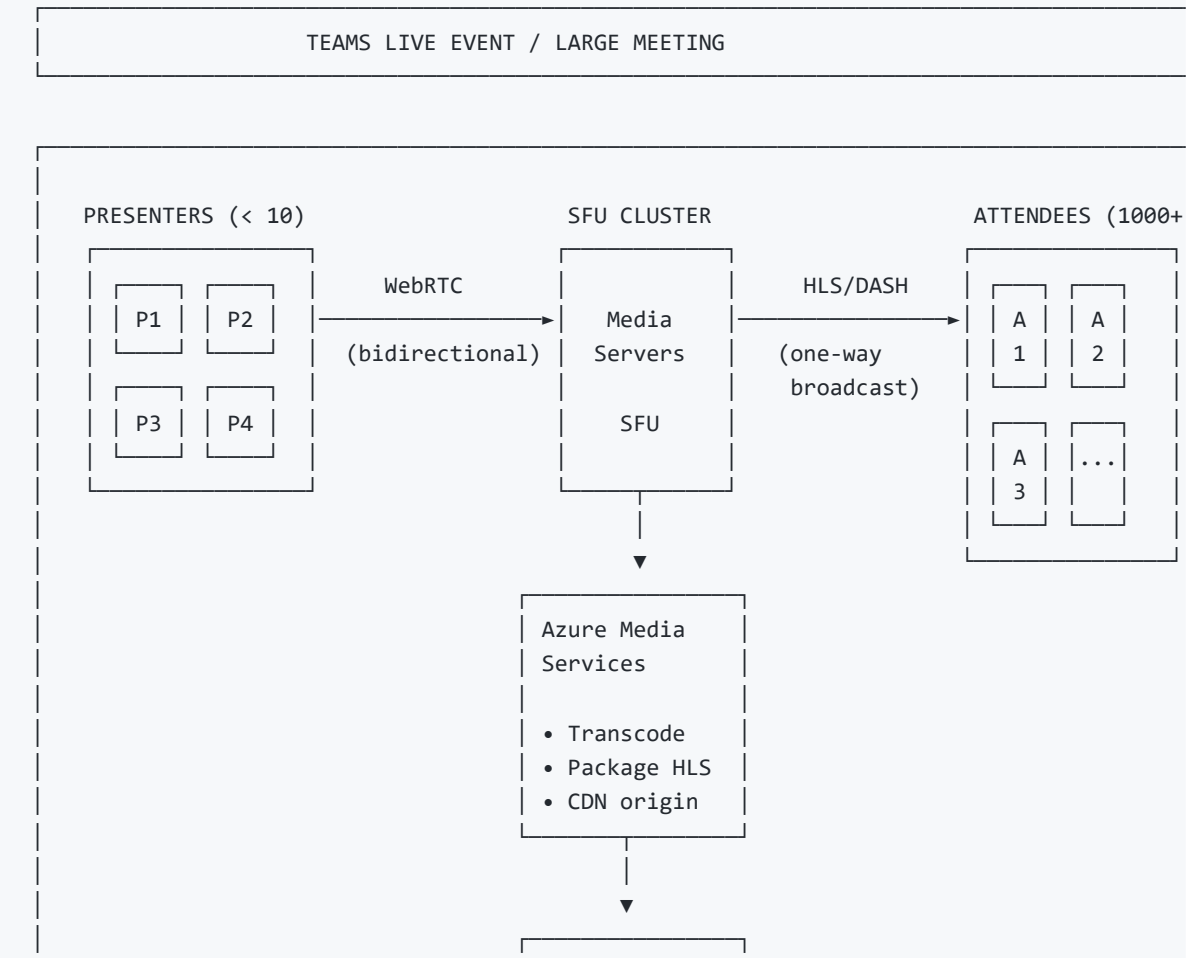

SFU vs MCU vs MESH
MESH (P2P) - Bad for groups > 4
4 participants
A
B
C
D
Each sends N-1 streams
Each receives N-1 streams
Total: N*(N-1) streams
For 4 users: 12 streams
Doesn't scale
MCU (Multipoint Control Unit) - Expensive
A
B
C
D
MCU
(Decode +
Compose +
Encode)
All
Each sends 1 stream
Server decodes ALL
Server composes 1 video
Server encodes & sends
High server CPU
Added latency
SFU (Selective Forwarding Unit) - Teams uses this ✓
SFU
(No transcoding!)
User A
Sends:
• 1080p
• 720p
• 360p
(Simulcast)
User B
A's streams:
• 1080p
• 720p
• 360p
(Simulcast)
B's streams
C's streams
User B
Receives:
• A: 720p
• C: 360p
• D: 360p
User C
(Mobile)
Receives:
• A: 360p
• B: 360p
• D: 360p
Low BW

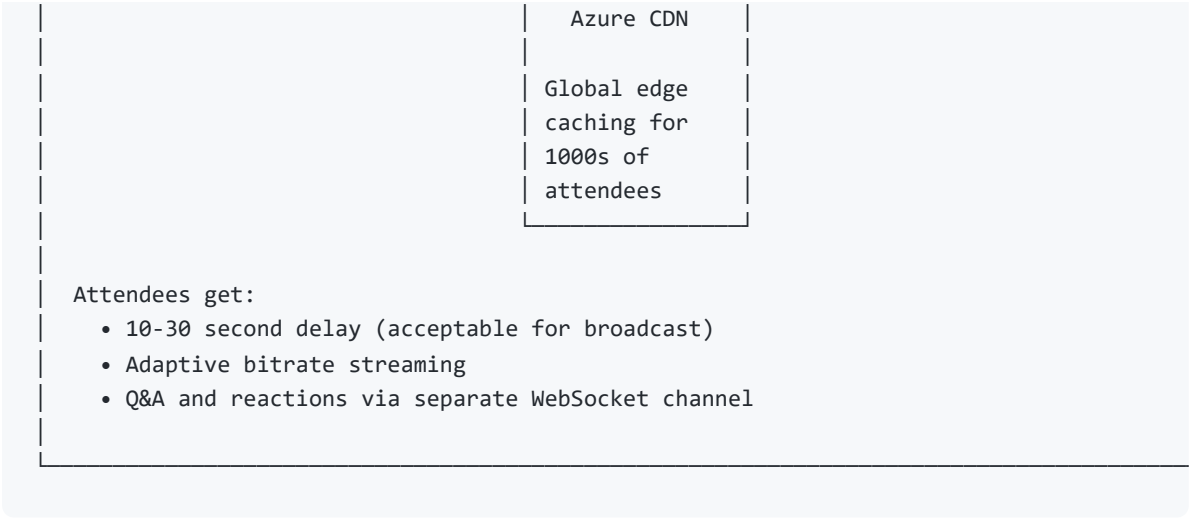


SIMULCAST: Sender encodes 3 quality layers
- High: 1080p @ 2.5 Mbps
- Medium: 720p @ 1.0 Mbps
- Low: 360p @ 0.3 Mbps

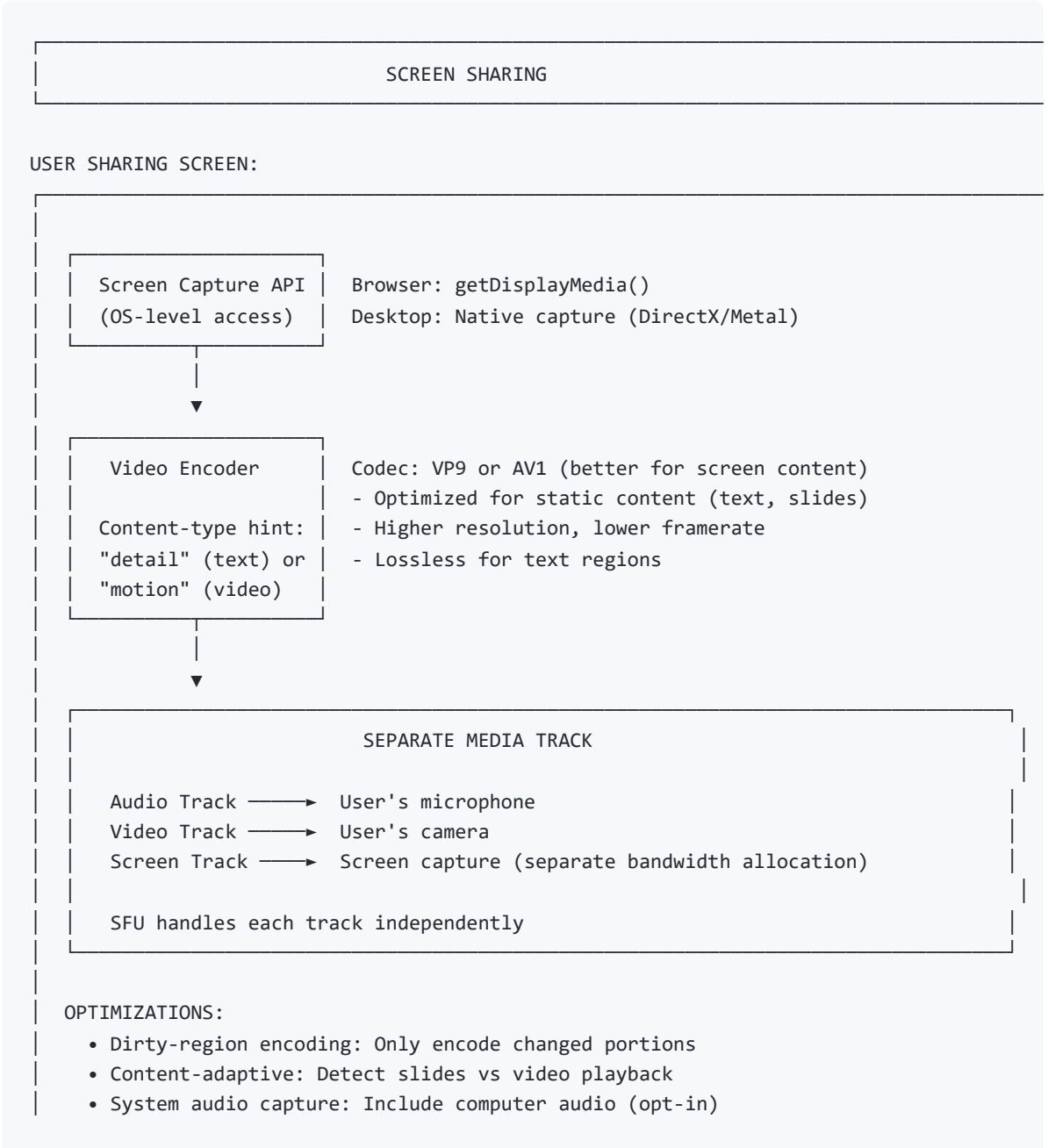
SFU selects which layer to forward based on:
- Receiver's available bandwidth
- Receiver's viewport size (is participant thumbnail or spotlight?)
- Active speaker detection (prioritize current speaker)

Large Meeting Optimization (1000+ participants)

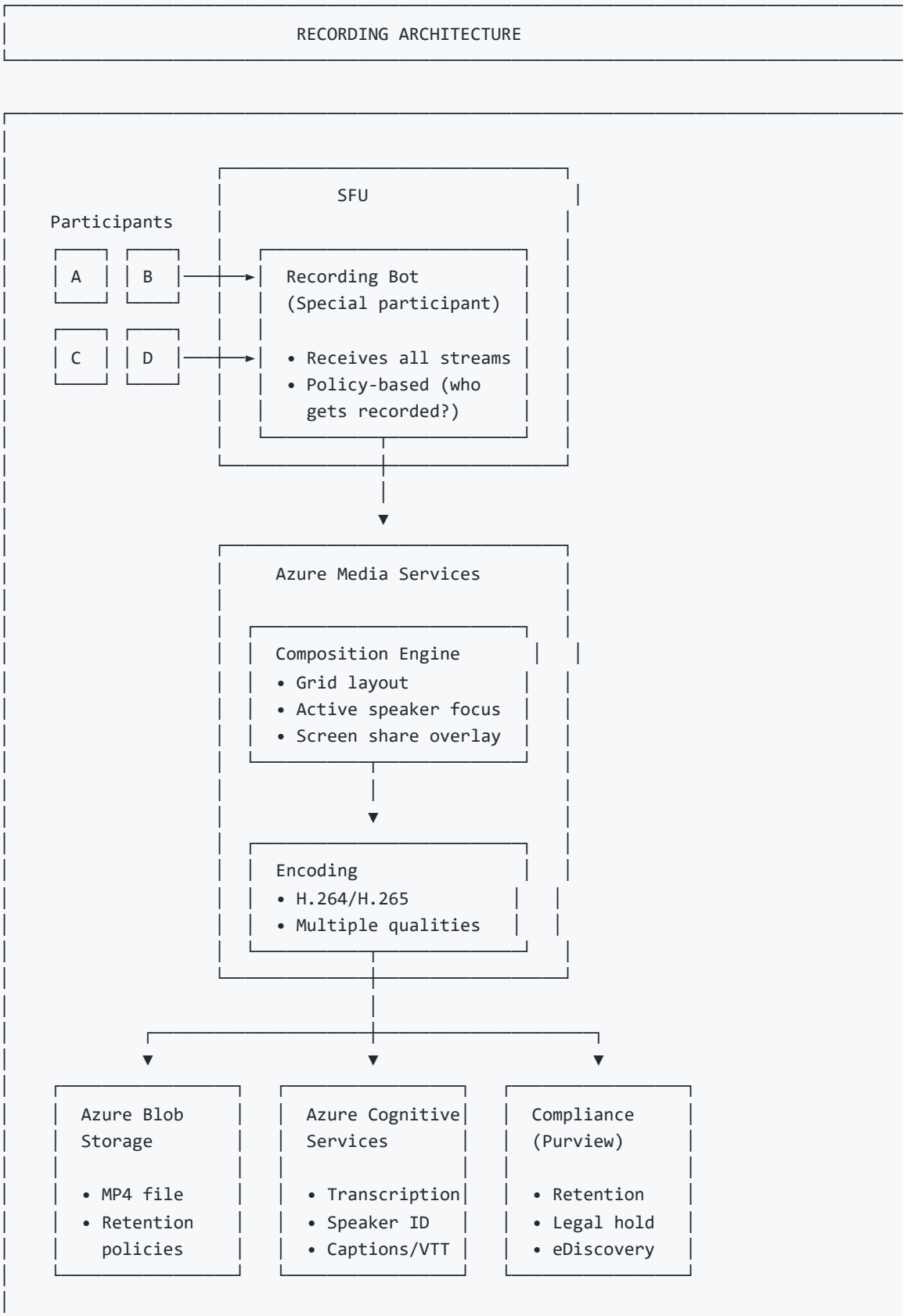




Screen Sharing Architecture



Call Recording Architecture



```
│  POST-PROCESSING:
│    • Auto-generated chapters (AI scene detection)
│    • Searchable transcript
│    • Action items extraction (Copilot integration)
│
└──────────────────────────────────────────────────────────────────────────────────────────────────
```

Quality Adaptation (Bandwidth Estimation)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│                                ADAPTIVE BITRATE CONTROL
└──────────────────────────────────────────────────────────────────────────────────────────────────

SENDER SIDE (Congestion Control):
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│
│   Available
│   Bandwidth     ┌──────────────────────────────────────────────────────────┐
│       ▲         │  GCC (Google Congestion Control) / BBR                   │
│       │         │                                                          │
│   5 Mbps  - - -|- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - |        |
│       │         │          ████                                            │
│   3 Mbps  - - -|- - - - ████████████████                  ████████████████|
│       │         │    ████                ████████████████████              │
│   1 Mbps  - - -|████                                                      │
│       │         │                                                          │
│       └─────────└──────────────────────────────────────────────────────────┘
│                 Time ──────────────────────────────────────────────►
│
│   Signals used:
│     • RTCP Receiver Reports (packet loss %, jitter)
│     • Transport-CC (precise packet timing feedback)
│     • REMB (Receiver Estimated Max Bitrate)
│
└──────────────────────────────────────────────────────────────────────────────────────────────────

SFU ADAPTATION DECISIONS:
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│
│   RECEIVER BANDWIDTH: 2 Mbps available
│
│   Meeting has 9 participants in gallery view:
│    ┌──────────┬──────────┬──────────┐
│    │ Speaker  │    P2    │    P3    │  Speaker gets 720p (1.0 Mbps)
│    │  720p    │  180p    │  180p    │  Others get 180p thumbnails (8 × 0.1 Mbps)
│    ├──────────┼──────────┼──────────┤  Total: ~1.8 Mbps
│    │    P4    │    P5    │    P6    │
│    │  180p    │  180p    │  180p    │  When bandwidth drops:
│    ├──────────┼──────────┼──────────┤    • Reduce speaker to 360p
│    │    P7    │    P8    │    P9    │    • Reduce thumbnails to 90p
│    │  180p    │  180p    │  180p    │    • Pause off-screen participants
│    └──────────┴──────────┴──────────┘
│
```

```
│
└──────────────
```

```
  "id": "msg_123456789",
  "partitionKey": "channel_abc",  -- Partition by channel for locality
  "type": "message",
  "channelId": "channel_abc",
  "threadId": "thread_xyz",       -- null for top-level messages
  "senderId": "user_123",
  "content": {
    "text": "Hello team!",
    "mentions": ["user_456"],
    "format": "html"
  },
  "attachments": [
    {
      "type": "file",
      "blobUrl": "https://...",
      "name": "report.pdf",
      "size": 1024000
    }
  ],
  "reactions": {
    "👍": ["user_456", "user_789"],
    "❤️": ["user_456"]
  },
  "createdAt": "2024-01-15T10:30:00Z",
  "editedAt": null,
  "deletedAt": null,
  "_ts": 1705314600  -- Cosmos DB timestamp for TTL/sync
}

-- Azure SQL for relational data
CREATE TABLE Teams (
    id UNIQUEIDENTIFIER PRIMARY KEY,
    name NVARCHAR(255),
    tenant_id UNIQUEIDENTIFIER,
    created_at DATETIME2,
    settings NVARCHAR(MAX)  -- JSON
);

CREATE TABLE Channels (
    id UNIQUEIDENTIFIER PRIMARY KEY,
    team_id UNIQUEIDENTIFIER FOREIGN KEY REFERENCES Teams(id),
    name NVARCHAR(255),
    type VARCHAR(20),  -- 'standard', 'private', 'shared'
    created_at DATETIME2
);

CREATE TABLE ChannelMembers (
    channel_id UNIQUEIDENTIFIER,
    user_id UNIQUEIDENTIFIER,
    role VARCHAR(20),  -- 'owner', 'member'
    joined_at DATETIME2,
    PRIMARY KEY (channel_id, user_id)
);
```

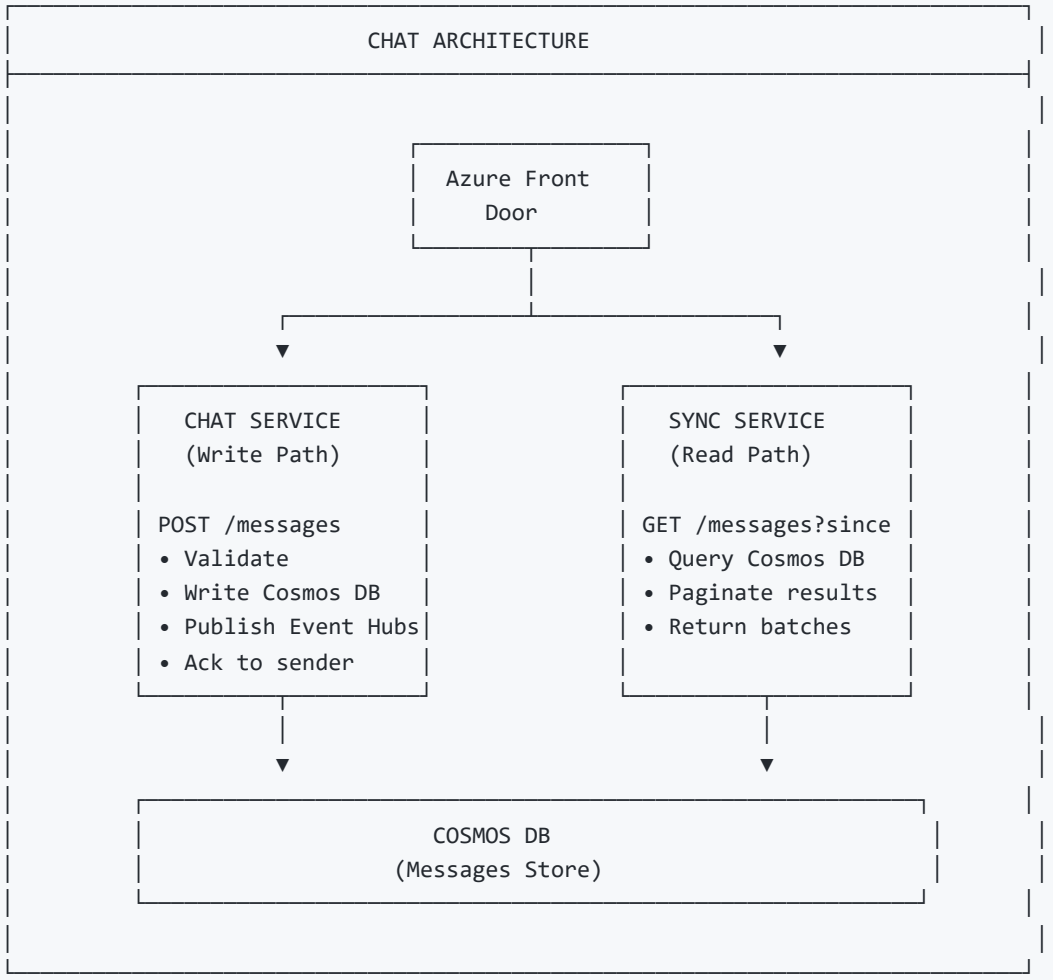
Key Components Summary

Component	Purpose	Azure Service	Why This Choice
Global Load Balancer	Route to nearest region	Azure Front Door	Anycast, WAF, SSL
API Gateway	Auth, rate limiting	Azure API Management	Enterprise features
Compute	Run microservices	AKS (Kubernetes)	Auto-scale, density
Real-time Messaging	WebSocket management	Azure SignalR Service	100K connections/unit
Message Queue	Async processing	Azure Event Hubs	Kafka API, partitioned
Task Queue	Reliable job queue	Azure Service Bus	Dead letter, retry
Chat Storage	Messages, threads	Cosmos DB	Multi-region writes
Relational Data	Users, teams, orgs	Azure SQL Database	ACID, joins
Cache	Sessions, presence	Azure Cache for Redis	Sub-ms latency
File Storage	Attachments, recordings	Azure Blob Storage	Tiered, cheap
Search	Message search	Azure AI Search	Full-text, faceted
Video/Calls	WebRTC infrastructure	Azure Comm. Services	SFU, TURN included
Recording	Call recording	Azure Media Services	Encode, stream
Identity	SSO, MFA	Azure AD / Entra ID	Enterprise SSO
Secrets	Keys, certs	Azure Key Vault	HSM-backed
Monitoring	Logs, metrics	Azure Monitor	Unified observability
Compliance	Data governance	Azure Purview	GDPR, eDiscovery

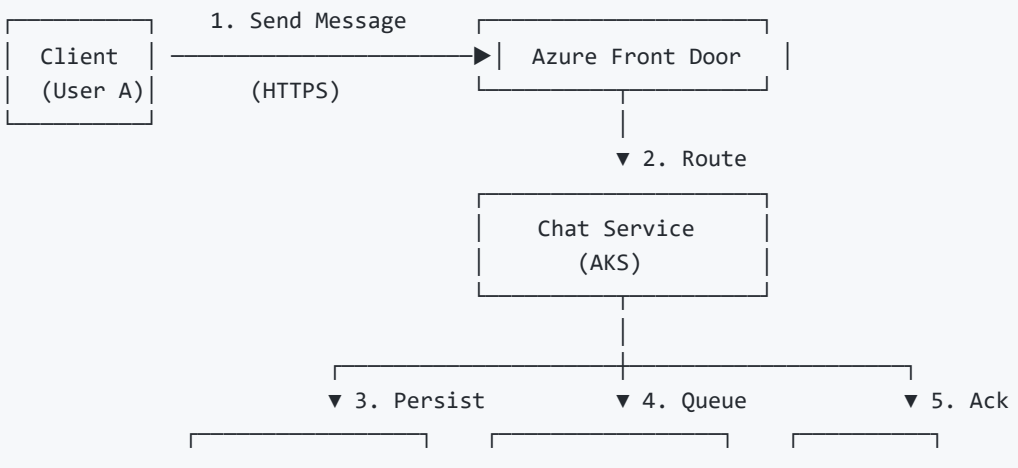
Feature Flows

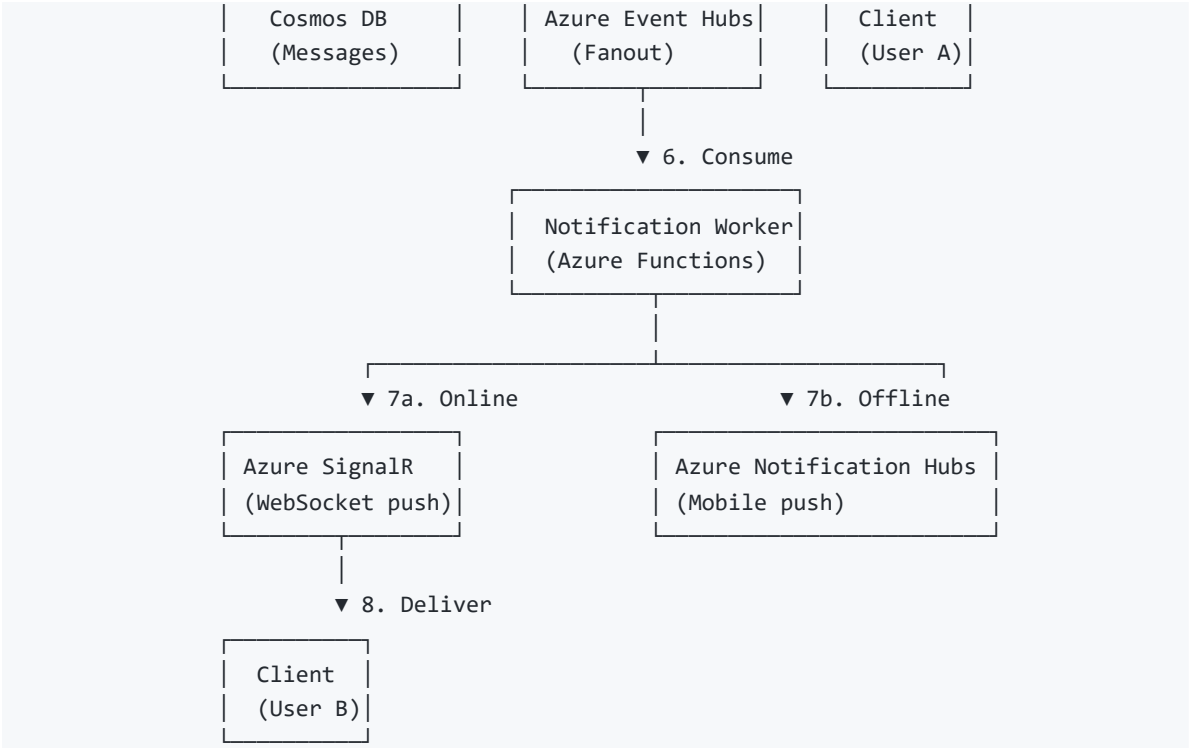
1. Chat Message Flow

Two Paths: Write (Send) vs Read (Sync)



Write Path: Sending a Message (Real-time)



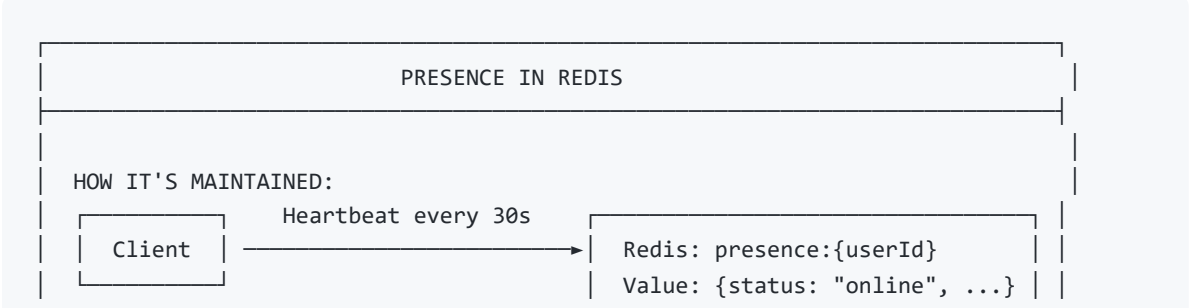


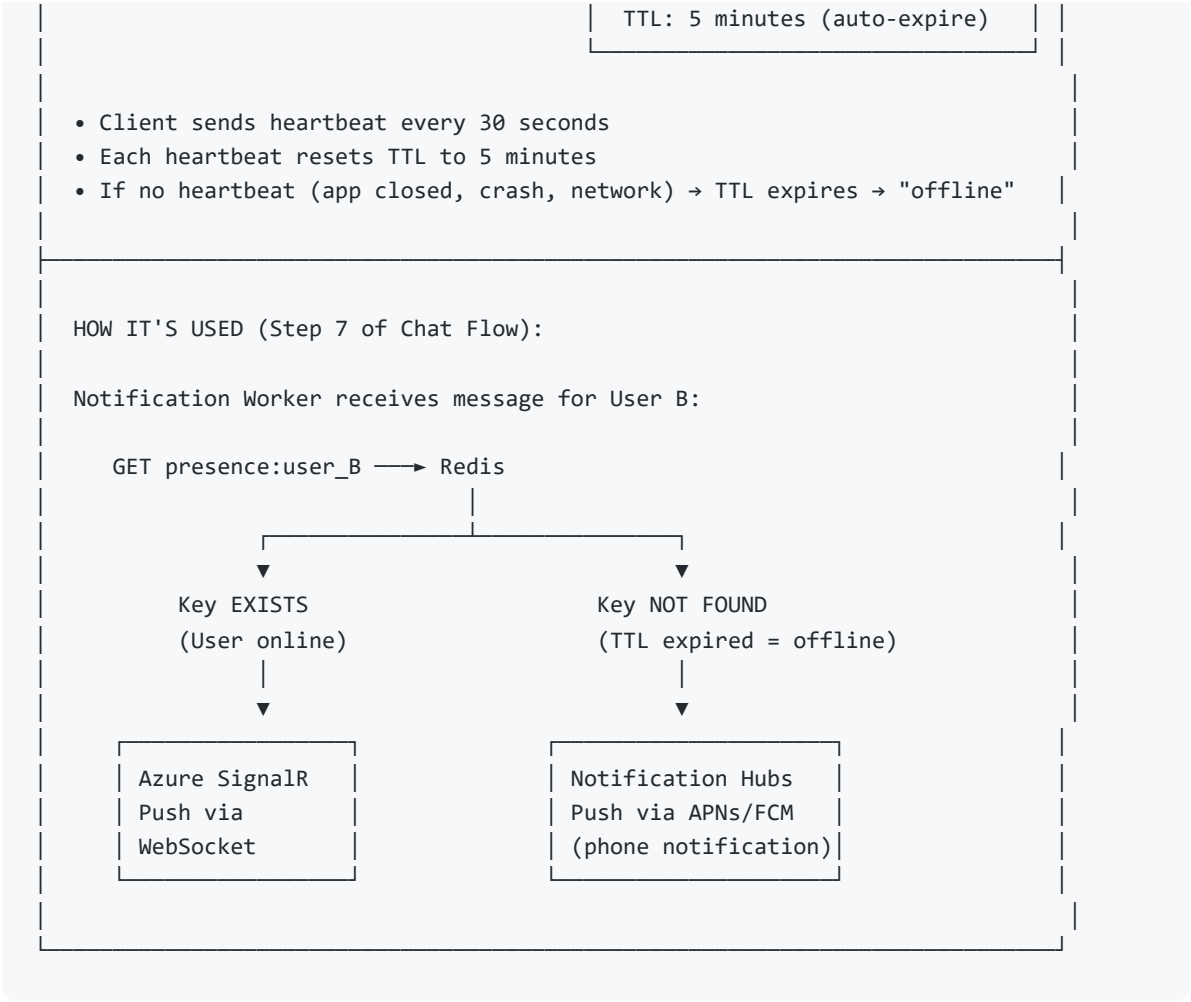
Write Path Steps:

1. User A sends message via HTTPS
2. Front Door routes to Chat Service in AKS
3. Message persisted to Cosmos DB (durable)
4. Message published to Event Hubs for fanout
5. Client A gets acknowledgment (message sent ✓)
6. Azure Functions worker consumes from Event Hubs
7. Check recipient status in Redis → route message:
   - **Online** → Push via SignalR WebSocket
   - **Offline** → Push via Notification Hubs (mobile)
8. User B receives message in real-time

Presence: How Step 7 Works

Presence determines **how to deliver** the message. It's stored in Redis and checked at delivery time.





Why TTL-based presence?

Approach	Problem
Explicit "disconnect" event	Unreliable - crashes, network loss don't trigger it
TTL + heartbeat ✓	Auto-expires; handles all failure modes

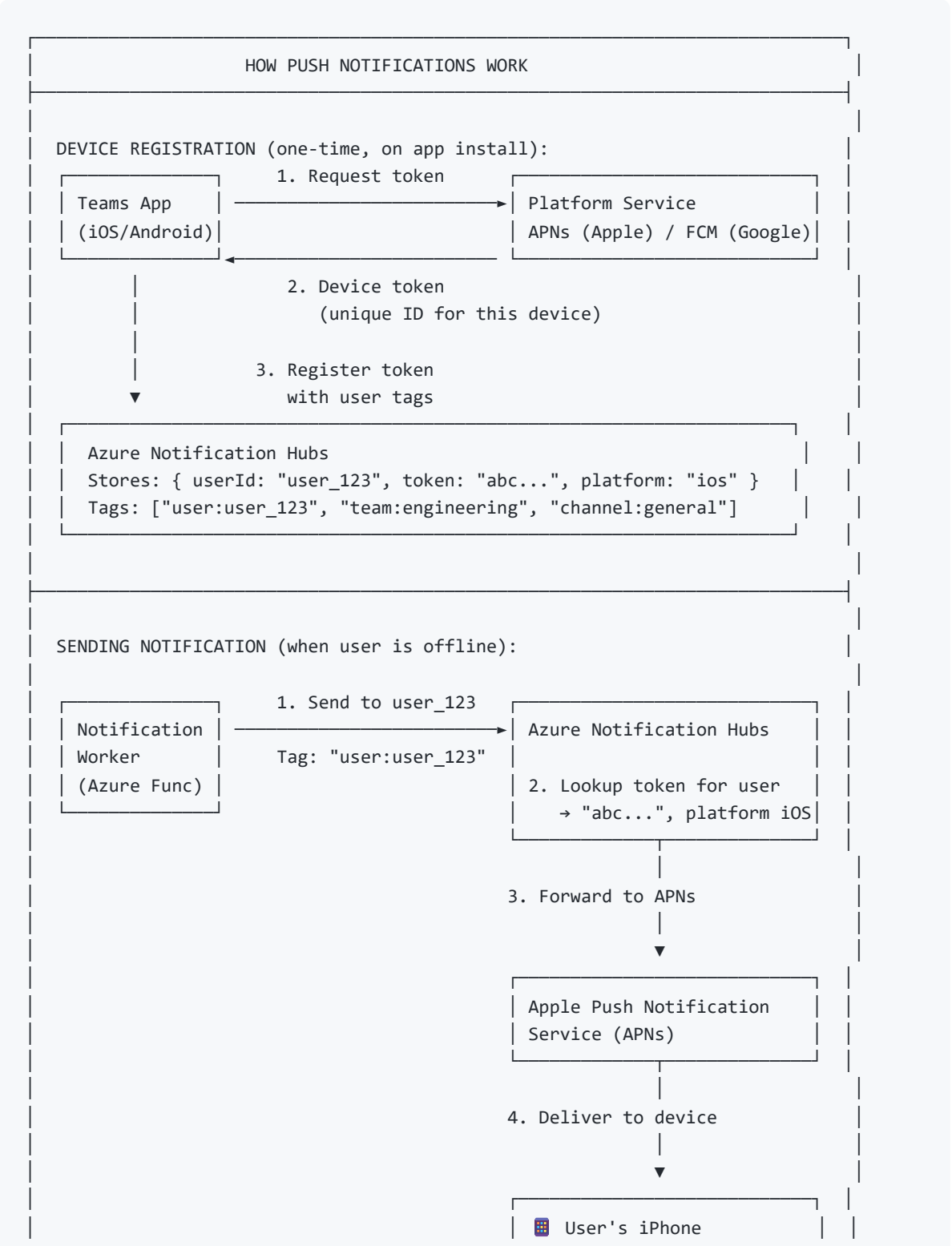
Design Decisions Explained:

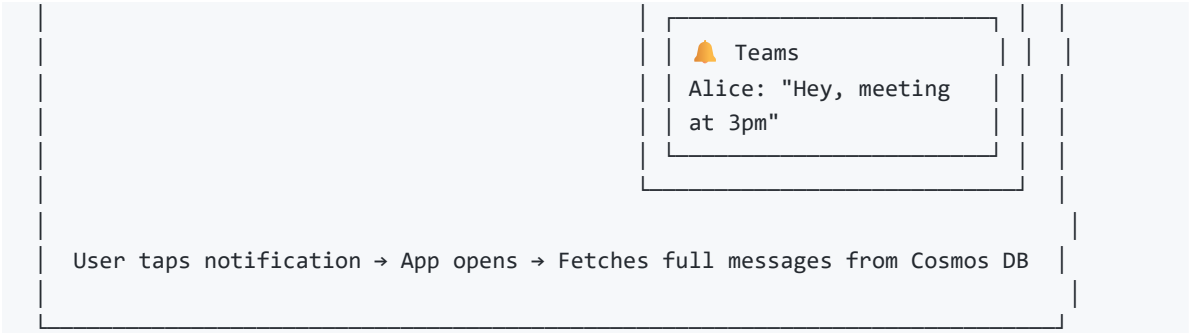
Decision	Why This Choice	Alternative Considered
Event Hubs for fanout	High throughput (millions/sec), partitioned by channelId, Kafka-compatible	Service Bus - better for task queues but lower throughput
Cosmos DB first, then queue	Durability - message is safe even if fanout fails; can replay from DB	Queue first risks message loss if DB write fails
Azure Functions as worker	Serverless, auto-scale with Event Hubs partitions, cost-efficient	AKS pods - more control but manual scaling

Redis TTL for presence	Auto-cleanup on crash/disconnect; no explicit "offline" event needed	Explicit disconnect - unreliable

Azure Notification Hubs: Reaching Offline Users

When a user is **offline** (app closed, phone locked, no WebSocket), we can't push via SignalR. Instead, we use **platform push notifications** (the same system that shows "You have a new email" on your lock screen).



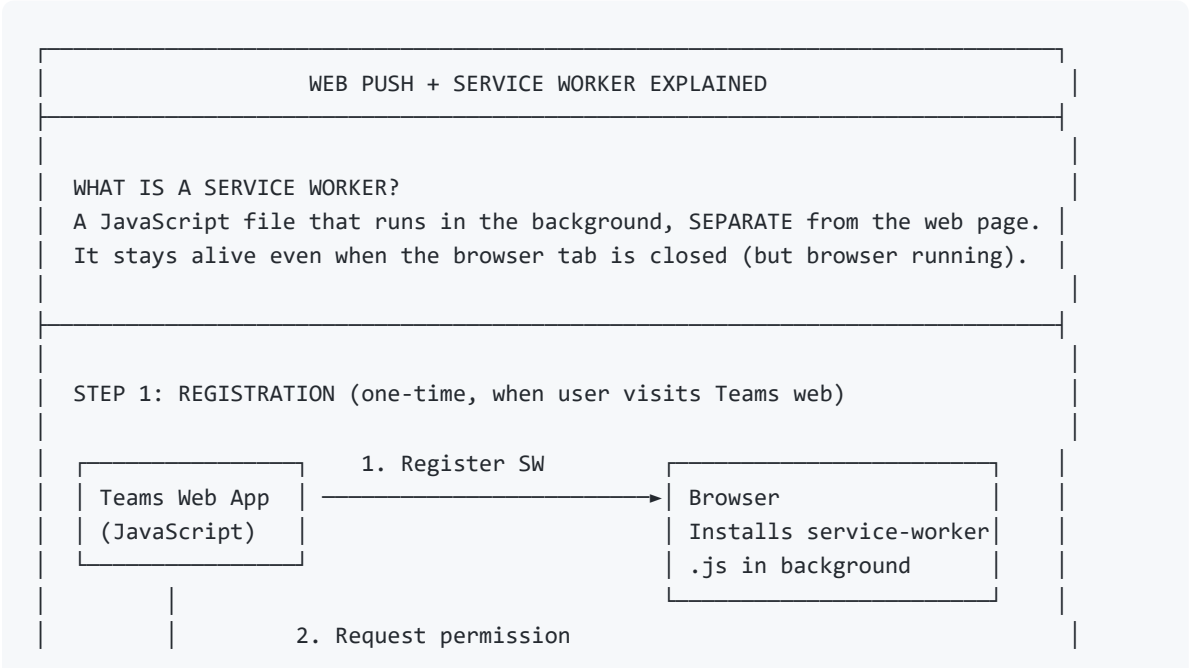


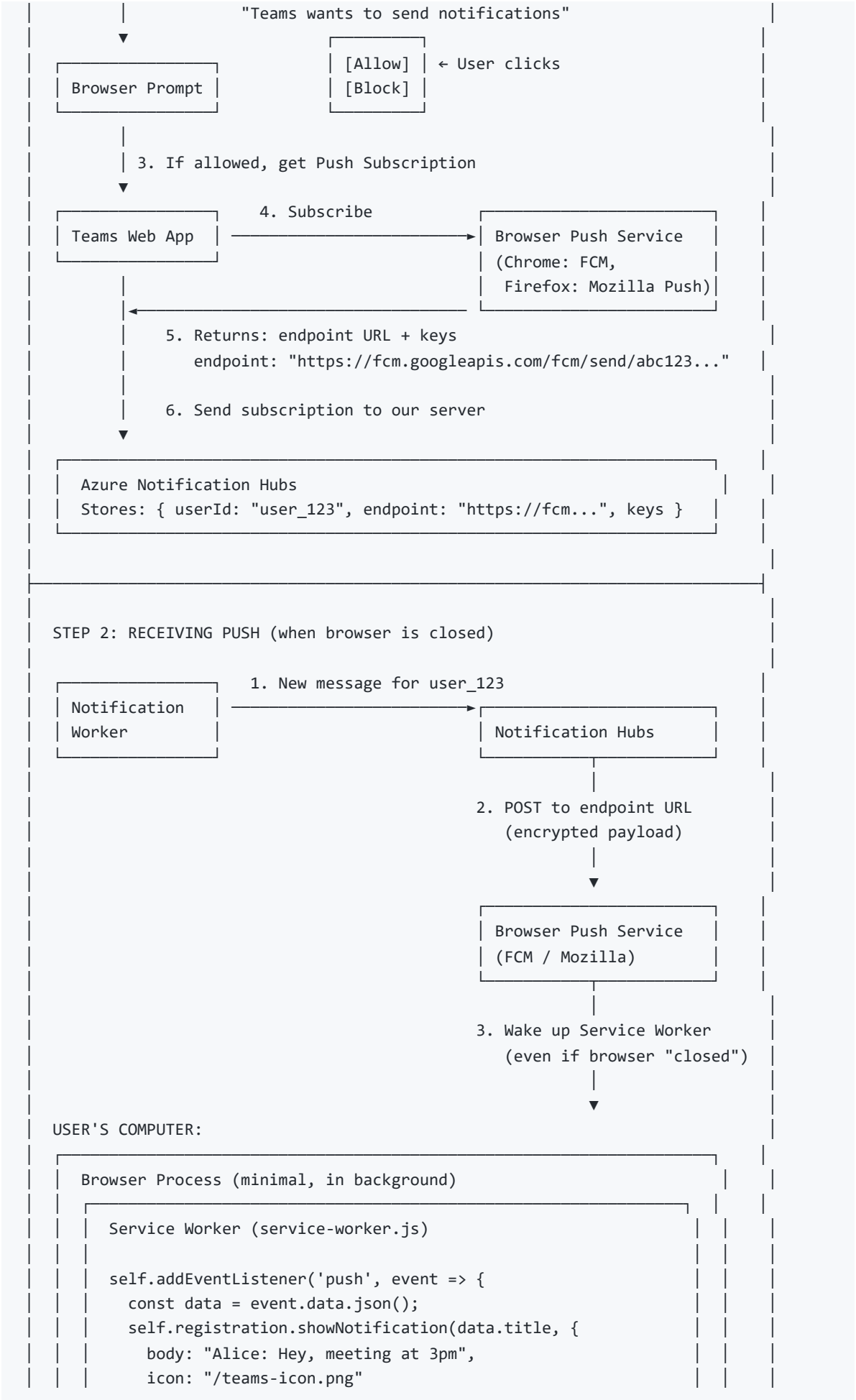
Notification Delivery by Platform:

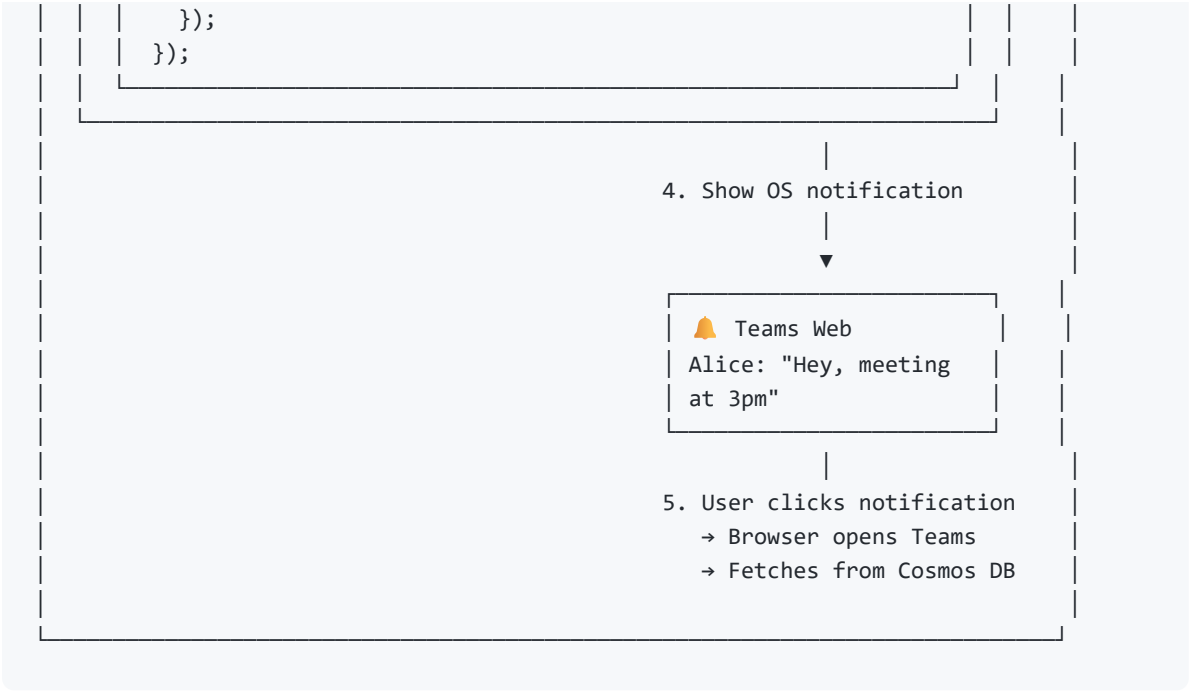
Platform	Push Service	How it works
iOS	APNs (Apple Push Notification)	Works even when app killed
Android	FCM (Firebase Cloud Messaging)	Works even when app killed
Windows Desktop	WNS (Windows Notification Service)	Toast notifications via Action Center
Mac Desktop	APNs (same as iOS)	Native macOS notifications
Web Browser	Web Push API + Service Worker	Works if browser open OR user allowed background

Web & Desktop: Key Differences from Mobile

How Web Push + Service Worker Works:



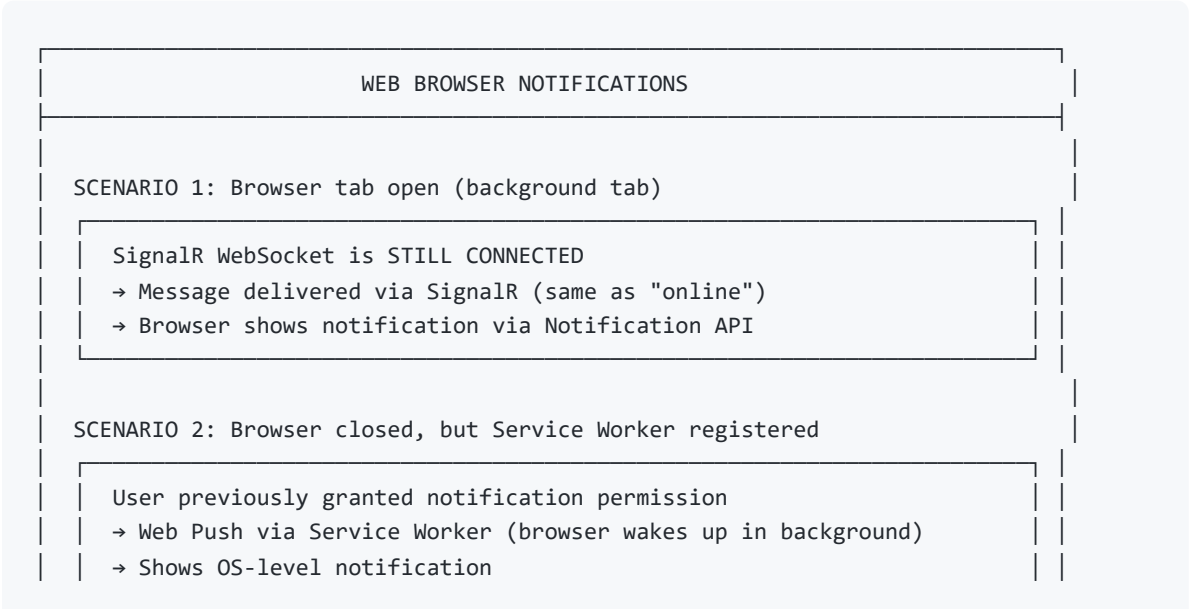

"Teams wants to send notifications"
Browser Prompt
[Allow]
[Block]
← User clicks
3. If allowed, get Push Subscription
Teams Web App
4. Subscribe
Browser Push Service
(Chrome: FCM,
Firefox: Mozilla Push)
5. Returns: endpoint URL + keys
endpoint: "https://fcm.googleapis.com/fcm/send/abc123..."
6. Send subscription to our server
Azure Notification Hubs
Stores: { userId: "user_123", endpoint: "https://fcm...", keys }
STEP 2: RECEIVING PUSH (when browser is closed)
Notification Worker
1. New message for user_123
Notification Hubs
2. POST to endpoint URL
(encrypted payload)
Browser Push Service
(FCM / Mozilla)
3. Wake up Service Worker
(even if browser "closed")
USER'S COMPUTER:
Browser Process (minimal, in background)
Service Worker (service-worker.js)
self.addEventListener('push', event => {
const data = event.data.json();
self.registration.showNotification(data.title, {
body: "Alice: Hey, meeting at 3pm",
icon: "/teams-icon.png"

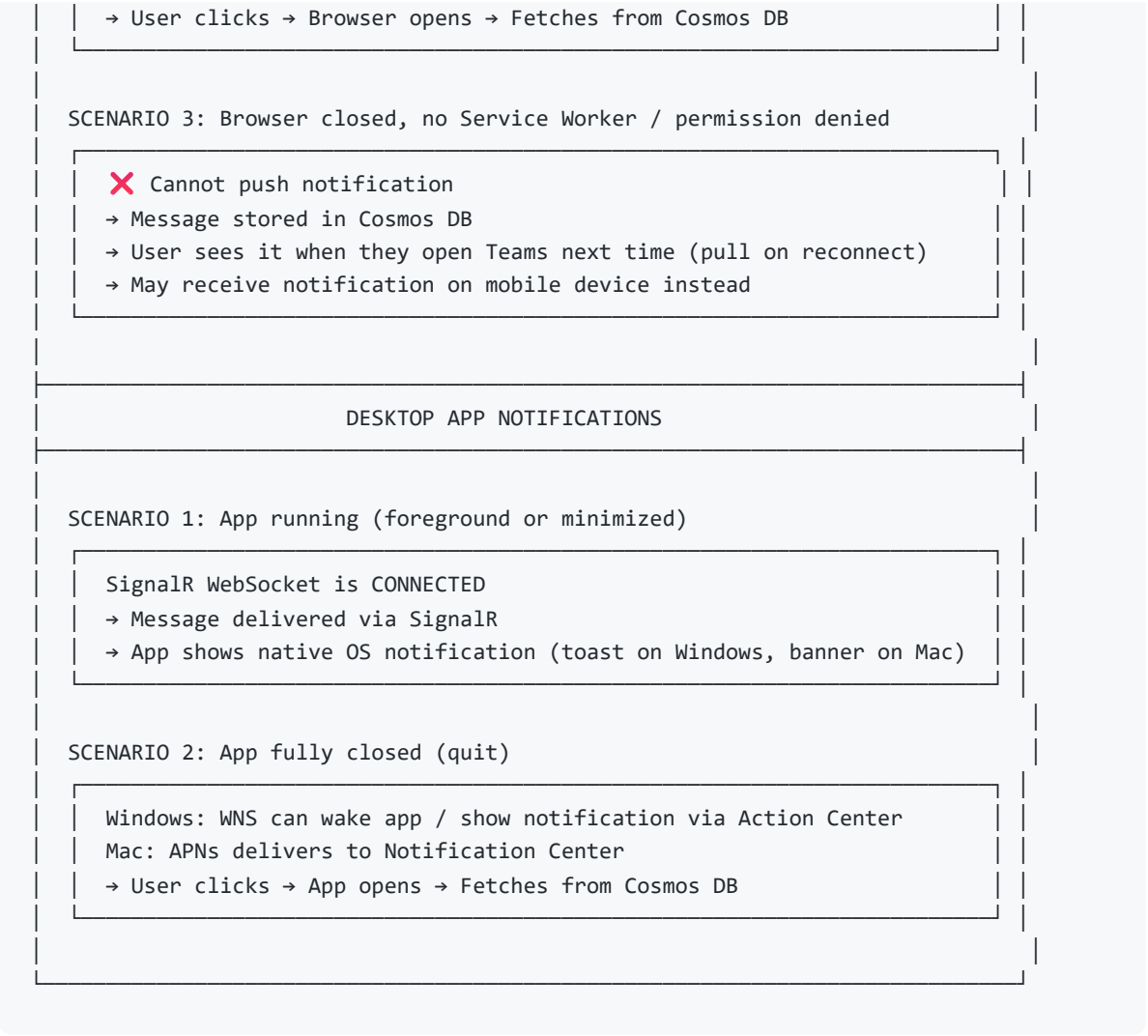


Limitations of Web Push:

Limitation	Impact
User must grant permission	Many users click "Block"
Browser must be installed	Obvious, but matters
Some browsers need to be running	Safari on Mac needs browser running
Doesn't work in Incognito	No Service Worker persistence
iOS Safari: No Web Push (until iOS 16.4+)	Major gap for iPhone web users

Web Browser Notification Scenarios:





Summary: Delivery Strategy by State

User State	Delivery Method	Reliability
App open (any platform)	SignalR WebSocket	✅ Instant
App closed - Mobile	APNs/FCM push	✅ Reliable
App closed - Windows	WNS push	✅ Reliable
App closed - Mac	APNs push	✅ Reliable
Browser closed + Service Worker	Web Push	⚠️ Depends on browser/OS
Browser closed, no permission	**No push** - pull on reconnect	❌ User must open app
Device shut down	**No push possible**	❌ Pull on next boot

What happens when device is shut down?



Key points:

- Push notification is just a "**wake up" alert** - it contains minimal data (sender, preview)
- Full message content is fetched from Cosmos DB when app opens
- **Cosmos DB is the source of truth** - push is best-effort delivery
- Device tokens are managed by Apple/Google/Microsoft, not us
- Notification Hubs abstracts the complexity of APNs/FCM/WNS APIs
- **Web is the least reliable** for offline push - encourage users to install desktop/mobile app
- **Multi-device**: If one device is off, another may receive the push

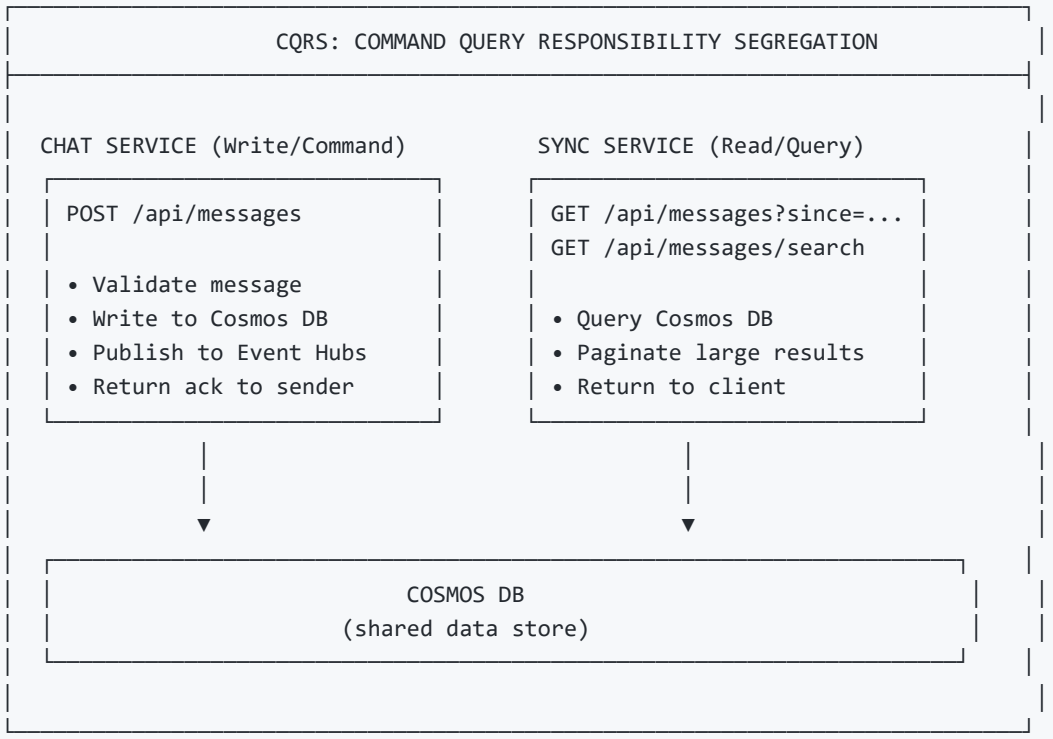
Why Notification Hubs (not direct APNs/FCM)?

Without Notification Hubs	With Notification Hubs
Integrate 3+ APIs (APNs, FCM, WNS, Web Push) separately	Single unified API
Manage device tokens yourself	Automatic token management
Handle token expiry/refresh	Handles automatically
Build retry/failover logic	Built-in reliability
Scale infrastructure yourself	Scales to millions

Offline → Online: Message Sync (Sync Service)

When a user reconnects after being offline, they need to fetch missed messages. This is handled by a **separate Sync Service**, not the Chat Service.

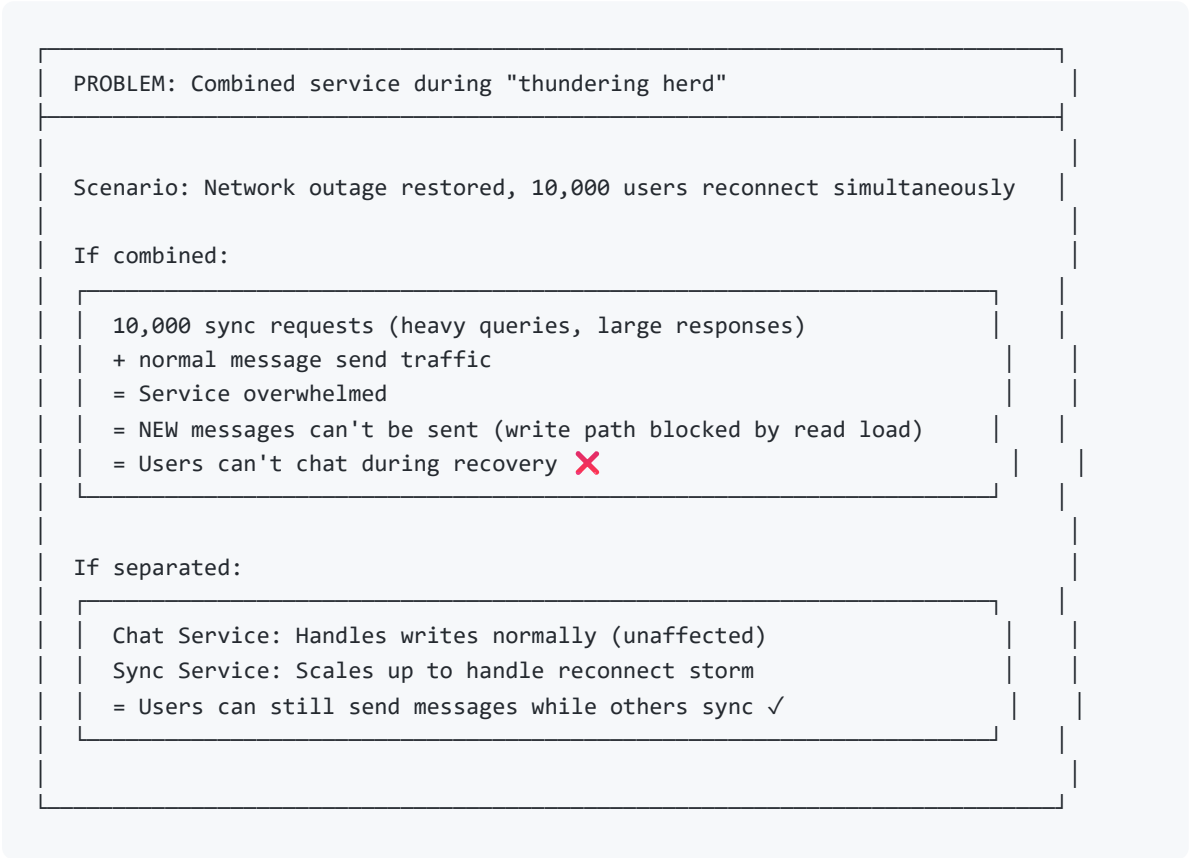
Why Separate Services? (CQRS Pattern)



Aspect	Chat Service (Write)	Sync Service (Read)
Pattern	Async, fire-and-forget	Sync, request-response
Operation	Write + publish event	Query + return data

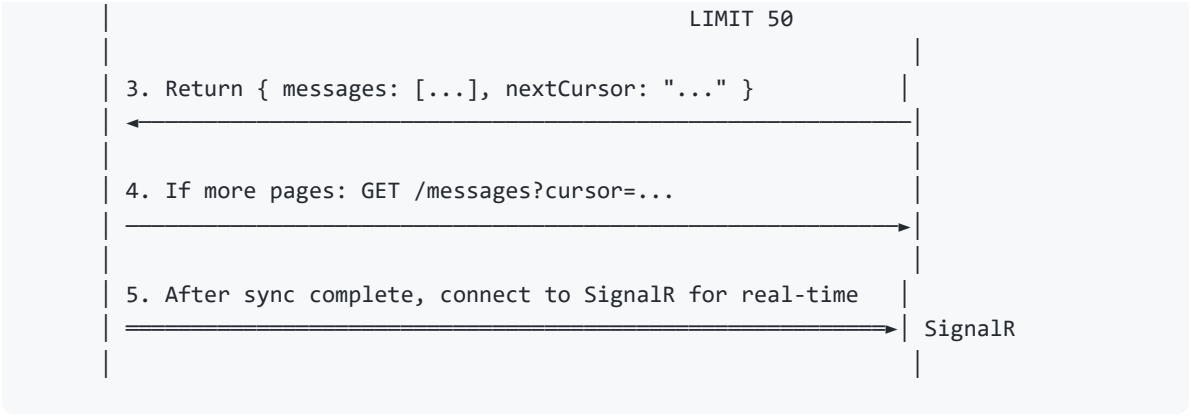
Payload	Small (one message)	Large (hundreds of messages)
Latency	<100ms ack required	Seconds acceptable
Scaling trigger	Messages/second	Concurrent reconnects + data size
Resource needs	CPU (validation, publish)	Memory + IO (large queries)
Failure impact	Message not sent (critical)	User retries (less critical)

Why NOT combine them?



Message Sync Flow:





Sync Strategy:

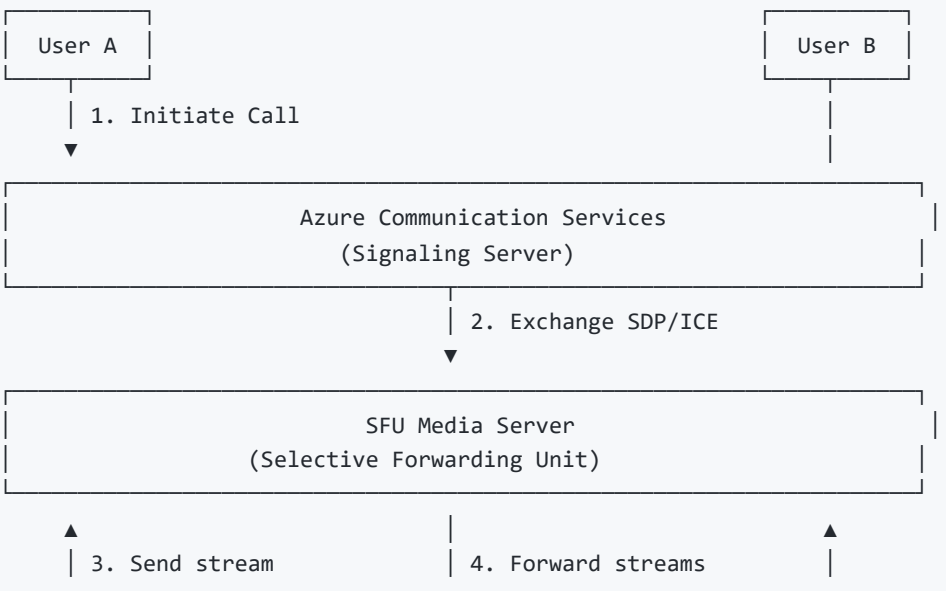
1. Client stores `lastSeenTimestamp` locally (IndexedDB/SQLite)
2. On reconnect, call Sync Service with `lastSeenTimestamp`
3. Sync Service queries Cosmos DB for `timestamp > lastSeen`
4. Return in batches (50 msgs) with cursor for pagination
5. After sync complete, subscribe to SignalR for real-time updates

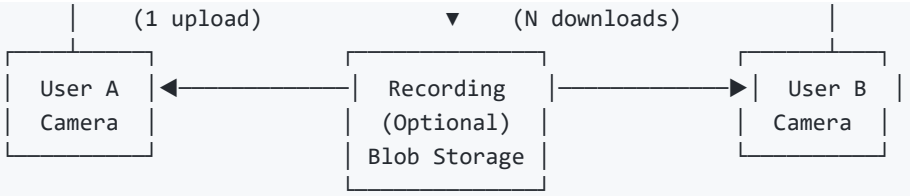
Edge Cases:

- Long offline (days): Paginate, fetch on scroll (don't load all at once)
- Conflicting edits: Server timestamp wins (last-write-wins)
- Deleted messages: Soft-delete flag, client removes from UI
- Multiple devices: Each device tracks own `lastSeenTimestamp`
- Thundering herd: Sync Service auto-scales independently of Chat Service

---

2. Video Call Flow





Steps:

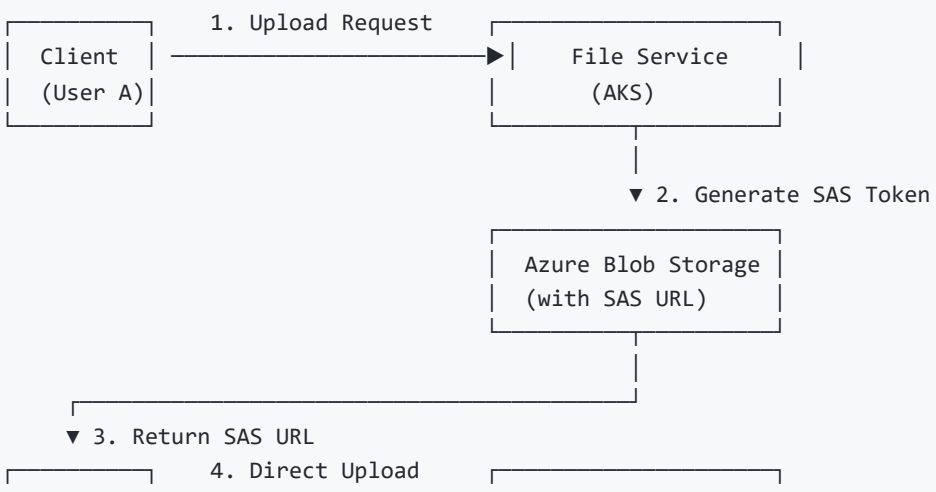
1. User A initiates call → Signaling server notifies User B
2. Exchange SDP offers and ICE candidates for connection setup
3. Each user uploads ONE stream to SFU (simulcast: 1080p/720p/360p)
4. SFU selectively forwards streams to participants based on bandwidth
5. Optional: Recording bot captures and stores to Azure Blob Storage

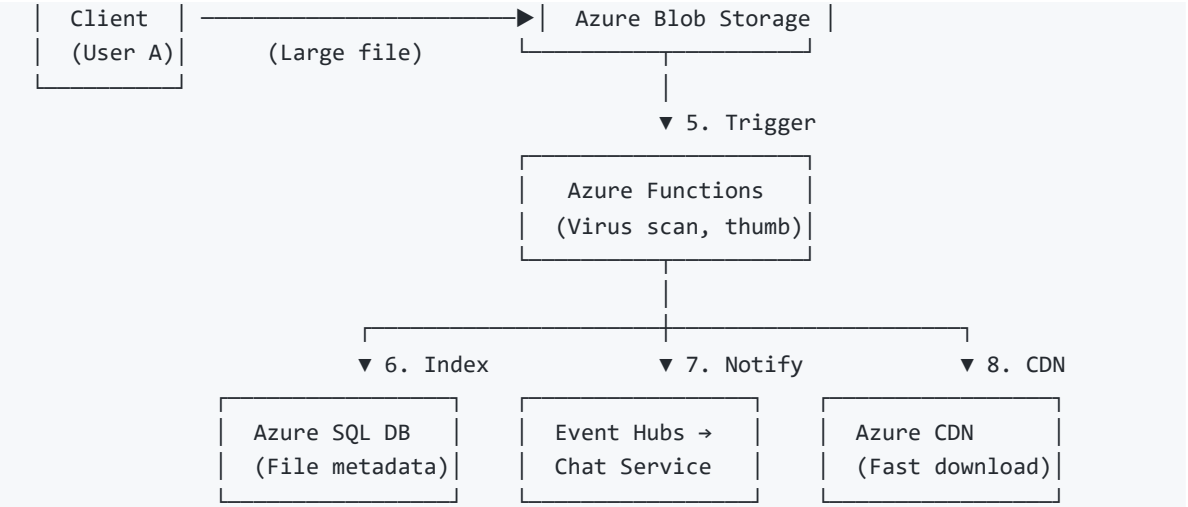
Design Decisions:

Decision	Why This Choice
SFU over Mesh	Mesh = N^2 streams (doesn't scale). SFU = N uploads, selective downloads
SFU over MCU	MCU decodes/re-encodes = high CPU + latency. SFU just forwards
Simulcast (3 qualities)	Sender encodes once at 3 levels; SFU picks per receiver bandwidth
Azure Communication Services	Managed TURN/STUN + SFU, no infrastructure to maintain

---

3. File Sharing Flow





Steps:

1. Client requests upload URL from File Service
2. Service generates time-limited SAS token for Blob Storage
3. Client receives secure upload URL
4. Client uploads directly to Blob Storage (offloads server)
5. Blob trigger fires Azure Function for processing (virus scan, thumbnail)
6. Metadata stored in Azure SQL (filename, size, owner, channel)
7. Notification sent to chat participants via Event Hubs
8. File cached at Azure CDN edge for fast downloads

Design Decisions:

Decision	Why This Choice
SAS token + direct upload	Offloads server; client uploads directly to Blob (scales better)
Blob trigger → Functions	Serverless processing; scales with upload volume
Metadata in SQL, file in Blob	SQL for queries/joins, Blob for cheap large file storage
CDN for downloads	Edge caching; fast delivery globally

Database Configurations

1. Azure Cosmos DB (Messages & Threads)

Setting	Value
Container	`messages`
Partition Key	`/channelId`
Consistency	Session (user sees own writes)
Throughput	Autoscale 400 - 100,000 RU/s
TTL	7 years (compliance) or per-tenant
Multi-region	Enabled (US East, EU West, Asia East)

Document Structure:

```
{
  "id": "msg_uuid",
  "channelId": "channel_123",      // Partition Key
  "senderId": "user_456",
  "content": "Hello team!",
  "timestamp": "2026-01-26T17:00:00Z",
  "threadId": "thread_789",        // For replies
  "type": "text",
  "reactions": [],
  "mentions": ["user_111", "user_222"]
}
```

Why `/channelId` as partition key?

- All messages in a channel queried together
- Even distribution across partitions
- Efficient range queries by timestamp within channel

Why Cosmos DB for messages?

- Write-heavy workload (millions of messages/min)
- Flexible schema (text, files, reactions, edits)
- Multi-region writes for global low latency
- Session consistency = user sees own writes immediately

---

2. Azure SQL Database (Users, Teams, Channels)

Table	Purpose	Key Columns

`Users`	User profiles	`userId (PK)`, `tenantId`, `displayName`, `email`
`Teams`	Team definitions	`teamId (PK)`, `tenantId`, `name`, `createdAt`
`Channels`	Channels in teams	`channelId (PK)`, `teamId (FK)`, `name`, `type`
`TeamMembers`	Team membership	`teamId`, `userId`, `role`, `joinedAt`
`ChannelMembers`	Channel access	`channelId`, `userId`, `role`
`Files`	File metadata	`fileId (PK)`, `channelId`, `blobUrl`, `size`, `uploadedBy`

Indexes:

- `tenantId` on all tables (tenant isolation)
- `teamId` on Channels, TeamMembers
- `channelId` on Files, ChannelMembers

Why SQL for metadata?

- Relational data with joins (user → teams → channels)
- ACID transactions (adding user to team = atomic)
- Complex queries (find all teams user belongs to)
- Read-heavy, infrequent writes

---

3. Azure Cache for Redis (Presence, Sessions, Cache)

Key Pattern	Value	TTL
`presence:{userId}`	`{status, lastSeen, device}`	5 min
`session:{connectionId}`	`{userId, deviceType, region}`	30 min
`typing:{channelId}`	`Set of userIds`	10 sec
`unread:{userId}:{channelId}`	`count`	None
`user:{userId}`	`{profile JSON}`	1 hour

Configuration:

- Cluster: 3 shards with replicas for HA
- Eviction: `allkeys-lru` for cache keys

- Pub/Sub: Used for presence broadcast across instances

Why Redis?

- Sub-millisecond latency for presence checks
- TTL for automatic offline detection
- Pub/Sub for real-time broadcast to all server instances
- Ephemeral data that doesn't need durability

---

4. Azure Blob Storage (Files, Media, Recordings)

Container	Purpose	Access
`attachments`	Chat file attachments	Private (SAS tokens)
`profile-images`	User avatars	Public read via CDN
`recordings`	Meeting recordings	Private (SAS tokens)
`thumbnails`	File/video previews	Public read via CDN
`transcripts`	Meeting transcripts	Private (SAS tokens)

Configuration:

- Lifecycle Policy: Hot → Cool after 30 days, Archive after 1 year
- Soft delete: 14 days retention
- Versioning: Enabled for compliance

Why Blob Storage?

- Cheapest storage for large files (pennies per GB)
- Tiered storage (hot/cool/archive) for cost optimization
- Direct upload via SAS tokens (offloads application servers)
- Integrates with CDN for global delivery

---

5. Azure AI Search (Message & File Search)

Index	Source	Searchable Fields
`messages-index`	Cosmos DB change feed	`content`, `senderId`, `channelId`, `timestamp`

`files-index`	Azure SQL + Blob metadata	`fileName`, `content` (extracted), `uploadedBy`
`people-index`	Azure AD sync	`displayName`, `email`, `department`

Configuration:

- Indexer: Runs every 5 minutes from Cosmos DB change feed
- Security: Filter by `tenantId` in queries
- Features: Fuzzy search, highlighting, autocomplete

Why dedicated Search (not SQL LIKE)?

- Full-text search with ranking and relevance
- Fuzzy matching (typos, synonyms)
- Fast even with millions of messages
- Faceted search (filter by date, sender, channel)

Scalability Considerations

- **Horizontal scaling**: Stateless services in AKS with HPA (Horizontal Pod Autoscaler)
- **Database sharding**: Cosmos DB partitioned by channel_id (messages stay together)
- **Message fanout**: Event Hubs partitioned by team_id, SignalR groups per channel
- **CDN**: Azure CDN for profile pictures, file downloads, static assets
- **Regional deployment**: Active-active in 3+ regions, Cosmos DB multi-master
- **Connection limits**: SignalR service scales to millions of concurrent connections

---

Trade-offs

Decision	Option A	Option B	Recommendation
Message storage	SQL	NoSQL (Cosmos)	**Cosmos** - flexible schema, multi-region
Real-time delivery	Polling	WebSocket	**WebSocket** - low latency
Video architecture	Mesh	SFU	**SFU** - scales to 1000s

Large meetings	SFU only	SFU + CDN broadcast	**Hybrid** - CDN for 100+ attendees
Consistency	Strong	Eventual	**Eventual** - availability > consistency
Recording	Client-side	Server-side	**Server-side** - reliable, compliant
Search	SQL LIKE	Dedicated search	**Azure AI Search** - full-text, fast

---

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
WebSocket	Persistent connections for real-time messaging	Tech Guide
WebRTC	Peer-to-peer video/audio for calls	Tech Guide
Redis	Presence caching, pub/sub for real-time updates	Tech Guide
Cosmos DB	Message storage with global distribution	Tech Guide
Kafka	Event streaming for message ordering	Tech Guide
SignalR	Managed WebSocket service for real-time	Tech Guide
CDN	Static asset delivery, meeting broadcasts	Tech Guide

How to Explain It Well

> **Opening (30 sec)**: "Teams is a real-time collaboration platform. I'll design for 300M DAU supporting chat, video calls up to 1000 participants, and file sharing with 99.99% availability across multiple Azure regions."

> **Structure your answer**:
>
> 1. Clarify requirements (scale, features, compliance)
> 2. Draw high-level architecture with Azure services
> 3. Deep dive into real-time messaging (SignalR + Cosmos)
> 4. Explain video architecture (WebRTC + SFU + simulcast)

5. Discuss large meeting optimization (CDN broadcast)
6. Address scalability and trade-offs

Key talking points:

- "Real-time messaging uses Azure SignalR Service - managed WebSocket at scale"
- "Messages stored in Cosmos DB partitioned by channel for locality"
- "Video uses SFU architecture with simulcast - sender encodes 3 qualities, SFU selects per receiver"
- "Large meetings (100+) switch to CDN broadcast mode for cost efficiency"
- "TURN servers handle NAT traversal for ~20% of calls that can't go P2P"
- "Recording bot joins as participant, streams to Media Services for encoding"

Common Follow-up Questions

1. **How do you handle message ordering?**
    - Cosmos DB timestamp + client-side reordering
    - Event Hubs partitioned by channel ensures order within partition
2. **What if SFU server crashes mid-call?**
    - Client reconnects to different SFU
    - ICE restart negotiates new connection
    - Brief interruption (~2-3 seconds)
3. **How do you prevent echo in calls?**
    - Acoustic Echo Cancellation (AEC) on client
    - WebRTC includes AEC in getUserMedia
4. **How do you handle 1000-person meeting?**
    - Presenters on SFU (bidirectional)
    - Attendees receive via CDN (HLS/DASH broadcast)
    - Q&A/reactions via separate WebSocket channel
5. **How is end-to-end encryption handled?**
    - Currently Teams uses TLS in transit + encryption at rest
    - E2EE for 1:1 calls available (keys never leave client)
    - Group E2EE challenging with SFU (server can't forward what it can't read)

Design a Scalable Azure Service

Key Terms:

Term	Full Name	What it does
AKS	Azure Kubernetes Service	Managed container orchestration platform
VMSS	Virtual Machine Scale Set	Auto-scaling group of identical VMs
HPA	Horizontal Pod Autoscaler	Kubernetes component that scales pods based on metrics
RU	Request Unit	Cosmos DB's throughput currency (1 RU = 1 read of 1KB)
CDN	Content Delivery Network	Edge servers caching content globally
WAF	Web Application Firewall	Protects against web exploits (SQL injection, XSS)
L7 LB	Layer 7 Load Balancer	Routes traffic based on HTTP content (URL, headers)
TTL	Time To Live	How long cached data remains valid

Problem Statement

Design a cloud service on Azure that can handle millions of users with high availability, auto-scaling, and cost efficiency.

Clarifying Questions to Ask

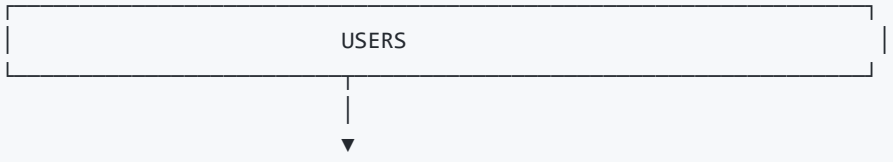
1. **Type of service**: API service, web app, data processing?
2. **Traffic pattern**: Steady load or spiky? Peak times?
3. **Data requirements**: Read-heavy or write-heavy?
4. **Latency requirements**: Real-time or batch acceptable?
5. **Compliance**: Any regulatory requirements (HIPAA, GDPR)?
6. **Budget constraints**: Cost sensitivity?
7. **Global reach**: Single region or multi-region?

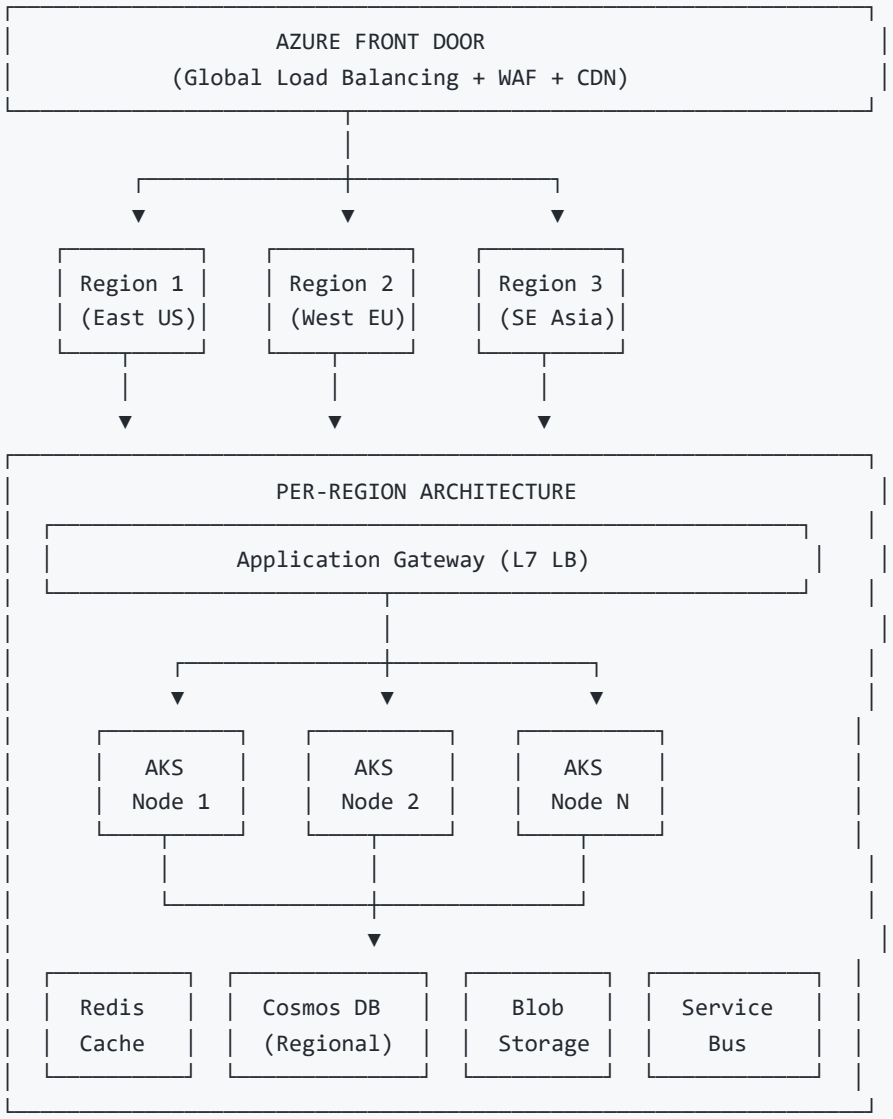
Design Reasoning

> **Core philosophy**: Maximize availability and minimize latency through multi-region active-active deployment on Azure, using managed services that auto-scale to handle millions of users with minimal operational overhead.

Decision	Choice	Why	Alternative Considered
Compute	AKS (Kubernetes)	Better pod density, faster auto-scaling, and portable workloads vs. raw VMs	VMSS (VMs) — slower scaling, lower density, more OS management
Database	Cosmos DB (multi-region)	Turnkey global distribution with tunable consistency and auto-scaling RUs	Azure SQL — single-region write primary, complex geo-replication setup
Global load balancing	Azure Front Door	Anycast routing, built-in WAF, DDoS protection, and CDN in one service	Traffic Manager — DNS-only routing, no L7 inspection or edge caching
Caching	Azure Cache for Redis	Cache-aside pattern reduces Cosmos DB RU consumption for read-heavy workloads	In-app memory cache — not shared across pods, inconsistent after restarts
Async processing	Azure Service Bus	Decouples services for resilience; dead-letter queues for failed message handling	Direct HTTP calls — tight coupling, cascading failures under load
Regional load balancing	Application Gateway (L7)	URL-based routing, SSL offload, and health probes within each region	Azure Load Balancer (L4) — no content-based routing or WAF capability

High-Level Design





API Design

Service APIs

Method	Endpoint	Description
`POST`	`/api/v1/resources`	Create a new resource
`GET`	`/api/v1/resources/{id}`	Get resource by ID
`GET`	`/api/v1/resources?page={n}&size={s}`	List resources with pagination
`PUT`	`/api/v1/resources/{id}`	Update a resource
`DELETE`	`/api/v1/resources/{id}`	Delete a resource

Health & Management APIs

Method	Endpoint	Description
`GET`	`/health`	Health check endpoint
`GET`	`/health/ready`	Readiness probe
`GET`	`/health/live`	Liveness probe
`GET`	`/metrics`	Prometheus metrics endpoint

Rate Limiting Headers

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 999
X-RateLimit-Reset: 1672531200
```

Key Components

Component	Purpose	Azure Service
Global Load Balancer	Route to nearest region	Azure Front Door
Regional Load Balancer	Distribute within region	Application Gateway
Compute	Run application code	AKS or App Service
Cache	Reduce DB load	Azure Cache for Redis
Database	Persistent storage	Cosmos DB (multi-region)
Queue	Async processing	Azure Service Bus
Storage	Files and blobs	Azure Blob Storage
Monitoring	Observability	Azure Monitor + App Insights

Scalability Considerations

- **Auto-scaling**: Use VMSS or AKS HPA based on CPU/memory/custom metrics
- **Database scaling**: Cosmos DB auto-scales RUs, use read replicas
- **Caching strategy**: Cache-aside pattern with Redis, TTL-based invalidation
- **Async processing**: Offload heavy tasks to queues and workers
- **CDN**: Cache static content at edge locations

Trade-offs

Decision	Option A	Option B	Recommendation
Compute	VMs	Containers (AKS)	**AKS** - better density, faster scaling
Database	SQL	Cosmos DB	**Cosmos** - for global distribution
Scaling	Manual	Auto	**Auto** - responds to demand
Regions	Single	Multi	**Multi** - for availability & latency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
AKS/Kubernetes	Container orchestration, auto-scaling	AKS Guide
Cosmos DB	Globally distributed database	Cosmos DB Guide
Redis	Caching layer, session storage	Redis Guide
Load Balancer	Traffic distribution (L7)	Load Balancer Guide
CDN	Edge caching for static content	CDN Guide

How to Explain It Well

> **Opening**: "I'll design a globally distributed Azure service using AKS for compute, Cosmos DB for data, with auto-scaling and multi-region deployment."

> **Key talking points**:
>
> - "Azure Front Door provides global load balancing and routes users to nearest region"
> - "Each region runs independently with AKS clusters that auto-scale"
> - "Cosmos DB with multi-region writes ensures data availability"
> - "Redis cache reduces database load for read-heavy patterns"
> - "Service Bus decouples components for resilience"

---

3. Architect a Multi-Tenant Cloud Solution

Key Terms:

Term	Full Name	What it does
SaaS	Software as a Service	Cloud-hosted software delivered over internet
Tenant	Customer/Organization	A single customer sharing the platform with others
RLS	Row-Level Security	Database feature filtering rows by tenant automatically
JWT	JSON Web Token	Signed token containing user/tenant identity claims
Noisy Neighbor	Resource contention	One tenant consuming resources affecting others

Problem Statement

Design a SaaS platform that serves multiple customers (tenants) with data isolation, customization, and efficient resource utilization.

Clarifying Questions to Ask

1. **Tenant count**: How many tenants? (100s, 1000s, millions?)
2. **Tenant size variance**: Similar sizes or huge variance?
3. **Isolation requirements**: Strict data isolation or shared acceptable?
4. **Customization**: Do tenants need custom features/branding?
5. **Compliance**: Different compliance needs per tenant?
6. **Pricing model**: Per-user, per-feature, usage-based?
7. **Noisy neighbor**: How to prevent one tenant affecting others?

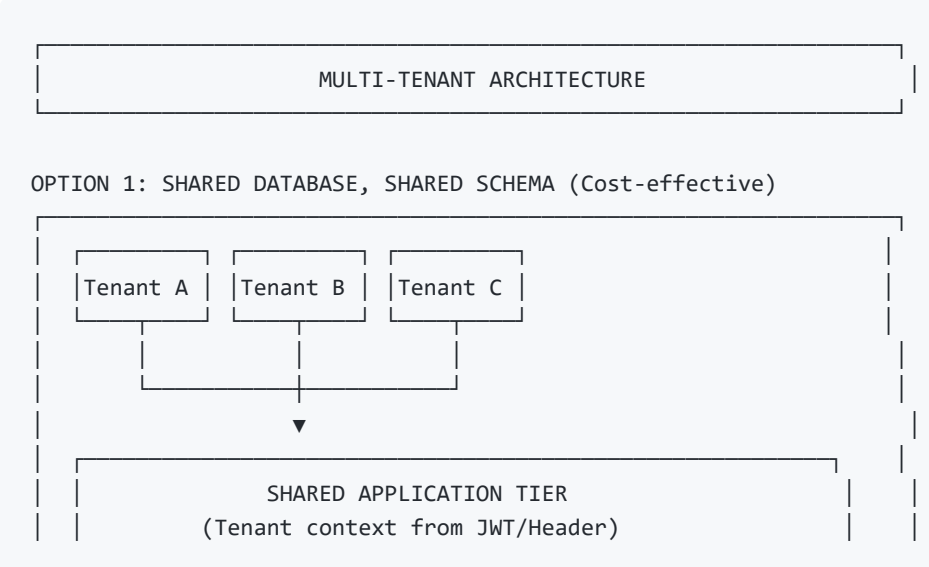
Design Reasoning

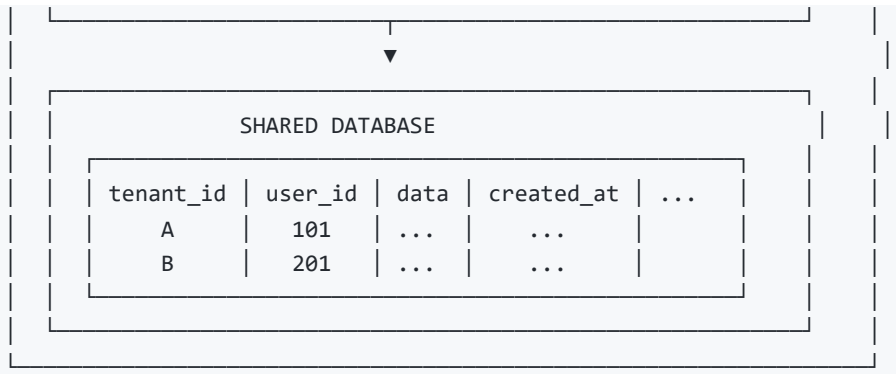
> **Core philosophy**: Use a hybrid tenant isolation model that balances cost efficiency for small tenants with strict isolation for enterprise customers, ensuring no tenant can degrade another's experience.

Decision	Choice	Why	Alternative Considered
Isolation model	Hybrid (shared pool + dedicated)	Small tenants share resources for cost; enterprise tenants get	Shared-only — insufficient isolation for regulated industries

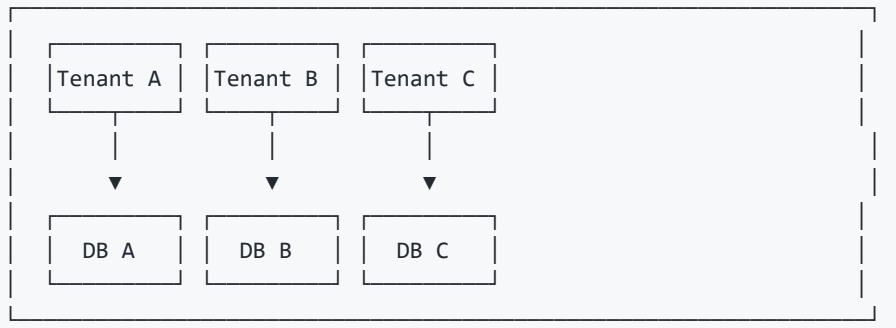
		dedicated DBs for compliance	
Data isolation	Row-Level Security (RLS)	Database-enforced tenant filtering prevents cross-tenant leaks without app-level bugs	Application-level filtering — one missed WHERE clause leaks data
Tenant identification	JWT claims + subdomain	Stateless, verifiable tenant context on every request without extra DB lookup	Header-only (X-Tenant-ID) — easily spoofed without cryptographic verification
Database (shared pool)	Cosmos DB with partition key per tenant	Natural tenant isolation via partition keys; auto-scales RUs across tenants	Single SQL database — hot tenants cause connection pool exhaustion
Resource protection	Per-tenant rate limiting	Prevents noisy neighbor by capping each tenant's request rate and resource consumption	No limits — one tenant's spike degrades service for all others
Configuration	Feature flags per tenant	Enables per-tenant customization and tiered features without separate code deployments	Code branching — unmaintainable as tenant count grows

High-Level Design

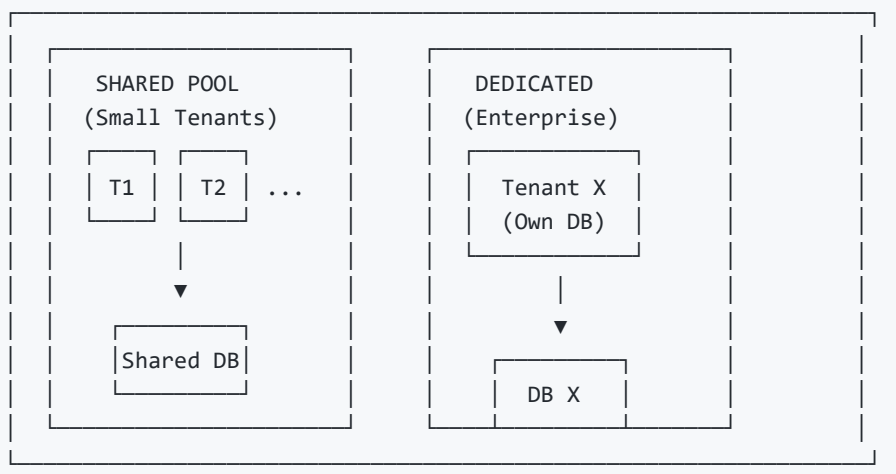




OPTION 2: DATABASE PER TENANT (Better isolation)



OPTION 3: HYBRID (Enterprise tenants get dedicated)



API Design

Tenant Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/tenants`	Provision a new tenant
`GET`	`/api/v1/tenants/{tenantId}`	Get tenant configuration
`PUT`	`/api/v1/tenants/{tenantId}/tier`	Update tenant tier (Basic/Standard/Premium)

`DELETE`	`/api/v1/tenants/{tenantId}`	Deprovision a tenant

Tenant-Scoped Resource APIs

Method	Endpoint	Description
`GET`	`/api/v1/{tenantId}/resources`	List tenant resources
`POST`	`/api/v1/{tenantId}/resources`	Create resource within tenant
`GET`	`/api/v1/{tenantId}/usage`	Get tenant usage metrics

All requests include the `X-Tenant-ID` header for tenant isolation. The API gateway validates tenant context and routes to the appropriate data partition.

Key Components

Component	Purpose	Implementation
Tenant Resolver	Identify tenant from request	Subdomain, header, or JWT claim
Data Isolation	Prevent cross-tenant access	Row-level security or separate DBs
Resource Quotas	Prevent noisy neighbor	Rate limiting per tenant
Config Service	Tenant-specific settings	Feature flags, branding
Billing Service	Usage tracking	Metering and invoicing

Scalability Considerations

- **Tenant sharding**: Distribute tenants across database shards
- **Connection pooling**: Share DB connections across tenants
- **Caching**: Tenant-aware cache keys (tenant:resource:id)
- **Queue isolation**: Separate queues for large tenants
- **Compute isolation**: Dedicated pods for enterprise tenants

Trade-offs

Approach	Pros	Cons	Best For
Shared DB + Schema	Cost-effective, simple	Less isolation	Many small tenants

Schema per Tenant	Good isolation	Migration complexity	Medium tenants
DB per Tenant	Best isolation	Expensive, complex	Enterprise/compliance
Hybrid	Flexible	Complex routing	Mixed tenant sizes

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Cosmos DB	Multi-tenant NoSQL database with partition keys per tenant	Tech Guide
Redis	Tenant-aware caching with prefixed keys (tenant:resource:id)	Tech Guide
AKS/Kubernetes	Compute isolation with dedicated pods for enterprise tenants	Tech Guide
Consistent Hashing	Distribute tenants across database shards evenly	Tech Guide
Rate Limiter	Per-tenant rate limiting to prevent noisy neighbor issues	Tech Guide

How to Explain It Well

> **Opening**: "Multi-tenancy is about serving multiple customers from shared infrastructure while ensuring isolation. I'll discuss three approaches and recommend a hybrid model."

> **Key talking points**:
>
> - "Tenant identification via subdomain (acme.service.com) or JWT claims"
> - "Row-level security in shared DB: every query includes tenant_id filter"
> - "Noisy neighbor prevention: per-tenant rate limits and resource quotas"
> - "Enterprise tenants get dedicated resources for compliance/performance"
> - "Feature flags enable per-tenant customization without code branches"

---

10. Design Collaborative Document Editing

Key Terms:

Term	Full Name	What it does
OT	Operational Transformation	Algorithm transforming concurrent edits to maintain consistency
CRDT	Conflict-free Replicated Data Type	Data structure that auto-merges without conflicts
WebSocket	Full-duplex protocol	Real-time bidirectional communication
Tombstone	Deleted marker	Marks deleted items instead of removing (for sync)
Pub/Sub	Publish/Subscribe	Messaging pattern where senders broadcast to subscribers

Problem Statement

Design a real-time collaborative document editor like Google Docs or Microsoft Word Online where multiple users can edit simultaneously.

Clarifying Questions to Ask

1. **Concurrency**: How many simultaneous editors? (50? 100?)
2. **Document size**: Max document size? (10MB?)
3. **Latency**: Max acceptable delay for seeing others' edits? (<100ms?)
4. **Offline support**: Allow offline editing and sync later?
5. **Features**: Comments, suggestions, version history?
6. **Conflict resolution**: How to handle simultaneous edits?
7. **Rich content**: Text only or images, tables, formatting?

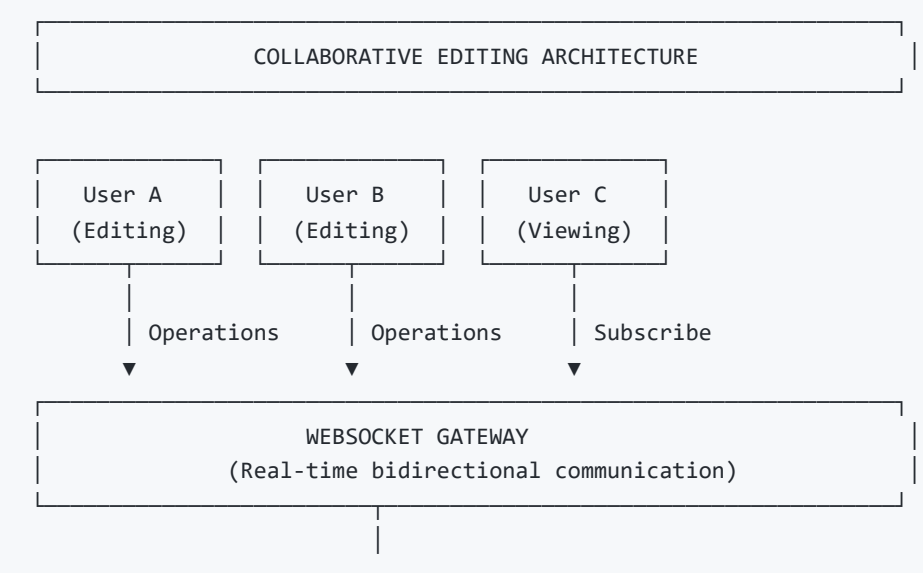
Design Reasoning

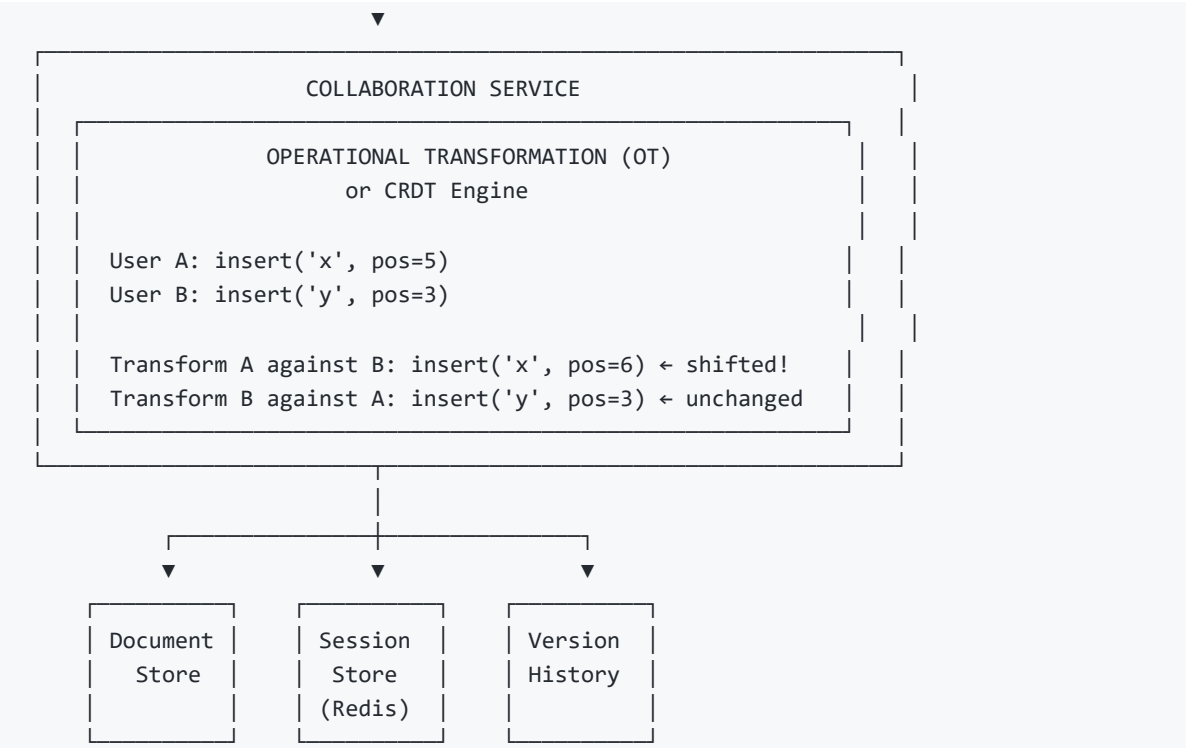
> Core philosophy: Real-time collaboration demands conflict-free convergence across all clients, making eventual consistency and low-latency change propagation the primary drivers of every architectural choice.

Decision	Choice	Why	Alternative Considered

Conflict Resolution	CRDT	Auto-merges without central coordination; supports offline editing and sync	OT — requires a central server to transform operations, harder to scale horizontally
Real-time Transport	WebSocket (SignalR)	Full-duplex, sub-100ms latency needed for keystroke-level collaborative edits	HTTP polling — too high latency for real-time multi-user editing
Document Store	Cosmos DB	Flexible schema for CRDT tree structures; low-latency global reads	PostgreSQL — rigid relational schema poorly fits nested CRDT state
Session & Presence Store	Redis Pub/Sub	In-memory speed for cursor tracking and active editor lists across servers	Database polling — too slow for real-time cursor position updates
Version History	Event Store (append-only log)	Natural fit for operation history; enables replay, undo, and time-travel	Full-document snapshots only — expensive storage, loses granular edit history
Sync Strategy	Batched operations (50ms)	Reduces network overhead vs. per-keystroke while still feeling real-time	Per-keystroke sync — excessive bandwidth and server load for large sessions

High-Level Design





API Design

Document APIs

Method	Endpoint	Description
`POST`	`/api/v1/documents`	Create a new document
`GET`	`/api/v1/documents/{docId}`	Get document content and metadata
`GET`	`/api/v1/documents/{docId}/history`	Get revision history
`GET`	`/api/v1/documents/{docId}/history/{version}`	Get specific version
`DELETE`	`/api/v1/documents/{docId}`	Delete a document

Collaboration APIs

Method	Endpoint	Description
`POST`	`/api/v1/documents/{docId}/share`	Share document with users
`GET`	`/api/v1/documents/{docId}/collaborators`	List active collaborators

`GET`	`/api/v1/documents/{docId}/cursors`	Get collaborator cursor positions

WebSocket Events (Real-Time Editing)

Event	Direction	Payload
`op.apply`	Client → Server	`{ docId, operation, baseVersion }`
`op.broadcast`	Server → Clients	`{ docId, operation, version, userId }`
`cursor.move`	Client → Server	`{ docId, position, selection }`
`cursor.update`	Server → Clients	`{ userId, position, selection, color }`
`user.joined`	Server → Clients	`{ userId, name }`
`user.left`	Server → Clients	`{ userId }`

Key Components

Component	Purpose	Technology
WebSocket Gateway	Real-time sync	SignalR / Socket.io
OT/CRDT Engine	Conflict resolution	Custom implementation
Document Store	Persist documents	Cosmos DB / MongoDB
Session Store	Active editors	Redis
Version History	Track changes	Event store
Presence Service	Show active cursors	Redis pub/sub

Operational Transformation (OT)

```
PROBLEM: Concurrent edits can conflict

Initial: "HELLO"
User A: insert('X', 2) → "HEXLLO"
User B: delete(3)      → "HELO"

If both applied blindly:
  A then B: "HEXLLO" → delete(3) → "HEXLO" (wrong!)
  B then A: "HELO" → insert('X', 2) → "HEXLO" (wrong!)

OT SOLUTION: Transform operations
```

```
When A receives B's operation:
  transform(insert('X', 2), delete(3))
  → insert('X', 2) (position unaffected, deletion was after)

When B receives A's operation:
  transform(delete(3), insert('X', 2))
  → delete(4) (position shifted right by 1)

Result: Both converge to "HEXLO" correctly!
```

CRDT Alternative

```
CRDT (Conflict-free Replicated Data Type):

Each character has a unique, ordered ID:
  "HELLO" → [(id1,'H'), (id2,'E'), (id3,'L'), (id4,'L'), (id5,'O')]

Insertions use IDs between existing characters:
  User A inserts 'X' after id2: (id2.5, 'X')
  User B deletes id3: mark (id3,'L') as tombstone

Merge is automatic - no transformation needed!
  Final: [(id1,'H'), (id2,'E'), (id2.5,'X'), (id4,'L'), (id5,'O')]
  = "HEXLO"

Pros: Simpler, works offline
Cons: Storage overhead (IDs), garbage collection
```

Data Model

```
  // Document
  {
    "doc_id": "doc123",
    "title": "Project Plan",
    "content": [...],  // OT operations or CRDT state
    "version": 1542,
    "owner_id": "user456",
    "collaborators": ["user789", "user012"],
    "created_at": "2024-01-15T10:00:00Z",
    "updated_at": "2024-01-15T14:30:00Z"
  }

  // Operation (for OT)
  {
    "op_id": "op789",
    "doc_id": "doc123",
    "user_id": "user456",
    "base_version": 1541,
```

```
    "operation": {
      "type": "insert",
      "position": 42,
      "text": "hello"
    },
    "timestamp": "2024-01-15T14:30:00Z"
  }
```

Scalability Considerations

- **Document sharding**: One collaboration session per server
- **Sticky sessions**: Route users of same doc to same server
- **Batching**: Batch operations (send every 50ms, not per keystroke)
- **Compression**: Delta compression for operation history
- **Snapshots**: Periodic snapshots to limit operation replay

Trade-offs

Decision	Option A	Option B	Recommendation
Algorithm	OT	CRDT	**CRDT** - simpler, offline-friendly
Sync	Per-keystroke	Batched	**Batched** - less overhead
Storage	Full doc	Operations	**Hybrid** - snapshots + ops
Cursors	Sync positions	Don't sync	**Sync** - better collaboration

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
WebSocket	Real-time operation sync between clients	Tech Guide
Redis	Presence tracking and cursor pub/sub	Tech Guide
Raft	Consensus for operation ordering	Tech Guide
SignalR	Managed real-time connection service	Tech Guide

How to Explain It Well

> **Opening**: "Collaborative editing requires real-time sync with conflict resolution. I'll use CRDTs to ensure all users converge to the same state."

> **Key talking points**:
>
> - "CRDTs give each character a unique ID - merges are automatic"
> - "WebSocket for real-time, batch operations every 50ms"
> - "Presence shows other users' cursors using Redis pub/sub"
> - "Periodic snapshots avoid replaying entire operation history"
> - "Offline: queue operations locally, sync when reconnected"

---

12. Design Distributed File Storage

Key Terms:

Term	Full Name	What it does
CDC	Content-Defined Chunking	Splits files at content-based boundaries (not fixed size)
SHA-256	Secure Hash Algorithm 256-bit	Hash function for content-addressable storage
Dedup	Deduplication	Storing identical content once, referenced by hash
Delta Sync	Incremental synchronization	Only sync changed blocks, not entire file
EB	Exabyte	1 million terabytes (10^18 bytes)
LSM Tree	Log-Structured Merge Tree	Write-optimized storage engine

Problem Statement

Design a cloud file storage system like OneDrive or Dropbox that syncs files across devices with versioning and sharing capabilities.

Clarifying Questions to Ask

1. **Scale**: How many users? Total storage? (500M users, 1EB?)
2. **File sizes**: Average and max file size? (avg 1MB, max 50GB?)
3. **Sync latency**: How fast should changes sync? (<5 seconds?)
4. **Versioning**: How many versions to keep? (30 days?)
5. **Sharing**: Public links, user sharing, permissions?
6. **Offline**: Support offline access and sync?
7. **Deduplication**: Dedupe identical files across users?

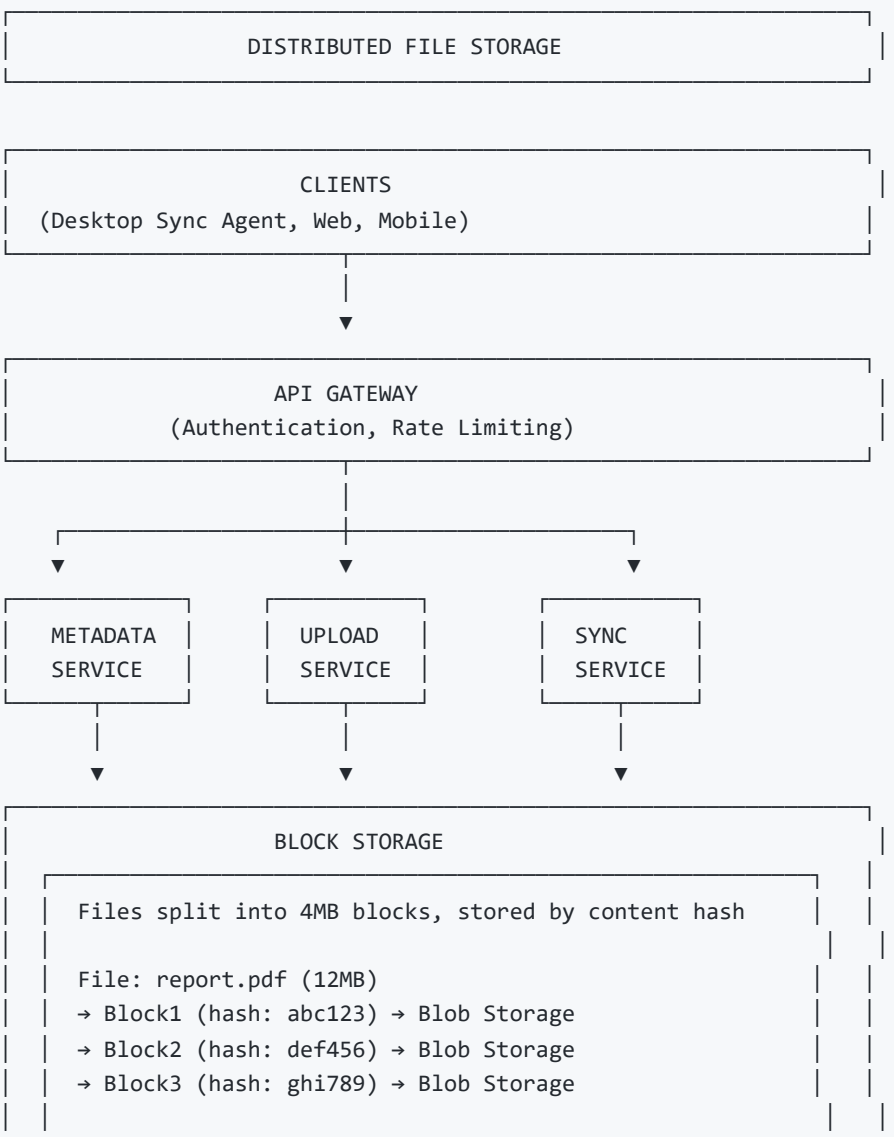
Design Reasoning

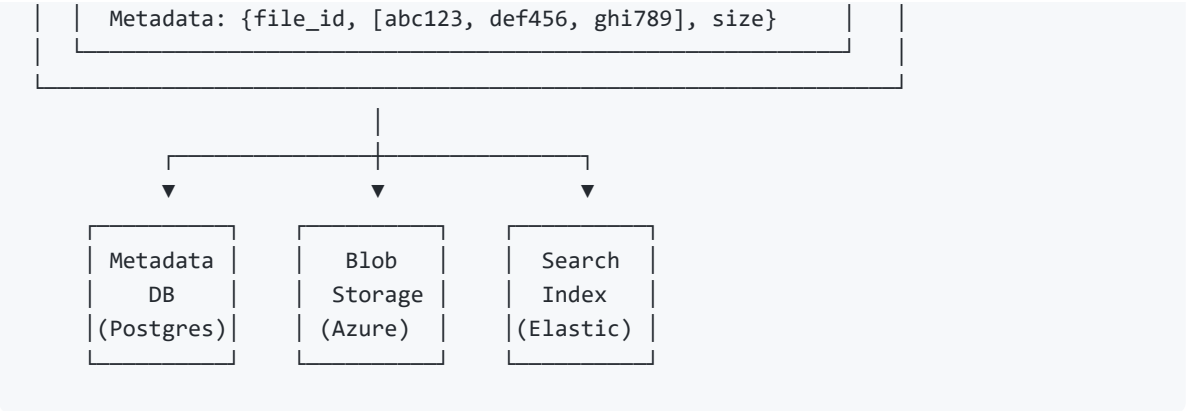
> **Core philosophy**: Split files into content-addressed blocks to enable deduplication, efficient delta sync, and scalable storage — treating file management as a metadata problem layered on top of cheap blob storage.

Decision	Choice	Why	Alternative Considered
Chunking Strategy	Content-Defined Chunking (CDC)	Small edits change only 1-2 chunks; stable boundaries maximize dedup	Fixed-size chunks — single byte insert shifts all boundaries, invalidating dedup
Block Storage	Azure Blob Storage	Virtually unlimited scale, built-in tiering (hot/cool/archive), 99.999% durability	Self-managed distributed storage — high operational burden, harder to scale
Metadata Database	PostgreSQL	Strong consistency for file hierarchy, sharing permissions, and relational joins	NoSQL — file trees need parent-child joins that document stores handle poorly
Deduplication	SHA-256 content hashing	Store each unique block once across all users; ~50% storage savings in practice	No dedup — doubles storage cost, wastes bandwidth re-uploading identical blocks
Sync Mechanism	WebSocket push + Kafka events	Instant cross-device notification; Kafka	Client polling — delays sync, wastes

		provides durable change log for reliability	bandwidth with empty poll responses
File Search	Elasticsearch	Full-text content search with fast indexing across user's file library	Database LIKE queries — too slow for content search at 500M-user scale
Storage Tiering	Hot/Cool/Archive lifecycle policies	80% cost reduction for old file versions while keeping recent files on fast storage	Single hot tier — pays premium prices for rarely accessed historical versions

High-Level Design





API Design

File APIs

Method	Endpoint	Description
`POST`	`/api/v1/files/upload`	Upload a file (multipart)
`POST`	`/api/v1/files/upload/init`	Initialize chunked upload
`PUT`	`/api/v1/files/upload/{uploadId}/chunk/{n}`	Upload a chunk
`POST`	`/api/v1/files/upload/{uploadId}/complete`	Complete chunked upload
`GET`	`/api/v1/files/{fileId}`	Download a file
`GET`	`/api/v1/files/{fileId}/metadata`	Get file metadata
`DELETE`	`/api/v1/files/{fileId}`	Delete a file

Folder APIs

Method	Endpoint	Description
`POST`	`/api/v1/folders`	Create a folder
`GET`	`/api/v1/folders/{folderId}/contents`	List folder contents
`PUT`	`/api/v1/files/{fileId}/move`	Move file to another folder

Sync APIs

Method	Endpoint	Description
`GET`	`/api/v1/sync/changes?since={timestamp}`	Get changes since last sync

`POST`	`/api/v1/sync/conflict/resolve`	Resolve sync conflict

Key Components

Component	Purpose	Technology
Metadata Service	File/folder hierarchy	PostgreSQL
Block Service	Store file chunks	Azure Blob Storage
Sync Service	Track changes, notify	WebSocket + Kafka
Dedup Service	Content-addressable storage	Hash-based lookup
Sharing Service	Permissions, links	PostgreSQL
Search Service	File content search	Elasticsearch

Chunking and Deduplication

```
FILE CHUNKING (Rolling Hash / Rabin Fingerprint):
┌─────────────────────────────────────────────────────────────────┐
│  Instead of fixed 4MB blocks, use content-defined chunking:     │
│                                                                 │
│  File content: [====================]                          │
│  Fixed chunks: [====][====][====][==]  (inefficient for edits) │
│  CDC chunks:   [===][=====][====][===] (boundaries by content)  │
│                                                                 │
│  Benefit: Small edit = only 1-2 chunks change, not all          │
└─────────────────────────────────────────────────────────────────┘

DEDUPLICATION:
┌─────────────────────────────────────────────────────────────────┐
│  Block stored by content hash (SHA-256)                         │
│                                                                 │
│  User A uploads: file1.pdf → blocks [abc, def, ghi]             │
│  User B uploads: file2.pdf → blocks [abc, xyz, ghi]             │
│                                                                 │
│  Storage: [abc, def, ghi, xyz] ← only 4 blocks, not 6           │
│  Savings: ~50% storage reduction in practice                    │
└─────────────────────────────────────────────────────────────────┘
```

Sync Protocol

```
CLIENT-SERVER SYNC:

1. Client tracks local changes (file watcher)
```

```
2. Client sends metadata diff to server:
   {
     "changes": [
       {"path": "/docs/file.txt", "action": "modified", "hash": "abc123"}
     ],
     "last_sync": 1705312345
   }

3. Server responds with:
   {
     "upload_required": ["/docs/file.txt"],
     "download_required": ["/photos/new.jpg"],
     "conflicts": []
   }

4. Client uploads changed blocks
5. Server notifies other clients via WebSocket

CONFLICT RESOLUTION:
  - Last-write-wins (simple)
  - Create conflict copy: "file (conflict).txt"
  - Let user resolve manually
```

Database Schema

```sql
-- Files and Folders
CREATE TABLE files (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    parent_id BIGINT,         -- folder hierarchy
    name VARCHAR(255),
    is_folder BOOLEAN,
    size BIGINT,
    content_hash VARCHAR(64), -- for quick comparison
    version INT,
    created_at TIMESTAMP,
    modified_at TIMESTAMP,
    deleted_at TIMESTAMP      -- soft delete for trash
);

-- Block References
CREATE TABLE file_blocks (
    file_id BIGINT,
    block_index INT,
    block_hash VARCHAR(64),   -- content-addressable
    PRIMARY KEY (file_id, block_index)
);

-- Sharing
CREATE TABLE shares (
    id BIGINT PRIMARY KEY,
    file_id BIGINT,
```

```
    shared_with_user_id BIGINT,
    permission VARCHAR(20),   -- view, edit
    link_token VARCHAR(64),   -- for public links
    expires_at TIMESTAMP
);
```

Scalability Considerations

- **Chunking**: 4MB chunks balance dedup vs. overhead
- **Metadata sharding**: Shard by user_id
- **Blob storage**: Azure Blob with hot/cool/archive tiers
- **Sync optimization**: Delta sync (only changed blocks)
- **CDN**: Cache popular shared files at edge

Trade-offs

Decision	Option A	Option B	Recommendation
Chunking	Fixed size	Content-defined	**CDC** - better dedup
Sync	Polling	Push (WebSocket)	**Push** - real-time
Conflicts	Auto-merge	User resolve	**Conflict copy** - safe
Storage tiers	Single tier	Hot/cool/archive	**Tiered** - cost savings

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Consistent Hashing	Chunk distribution across storage nodes	Tech Guide
Raft	Consensus for metadata and replication	Tech Guide
LSM Tree	Efficient metadata storage engine	Tech Guide

How to Explain It Well

> **Opening**: "File storage needs efficient sync, deduplication, and versioning. I'll design for 500M users with petabytes of data."

> **Key talking points**:
>
> - "Content-defined chunking: edit 1 byte, upload 1 chunk not whole file"
> - "Deduplication by content hash saves ~50% storage"
> - "WebSocket push for instant sync across devices"
> - "Blob storage tiers: hot for recent, archive for old versions"
> - "Conflict detection via content hash comparison"

---

31. Design Teams Video Conferencing

Key Terms:

Term	Full Name	What it does
WebRTC	Web Real-Time Communication	Browser API for peer-to-peer audio/video
SFU	Selective Forwarding Unit	Server that forwards video without transcoding
MCU	Multipoint Control Unit	Server that mixes/transcodes all streams
TURN	Traversal Using Relays around NAT	Relay server when direct connection fails
ICE	Interactive Connectivity Establishment	Protocol to find best connection path
Simulcast	Multiple quality streams	Sender encodes multiple resolutions
NAT	Network Address Translation	Router hiding local IPs behind public IP

Problem Statement

Design a video conferencing system like Microsoft Teams Meetings that supports thousands of concurrent meetings with features like screen sharing, recording, and live captions.

Clarifying Questions to Ask

1. **Scale**: Max participants per meeting? Concurrent meetings globally?
2. **Features**: Video, audio, screen share, recording, live captions?

3. **Latency**: Max acceptable delay? (<150ms for real-time?)
4. **Quality**: Adaptive bitrate? Resolution options?
5. **Recording**: Where stored? Who can access?
6. **Breakout rooms**: Support splitting into smaller groups?
7. **Integration**: Calendar, chat, file sharing during meeting?

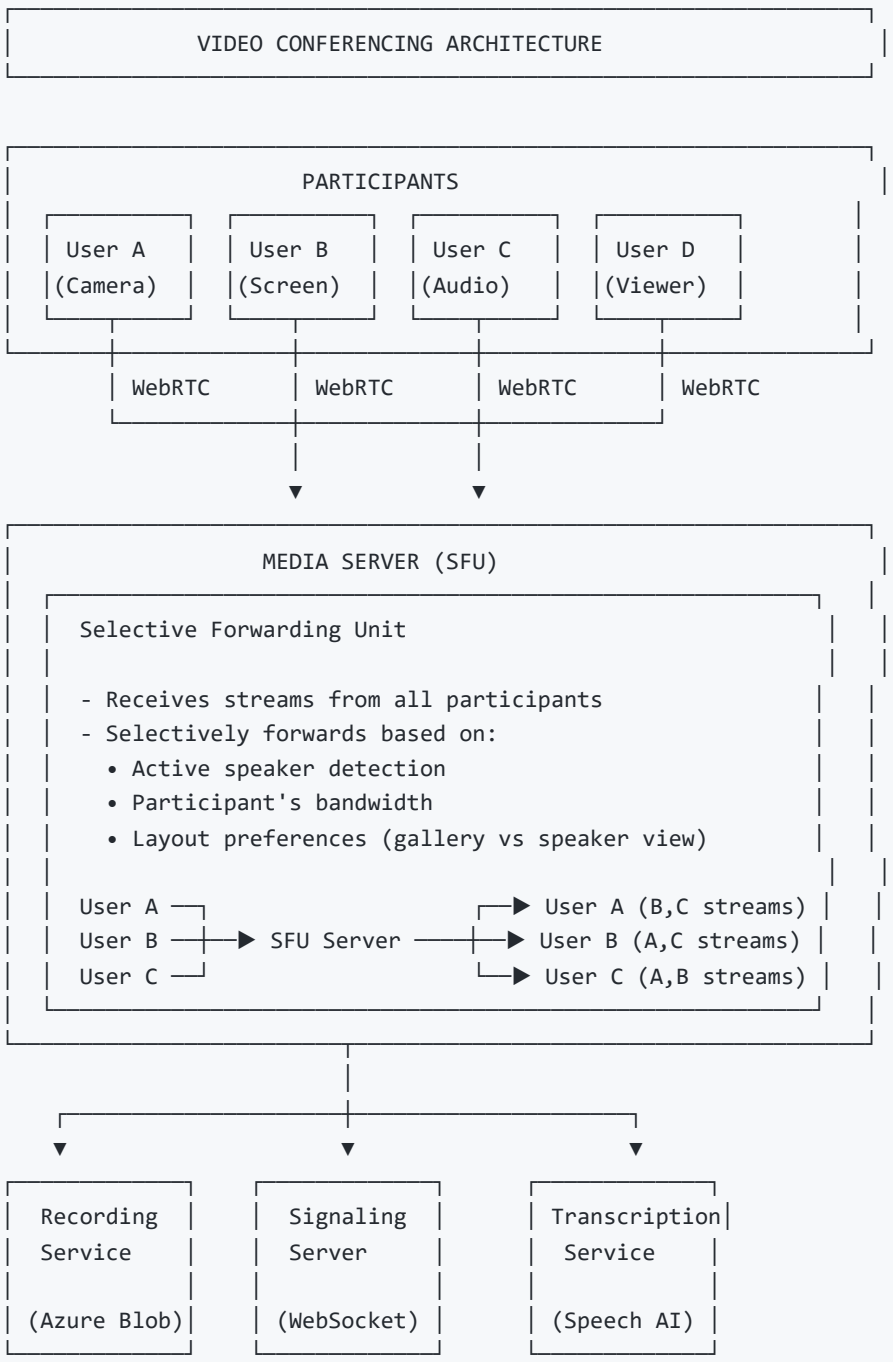
Design Reasoning

> **Core philosophy**: Minimize latency and maximize video quality through selective forwarding and adaptive streaming, keeping media routing on the server while avoiding costly transcoding.

Decision	Choice	Why	Alternative Considered
Media architecture	SFU (Selective Forwarding Unit)	Lower server CPU, no quality loss from transcoding; scales to 1000 participants	MCU (Multipoint Control Unit) — high CPU cost, quality loss from mixing streams
Streaming protocol	WebRTC	Browser-native, no plugin required, built-in NAT traversal and encryption	Custom protocol — requires client installation, longer development cycle
Signaling transport	WebSocket (SignalR)	Low-latency bidirectional signaling for ICE negotiation and session setup	HTTP polling — too slow for real-time session negotiation
Quality adaptation	Simulcast (multiple resolutions)	Receivers pick quality based on available bandwidth; no server transcoding needed	Single stream — poor experience on slow connections, no adaptation
NAT traversal	TURN relay servers	Guarantees connectivity when direct P2P fails (~15% of users behind symmetric NAT)	STUN only — fails for users behind symmetric NAT
Recording	Server-side composite	Reliable capture regardless of participant's client state or disconnection	Client-side recording — unreliable, depends on participant's device and connection

Live captions	Azure Speech Services	Real-time transcription with enterprise-grade accuracy and multi-language support	On-device speech-to-text — inconsistent quality, high client CPU usage

High-Level Design



API Design

Meeting APIs

Method	Endpoint	Description
`POST`	`/api/v1/meetings`	Create a meeting
`POST`	`/api/v1/meetings/{meetingId}/join`	Join a meeting
`POST`	`/api/v1/meetings/{meetingId}/leave`	Leave a meeting
`GET`	`/api/v1/meetings/{meetingId}`	Get meeting details
`GET`	`/api/v1/meetings/{meetingId}/participants`	List participants

Media Control APIs

Method	Endpoint	Description
`PUT`	`/api/v1/meetings/{meetingId}/media`	Toggle audio/video
`POST`	`/api/v1/meetings/{meetingId}/screenshare/start`	Start screen sharing
`POST`	`/api/v1/meetings/{meetingId}/screenshare/stop`	Stop screen sharing
`PUT`	`/api/v1/meetings/{meetingId}/layout`	Change video layout

Signaling WebSocket Events

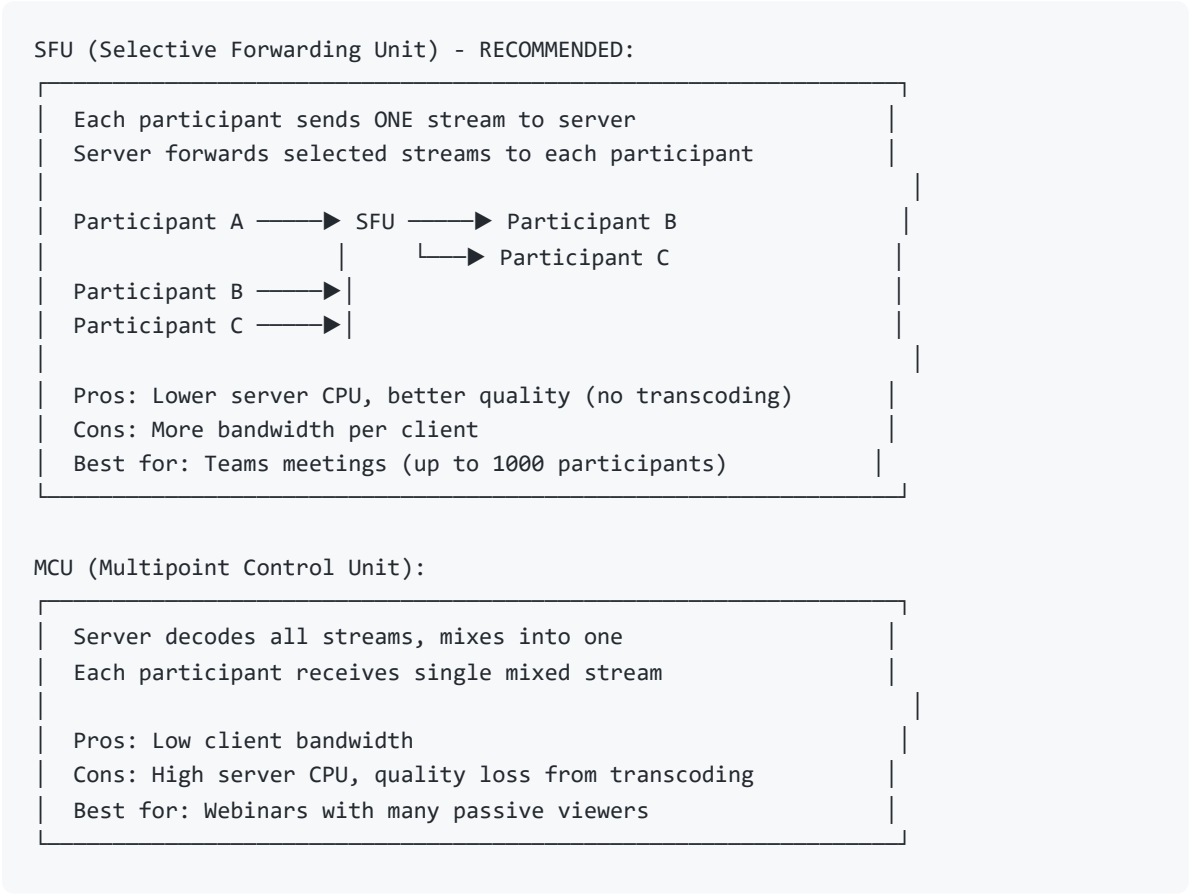
Event	Direction	Payload
`participant.joined`	Server → Client	`{ userId, name, mediaState }`
`participant.left`	Server → Client	`{ userId }`
`media.offer`	Client → Server	`{ sdp }`
`media.answer`	Server → Client	`{ sdp }`
`media.ice`	Both	`{ candidate }`
`media.state`	Server → Client	`{ userId, audio, video }`

Key Components

Component	Purpose	Technology
Signaling Server	Session setup, ICE negotiation	WebSocket

SFU (Selective Forwarding)	Route media streams	Mediasoup, Janus
TURN Server	NAT traversal relay	CoTURN
Recording Service	Capture and store meetings	FFmpeg + Blob Storage
Transcription	Live captions	Azure Speech Services
Quality Monitor	Adapt to network conditions	Custom metrics

Media Architecture: SFU vs MCU



Scalability Considerations

- **Regional SFU servers**: Deploy media servers close to participants
- **Cascading**: Connect SFUs across regions for global meetings
- **Simulcast**: Sender transmits multiple quality levels
- **Bandwidth estimation**: Adapt quality based on network conditions
- **Large meetings**: Switch to broadcast mode for 100+ participants

Trade-offs

Decision	Option A	Option B	Recommendation
Architecture	SFU	MCU	**SFU** - better quality
Protocol	WebRTC	Custom	**WebRTC** - browser native
Quality	Fixed	Adaptive	**Adaptive** - handles poor networks
Recording	Client-side	Server-side	**Server** - reliable

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
WebRTC	Peer-to-peer media streaming protocol	Tech Guide
SignalR	Real-time signaling and session management	Tech Guide
CDN	Media relay and content distribution	Tech Guide

How to Explain It Well

> **Opening**: "Video conferencing needs low latency and adaptive quality. I'll use SFU architecture with WebRTC for peer-to-peer media."

> **Key talking points**:
>
> - "SFU: each user sends one stream, server selectively forwards"
> - "Simulcast: sender transmits 3 qualities, receiver picks based on bandwidth"
> - "TURN servers for NAT traversal when direct connection fails"
> - "Active speaker detection reduces streams to forward"
> - "Recording is server-side composite for reliability"

---

32. Design Xbox Live Leaderboard

Key Terms:

Term	Full Name	What it does

ZADD	Redis Sorted Set Add	Adds member with score to sorted set
ZRANK	Redis Sorted Set Rank	Gets rank of member (0-indexed)
ZREVRANGE	Redis Reverse Range	Gets members by rank, highest first
Sorted Set	Redis data structure	Collection sorted by score, O(log N) operations
TTL	Time To Live	Auto-expiration for time-based leaderboards
Anti-cheat	Score validation	ML/rules to detect invalid scores

Problem Statement

Design a global leaderboard system for Xbox Live that ranks millions of players in real-time across multiple games and categories.

Clarifying Questions to Ask

1. **Scale**: How many players? Games? (100M players, 1000 games?)
2. **Update frequency**: Real-time or periodic updates?
3. **Leaderboard types**: Global, friends, regional, time-based?
4. **Query patterns**: Top N, rank of specific player, nearby ranks?
5. **Score updates**: Frequency per player? (every match?)
6. **Historical**: Keep historical leaderboards? (weekly, monthly?)
7. **Cheating**: How to handle cheating/invalid scores?

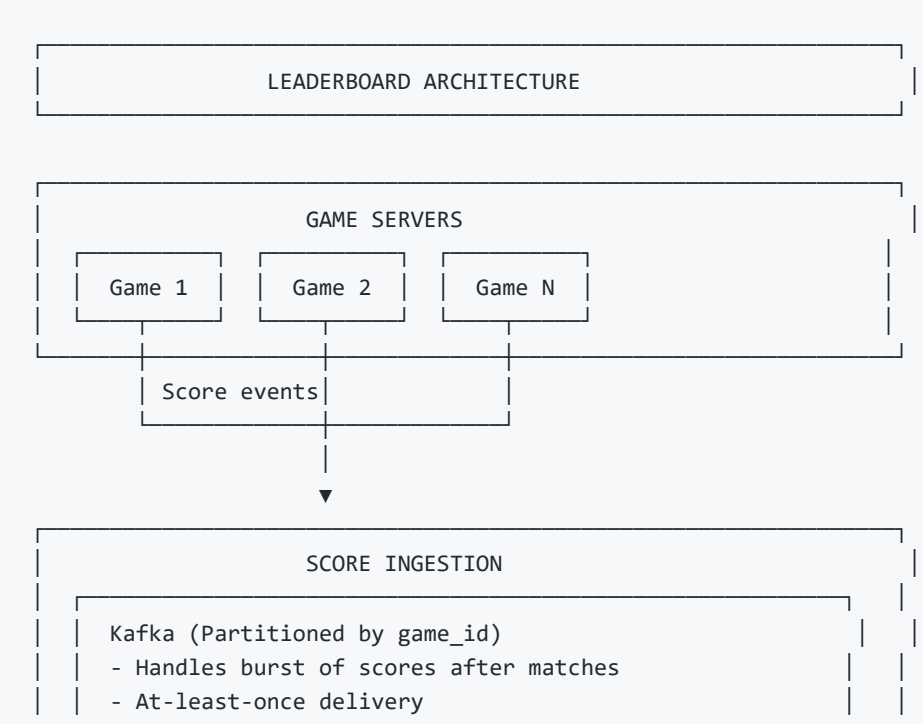
Design Reasoning

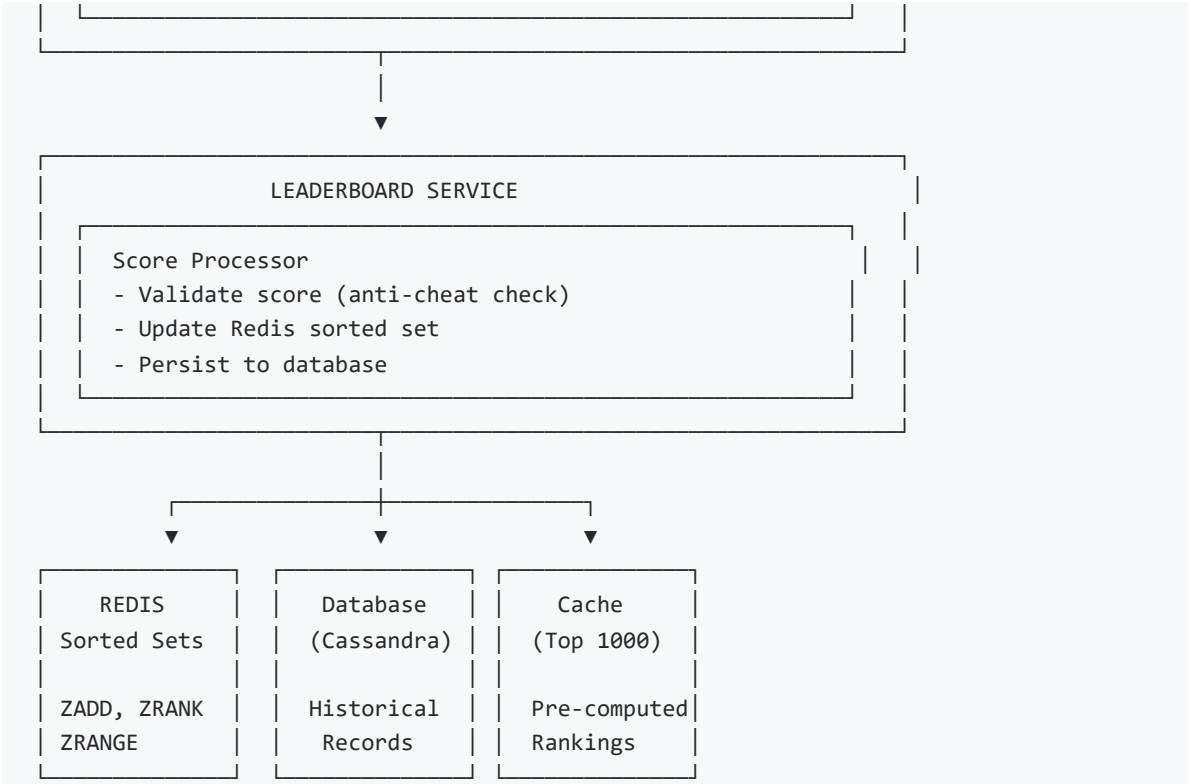
> **Core philosophy**: Use Redis sorted sets as the primary ranking engine for O(log N) rank operations, with event-driven ingestion to absorb post-match score bursts at massive scale.

Decision	Choice	Why	Alternative Considered
Ranking store	Redis Sorted Sets	O(log N) ZADD/ZRANK for millions of players; purpose-built for leaderboards	SQL database — O(N) rank calculation with COUNT(*), too slow at 100M players

Score ingestion	Kafka (partitioned by game_id)	Buffers post-match score bursts without overwhelming Redis	Direct Redis writes — risks overload when thousands of matches end simultaneously
Historical persistence	Cassandra	Append-friendly, handles high write throughput for historical score records	PostgreSQL — single-writer bottleneck at global scale
Friends leaderboard	On-demand (ZSCORE per friend, client-side sort)	Friend lists change frequently; pre-computation is stale immediately	Pre-computed per user — wasted resources, stale when friendships change
Time-based leaderboards	Separate Redis keys per period	Clean reset via key rotation; no data corruption or complex filtering	Single key with timestamp filters — complex range queries, no atomic reset
Anti-cheat	ML anomaly detection at ingestion	Catches statistical outliers before they reach the leaderboard	Manual review — too slow for millions of score submissions per day

High-Level Design





API Design

Leaderboard APIs

Method	Endpoint	Description
`GET`	`/api/v1/leaderboards/{boardId}/top?limit=100`	Get top players
`GET`	`/api/v1/leaderboards/{boardId}/around/{userId}?range=10`	Get players around a user
`GET`	`/api/v1/leaderboards/{boardId}/rank/{userId}`	Get user's rank and score
`GET`	`/api/v1/leaderboards/{boardId}/friends/{userId}`	Get friends leaderboard

Score Submission APIs

Method	Endpoint	Description
`POST`	`/api/v1/scores`	Submit a score
`GET`	`/api/v1/scores/{userId}/history?game={id}`	Get user's score history

Submit Score Request

```
POST /api/v1/scores
{
  "user_id": "player_123",
  "leaderboard_id": "halo_weekly",
  "score": 15000,
  "metadata": {
    "game_id": "match_456",
    "level": 42,
    "timestamp": 1672531200
  }
}

Response: 200 OK
{
  "new_rank": 42,
  "previous_rank": 55,
  "percentile": 98.5
}
```

Leaderboard Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/leaderboards`	Create a leaderboard
`POST`	`/api/v1/leaderboards/{boardId}/reset`	Reset leaderboard (season end)

Key Components

Component	Purpose	Technology
Score Ingestion	Buffer score updates	Kafka
Leaderboard Service	Process and rank	Custom service
Real-time Rankings	Fast rank queries	Redis Sorted Sets
Persistence	Historical data	Cassandra
Cache	Top N pre-computed	Redis/CDN
Anti-cheat	Validate scores	ML anomaly detection

Redis Sorted Set Operations

```
REDIS COMMANDS FOR LEADERBOARD:
┌─────────────────────────────────────────────────────────────────┐
│  # Update player score                                          │
│  ZADD leaderboard:halo:weekly 15000 player_123                  │
│                                                                 │
│  # Get player's rank (0-indexed, so add 1)                      │
│  ZREVRANK leaderboard:halo:weekly player_123                    │
│  → 42 (meaning rank #43)                                        │
│                                                                 │
│  # Get top 10 players with scores                               │
│  ZREVRANGE leaderboard:halo:weekly 0 9 WITHSCORES               │
│  → [player_1, 50000, player_2, 48000, ...]                      │
│                                                                 │
│  # Get players around a specific rank (rank 40-50)              │
│  ZREVRANGE leaderboard:halo:weekly 39 49 WITHSCORES             │
│                                                                 │
│  # Get player's score                                           │
│  ZSCORE leaderboard:halo:weekly player_123                      │
│  → 15000                                                        │
│                                                                 │
│  # Count total players                                          │
│  ZCARD leaderboard:halo:weekly                                  │
│  → 5000000                                                      │
└─────────────────────────────────────────────────────────────────┘

Time Complexity:
  ZADD:     O(log N)
  ZRANK:    O(log N)
  ZREVRANGE: O(log N + M) where M is result size
```

Leaderboard Types

```
1. GLOBAL LEADERBOARD (All players)
   Key: leaderboard:{game}:global

2. FRIENDS LEADERBOARD
   - Fetch friend list
   - ZSCORE for each friend
   - Sort client-side (small list)

3. TIME-BASED LEADERBOARDS
   Keys: leaderboard:{game}:daily:{date}
         leaderboard:{game}:weekly:{week}
         leaderboard:{game}:monthly:{month}

   Reset: Scheduled job creates new key, archives old

4. REGIONAL LEADERBOARDS
   Keys: leaderboard:{game}:{region}
   Sharding by region reduces single key size
```

Scalability Considerations

- **Sharding**: Separate Redis instances per game or region
- **Pre-computation**: Cache top 1000 with short TTL
- **Batch updates**: Aggregate scores before writing
- **Archival**: Move old leaderboards to cold storage
- **Eventually consistent**: Rank updates can be slightly delayed

Trade-offs

Decision	Option A	Option B	Recommendation
Storage	SQL	Redis Sorted Set	**Redis** - O(log N) rank
Update	Real-time	Batched	**Batched** - handle spikes
Friends LB	Pre-compute	On-demand	**On-demand** - friend lists change
Historical	Keep all	Sample	**Archive + sample**

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Sorted sets for O(log N) rank operations	Tech Guide
Kafka	Score ingestion and event streaming	Tech Guide
Cassandra	Historical leaderboard persistence	Tech Guide

How to Explain It Well

> **Opening**: "Leaderboards need fast rank lookups for millions of players. I'll use Redis Sorted Sets for O(log N) operations."

> **Key talking points**:
>
> - "Redis ZREVRANK gives player's rank in O(log N)"
> - "Shard by game/region to distribute load"
> - "Time-based leaderboards: new key per period, archive old"

> - "Friends leaderboard: fetch scores on-demand, sort client-side"
> - "Batch score updates during high traffic (end of match)"

---

33. Design Xbox Matchmaking

Key Terms:

Term	Full Name	What it does
ELO	Rating system	Chess-derived skill rating (0-3000 scale)
TrueSkill	Microsoft's rating	Bayesian skill rating with uncertainty (μ, σ)
MMR	Matchmaking Rating	Hidden skill score for matchmaking
Ticket	Queue entry	Player's matchmaking request with criteria
RTT	Round-Trip Time	Latency between player and server
Backfill	Mid-match join	Filling spots when players leave

Problem Statement

Design a matchmaking system that pairs players based on skill level, latency, and preferences for fair and fast matches.

Clarifying Questions to Ask

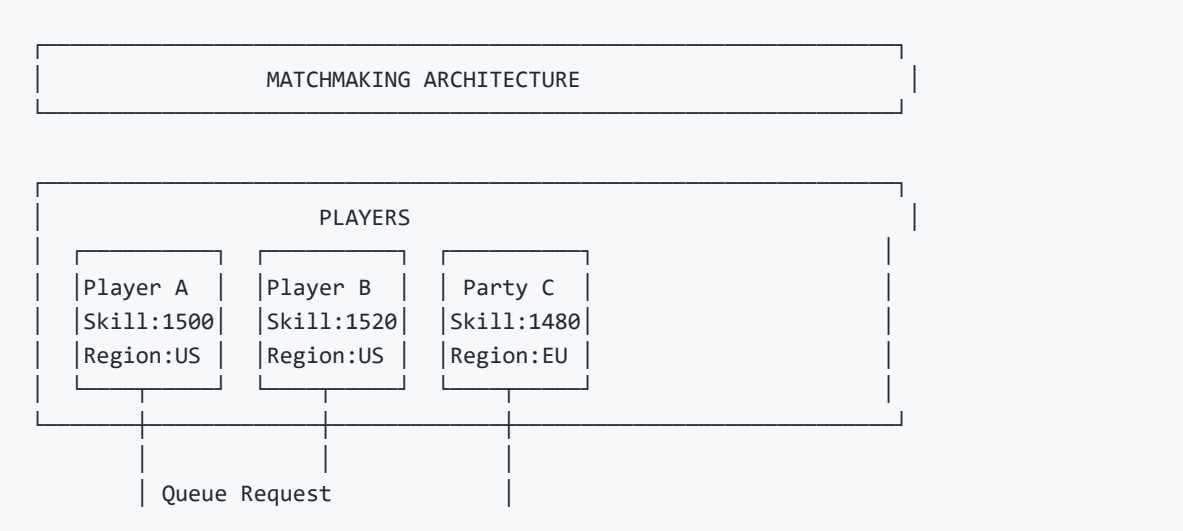
1. **Scale**: Concurrent players searching? (1M+ concurrent?)
2. **Match size**: 1v1, teams, battle royale (100 players)?
3. **Criteria**: Skill, latency, party size, game mode?
4. **Wait time**: Max acceptable queue time? (2 minutes?)
5. **Fairness**: How balanced should matches be?
6. **Parties**: Handle groups of friends?
7. **Regions**: Global or region-locked?

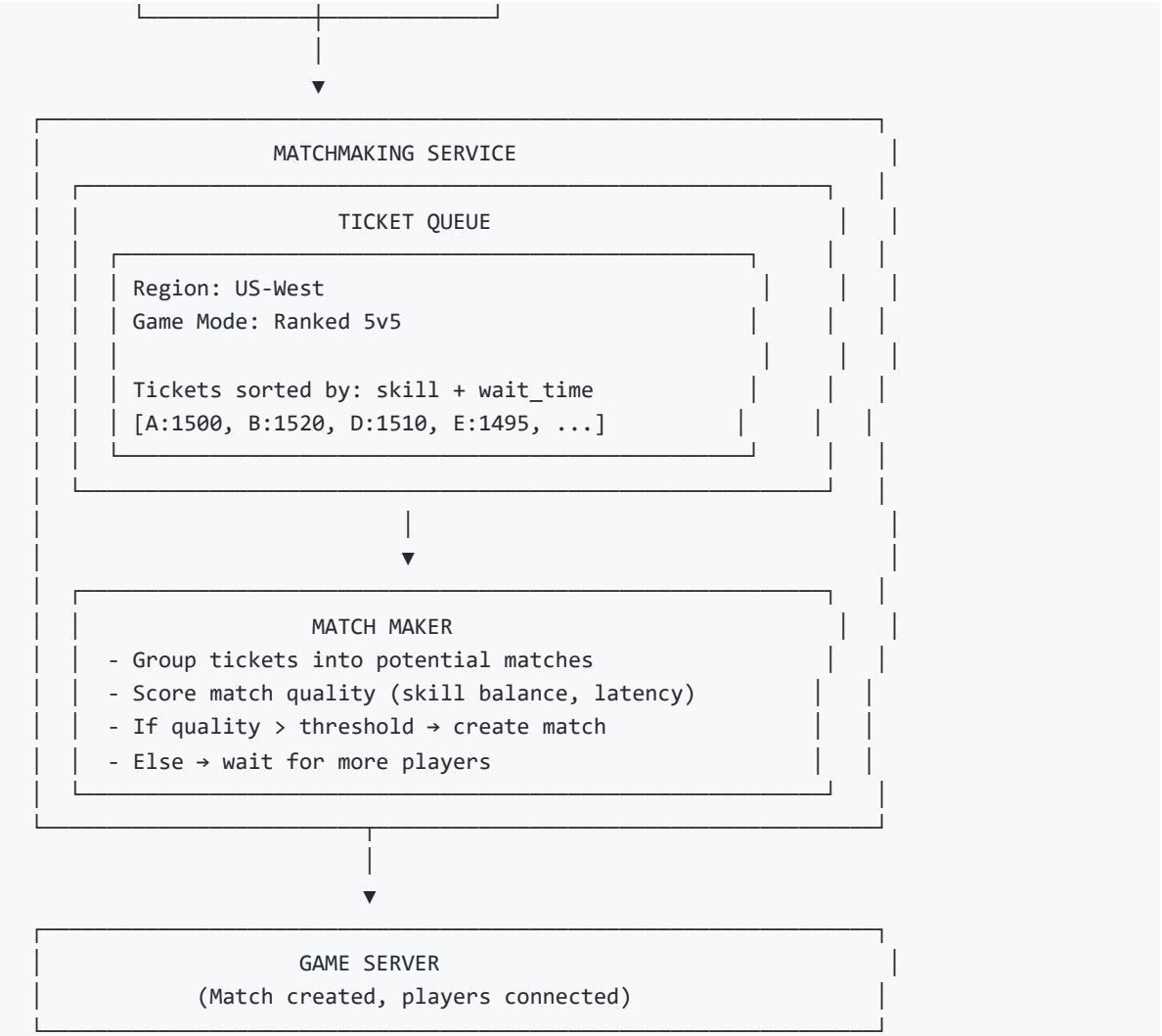
Design Reasoning

> **Core philosophy**: Balance match quality against wait time using adaptive skill-range expansion and multi-factor scoring that weighs skill, latency, and queue duration.

Decision	Choice	Why	Alternative Considered
Skill rating system	TrueSkill (μ, σ)	Models uncertainty, handles team games natively, converges faster for new players	ELO — designed for 1v1 chess, poor support for team-based games
Queue storage	Redis	Low-latency sorted ticket access for the 1-2 second matching loop	SQL database — query latency too high for real-time matching cycles
Matching strategy	Adaptive threshold expansion	Starts strict then relaxes over time; balances quality and wait time	Fixed threshold — either too strict (long waits) or too loose (unfair matches)
Server allocation	Kubernetes (AKS)	Auto-scales game servers during peak hours, pre-warms capacity	Fixed server pool — wastes resources off-peak, insufficient during spikes
Queue partitioning	By region + game mode	Reduces search space per matching cycle, ensures low-latency matches	Global queue — cross-region matches produce unacceptable latency
Party handling	Match by highest skill in party	Fairer for opponents, prevents skill-boosting exploits	Average party skill — unfair to opponents facing one high-skill player

High-Level Design





API Design

Matchmaking APIs

Method	Endpoint	Description
`POST`	`/api/v1/matchmaking/queue`	Join matchmaking queue
`DELETE`	`/api/v1/matchmaking/queue/{ticketId}`	Cancel matchmaking
`GET`	`/api/v1/matchmaking/queue/{ticketId}/status`	Get queue status

Queue Request

```
POST /api/v1/matchmaking/queue
{
  "player_id": "player_123",
  "game_mode": "ranked_5v5",
```

```
    "skill_rating": 1850,
    "region": "us-west",
    "party": ["player_124", "player_125"],
    "preferences": {
      "max_ping": 80,
      "map_pool": ["map_a", "map_b"]
    }
  }

  Response: 202 Accepted
  {
    "ticket_id": "ticket_789",
    "estimated_wait": 45,
    "queue_position": 12
  }
```

Match APIs

Method	Endpoint	Description
`GET`	`/api/v1/matches/{matchId}`	Get match details
`POST`	`/api/v1/matches/{matchId}/accept`	Accept match
`POST`	`/api/v1/matches/{matchId}/results`	Submit match results

WebSocket Events

Event	Direction	Payload
`match.found`	Server → Client	`{ matchId, players, server, map }`
`match.ready`	Server → Client	`{ matchId, allAccepted }`
`queue.update`	Server → Client	`{ position, estimatedWait }`

Key Components

Component	Purpose	Technology
Ticket Service	Create/cancel queue tickets	REST API
Queue Manager	Organize tickets by criteria	Redis
Match Maker	Find optimal player groups	Custom algorithm
Skill Service	Player ratings (ELO/TrueSkill)	Dedicated service

Game Server Allocator	Spin up game servers	Kubernetes
Latency Service	Measure player-to-server RTT	Ping service

Matchmaking Algorithm

```
MATCHMAKING LOOP (runs every 1-2 seconds):
┌─────────────────────────────────────────────────────────────┐
│  For each queue (game_mode + region):                       │
│                                                             │
│  1. Get all tickets sorted by (skill, wait_time)            │
│                                                             │
│  2. For each potential match combination:                   │
│     score = calculate_match_quality(team1, team2)           │
│                                                             │
│     Match Quality Score:                                    │
│     ┌─────────────────────────────────────────────────────┐ │
│     │ skill_balance = 1 - |avg(team1) - avg(team2)| / 500  │ │
│     │ wait_penalty = avg_wait_time / max_wait_time        │ │
│     │ latency_score = 1 - avg_latency / max_acceptable    │ │
│     │                                                     │ │
│     │ score = 0.5 * skill_balance                         │ │
│     │       + 0.3 * wait_penalty    (longer wait = less picky)│ │
│     │       + 0.2 * latency_score                         │ │
│     └─────────────────────────────────────────────────────┘ │
│                                                             │
│  3. If score > threshold (e.g., 0.7):                       │
│     - Remove tickets from queue                             │
│     - Create match                                          │
│     - Allocate game server                                  │
│     - Notify players                                        │
│                                                             │
│  4. Tickets waiting > 2 min: expand skill range             │
└─────────────────────────────────────────────────────────────┘
```

Skill Rating Systems

```
ELO RATING:
┌─────────────────────────────────────────────────────────────┐
│  New Rating = Old Rating + K * (Actual - Expected)          │
│                                                             │
│  Expected = 1 / (1 + 10^((opponent - player) / 400))        │
│                                                             │
│  Example:                                                   │
│    Player: 1500, Opponent: 1600                             │
│    Expected win probability: 36%                            │
│    If player wins: +23 points                               │
│    If player loses: -13 points                              │
│                                                             │
│  Pros: Simple, well-understood                              │
```

```
│  Cons: Doesn't handle team games well                          │
└─────────────────────────────────────────────────────────────────┘

TRUESKILL (Microsoft's system):
┌─────────────────────────────────────────────────────────────────┐
│  Rating = (μ, σ) where:                                         │
│    μ = skill estimate (mean)                                    │
│    σ = uncertainty (standard deviation)                         │
│                                                                 │
│  Conservative estimate = μ - 3σ                                 │
│                                                                 │
│  After each game:                                               │
│    - Winners' μ increases, σ decreases                          │
│    - Losers' μ decreases, σ decreases                           │
│    - Upsets cause larger updates                                │
│                                                                 │
│  Pros: Handles teams, converges faster                          │
│  Cons: More complex                                             │
└─────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Sharding**: Separate queues per region and game mode
- **Ticket expiry**: Auto-remove stale tickets (timeout)
- **Party handling**: Treat party as single ticket, match by highest skill
- **Server allocation**: Pre-warm game servers during peak hours
- **Backfill**: Fill spots when players leave mid-match

Trade-offs

Decision	Option A	Option B	Recommendation
Skill rating	ELO	TrueSkill	**TrueSkill** - better for teams
Match speed	Fast (loose)	Slow (strict)	**Adaptive** - expand over time
Queue	Global	Regional	**Regional** - lower latency
Party matching	By avg skill	By highest	**Highest** - fairer for opponents

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference

Redis	Ticket queue and player state management	Tech Guide
AKS/Kubernetes	Game server allocation and scaling	Tech Guide

How to Explain It Well

> **Opening**: "Matchmaking balances match quality with wait time. I'll use TrueSkill for ratings and adaptive matching that loosens criteria over time."

> **Key talking points**:
>
> - "TrueSkill tracks skill (μ) and uncertainty (σ) - better for team games"
> - "Match quality score combines skill balance, wait time, and latency"
> - "Adaptive: after 2 minutes, expand acceptable skill range"
> - "Shard queues by region for low latency matches"
> - "Pre-warm game servers during peak hours"

---

34. Design LinkedIn News Feed

Key Terms:

Term	Full Name	What it does
Fanout	Content distribution	Pushing posts to followers' feeds
Fanout-on-Write	Push model	Pre-compute feeds when content is posted
Fanout-on-Read	Pull model	Build feed when user requests it
Celebrity Problem	High-follower users	Users with millions of followers (expensive fanout)
Feed Ranking	Personalization	ML model ordering content by relevance
Time Decay	Freshness signal	Older posts score lower in ranking

Problem Statement

Design a personalized news feed for LinkedIn that shows relevant posts, articles, and updates from connections and followed entities.

Clarifying Questions to Ask

1. **Scale**: How many users? Posts per day? (800M users, 2M posts/day?)
2. **Content types**: Posts, articles, job updates, ads?
3. **Personalization**: How personalized should it be?
4. **Freshness**: Real-time or eventual consistency OK?
5. **Engagement**: Prioritize engagement or chronological?
6. **Connections**: Average connections per user? (500?)
7. **Read/write ratio**: Mostly reading feeds?

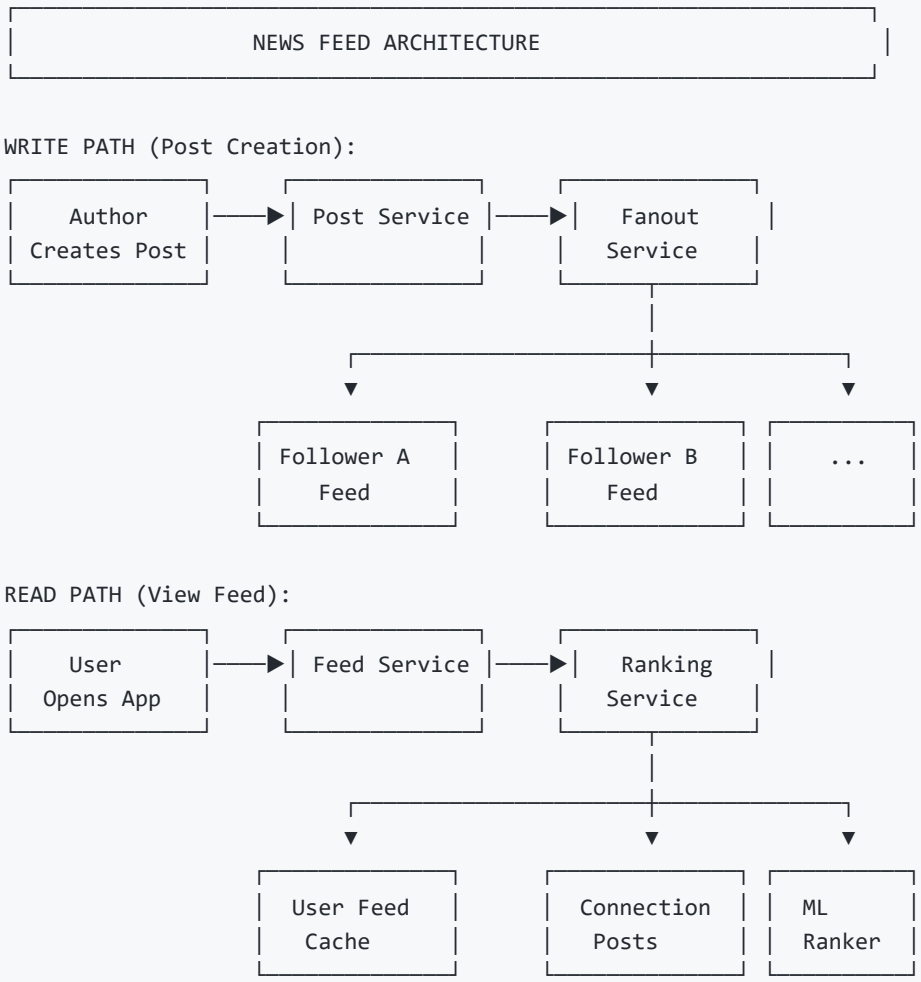
Design Reasoning

> **Core philosophy**: Hybrid fanout combined with ML-based ranking delivers a personalized, fresh feed that scales for both regular users and high-follower celebrities without write amplification.

Decision	Choice	Why	Alternative Considered
Fanout strategy	Hybrid (push for <10K followers, pull for celebrities)	Avoids expensive fanout for high-follower accounts like Bill Gates	Pure fanout-on-write — celebrity posts fan out to millions, massive write amplification
Feed ranking	ML-ranked (relevance model)	Better engagement than chronological; combines relationship, content, and freshness signals	Chronological feed — misses personalization opportunity, lower engagement
Feed cache	Post IDs in Redis (top 500 per user)	Much less memory than caching full post objects at 800M users	Full post objects in cache — memory explodes at scale
Post distribution	Kafka + async workers	Handles post burst without blocking author; rate-limited fanout to Redis	Synchronous fanout — blocks post creation, single failure cascades
Post storage	Cassandra	Handles high write throughput for 2M posts/day across regions	PostgreSQL — single-writer bottleneck at LinkedIn's post volume

Graph queries	Graph DB for connection traversal	Efficient multi-hop queries for social graph features used in ranking	Relational joins — $O(N^2)$ for multi-hop connection lookups

High-Level Design



API Design

Feed APIs

Method	Endpoint	Description
`GET`	`/api/v1/feed?cursor={cursor}&limit=20`	Get personalized feed
`GET`	`/api/v1/feed/refresh`	Get new posts since last fetch

Post APIs

Method	Endpoint	Description
`POST`	`/api/v1/posts`	Create a post
`GET`	`/api/v1/posts/{postId}`	Get post details
`PUT`	`/api/v1/posts/{postId}`	Edit a post
`DELETE`	`/api/v1/posts/{postId}`	Delete a post

Engagement APIs

Method	Endpoint	Description
`POST`	`/api/v1/posts/{postId}/like`	Like a post
`DELETE`	`/api/v1/posts/{postId}/like`	Unlike a post
`POST`	`/api/v1/posts/{postId}/comments`	Comment on a post
`POST`	`/api/v1/posts/{postId}/share`	Share/repost
`GET`	`/api/v1/posts/{postId}/comments?cursor={c}`	Get comments

Create Post Request

```
POST /api/v1/posts
{
  "content": "Excited to share...",
  "media": [{ "type": "image", "url": "..." }],
  "visibility": "connections",  // public, connections, private
  "mentions": ["user_456"],
  "hashtags": ["systemdesign"]
}
```

Key Components

Component	Purpose	Technology
Post Service	Create/store posts	PostgreSQL
Fanout Service	Distribute to feeds	Kafka + Workers
Feed Cache	Pre-computed feeds	Redis
Graph Service	Connections, follows	Graph DB

Ranking Service	Personalized ordering	ML model
Search Service	Content discovery	Elasticsearch

Fanout Strategies

```
FANOUT ON WRITE (Push model):
┌─────────────────────────────────────────────────────────────┐
│  When user posts:                                           │
│    For each follower:                                       │
│      Add post_id to follower's feed cache                   │
│                                                             │
│  Pros: Fast reads (feed is pre-built)                       │
│  Cons: Expensive for users with many followers (celebrities)│
│                                                             │
│  Best for: Users with < 10K followers                       │
└─────────────────────────────────────────────────────────────┘

FANOUT ON READ (Pull model):
┌─────────────────────────────────────────────────────────────┐
│  When user opens feed:                                      │
│    Fetch posts from all followed users                      │
│    Merge and rank                                           │
│                                                             │
│  Pros: No fanout cost for celebrities                       │
│  Cons: Slower reads, more compute per request               │
│                                                             │
│  Best for: Following celebrities (Bill Gates, etc.)         │
└─────────────────────────────────────────────────────────────┘

HYBRID (LinkedIn approach):
┌─────────────────────────────────────────────────────────────┐
│  - Regular users: Fanout on write                           │
│  - Celebrities (>10K followers): Fanout on read             │
│  - Merge at read time                                       │
│                                                             │
│  User opens feed:                                           │
│    1. Fetch pre-built feed (from regular connections)       │
│    2. Fetch recent posts from followed celebrities          │
│    3. Merge and rank                                        │
└─────────────────────────────────────────────────────────────┘
```

Feed Ranking

```
RANKING SIGNALS:
┌─────────────────────────────────────────────────────────────┐
│  Relevance Score = Σ (weight_i * signal_i)                  │
│                                                             │
│  Signals:                                                   │
│  ┌──────────────────────────────────────────────────────────┐
```

```
|   | Relationship strength:                                          ||
|   |   - Connection degree (1st, 2nd, 3rd)                          ||
|   |   - Interaction history (messages, profile views)              ||
|   |   - Common connections                                          ||
|   |                                                                 ||
|   | Content quality:                                                ||
|   |   - Engagement rate (likes, comments, shares)                  ||
|   |   - Content type affinity (user prefers articles vs posts)     ||
|   |   - Topic relevance (matches user interests)                   ||
|   |                                                                 ||
|   | Freshness:                                                      ||
|   |   - Time decay (recent posts score higher)                     ||
|   |   - Trending boost (viral content)                             ||
|   |                                                                 ||
|   | Diversity:                                                      ||
|   |   - Don't show 5 posts from same author in a row               ||
|   |   - Mix content types (posts, articles, jobs)                  ||
|   └─────────────────────────────────────────────────────────────────┘|
└──────────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Feed cache**: Store top 500 post IDs per user in Redis
- **Sharding**: Shard by user_id for feed storage
- **Async fanout**: Queue-based fanout with rate limiting
- **Read replicas**: Separate read/write paths
- **Edge caching**: Cache popular content at CDN

Trade-offs

Decision	Option A	Option B	Recommendation
Fanout	On write	On read	**Hybrid** - best of both
Ranking	Chronological	ML-ranked	**ML-ranked** - better engagement
Cache	Full posts	Post IDs only	**Post IDs** - less memory
Freshness	Real-time	Near-real-time	**Near-real-time** - simpler

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference

Redis	Pre-computed feed cache storage	Tech Guide
Kafka	Async fanout and event distribution	Tech Guide
Cassandra	Post and feed persistence	Tech Guide

How to Explain It Well

> **Opening**: "LinkedIn feed needs fast reads with personalized ranking. I'll use hybrid fanout and ML-based ranking."

> **Key talking points**:
>
> - "Hybrid fanout: push for regular users, pull for celebrities"
> - "ML ranking combines relationship strength, content quality, freshness"
> - "Feed cache stores top 500 post IDs, hydrate on read"
> - "Async fanout via Kafka to handle post burst"
> - "Diversity rules prevent feed dominated by single author"

---

35. Design LinkedIn People You May Know

Key Terms:

Term	Full Name	What it does
PYMK	People You May Know	LinkedIn's connection recommendation feature
FOF	Friends of Friends	2-hop connections in social graph
Graph DB	Graph Database	Database optimized for relationship queries
Jaccard Similarity	Set similarity metric	Overlap of two sets divided by union
Bloom Filter	Probabilistic data structure	Fast "already connected?" check
Feature Store	ML feature storage	Pre-computed features for model inference

Problem Statement

Design a system that suggests relevant professional connections based on social graph analysis, shared attributes, and user behavior.

Clarifying Questions to Ask

1. **Scale**: Users, connections per user? (800M users, 500 avg connections)
2. **Update frequency**: How fresh should suggestions be?
3. **Signals**: What factors determine relevance?
4. **Privacy**: Respect user privacy settings?
5. **Diversity**: Mix of signal types in suggestions?
6. **Explanation**: Show why someone is recommended?
7. **Negative feedback**: Handle "Don't show" actions?

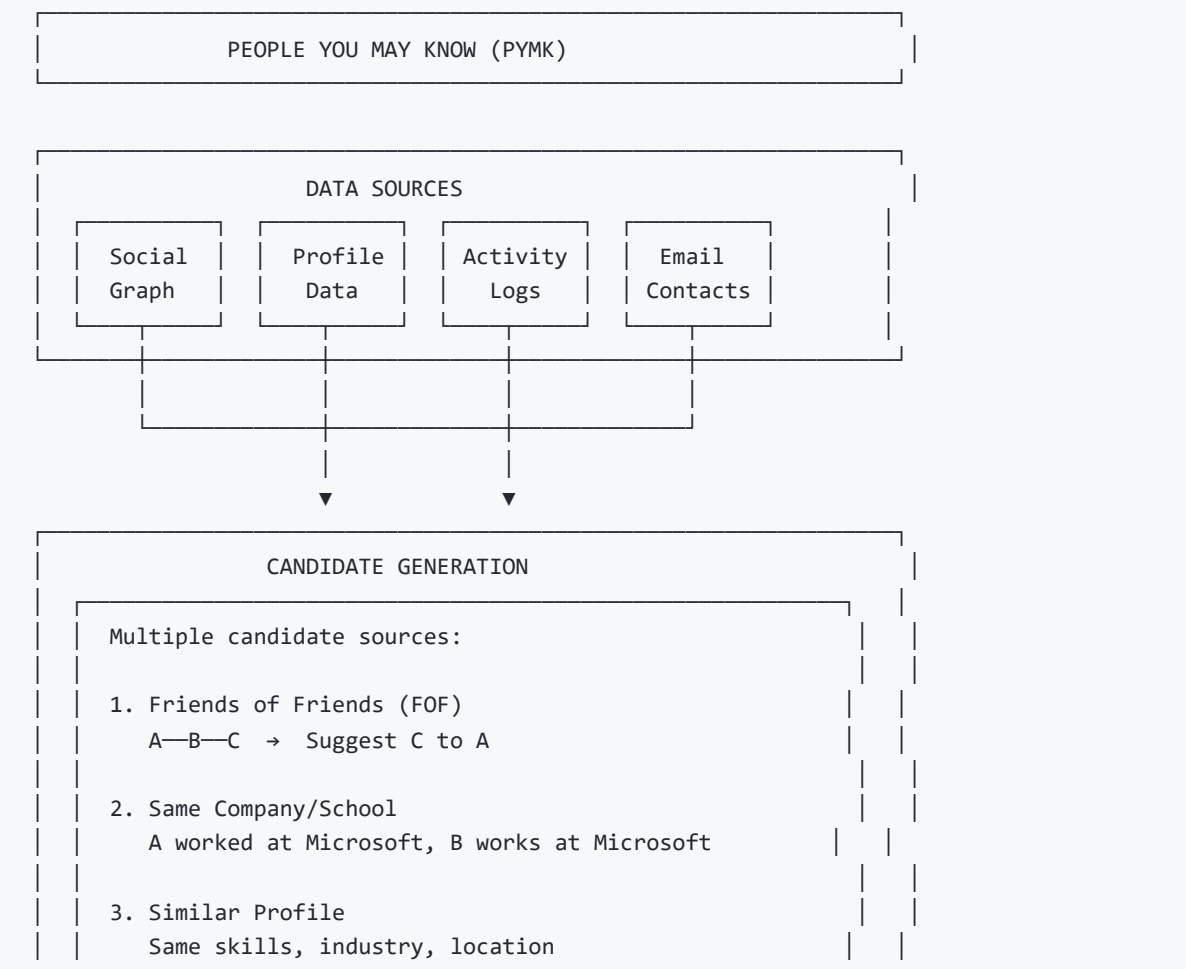
Design Reasoning

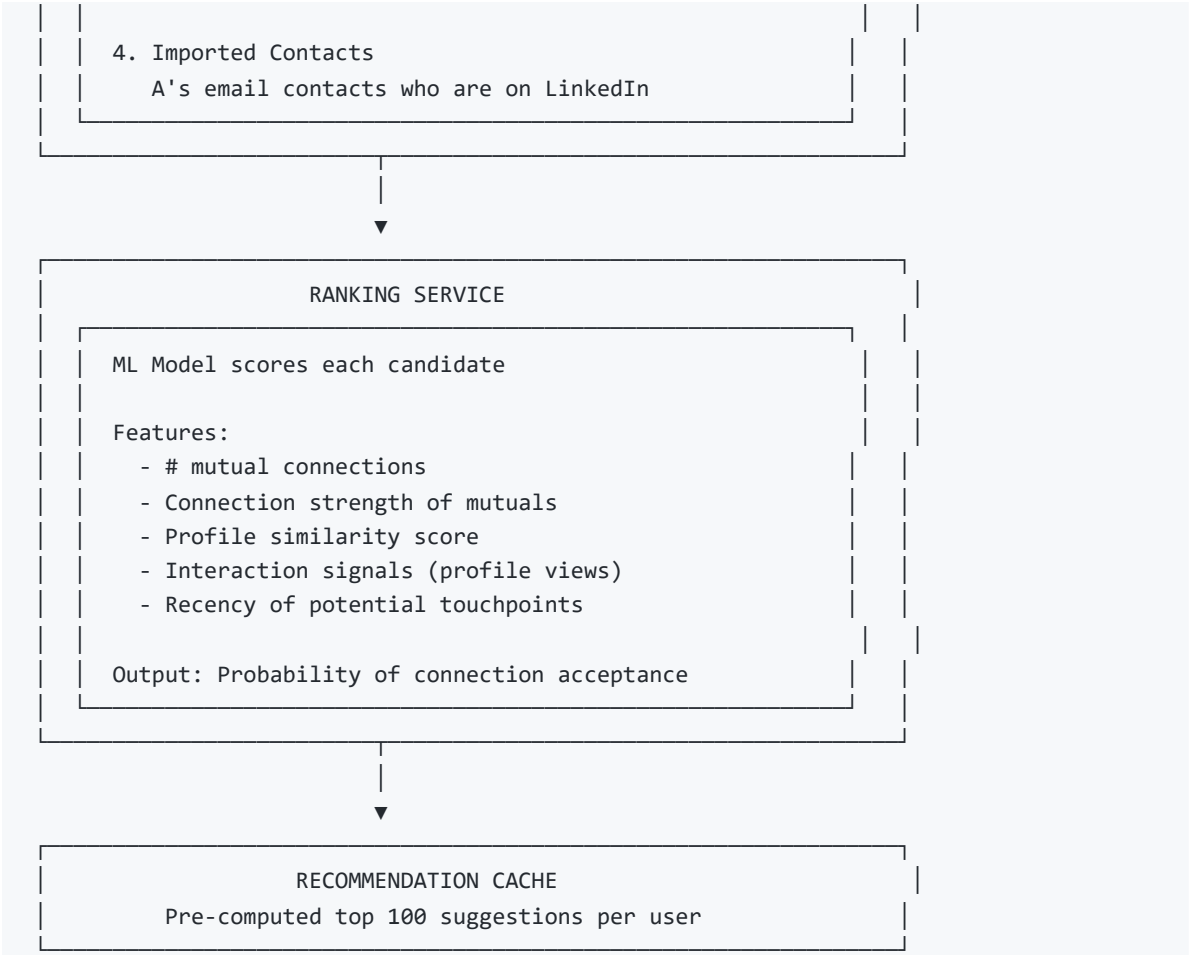
> **Core philosophy**: Combine graph-based candidate generation with ML ranking to surface relevant connections, pre-computing recommendations in batch while incrementally updating on new signals.

Decision	Choice	Why	Alternative Considered
Graph storage	Neo4j / JanusGraph	Optimized for 2-hop FOF traversal on an 800M-node social graph	SQL with joins — prohibitively slow for multi-hop graph queries at scale
Candidate generation	Batch Spark jobs	Pre-compute FOF and profile similarity for all users cost-effectively	Real-time graph traversal — too expensive per request at 800M users
Ranking model	ML model (TensorFlow/PyTorch)	Trained on historical accept/ignore actions; captures complex interaction patterns	Rule-based scoring — can't learn nuanced relevance signals

Dedup check	Bloom filter	O(1) "already connected?" check with minimal memory overhead	Database lookup per candidate — adds latency to every candidate evaluation
Profile search	Elasticsearch	Fast attribute matching (company, skills, location) for candidate sourcing	SQL LIKE queries — poor full-text performance at LinkedIn's data scale
Recommendation cache	Redis (top 100 per user)	Instant serving without re-computation; updated incrementally on new connections	On-demand computation — seconds of latency per suggestion request

High-Level Design





API Design

Suggestion APIs

Method	Endpoint	Description
`GET`	`/api/v1/suggestions?userId={id}&limit=20`	Get "People You May Know" suggestions
`GET`	`/api/v1/suggestions/reasons/{suggestedUserId}`	Get why this person was suggested

Connection APIs

Method	Endpoint	Description
`POST`	`/api/v1/connections/request`	Send connection request
`PUT`	`/api/v1/connections/request/{requestId}/accept`	Accept request

`PUT`	`/api/v1/connections/request/{requestId}/reject`	Reject request
`DELETE`	`/api/v1/connections/{connectionId}`	Remove connection

Feedback APIs (to improve suggestions)

Method	Endpoint	Description
`POST`	`/api/v1/suggestions/{suggestedUserId}/dismiss`	Dismiss suggestion
`POST`	`/api/v1/suggestions/{suggestedUserId}/feedback`	Report irrelevant suggestion

Suggestion Response

```
GET /api/v1/suggestions?userId=user_123&limit=5

Response: 200 OK
{
  "suggestions": [
    {
      "user_id": "user_456",
      "name": "Jane Doe",
      "headline": "Senior Engineer at Microsoft",
      "mutual_connections": 12,
      "reasons": ["12 mutual connections", "Same company"],
      "score": 0.92
    }
  ]
}
```

Key Components

Component	Purpose	Technology
Graph Database	Store social connections	Neo4j / JanusGraph
Profile Index	Search by attributes	Elasticsearch
Candidate Generator	Find potential connections	Batch Spark jobs
Ranking Model	Score candidates	TensorFlow / PyTorch
Cache	Store recommendations	Redis
Feedback Service	Handle dismissals	Event store

Graph-Based Candidate Generation

```
FRIENDS OF FRIENDS (FOF):
┌─────────────────────────────────────────────────────────────────┐
│  Query: Find people within 2 hops who aren't connected          │
│                                                                 │
│  MATCH (user:Person {id: $userId})                              │
│        -[:CONNECTED_TO]-(friend)                                │
│        -[:CONNECTED_TO]-(fof)                                   │
│  WHERE NOT (user)-[:CONNECTED_TO]-(fof)                         │
│    AND fof <> user                                              │
│  RETURN fof, COUNT(friend) AS mutualCount                       │
│  ORDER BY mutualCount DESC                                      │
│  LIMIT 100                                                      │
│                                                                 │
│  Result: People with most mutual connections                    │
└─────────────────────────────────────────────────────────────────┘

SCORING MUTUAL CONNECTIONS:
┌─────────────────────────────────────────────────────────────────┐
│  Not all mutuals are equal:                                     │
│                                                                 │
│  Strong mutual: Recent interaction, similar industry            │
│  Weak mutual: Connected 5 years ago, never interacted           │
│                                                                 │
│  Weighted score = Σ (connection_strength[mutual])               │
└─────────────────────────────────────────────────────────────────┘
```

Ranking Features

```
FEATURE CATEGORIES:
┌─────────────────────────────────────────────────────────────────┐
│  GRAPH FEATURES:                                                │
│    - Mutual connection count                                    │
│    - Weighted mutual score                                      │
│    - Path length (2nd vs 3rd degree)                            │
│    - Common groups/companies                                    │
│                                                                 │
│  PROFILE FEATURES:                                              │
│    - Industry match                                             │
│    - Location proximity                                         │
│    - Skill overlap (Jaccard similarity)                         │
│    - Seniority level match                                      │
│    - Company size similarity                                    │
│                                                                 │
│  BEHAVIORAL FEATURES:                                           │
│    - Did user view candidate's profile?                         │
│    - Did candidate view user's profile?                         │
│    - Common content engagement                                  │
│                                                                 │
│  HISTORICAL FEATURES:                                           │
```

```
│    - Past connection acceptance rate (similar profiles)       │
│    - Time since candidate joined LinkedIn                     │
└──────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Batch computation**: Pre-compute recommendations daily
- **Incremental updates**: Update when new connections made
- **Graph partitioning**: Shard graph by user geography
- **Bloom filters**: Quick "already connected?" check
- **Caching**: Store top 100 recommendations per user

Trade-offs

Decision	Option A	Option B	Recommendation
Computation	Real-time	Batch	**Batch + incremental**
Graph storage	SQL	Graph DB	**Graph DB** - FOF queries
Ranking	Rules	ML	**ML** - better accuracy
Explanation	Show	Hide	**Show** - builds trust

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Recommendation cache storage	Tech Guide
Elasticsearch	Profile search and attribute indexing	Tech Guide
Flink/Spark	Batch candidate generation	Tech Guide
Bloom Filter	Quick "already connected?" checks	Tech Guide

How to Explain It Well

> **Opening**: "PYMK uses graph analysis and ML ranking. Primary signal is mutual connections, enhanced with profile similarity and behavioral data."

> **Key talking points**:
>
> - "Friends-of-friends is primary source: 2-hop graph traversal"
> - "Weight mutual connections by interaction strength"
> - "ML model trained on historical accept/ignore actions"
> - "Batch compute daily, incremental update on new connections"
> - "Show explanation ('12 mutual connections') to build trust"

---

36. Design Azure Key Vault

Key Terms:

Term	Full Name	What it does
HSM	Hardware Security Module	Tamper-resistant device protecting keys
FIPS 140-2	Federal Information Processing Standard	Security certification for crypto modules
KEK	Key Encryption Key	Key that encrypts other keys
DEK	Data Encryption Key	Key that encrypts actual data
RBAC	Role-Based Access Control	Permissions by role assignment
Managed Identity	Azure AD identity	Automatic credential management for Azure services
Soft Delete	Recoverable deletion	Deleted secrets recoverable for retention period

Problem Statement

Design a secure secrets management service that stores and retrieves encryption keys, certificates, and secrets with fine-grained access control.

Clarifying Questions to Ask

1. **Secret types**: Keys, certificates, connection strings, API keys?
2. **Scale**: How many secrets? Access frequency?

3. **Compliance**: FIPS 140-2, HSM requirements?
4. **Access control**: Who can access what?
5. **Rotation**: Automatic key rotation?
6. **Audit**: Track all access attempts?
7. **Availability**: Multi-region replication?

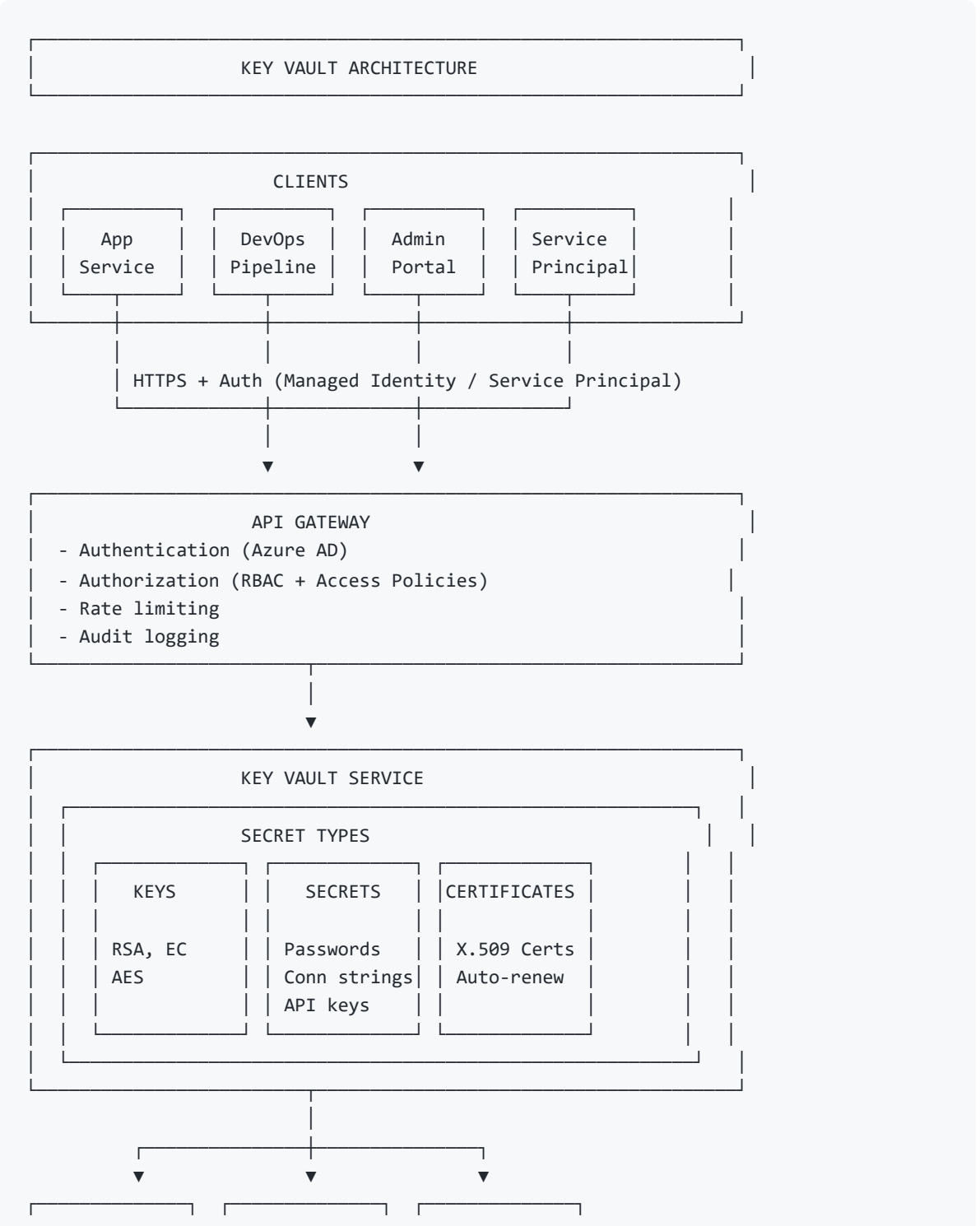
Design Reasoning

> **Core philosophy**: Defense in depth with HSM-backed key hierarchy, where sensitive key material never leaves hardware boundaries and every operation is authenticated, authorized, and audited.

Decision	Choice	Why	Alternative Considered
Key protection	Hardware Security Module (HSM)	Key material never exported; meets FIPS 140-2 Level 2/3 compliance requirements	Software-based encryption — keys reside in memory, vulnerable to extraction
Authentication	Azure AD + Managed Identity	Eliminates the "secret to get a secret" problem; credentials auto-rotated	API keys / shared secrets — secrets end up in config files, manual rotation required
Authorization	RBAC + Access Policies (dual plane)	Management plane separated from data plane; enforces least privilege per principal	Single auth model — can't differentiate vault admin from secret consumer
Key hierarchy	Master → KEK → DEK	Compartmentalized encryption; compromised DEK doesn't expose other secrets	Single encryption key — one key compromise exposes all secrets in the vault
Deletion model	Soft delete + purge protection	Prevents accidental or malicious permanent deletion; configurable retention period	Hard delete — no recovery from mistakes, insider threats, or operational errors
Audit	Immutable append-only	Captures every access attempt for compliance	Application-level logging — gaps in

	log	forensics and anomaly detection	coverage, logs can be tampered with
Replication	Multi-region geo-replication	Ensures secret availability during regional outages for critical workloads	Single region — single point of failure for secrets that all services depend on

High-Level Design





API Design

Secret APIs

Method	Endpoint	Description
`PUT`	`/api/v1/secrets/{secretName}`	Create or update a secret
`GET`	`/api/v1/secrets/{secretName}`	Get secret value
`GET`	`/api/v1/secrets/{secretName}/versions`	List secret versions
`GET`	`/api/v1/secrets/{secretName}/{version}`	Get specific version
`DELETE`	`/api/v1/secrets/{secretName}`	Soft-delete a secret
`POST`	`/api/v1/secrets/{secretName}/recover`	Recover deleted secret

Key APIs (Cryptographic Operations)

Method	Endpoint	Description
`POST`	`/api/v1/keys/{keyName}/create`	Create a cryptographic key
`POST`	`/api/v1/keys/{keyName}/encrypt`	Encrypt data
`POST`	`/api/v1/keys/{keyName}/decrypt`	Decrypt data
`POST`	`/api/v1/keys/{keyName}/sign`	Sign data
`POST`	`/api/v1/keys/{keyName}/verify`	Verify signature
`POST`	`/api/v1/keys/{keyName}/rotate`	Rotate key

Certificate APIs

Method	Endpoint	Description
`POST`	`/api/v1/certificates/{certName}/create`	Create/import certificate
`GET`	`/api/v1/certificates/{certName}`	Get certificate

`POST`	`/api/v1/certificates/{certName}/renew`	Renew certificate

Access Policy APIs

Method	Endpoint	Description
`PUT`	`/api/v1/policies/{principalId}`	Set access policy
`GET`	`/api/v1/audit/logs?objectName={name}`	Get audit trail

Key Components

Component	Purpose	Technology
Auth Service	Verify caller identity	Azure AD
Policy Engine	Check permissions	RBAC + Access Policies
HSM	Protect key material	FIPS 140-2 Level 2/3
Secret Store	Encrypted secret storage	Encrypted database
Version Store	Secret history	Append-only storage
Audit Log	Track all operations	Immutable log

Encryption Architecture

```
KEY HIERARCHY:
┌─────────────────────────────────────────────────────────────┐
│                                                             │
│  HSM Master Key (never exported)                            │
│         │                                                   │
│         │ Wraps                                             │
│         ▼                                                   │
│  Key Encryption Key (KEK) - per vault                       │
│         │                                                   │
│         │ Wraps                                             │
│         ▼                                                   │
│  Data Encryption Key (DEK) - per secret                     │
│         │                                                   │
│         │ Encrypts                                          │
│         ▼                                                   │
│  Secret Value (encrypted at rest)                           │
│                                                             │
└─────────────────────────────────────────────────────────────┘

CRYPTO OPERATIONS:
```

```
┌─────────────────────────────────────────────────────────────┐
│  Keys can perform:                                          │
│    - Encrypt/Decrypt (data protection)                      │
│    - Sign/Verify (authentication)                           │
│    - Wrap/Unwrap (key wrapping)                             │
│                                                             │
│  Key material NEVER leaves HSM                              │
│  Operations happen IN the HSM                               │
└─────────────────────────────────────────────────────────────┘
```

Access Control

```
RBAC + ACCESS POLICIES:
┌─────────────────────────────────────────────────────────────┐
│  Management Plane (RBAC):                                   │
│    - Who can create/delete vaults                           │
│    - Who can set access policies                            │
│    - Roles: Owner, Contributor, Reader                      │
│                                                             │
│  Data Plane (Access Policies):                              │
│    - Who can read/write specific secrets                    │
│    - Granular permissions per principal                     │
│                                                             │
│  Example Policy:                                            │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  Principal: app-service-prod                         │   │
│  │  Permissions:                                        │   │
│  │    Keys: Get, List                                   │   │
│  │    Secrets: Get (NOT Set, Delete)                    │   │
│  │    Certificates: Get                                 │   │
│  └──────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘

PRINCIPLE OF LEAST PRIVILEGE:
┌─────────────────────────────────────────────────────────────┐
│  - Apps: Only Get secrets they need                         │
│  - DevOps: Set secrets, no Delete                           │
│  - Admins: Full control, requires MFA                       │
│  - No one: Export private keys                              │
└─────────────────────────────────────────────────────────────┘
```

Secret Rotation

```
AUTOMATIC ROTATION:
┌─────────────────────────────────────────────────────────────┐
│  1. Key Vault triggers rotation (schedule or event)         │
│  2. Generate new secret value                               │
│  3. Update dependent service (e.g., SQL password)           │
│  4. Store new version in Key Vault                          │
│  5. Apps automatically get new value (no redeploy)          │
```

```
│                                                                  │
│  Version History:                                                │
│    secret/db-password                                            │
│      ├── v1 (expired)                                            │
│      ├── v2 (expired)                                            │
│      └── v3 (current) ← Apps get this                            │
└──────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Caching**: Apps should cache secrets (with TTL)
- **Rate limits**: Throttle excessive requests
- **Geo-replication**: Replicate to DR region
- **Soft delete**: Recover accidentally deleted secrets
- **Purge protection**: Prevent permanent deletion

Trade-offs

Decision	Option A	Option B	Recommendation
HSM	Software	Hardware	**Hardware** - compliance
Auth	API keys	Managed Identity	**Managed Identity** - no secrets
Caching	None	Client-side	**Client-side** - reduce latency
Replication	Single region	Multi-region	**Multi** - availability

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
HSM	Hardware security for key material protection	Tech Guide

How to Explain It Well

> **Opening**: "Key Vault provides secure secrets storage with HSM protection. Key material never leaves the HSM - crypto operations happen inside."

> **Key talking points**:

> - "Key hierarchy: HSM Master → KEK → DEK → encrypted secret"
> - "Managed Identity for auth - apps don't need secrets to get secrets"
> - "Access policies: least privilege, apps only get what they need"
> - "Automatic rotation with version history"
> - "Audit log captures every access attempt"

---

48. Design Azure Entra Identity Platform

---

0) Requirements & Sizing

Functional

- **Authentication**: Authenticate users, services, and workloads across VMs, Kubernetes, and external systems
- **Authorization**: Role-Based Access Control (RBAC) with fine-grained permissions
- **Managed Identities**: Passwordless authentication for Azure resources (VMs, AKS, App Services)
- **Workload Identity**: Federated credentials for Kubernetes pods and external workloads (GitHub Actions, AWS)
- **External Identities**: B2B (partners) and B2C (customers) identity federation
- **Conditional Access**: Policy-based access control (location, device, risk level)
- **Single Sign-On (SSO)**: Seamless access across applications
- **Multi-Factor Authentication (MFA)**: Additional security layer
- **Token Management**: OAuth 2.0, OIDC, SAML token issuance and validation

Non-functional

Metric	Target
Authentication latency	< **100ms** p99 for token validation
Token issuance	< **500ms** for new token acquisition
Availability	**99.99%** (four 9s) - identity is critical path
Scale	**Millions of authentications/day** per tenant
Global	Multi-region with <50ms to nearest endpoint

Security	Zero-trust, continuous verification

Capacity

```
Large Enterprise Scenario:
- 100,000 employees
- 10,000 VMs with managed identities
- 500 AKS clusters, 50,000 pods with workload identity
- 1M external partners (B2B)
- 10M customers (B2C)

Token operations:
- Each workload: ~100 token acquisitions/day (cached, refreshed hourly)
- Peak: 10M token operations/hour
- Token size: ~2-4KB (JWT)
```

Design Reasoning

> **Core philosophy**: Zero Trust, passwordless-first identity — never trust, always verify using multi-signal context (user, device, location, risk), with credential-free authentication for Azure resources and OIDC federation for external systems, all built on short-lived tokens with continuous access evaluation.

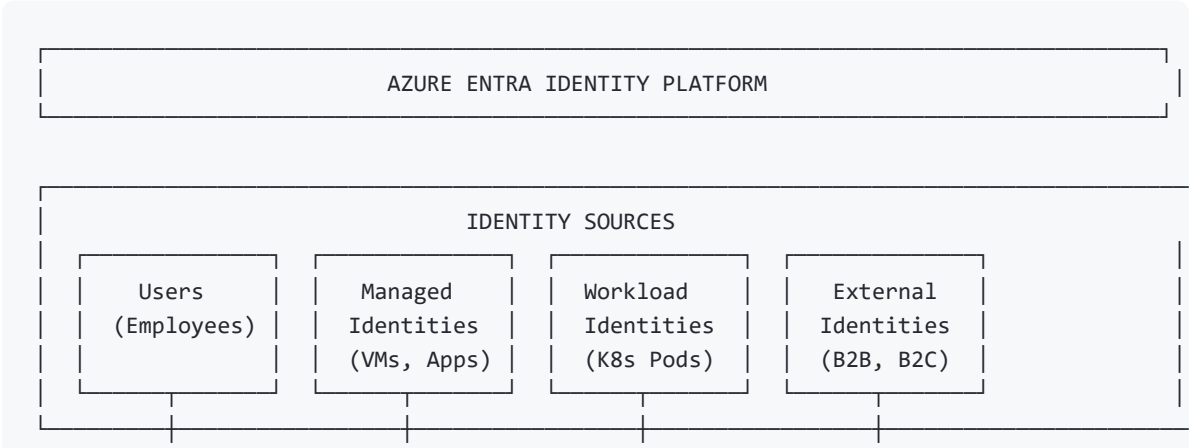
Decision	Choice	Why	Alternative Considered
Azure resource auth	Managed Identities (system/user-assigned)	Eliminates credential storage; tokens from local IMDS at <100ms; automatic lifecycle management	Service Principals with secrets — requires rotation, risk of exposure in CI/CD pipelines
Kubernetes auth	Workload Identity with OIDC federation	No secrets in cluster, pod-level granularity, multi-cluster support; replaces deprecated Pod Identity	Pod Identity (NMI DaemonSet) — required secrets in cluster, added networking overhead
External workload auth	Federated Identity Credentials (RFC 8693)	Zero credential management for GitHub Actions, AWS, GCP; external OIDC tokens exchanged for Azure tokens	Service Principals with shared secrets — requires rotation, potential exposure in external systems

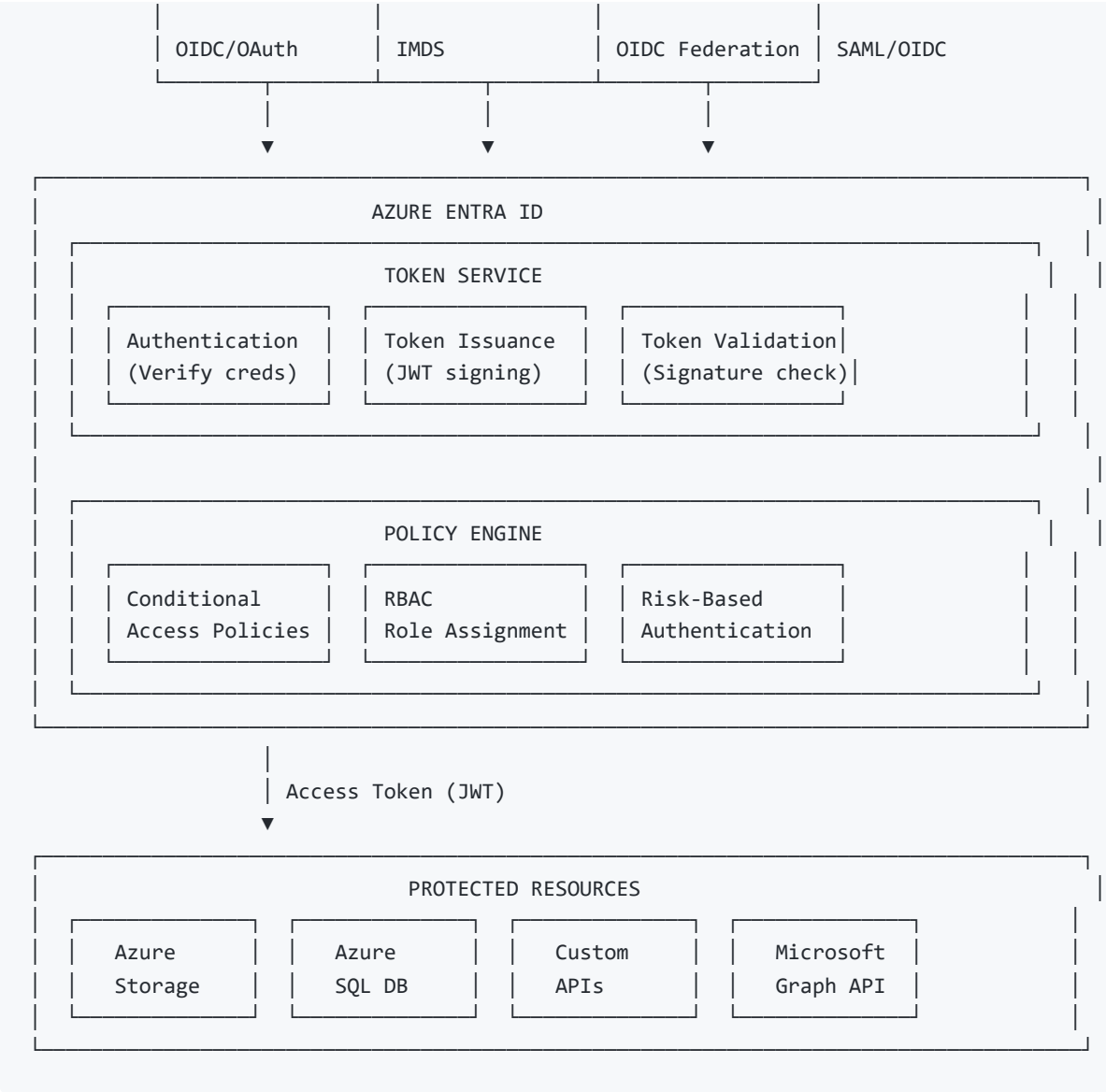
Token lifecycle	1-hour access tokens + refresh tokens	Minimizes blast radius from compromise; CAE enables immediate revocation on risk signal detection	Long-lived tokens (24h) — increases damage window, delays risk-based revocation
Policy engine	Conditional Access (multi-signal)	Implements Zero Trust with real-time evaluation of user, device, location, and risk signals	Static RBAC-only — doesn't account for context, can't respond to real-time threats
External identities	B2B guest accounts + B2C for consumers	B2B leverages existing directories; B2C provides 50K free MAU, social login, custom user journeys	Single directory for all — creates identity sprawl; Auth0 — higher cost, less Azure integration
Core protocol	OAuth 2.0 / OIDC with JWT tokens	Industry standard, stateless validation via JWKS, supports users + managed identities + workloads	SAML — heavier XML format, less suited for service-to-service, reserved for legacy B2B
Availability	Global multi-region distribution	<50ms to nearest endpoint, 99.99% SLA; JWKS cached locally for offline validation	Single-region — geographic latency, failover complexity; Entra's global fabric handles natively

---

1) High-Level Architecture

Authentication Flow Overview





Step	From	To	Action	Sync/Async
1	Identity Source	Azure Entra	Present credentials (password, certificate, token)	SYNC
2	Azure Entra	Policy Engine	Evaluate conditional access policies	SYNC
3	Policy Engine	MFA Service	Trigger MFA if required	SYNC
4	Azure Entra	Token Service	Issue signed JWT access token	SYNC
5	Client	Resource	Present token in Authorization header	SYNC

6	Resource	Azure Entra	Validate token signature (cached keys)	SYNC

API Design

Authentication APIs

Method	Endpoint	Description
`POST`	`/api/v1/oauth2/token`	Get access token (OAuth 2.0)
`GET`	`/api/v1/oauth2/authorize`	Authorization endpoint
`POST`	`/api/v1/oauth2/revoke`	Revoke a token
`GET`	`/api/v1/.well-known/openid-configuration`	OpenID Connect discovery

User APIs

Method	Endpoint	Description
`GET`	`/api/v1/users/{userId}`	Get user profile
`GET`	`/api/v1/users/me`	Get current user (from token)
`PATCH`	`/api/v1/users/{userId}`	Update user attributes
`POST`	`/api/v1/users`	Create a user
`DELETE`	`/api/v1/users/{userId}`	Delete a user

Application Registration APIs

Method	Endpoint	Description
`POST`	`/api/v1/applications`	Register an application
`GET`	`/api/v1/applications/{appId}`	Get application details
`PUT`	`/api/v1/applications/{appId}/permissions`	Configure API permissions

Token Request (Client Credentials Flow)

```
POST /api/v1/oauth2/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=app_123
&client_secret=secret_456
&scope=https://graph.microsoft.com/.default

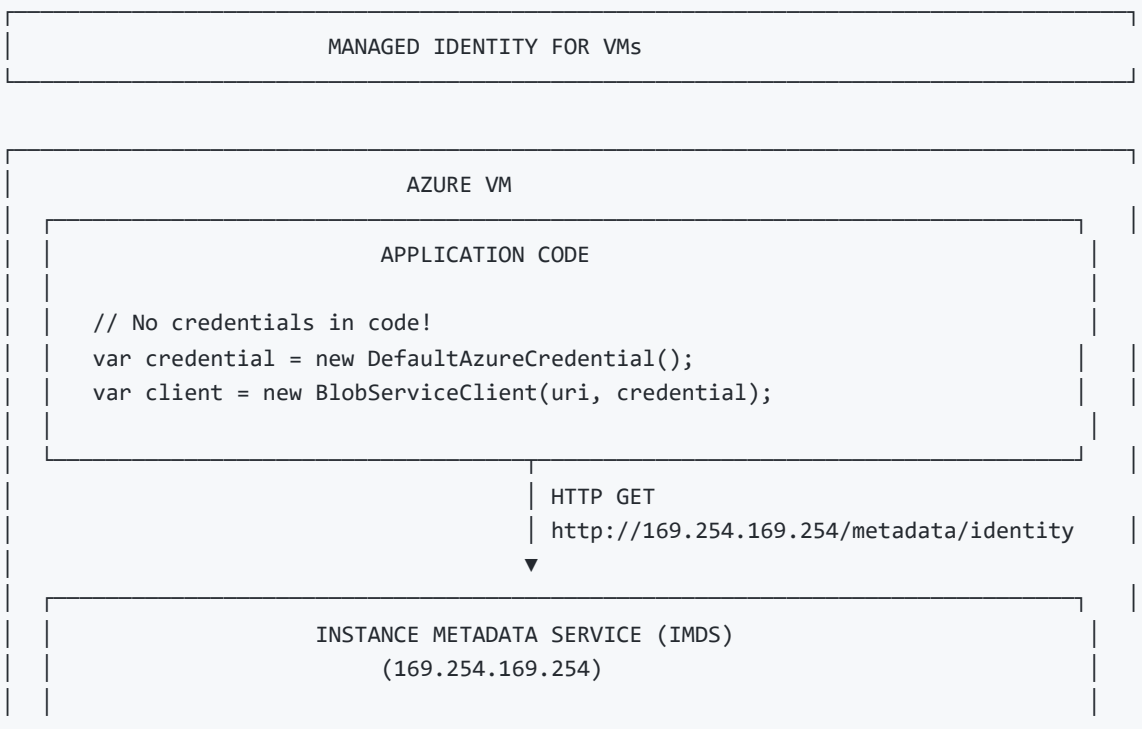
Response: 200 OK
{
  "access_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "User.Read"
}
```

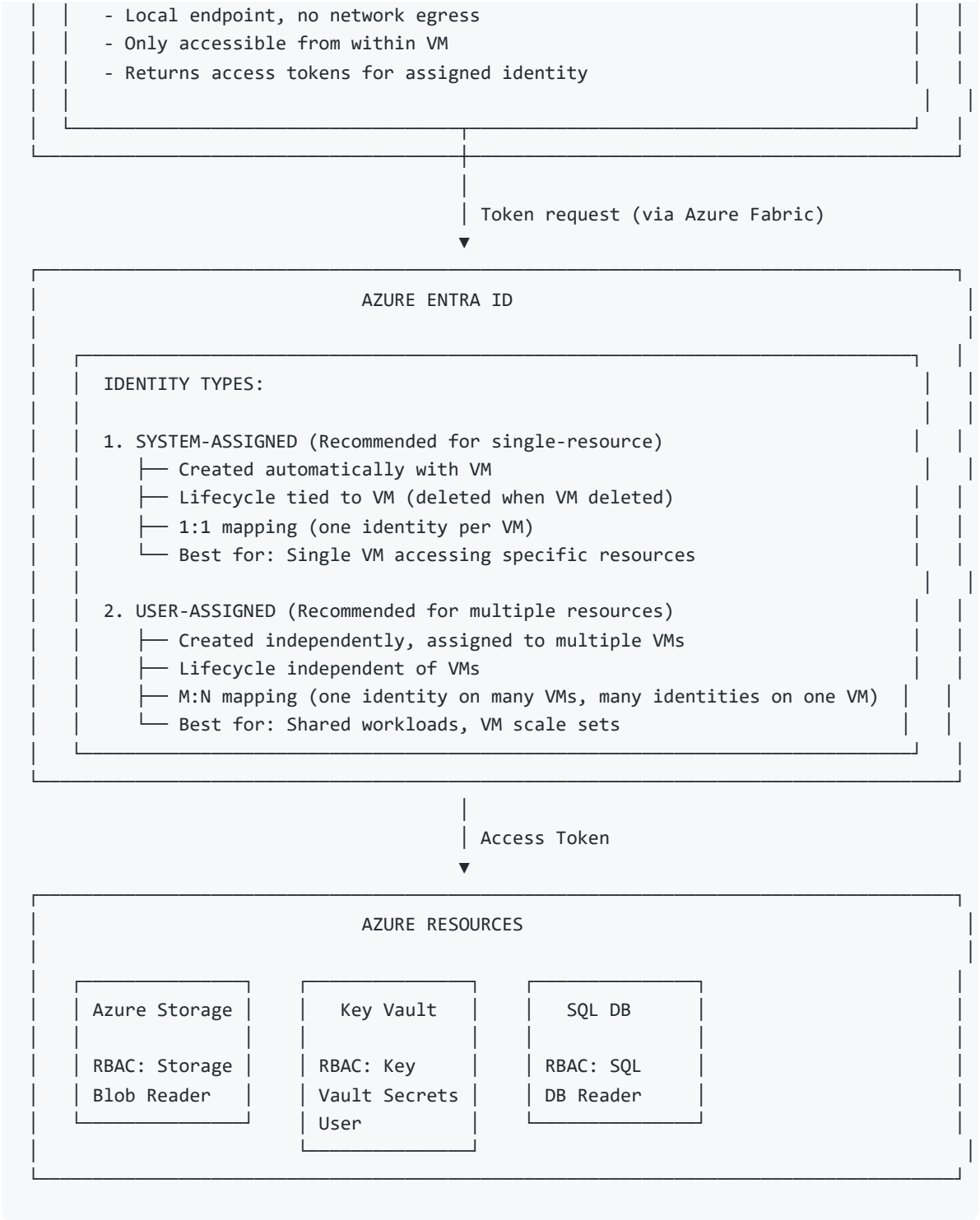
Managed Identity APIs

Method	Endpoint	Description
`GET`	`/metadata/identity/oauth2/token?resource={resource}`	Get token via managed identity (IMDS)

2) Managed Identities for VMs

Architecture





Token Acquisition Flow

Step	Component	Action	Details
1	App Code	Call `DefaultAzureCredential()`	SDK handles token acquisition
2	SDK	HTTP GET to IMDS	`http://169.254.169.254/metadata/id` `resource=https://storage.azure.com`

3	IMDS	Forward to Azure Fabric	Internal Azure infrastructure
4	Azure Entra	Issue token	Signed JWT with claims
5	SDK	Cache token	Until 5 min before expiry
6	SDK	Present to resource	`Authorization: Bearer <token>`

Code Example

```
// C# - Using DefaultAzureCredential (recommended)
using Azure.Identity;
using Azure.Storage.Blobs;

// Automatically uses:
// 1. Environment variables (for local dev)
// 2. Managed Identity (when deployed to Azure)
// 3. Azure CLI / Visual Studio credentials (for development)
var credential = new DefaultAzureCredential();

// Access Azure Storage - NO connection strings!
var blobClient = new BlobServiceClient(
    new Uri("https://mystorageaccount.blob.core.windows.net"),
    credential
);

// Access Key Vault - NO secrets in code!
var secretClient = new SecretClient(
    new Uri("https://mykeyvault.vault.azure.net"),
    credential
);
```

```
# Python - Using DefaultAzureCredential
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient

credential = DefaultAzureCredential()

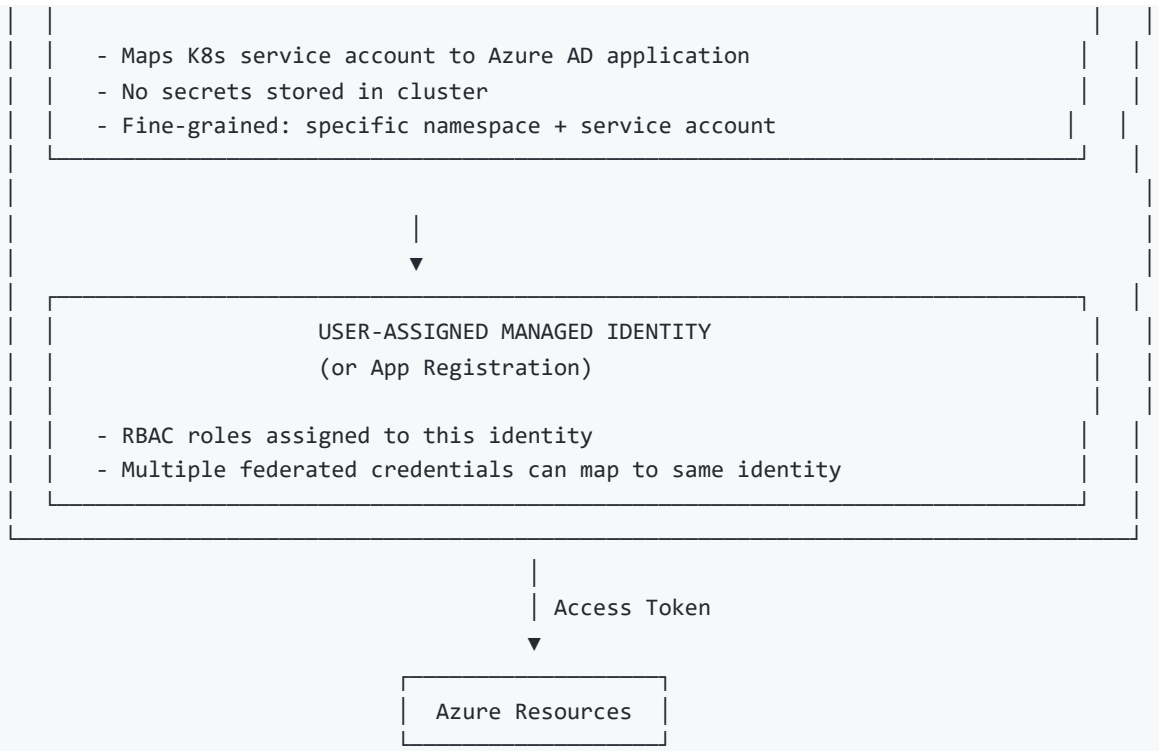
# Access Azure Storage
blob_service = BlobServiceClient(
    account_url="https://mystorageaccount.blob.core.windows.net",
    credential=credential
)
```

---

3) Workload Identity for Kubernetes (AKS)

Architecture





Setup Steps

```
# 1. Create User-Assigned Managed Identity
az identity create \
  --name "my-workload-identity" \
  --resource-group "my-rg" \
  --location "eastus"

# 2. Create Federated Credential
az identity federated-credential create \
  --name "aks-federation" \
  --identity-name "my-workload-identity" \
  --resource-group "my-rg" \
  --issuer "https://oidc.prod-aks.azure.com/{tenant-id}/{cluster-id}" \
  --subject "system:serviceaccount:my-namespace:my-service-account" \
  --audience "api://AzureADTokenExchange"

# 3. Assign RBAC Role
az role assignment create \
  --assignee-object-id "<managed-identity-object-id>" \
  --role "Storage Blob Data Reader" \
  --scope "/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Storage/stor
```

```
# 4. Kubernetes ServiceAccount (with annotations)
apiVersion: v1
kind: ServiceAccount
metadata:
```

```yaml
    name: my-service-account
    namespace: my-namespace
    annotations:
      azure.workload.identity/client-id: "<managed-identity-client-id>"
    labels:
      azure.workload.identity/use: "true"
---
# 5. Pod using the ServiceAccount
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
  labels:
    azure.workload.identity/use: "true"
spec:
  serviceAccountName: my-service-account
  containers:
  - name: app
    image: myapp:latest
    # Token automatically projected to /var/run/secrets/azure/tokens/
```

Comparison: Pod Identity vs Workload Identity

Aspect	Pod Identity (Deprecated)	Workload Identity (Recommended)
Secret storage	Identity binding CRDs	No secrets in cluster
Token exchange	NMI DaemonSet intercepts	Direct OIDC federation
Security	Node-level access	Pod-level, fine-grained
Multi-cluster	Complex	Easy (same federation)
Non-AKS K8s	No	Yes (any OIDC issuer)
Network	Requires NMI pod	No extra pods

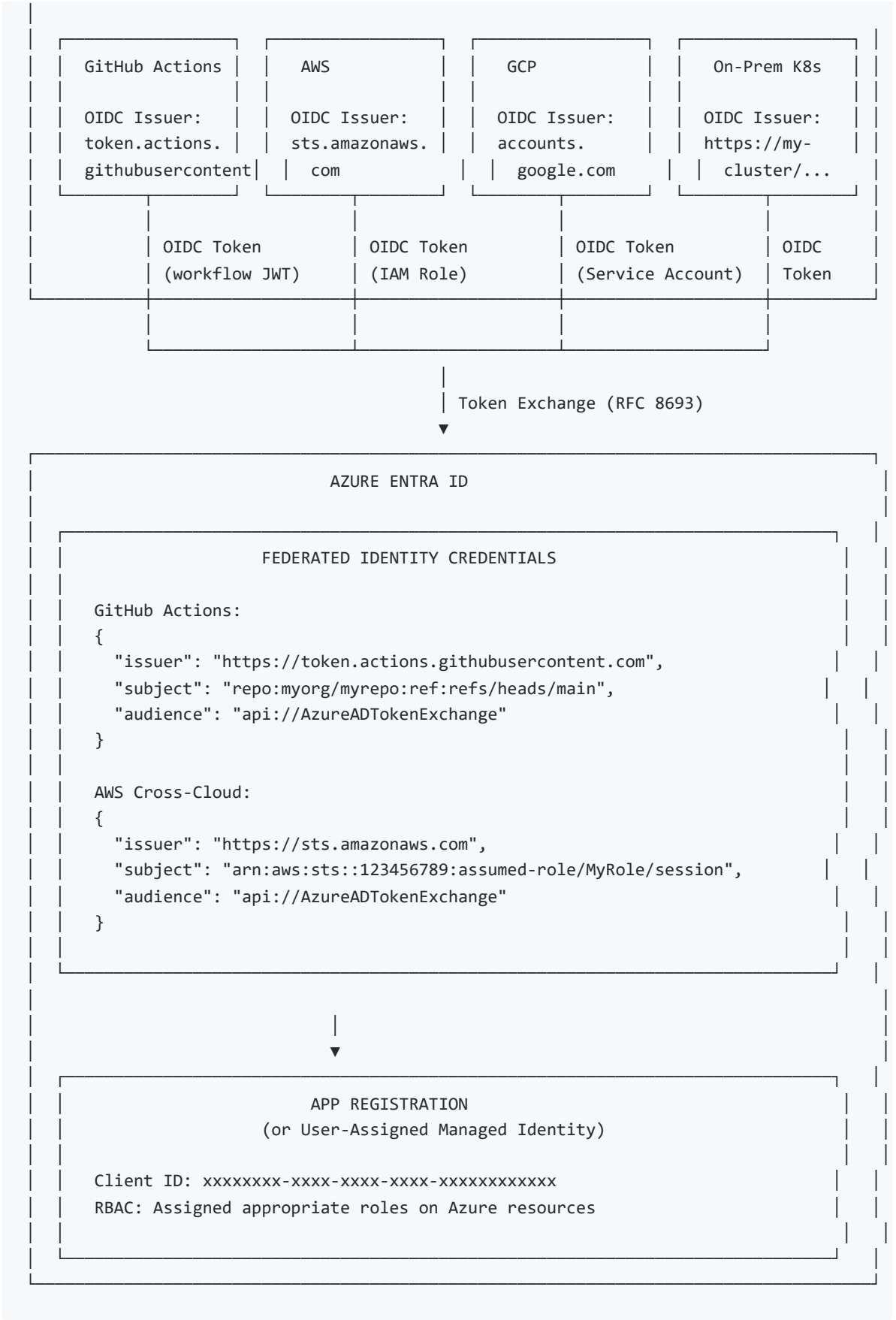
---

4) External/Federated Identities

Architecture for External Workloads

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    FEDERATED IDENTITY FOR EXTERNAL SYSTEMS                   │
└──────────────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────────────┐
│                           EXTERNAL IDENTITY PROVIDERS
```



GitHub Actions Example

```yaml
# .github/workflows/deploy.yml
name: Deploy to Azure
on:
  push:
    branches: [main]

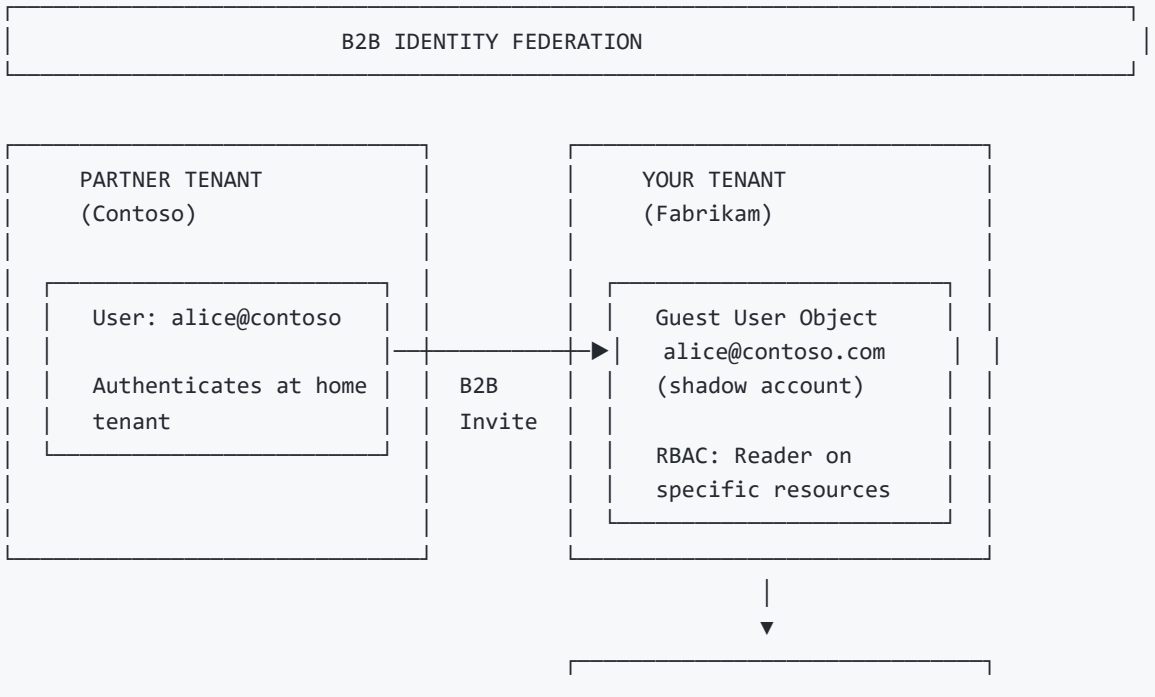
permissions:
  id-token: write   # Required for OIDC
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Login using OIDC - NO secrets stored in GitHub!
      - name: Azure Login
        uses: azure/login@v1
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

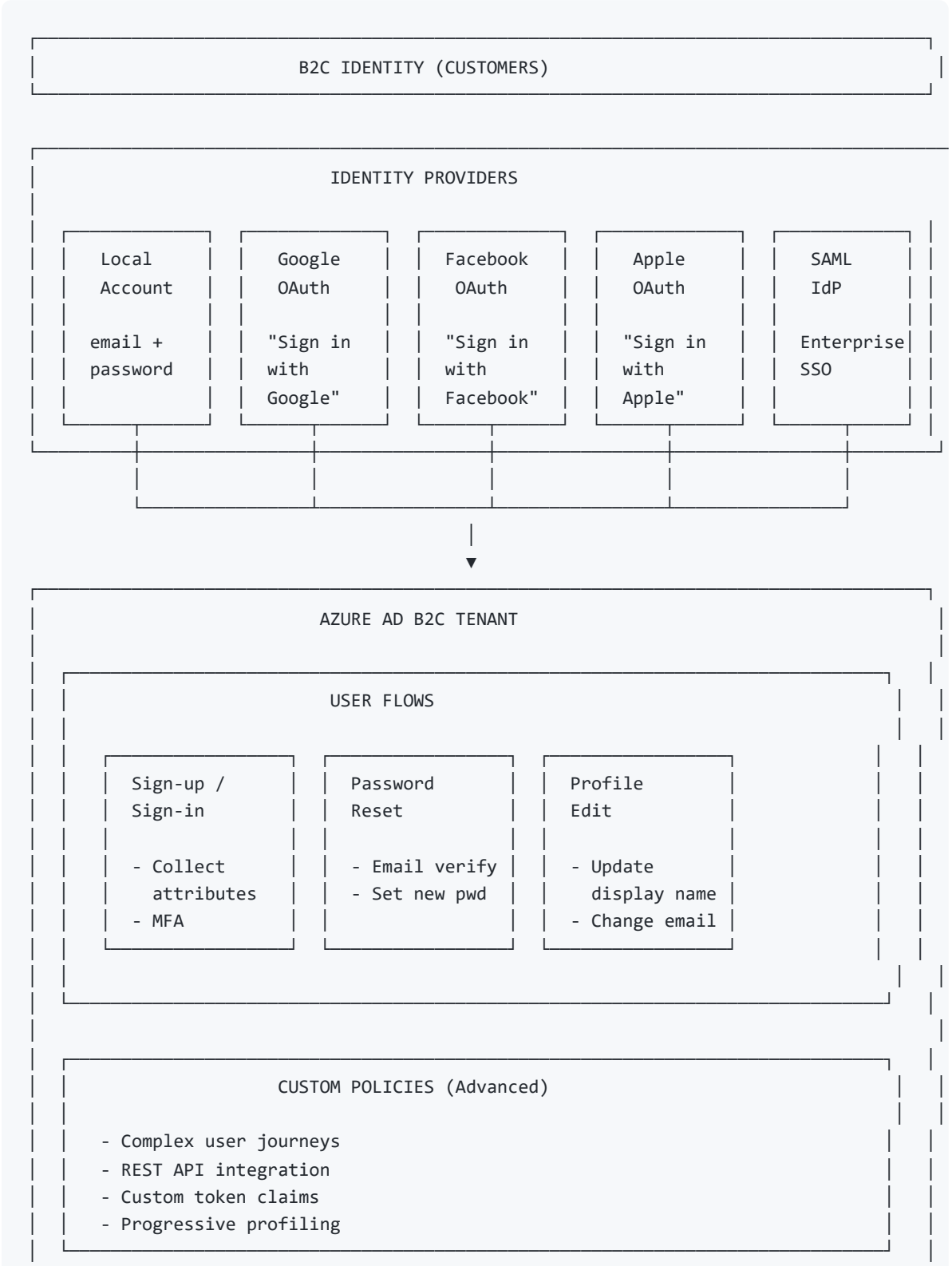
      # Deploy resources
      - name: Deploy
        run: |
          az webapp deploy --name myapp --src-path ./dist
```

B2B (Business Partners) Federation





B2C (Customer) Identity



```
└──────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        │ Access Token + ID Token
                                        ▼
                                ┌──────────────────────────────────┐
                                │   YOUR CUSTOMER-FACING APPLICATION   │
                                └──────────────────────────────────┘
```

5) Token Architecture

JWT Structure

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              JWT ACCESS TOKEN                                │
└──────────────────────────────────────────────────────────────────────────────┘

HEADER.PAYLOAD.SIGNATURE

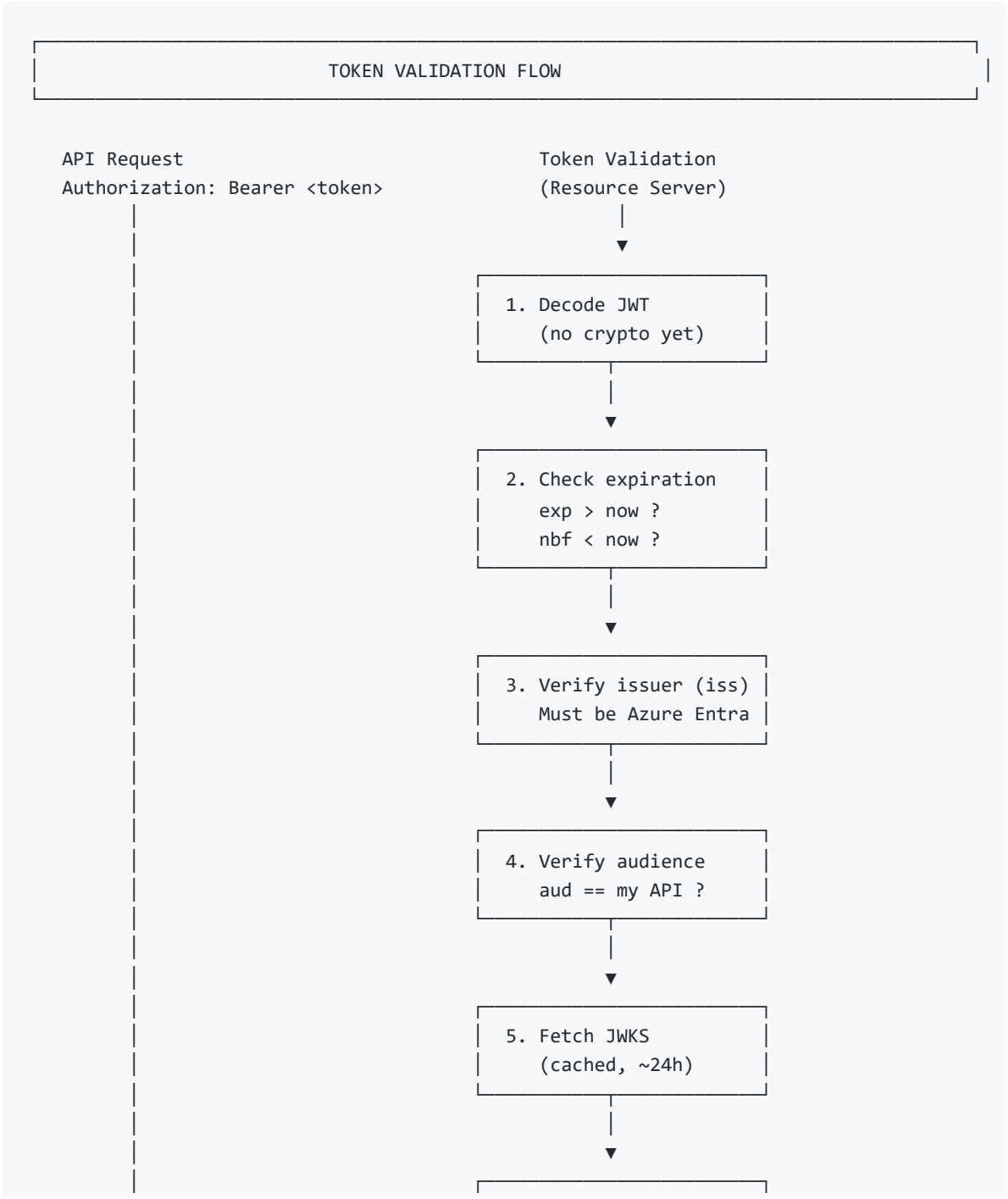
┌──────────────────────────────────────────────────────────────────────────────┐
│  HEADER (Base64URL encoded)                                                  │
│  {                                                                           │
│    "typ": "JWT",                                                             │
│    "alg": "RS256",                                                           │
│    "kid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"  // Key ID for signature verify   │
│  }                                                                           │
└──────────────────────────────────────────────────────────────────────────────┘

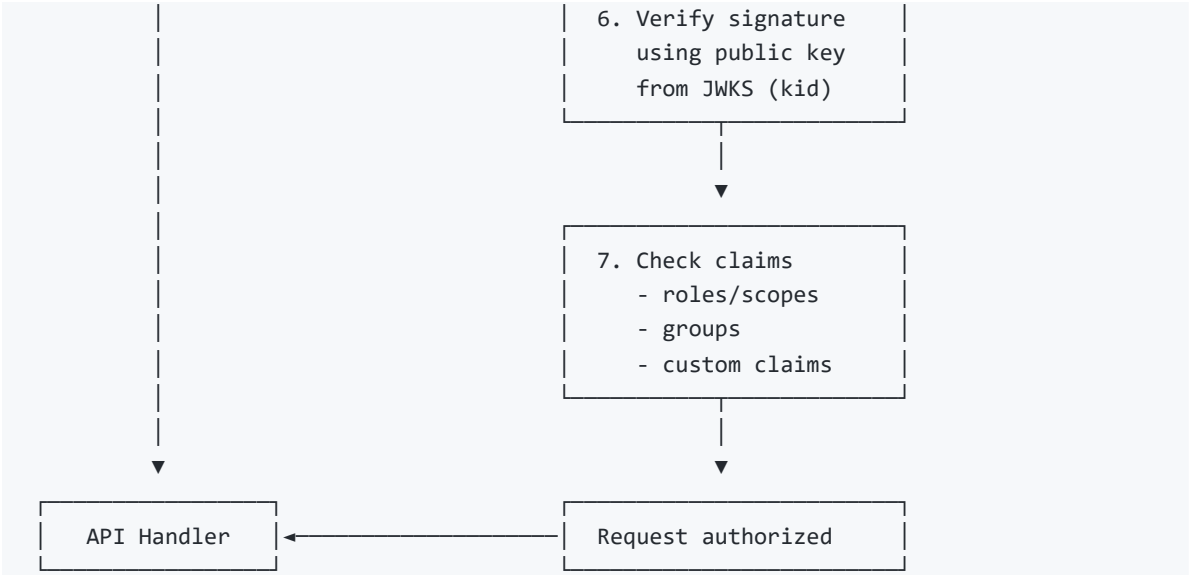
┌──────────────────────────────────────────────────────────────────────────────┐
│  PAYLOAD (Claims)                                                            │
│  {                                                                           │
│    // Standard claims                                                        │
│    "iss": "https://sts.windows.net/{tenant-id}/",     // Issuer              │
│    "sub": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Subject (user/app ID)     │
│    "aud": "https://storage.azure.com",                // Audience (target resource)│
│    "exp": 1706364000,                                  // Expiration (Unix timestamp)
│    "iat": 1706360400,                                  // Issued at           │
│    "nbf": 1706360400,                                  // Not before          │
│                                                                              │
│    // Azure-specific claims                                                  │
│    "tid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Tenant ID           │
│    "oid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Object ID           │
│    "azp": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Authorized party (client) │
│    "scp": "user_impersonation",                        // Scopes (delegated)  │
│    "roles": ["Storage.BlobReader"],                    // App roles           │
│                                                                              │
│    // Optional claims                                                        │
│    "name": "John Doe",                                                       │
│    "email": "john@contoso.com",                                              │
│    "groups": ["group-id-1", "group-id-2"],            // Group memberships   │
│    "xms_cc": ["CP1"]                                   // Client capabilities │
│  }                                                                           │
```

```
└──────────────────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────────────┐
│  SIGNATURE                                                                           │
│                                                                                      │
│  RS256(Base64URL(header) + "." + Base64URL(payload), private_key)                    │
│                                                                                      │
│  - Signed by Azure Entra's private key                                               │
│  - Verified using public key from JWKS endpoint                                      │
│  - JWKS URL: https://login.microsoftonline.com/{tenant}/discovery/v2.0/keys          │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

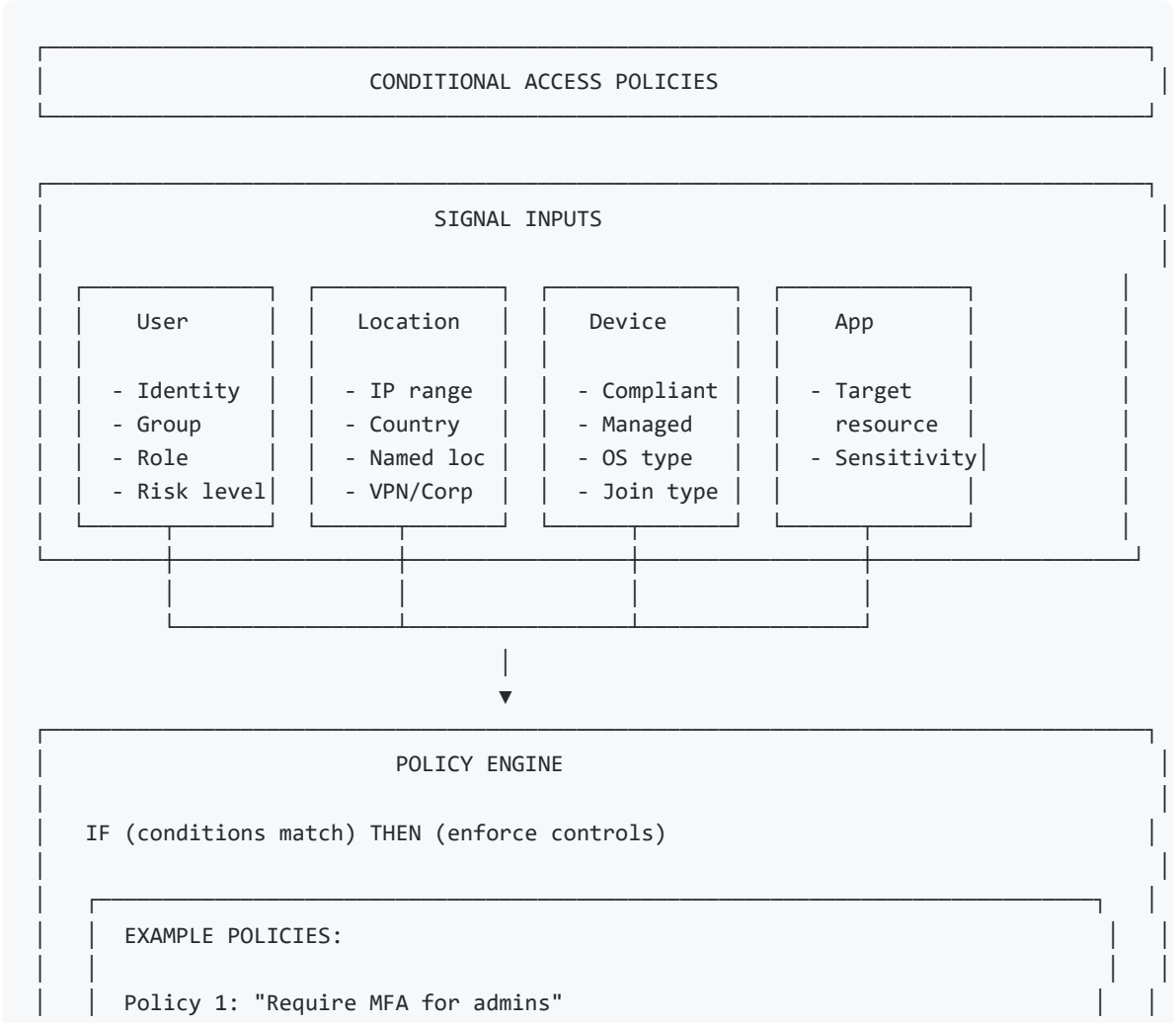
Token Validation Flow

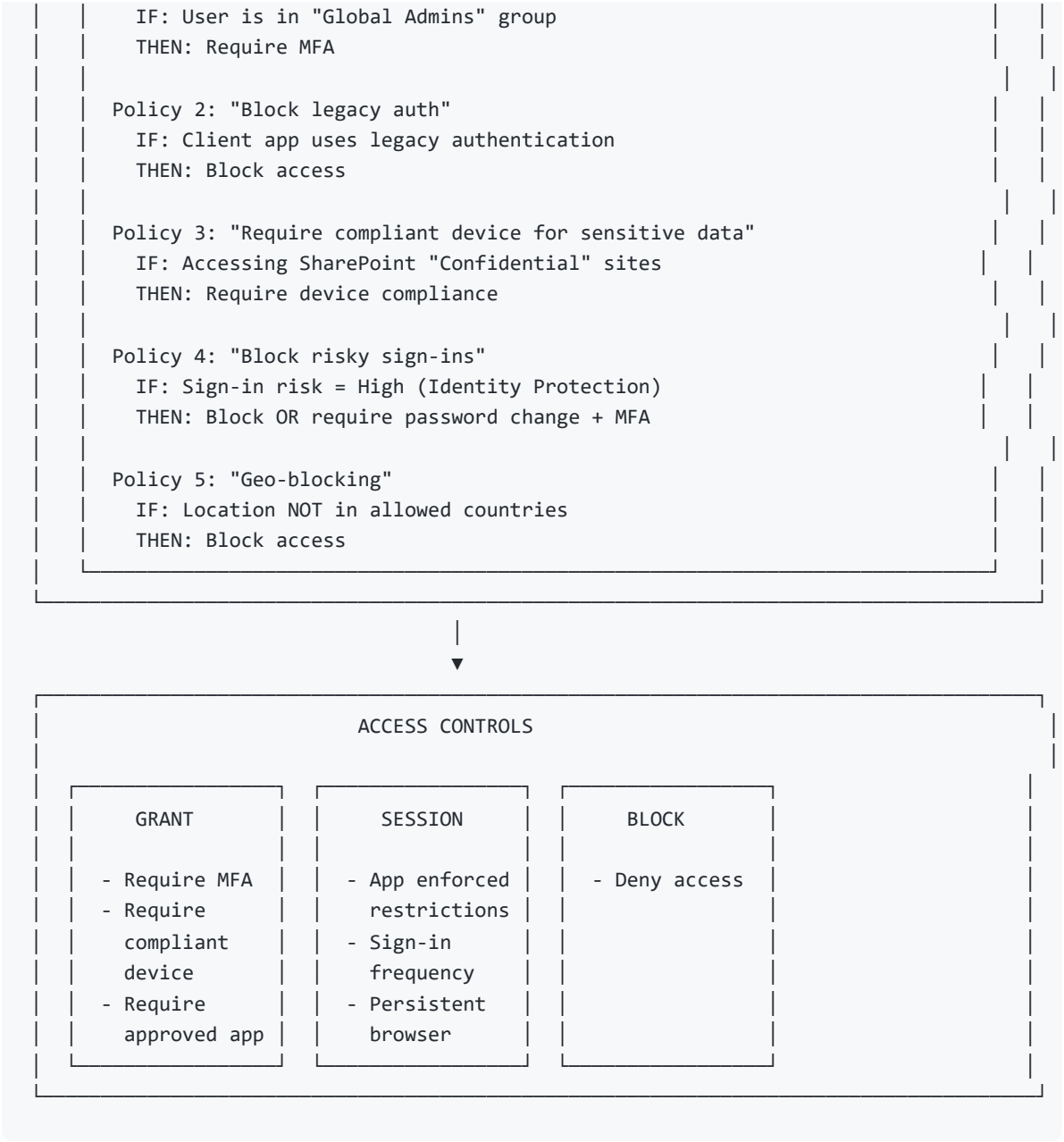




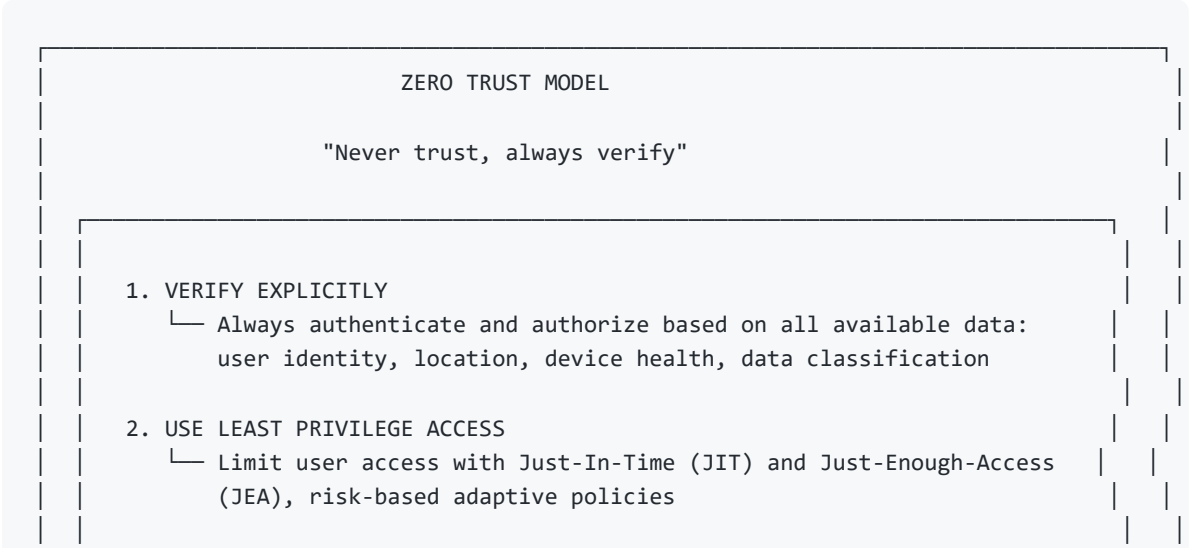
6) Conditional Access & Zero Trust

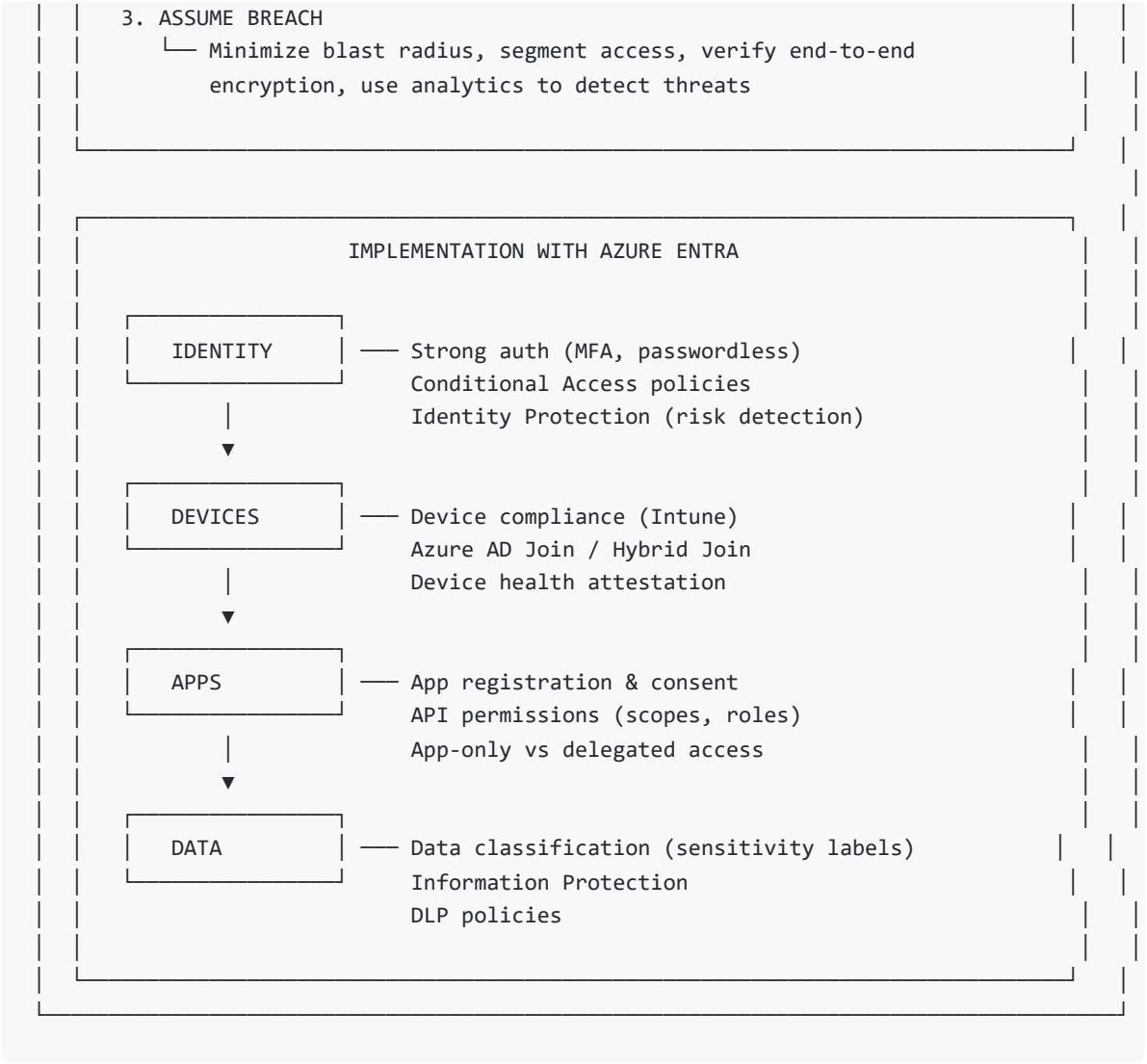
Policy Architecture





Zero Trust Principles



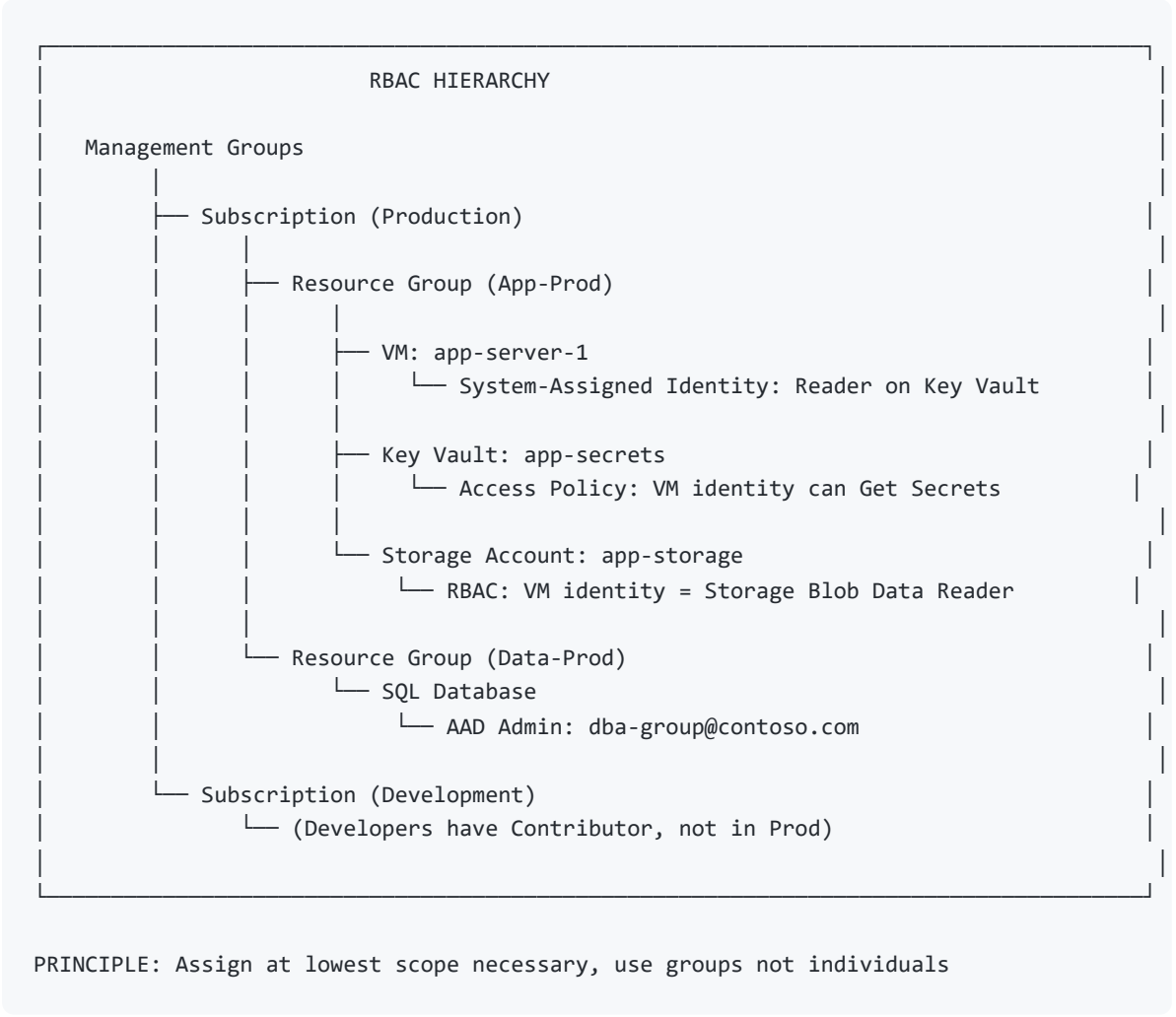


7) Security & Governance

Concern	Solution
Credential theft	Managed identities (no passwords), passwordless auth (FIDO2, Windows Hello)
Token theft	Short-lived tokens (1h), Continuous Access Evaluation (CAE), token binding
Lateral movement	Conditional Access, PIM for just-in-time admin access
Compromised identity	Identity Protection (risk detection), automated remediation
Over-privileged access	RBAC with least privilege, Access Reviews, PIM

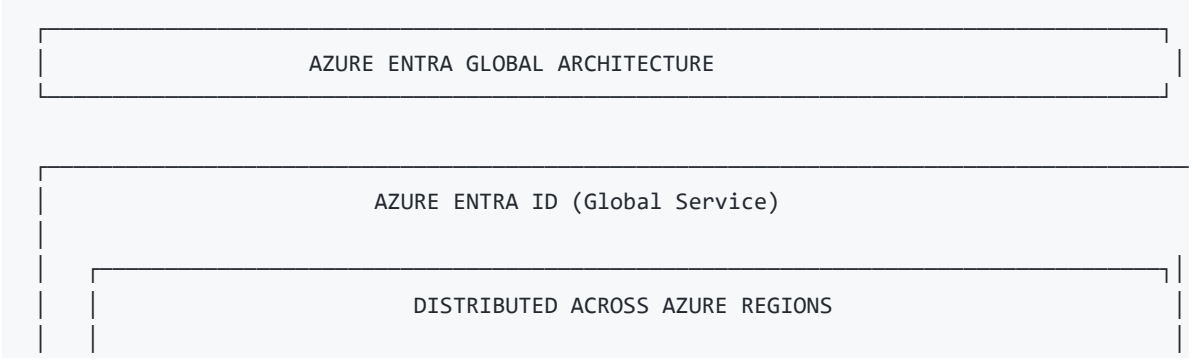
Audit trail	Sign-in logs, Audit logs, Azure Monitor integration
External threats	Geo-blocking, IP restrictions, risky sign-in policies
Legacy protocols	Block legacy authentication (IMAP, POP3, SMTP)

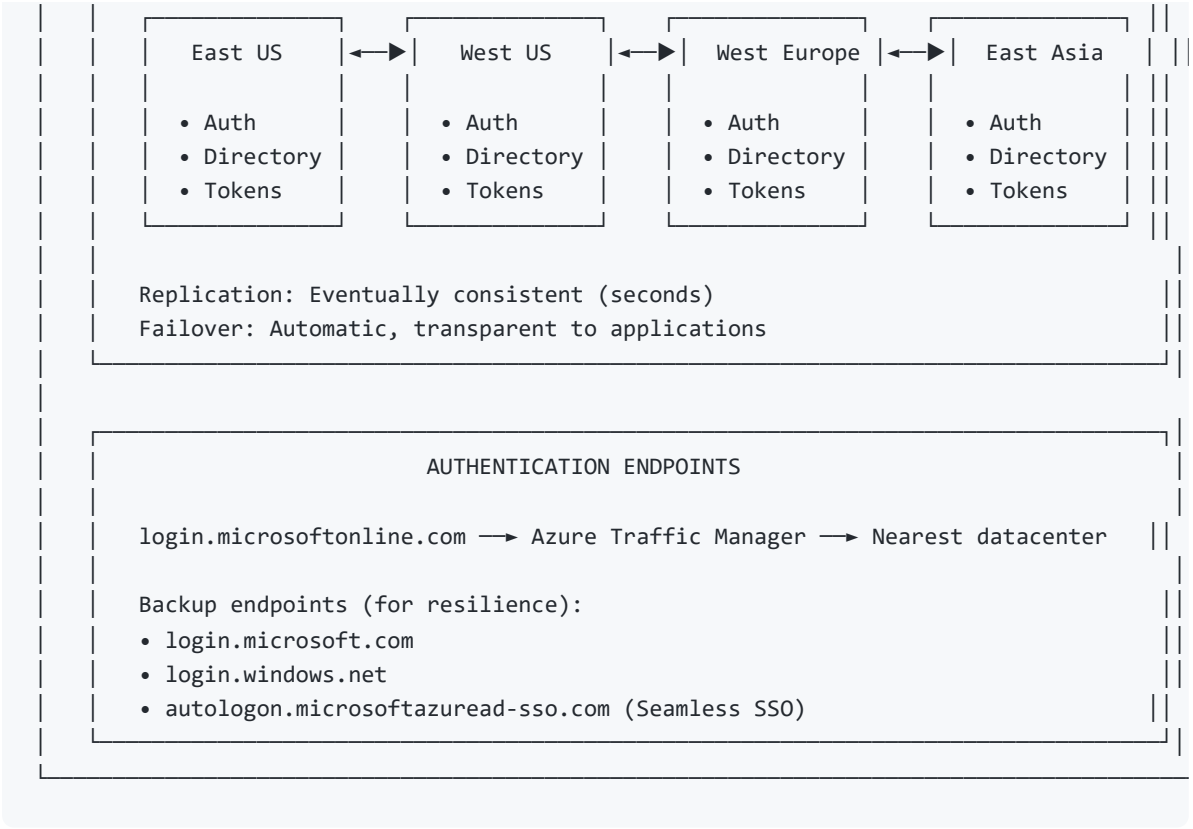
RBAC Best Practices



---

8) Multi-Region & High Availability





Resilience Best Practices

Scenario	Mitigation
Azure Entra partial outage	Token caching (MSAL default: 1h), fallback to cached credentials
Network connectivity	Multiple auth endpoints, retry with exponential backoff
Regional failure	Azure Entra auto-routes to healthy region
Token validation offline	Cache JWKS keys locally (24h TTL)

---

9) Failure Modes

Failure	Impact	Mitigation
Azure Entra outage	Cannot acquire new tokens	Token caching, long-lived refresh tokens (90 days)
IMDS unavailable	VMs can't get tokens	Retry logic, circuit breaker, fallback to service principal

AKS OIDC issuer down	Pods can't exchange tokens	Cache tokens aggressively, graceful degradation
Network partition	Can't reach auth endpoints	Validate tokens locally with cached JWKS
Token expired	401 Unauthorized	Proactive token refresh (5 min before expiry)
Credential rotation	Brief auth failures	Blue-green credential rotation, overlap validity period

Token Caching Strategy

```
// MSAL handles caching automatically, but you can configure it
var app = ConfidentialClientApplicationBuilder
    .Create(clientId)
    .WithAuthority(authority)
    .WithClientSecret(clientSecret)
    .Build();

// In-memory cache (default) or distributed cache (Redis)
app.AddDistributedTokenCache(services =>
{
    services.AddDistributedMemoryCache(); // Or Redis
    services.Configure<MsalDistributedTokenCacheAdapterOptions>(o =>
    {
        o.AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(24);
    });
});

// Acquire token (uses cache automatically)
var result = await app.AcquireTokenForClient(scopes)
    .ExecuteAsync();
```

---

10) Cost Estimates

Component	Tier	Monthly Cost (Est.)
Azure Entra ID P1	Per user/month	~$6/user
Azure Entra ID P2	Per user/month (includes P1 + Identity Protection, PIM)	~$9/user

B2C	First 50K MAU free, then $0.00325/MAU	Variable
Managed Identities	Free	$0
Workload Identity	Free	$0
Conditional Access	Included in P1/P2	$0 (with P1/P2)

Example: 10,000 employee enterprise

- Azure Entra ID P2: 10,000 × $9 = **$90,000/year**
- Managed Identities: Free
- B2C (1M customers): ~$3,250/month = **$39,000/year**
- **Total: ~$129,000/year**

---

11) Trade-offs Discussion

Decision	Option A	Option B	Choice	Reasoning
VM Identity	System-assigned	User-assigned	**Depends**	System for single VM; User for VMSS/shared
K8s auth	Pod Identity	Workload Identity	**Workload**	No secrets in cluster, future-proof
External workloads	Service Principal + Secret	Federated Credential	**Federated**	No secrets to manage/rotate
B2B	Guest accounts	External identities	**Guest**	Full integration with your tenant
B2C	Azure AD B2C	3rd party (Auth0)	**B2C**	Native Azure integration, cost
Token lifetime	Short (1h)	Long (24h)	**Short**	Security; use refresh tokens for UX

---

12) 60-Second Explanation

> "Azure Entra ID is Microsoft's cloud identity platform implementing **Zero Trust**: never trust, always verify.
>
> For **Azure VMs**, I use **Managed Identities** - the VM gets an identity automatically, acquires tokens via the Instance Metadata Service at 169.254.169.254, and authenticates to Azure resources without any credentials in code.
>
> For **Kubernetes**, **Workload Identity** federates the AKS OIDC issuer with Azure Entra. Pods get service account tokens that are exchanged for Azure AD tokens - again, no secrets in the cluster.
>
> For **external workloads** like GitHub Actions or AWS, I create **Federated Identity Credentials** that trust external OIDC issuers. The external system presents its token, Azure Entra validates it against the trusted issuer, and issues an Azure token.
>
> For **partners and customers**, **B2B** invites external users as guests (they authenticate at their home tenant), and **B2C** handles consumer identities with social logins.
>
> Everything flows through **Conditional Access** policies that evaluate signals (user, device, location, risk) and enforce controls (MFA, compliance). Combined with short-lived tokens and continuous access evaluation, this implements defense-in-depth."

Technologies Deep Dive

Technology	Role	Reference
OAuth 2.0 / OIDC	Token-based authentication protocols	Tech Guide
JWT	Token format for claims	Tech Guide
SAML	Legacy federation protocol	Tech Guide
FIDO2	Passwordless authentication	Tech Guide

Appendix: DefaultAzureCredential Chain

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    DefaultAzureCredential - Credential Chain                 │
└──────────────────────────────────────────────────────────────────────────────┘

The SDK tries each credential source in order until one succeeds:

┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
```

```
┌─────────────────────────────────────────────────────────────────────────────┐
│  1. ENVIRONMENT VARIABLES                                                   │
│     └── AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, AZURE_TENANT_ID               │
│     └── Used in: CI/CD pipelines, containers                                │
│                                                                             │
│  2. WORKLOAD IDENTITY (AKS)                                                 │
│     └── AZURE_FEDERATED_TOKEN_FILE, AZURE_CLIENT_ID, AZURE_TENANT_ID        │
│     └── Used in: Kubernetes pods with workload identity                     │
│                                                                             │
│  3. MANAGED IDENTITY                                                        │
│     └── System-assigned or user-assigned                                    │
│     └── Used in: VMs, App Service, Functions, AKS                           │
│                                                                             │
│  4. AZURE CLI                                                               │
│     └── az login credentials                                                │
│     └── Used in: Local development                                          │
│                                                                             │
│  5. AZURE POWERSHELL                                                        │
│     └── Connect-AzAccount credentials                                       │
│     └── Used in: Local development                                          │
│                                                                             │
│  6. VISUAL STUDIO                                                           │
│     └── Logged-in Azure account                                             │
│     └── Used in: Local development in VS                                    │
│                                                                             │
│  7. VISUAL STUDIO CODE                                                      │
│     └── Azure Account extension                                             │
│     └── Used in: Local development in VS Code                               │
│                                                                             │
│  8. INTERACTIVE BROWSER                                                     │
│     └── Opens browser for login (if enabled)                                │
│     └── Used in: Local development, interactive apps                        │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
```

```
// Code works everywhere without changes:
var credential = new DefaultAzureCredential();

// Local dev: Uses Azure CLI
// VM: Uses Managed Identity
// AKS: Uses Workload Identity
// CI/CD: Uses Environment Variables
```

Appendix: Common Scenarios

Scenario 1: VM accessing Key Vault

```
# 1. Enable system-assigned managed identity on VM
az vm identity assign --name myVM --resource-group myRG

# 2. Grant access to Key Vault
```

```
az keyvault set-policy --name myKeyVault \
  --object-id $(az vm show --name myVM --resource-group myRG --query identity.princi
  --secret-permissions get list

# 3. In VM, app code just works
```

Scenario 2: AKS pod accessing Storage

```
# 1. Create user-assigned managed identity
az identity create --name myWorkloadId --resource-group myRG

# 2. Create federated credential
az identity federated-credential create \
  --name k8s-federation \
  --identity-name myWorkloadId \
  --resource-group myRG \
  --issuer $(az aks show -n myCluster -g myRG --query "oidcIssuerProfile.issuerUrl"
  --subject system:serviceaccount:default:my-service-account

# 3. Assign RBAC role
az role assignment create \
  --assignee $(az identity show -n myWorkloadId -g myRG --query principalId -o tsv)
  --role "Storage Blob Data Reader" \
  --scope /subscriptions/.../storageAccounts/mystorageaccount

# 4. Annotate ServiceAccount and deploy
```

Scenario 3: GitHub Actions deploying to Azure

```
# 1. Create app registration
az ad app create --display-name "github-actions-deploy"

# 2. Create federated credential
az ad app federated-credential create --id <app-id> --parameters '{
  "name": "github-main-branch",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:myorg/myrepo:ref:refs/heads/main",
  "audiences": ["api://AzureADTokenExchange"]
}'

# 3. Assign RBAC role to service principal
az role assignment create \
  --assignee <app-id> \
  --role "Contributor" \
  --scope /subscriptions/.../resourceGroups/myRG

# 4. In GitHub Actions, use azure/login with OIDC
```

49. Design GitHub Copilot

Problem Statement

Design an AI-powered code completion and assistance platform like GitHub Copilot. The system should provide real-time code suggestions, multi-line completions, chat-based code generation, and contextual understanding of codebases — all integrated into IDEs at massive scale.

Clarifying Questions to Ask

1. **Latency**: What's the acceptable latency for inline suggestions? (< 500ms typical)
2. **Context window**: How much surrounding code context do we send to the model?
3. **Model hosting**: Self-hosted models or third-party LLM API (OpenAI)?
4. **Multi-modal**: Code completions only, or also chat, PR summaries, CLI assistance?
5. **Languages**: How many programming languages to support? (30+ typical)
6. **Privacy**: Can we use customer code for model training? Telemetry policies?
7. **Offline**: Should suggestions work offline or always require connectivity?
8. **Rate limiting**: Per-user or per-organization rate limits?

Design Reasoning

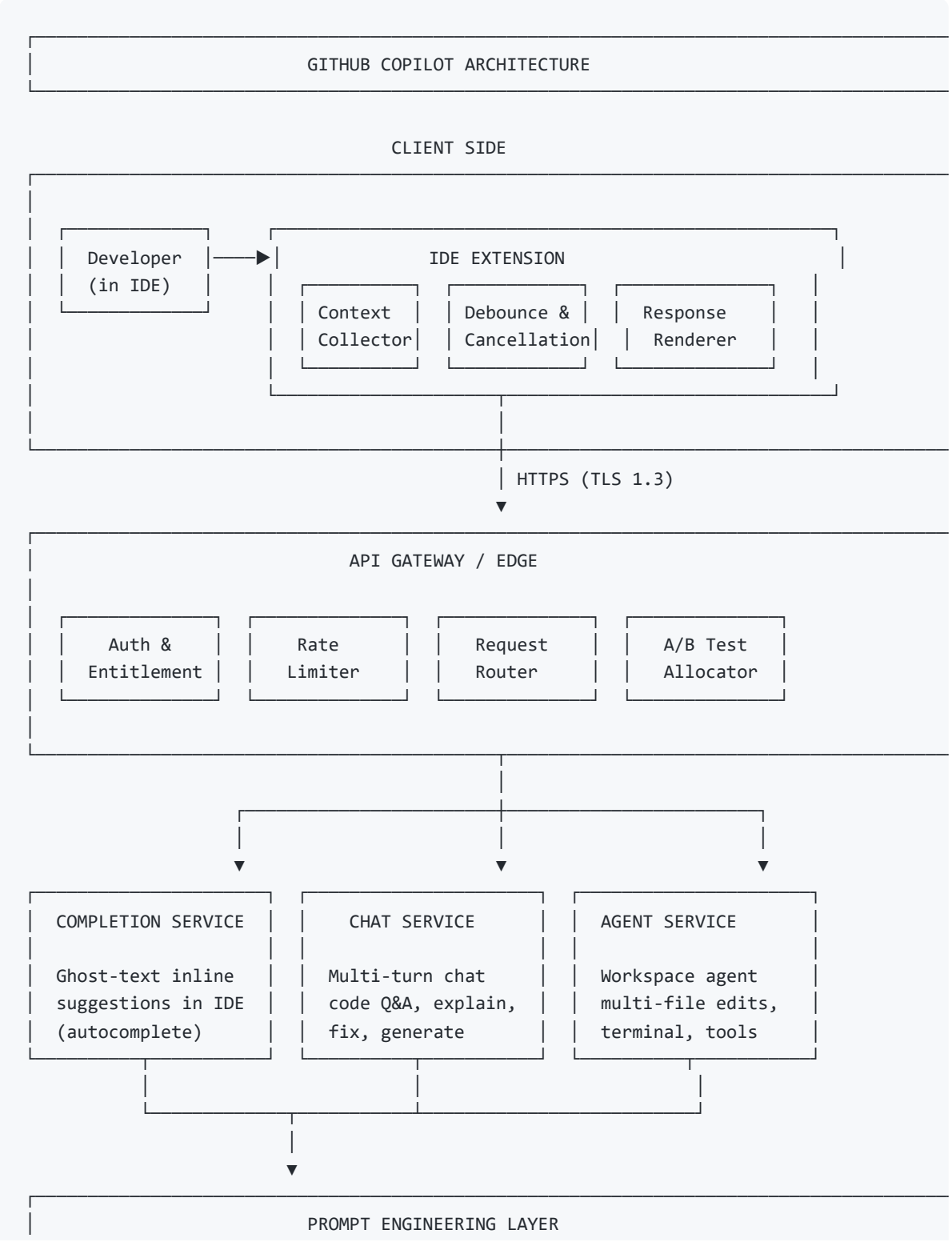
> **Core philosophy**: Optimize for latency, cost, and safety by cancelling most requests before GPU inference, routing tasks to appropriately-sized models, and operating a global edge network — recognizing that the user experience of "instant suggestions" matters as much as suggestion quality.

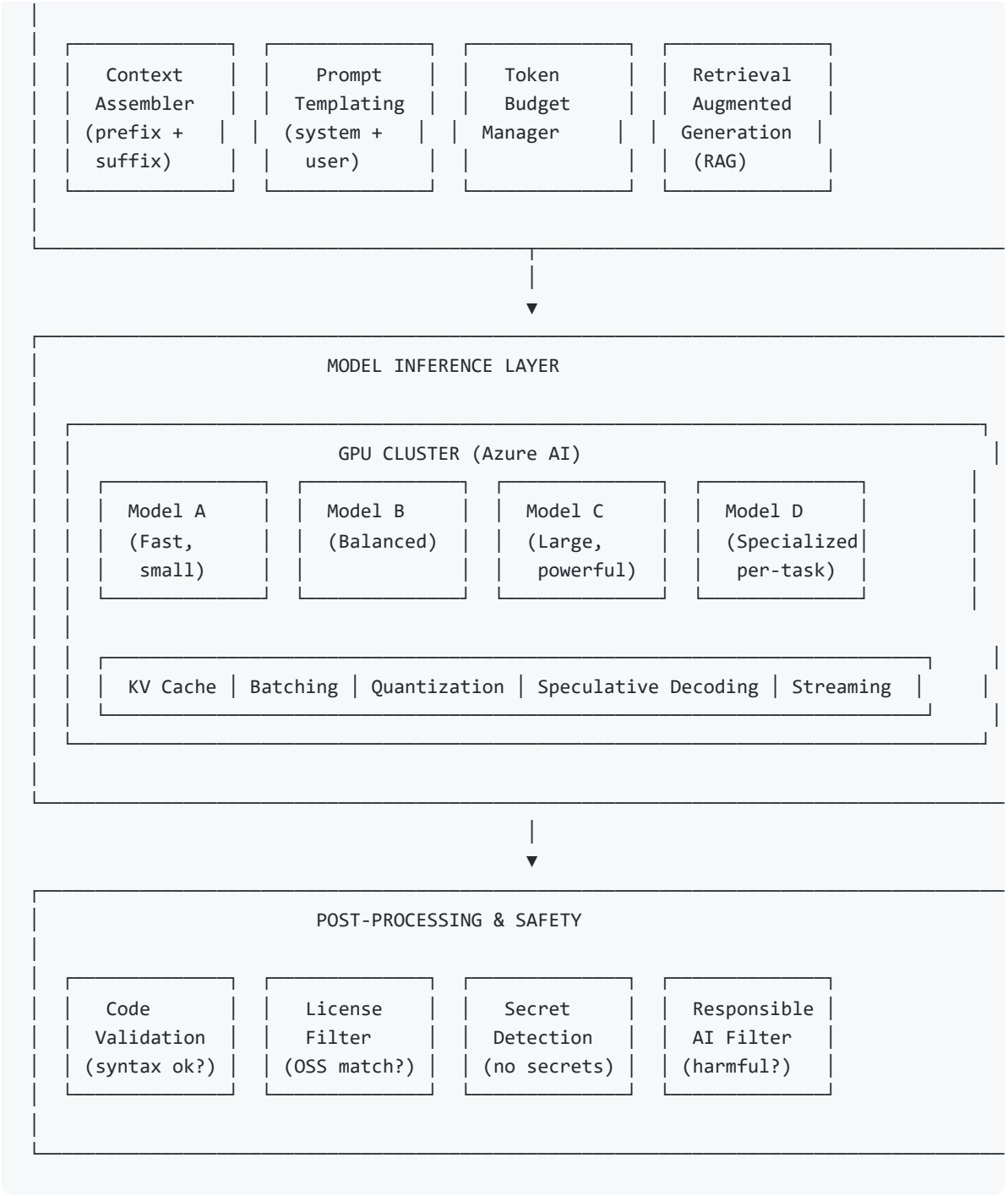
Decision	Choice	Why	Alternative Considered
Context encoding	Fill-in-the-Middle (FIM) `<PRE>{prefix}<SUF>{suffix}<MID>`	Sees code after cursor (function signatures, closing brackets), producing completions that fit surrounding context	Left-to-right completion — misses post-cursor context, lower quality for mid-function edits

Model strategy	Multi-tier: Fast (1-7B) for inline, Balanced (13-30B) for chat, Powerful (70B+) for complex generation	Matches latency SLAs to task complexity; avoids paying for large models on simple completions	Single model for all — economically wasteful, violates <300ms inline completion SLA
Request management	150ms client debounce + AbortController cancellation	Eliminates 70-80% of requests before GPU; makes 100M developers × 20 req/min economically tractable	Send every keystroke — astronomical GPU cost, wastes compute on suggestions user will never see
Model hosting	Hybrid: self-hosted GPU (A100/H100) for small models + OpenAI API for large	Self-hosting small models is cheaper at scale with latency control; API for large models avoids capital expenditure	Fully self-hosted — prohibitive for GPT-4 scale; fully API — premium rates, no latency control
GPU optimization	Continuous batching + KV cache reuse on common prefixes	Reduces latency (no queue wait) and increases utilization; common system prompts share cached computation	Traditional batching (wait for 32+) — increases latency for early-arriving requests
License compliance	Bloom filter (2GB RAM) + distributed code search index	O(1) fast negative check; handles hundreds of millions of OSS files for GPL/copyleft detection	No filtering — legal exposure from GPL code; regex-only — misses sophisticated matches
Global distribution	Multi-region GPU clusters with Azure Front Door latency-based routing	<30ms network RTT for global developers; auto-failover if region GPU utilization >80%	Single US region — 100-200ms+ RTT from distant regions, violates <500ms total latency SLA
Caching	Three-layer: IDE local + API gateway exact	Eliminates redundant GPU inference for	No caching — many developers

	match + semantic cache (cosine >0.95)	repeated/similar queries; ~5-10% GPU load reduction	write identical code (auth, CRUD), missing free efficiency gains

High-Level Design





API Design

Completion API (Inline Suggestions)

Method	Endpoint	Description
`POST`	`/v1/completions`	Get inline code completion
`POST`	`/v1/completions/accept`	Log acceptance telemetry

`POST`	`/v1/completions/reject`	Log rejection telemetry

Chat API

Method	Endpoint	Description
`POST`	`/v1/chat`	Send chat message (streaming SSE)
`GET`	`/v1/chat/{sessionId}/history`	Retrieve chat history
`DELETE`	`/v1/chat/{sessionId}`	Delete chat session

Agent API

Method	Endpoint	Description
`POST`	`/v1/agents/run`	Start an agent task
`GET`	`/v1/agents/{runId}/status`	Poll agent run status
`POST`	`/v1/agents/{runId}/feedback`	Submit tool-call approval/rejection

Completion Request

```
POST /v1/completions
{
  "prompt": {
    "prefix": "def fibonacci(n):\n    if n <= 1:\n        return n\n    ",
    "suffix": "\n\nprint(fibonacci(10))",
    "language": "python",
    "file_path": "math_utils.py"
  },
  "context": {
    "open_files": [
      {
        "path": "tests/test_math.py",
        "content": "...(relevant snippet)...",
        "relevance_score": 0.87
      }
    ],
    "repo_info": {
      "languages": ["python"],
      "framework": "pytest"
    }
  },
  "params": {
    "max_tokens": 256,
    "temperature": 0.0,
    "top_p": 1.0,
```

```
    "n": 3,
    "stop": ["\ndef ", "\nclass ", "\n\n"]
  },
  "user": {
    "ide": "vscode",
    "ide_version": "1.95.0",
    "extension_version": "1.200.0"
  }
}
```

Completion Response

```
{
  "id": "cmpl_abc123",
  "choices": [
    {
      "index": 0,
      "text": "return fibonacci(n-1) + fibonacci(n-2)",
      "finish_reason": "stop",
      "confidence": 0.94
    }
  ],
  "usage": {
    "prompt_tokens": 312,
    "completion_tokens": 14
  },
  "telemetry_id": "tel_xyz789"
}
```

Chat Request (Streaming)

```
POST /v1/chat
{
  "session_id": "sess_001",
  "messages": [
    { "role": "system", "content": "You are a helpful coding assistant..." },
    { "role": "user", "content": "Explain this function and suggest improvements" }
  ],
  "context": {
    "active_selection": "def fib(n): return n if n<=1 else fib(n-1)+fib(n-2)",
    "file_path": "math_utils.py",
    "language": "python"
  },
  "stream": true
}
```

Context Collection & Prompt Assembly

The most critical differentiator — how we build the prompt from surrounding code.

Context Sources (Priority Order)

```
┌──────────────────────────────────────────────────────────────┐
│                   CONTEXT ASSEMBLY PIPELINE                  │
│                                                              │
│  Priority 1: IMMEDIATE CONTEXT                               │
│  ├── Current file prefix (code before cursor)                │
│  ├── Current file suffix (code after cursor)                 │
│  └── Cursor position, indentation level                      │
│                                                              │
│  Priority 2: NEIGHBOR CONTEXT                                │
│  ├── Open editor tabs (ranked by recency)                    │
│  ├── Imports/includes in current file → resolve to definitions │
│  └── Symbols referenced near cursor → find declarations      │
│                                                              │
│  Priority 3: REPOSITORY CONTEXT (RAG)                        │
│  ├── Embedding search over repo index                        │
│  ├── Similar code patterns in the codebase                   │
│  └── README, docs, config files                              │
│                                                              │
│  Priority 4: LEARNED PATTERNS                                │
│  ├── User's accept/reject history                            │
│  ├── Organization coding standards                           │
│  └── Custom instructions (.github/copilot-instructions.md)   │
│                                                              │
│  ──────────────────────────────────────────                  │
│  Token Budget: ~8K context → model                           │
│  Strategy: Fill-in-the-Middle (FIM) encoding                 │
│  Format: <PRE> prefix <SUF> suffix <MID> → completion        │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Fill-in-the-Middle (FIM) Encoding

```
Standard autocompletion (left-to-right):
  prompt = prefix → generate next tokens

FIM (Copilot uses this):
  prompt = <PRE> {prefix} <SUF> {suffix} <MID>
  model generates tokens to fill the gap

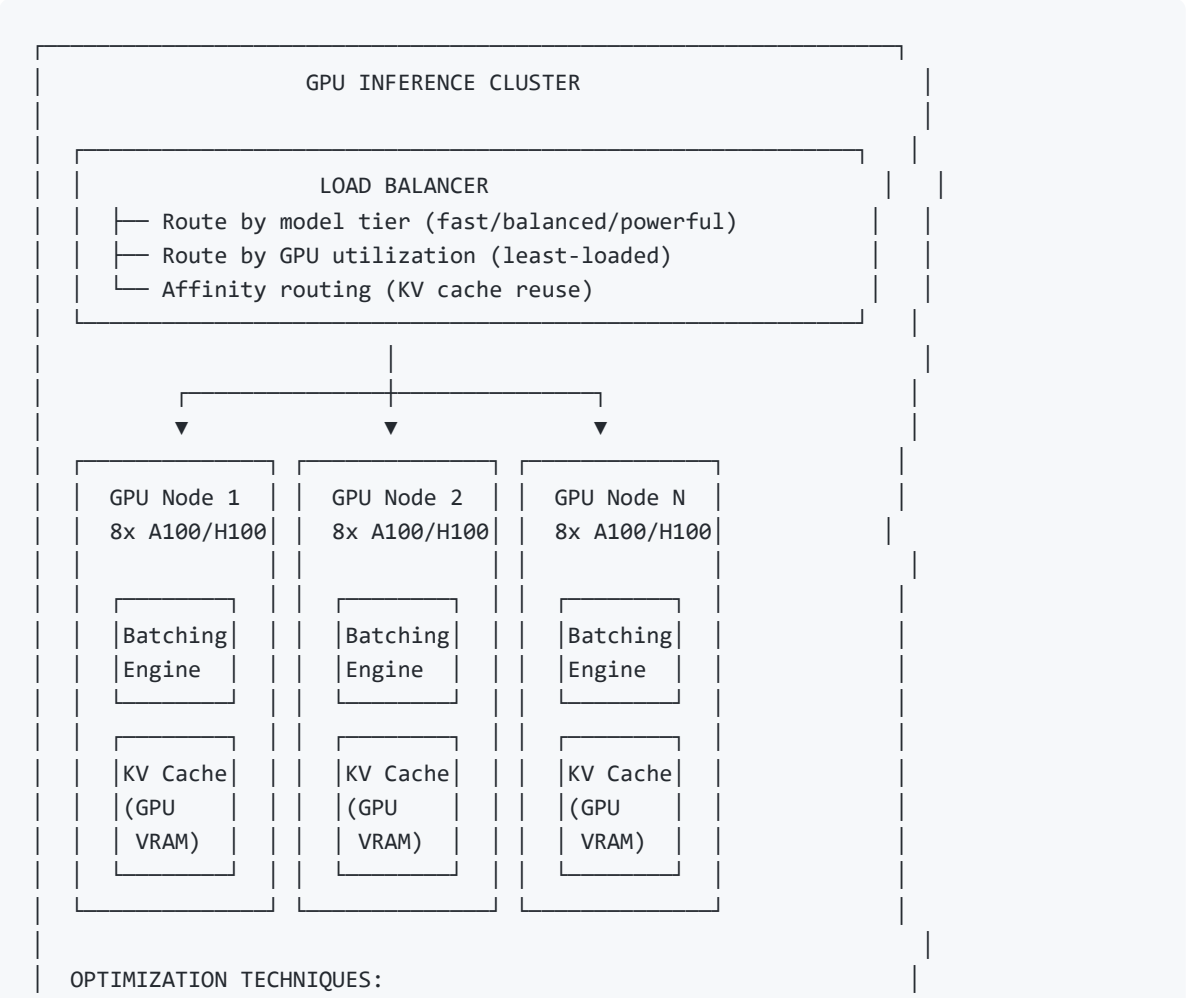
Why FIM is better:
  - Sees code AFTER the cursor (function signature, tests, etc.)
  - Produces completions that fit surrounding code
  - Better for mid-function edits, not just end-of-file
```

Model Inference Layer

Multi-Model Strategy

Model Tier	Use Case	Latency Target	Example
Fast	Inline completions, single-line	< 300ms	Small distilled model (1-7B params)
Balanced	Multi-line completions, chat	< 1s	Medium model (13-30B params)
Powerful	Complex generation, agent tasks	< 10s	Large model (70B+ / GPT-4 class)
Specialized	Code review, security scanning	Async	Fine-tuned per-task models

GPU Cluster Architecture



```
│  ├── Continuous batching (don't wait for full batch)            │
│  ├── Speculative decoding (draft model + verify)                │
│  ├── KV cache sharing (common prefixes across users)            │
│  ├── Quantization (INT8/INT4 for fast tier)                     │
│  └── Tensor parallelism (split large models across GPUs)        │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Latency Breakdown (Inline Completion)

```
End-to-end target: < 500ms

IDE Extension:
  ├── Debounce wait                    ~150ms (wait for typing pause)
  ├── Context collection                ~20ms
  └── Serialize & send request          ~5ms

Network:
  └── Round-trip to nearest region      ~30ms

API Gateway:
  ├── Auth + entitlement check          ~5ms
  ├── Rate limit check                  ~2ms
  └── Route to inference cluster        ~3ms

Prompt Engineering:
  ├── Context assembly (FIM encoding)   ~10ms
  ├── RAG retrieval (if enabled)        ~30ms
  └── Token counting & truncation       ~5ms

Model Inference:
  ├── Queuing (wait for batch slot)     ~20ms
  ├── Prefill (process prompt tokens)   ~30ms
  └── Decode (generate ~30 tokens)      ~80ms

Post-processing:
  ├── Syntax validation                 ~5ms
  ├── License filter                    ~10ms
  └── Secret detection                  ~5ms

Total server-side:                      ~235ms
Total with network:                     ~265ms
Total with debounce:                    ~415ms
```

Debounce, Cancellation & Streaming

Client-Side Intelligence



Why Cancellation Matters

```
Without cancellation:
  - 100M developers × 20 requests/min = 2B requests/min
  - GPU cost: astronomical

With cancellation:
  - 70-80% of requests cancelled before completion
  - Only ~20-30% actually reach inference
  - Massive cost savings
```

Post-Processing & Safety

License Filter (Duplicate Code Detection)

```
┌──────────────────────────────────────────────────────────────┐
│                  LICENSE FILTER PIPELINE                     │
│                                                              │
│  Model output: "function sort(arr) { ... 30 lines ... }"     │
│       │                                                      │
│       ▼                                                      │
│  1. FINGERPRINT: Normalize whitespace, compute n-gram hashes │
│       │                                                      │
│       ▼                                                      │
│  2. SEARCH: Compare against known open-source code index     │
│     (Bloom filter for fast pre-check, then exact match)      │
│       │                                                      │
│       ▼                                                      │
│  3. MATCH FOUND?                                             │
│     ├── No match → Pass through (safe to suggest)            │
│     ├── Permissive license (MIT, Apache) → Allow, cite source│
│     └── Copyleft license (GPL) → Block suggestion            │
│                                                              │
│  INDEX: ~hundreds of millions of public code files            │
│  Storage: Bloom filter (~2GB RAM) + search index (distributed)│
└──────────────────────────────────────────────────────────────┘
```

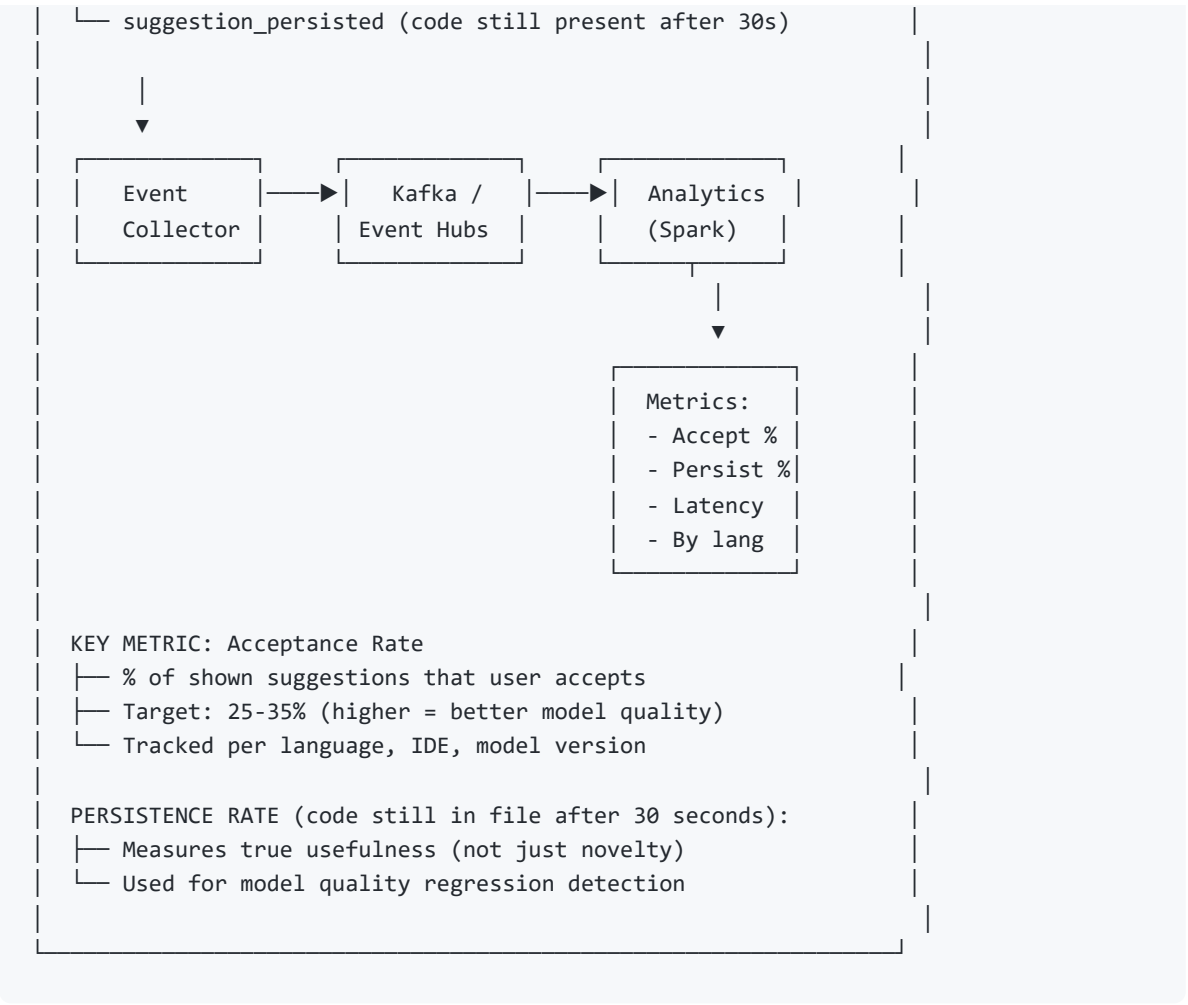
Secret Detection

```
Scan model output for:
├── API keys (pattern: /[A-Za-z0-9]{32,}/)
├── Connection strings (pattern: /Server=.*;Password=.*/)
├── Private keys (pattern: /-----BEGIN.*PRIVATE KEY-----/)
├── Tokens (pattern: /ghp_[A-Za-z0-9]{36}/)
└── Custom patterns per organization

Action: Redact or block suggestion if secrets detected
```

Telemetry & Feedback Loop

```
┌──────────────────────────────────────────────────────────────┐
│                  TELEMETRY PIPELINE                          │
│                                                              │
│  IDE Extension ──▶ Events:                                   │
│  ├── suggestion_shown    (ghost text displayed)              │
│  ├── suggestion_accepted (user pressed Tab)                  │
│  ├── suggestion_rejected (user kept typing / Esc)            │
│  ├── suggestion_partially_accepted (accepted first line only)│
│  ├── chat_message_sent                                       │
│  ├── chat_code_inserted  (copied code from chat to editor)   │
```



Key Components

Component	Technology	Purpose
IDE Extension	VS Code API, LSP	Collect context, render suggestions, handle accept/reject
API Gateway	Azure API Management / Envoy	Auth, rate limiting, routing, TLS termination
Completion Service	Go / Rust microservice	Orchestrate prompt assembly → inference → post-processing
Chat Service	Go / Rust microservice	Multi-turn conversation management, streaming SSE
Agent Service	Go / Python microservice	Tool orchestration, multi-step task execution

Prompt Engine	Python / Rust	FIM encoding, context ranking, token budget management
RAG Index	Vector DB (pgvector / Qdrant)	Semantic code search over repository embeddings
Model Serving	vLLM / TensorRT-LLM / Triton	GPU inference with continuous batching, KV caching
GPU Cluster	Azure ND-series (A100/H100)	Tensor parallel model hosting, auto-scaling
License Filter	Bloom filter + search index	Detect code matching open-source with restrictive licenses
Secret Scanner	Regex + ML classifier	Block suggestions containing secrets or credentials
Telemetry	Kafka / Azure Event Hubs + Spark	Track acceptance rate, latency, usage metrics
Feature Flags	LaunchDarkly / Azure App Config	A/B test model versions, prompt strategies, features
User Entitlement	GitHub subscription DB	Verify Copilot license (Individual, Business, Enterprise)

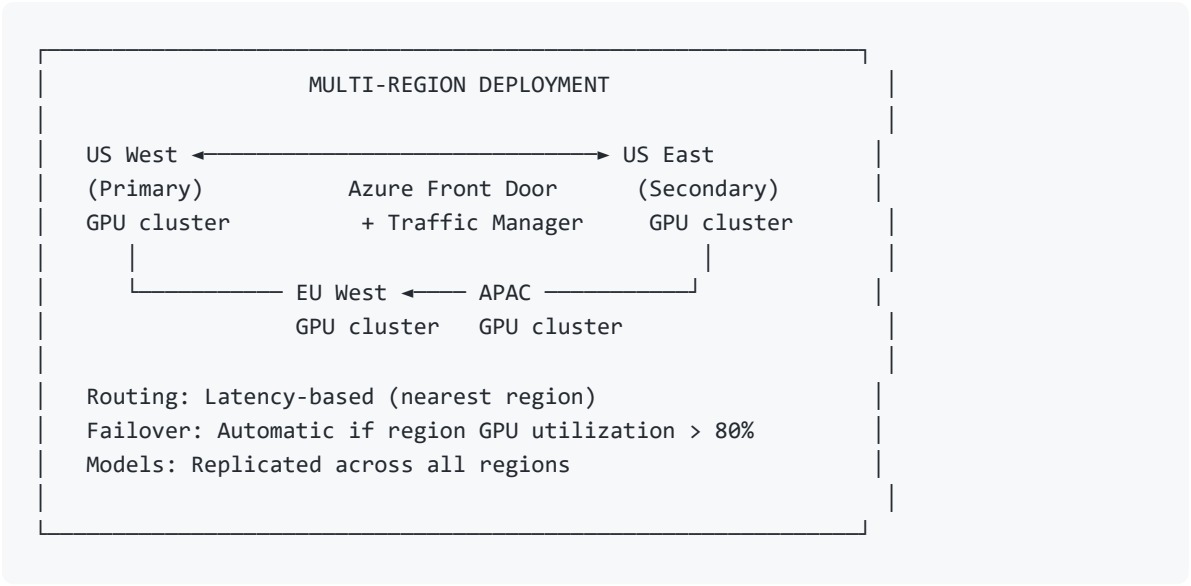
Scalability Considerations

1. GPU Cost Optimization

```
Problem: GPUs are the #1 cost driver (~$2-3 per GPU-hour for A100)

Solutions:
├── Request cancellation: 70-80% of requests never reach inference
├── Model tiering: Use small models for simple completions
├── Continuous batching: Pack multiple requests per GPU cycle
├── KV cache sharing: Common system prompts cached across requests
├── Quantization: INT8/INT4 for fast-tier models (2-4x throughput)
├── Speculative decoding: Small draft model + large verify (2x faster)
└── Auto-scaling: Scale GPU nodes based on queue depth
```

2. Global Distribution



3. Rate Limiting Strategy

```
Tier 1: Per-user token bucket
  ├── Individual: 100 requests/min
  ├── Business: 150 requests/min
  └── Enterprise: Custom limits

Tier 2: Per-organization aggregate
  └── Prevent one org from starving others

Tier 3: Global circuit breaker
  └── Shed load during GPU saturation (graceful degradation)

Graceful degradation order:
  1. Reduce max_tokens (shorter suggestions)
  2. Route to faster/smaller model
  3. Increase debounce delay on client
  4. Return empty suggestions (last resort)
```

4. Caching

```
Layer 1: Client-side (IDE extension)
  └── Cache recent completions, re-show if cursor returns to same position

Layer 2: Edge cache (API Gateway)
  └── Exact prompt match → return cached result (rare but free)

Layer 3: Semantic cache
  └── Similar prompts (cosine similarity > 0.95) → return cached result
  └── Reduces GPU load by ~5-10%

Layer 4: KV cache (GPU memory)
```

```
    └── Reuse computed attention for common prompt prefixes
    └── System prompt + common imports = pre-computed
```

Trade-offs

Decision	Option A	Option B	Copilot's Choice
Model hosting	Self-hosted GPUs	OpenAI API	**Hybrid** — use OpenAI for large models, self-hosted for fast models
Suggestion trigger	Every keystroke	After typing pause (debounce)	**Debounce** — dramatically reduces cost and noise
Context strategy	Current file only	Multi-file + RAG	**Multi-file + RAG** — better suggestions, higher accept rate
Streaming	Wait for full response	Stream tokens as generated	**Stream for chat**, wait for inline (ghost text needs full line)
Model selection	One model for all	Route by task complexity	**Route by task** — fast model for completions, large for chat/agents
Privacy	Send all code to cloud	Process locally	**Cloud with privacy controls** — users can exclude files, orgs control data retention
License filter	Always filter	Optional	**Optional per-org** — Enterprise can enable/disable duplicate detection
Offline support	Full offline mode	Cloud-only	**Cloud-only** with graceful degradation — local models emerging

---

Technologies Deep Dive

Model Serving Stack

Technology	Purpose	Why

vLLM	Inference engine	PagedAttention for efficient KV cache, continuous batching
TensorRT-LLM	NVIDIA optimization	Kernel fusion, quantization, 2-4x throughput vs vanilla
Triton Inference Server	Model serving	Multi-model management, metrics, health checks
CUDA	GPU compute	Direct GPU programming for custom kernels
ONNX Runtime	Cross-platform inference	CPU fallback, edge deployment

Infrastructure

Technology	Purpose	Why
Azure Kubernetes Service	Container orchestration	GPU node pools, auto-scaling, spot instances
Azure Front Door	Global load balancing	Latency-based routing, DDoS protection
Kafka / Event Hubs	Event streaming	Telemetry pipeline, async processing
Redis	Caching & rate limiting	Session state, token buckets, semantic cache
Cosmos DB	Chat history	Global distribution, low-latency reads
Azure Blob Storage	Code index	License filter bloom filters, RAG embeddings

Client-Side

Technology	Purpose	Why
VS Code Extension API	IDE integration	InlineCompletionProvider, Chat API
Language Server Protocol	Code intelligence	Symbol resolution, diagnostics, type info
Tree-sitter	Fast parsing	Extract syntax tree for context collection

AbortController	Request cancellation	Cancel in-flight requests on new keystrokes

---

How to Explain It Well

30-Second Pitch

> "Copilot is an AI pair programmer that provides real-time code suggestions. The IDE extension collects context — the code before and after the cursor plus relevant open files — and sends it to a GPU inference cluster using Fill-in-the-Middle encoding. We use a multi-model strategy: fast small models for inline completions under 300ms, and larger models for chat and agent tasks. The key challenges are latency optimization through debounce + cancellation, GPU cost management through request filtering and batching, and safety through license detection and secret scanning."

Key Talking Points

1. **Context is king**: The quality of suggestions depends on how well we assemble context (FIM, neighbor files, RAG)
2. **Cancellation saves millions**: 70-80% of requests are cancelled before inference — this is the biggest cost lever
3. **Multi-model routing**: Not every request needs GPT-4; route simple completions to fast small models
4. **Acceptance rate**: The north-star metric — measures both model quality and context quality
5. **Safety pipeline**: License filter + secret detection + responsible AI checks on every suggestion

Whiteboard Sequence

1. Start with IDE → API Gateway → Inference → Response flow
2. Zoom into context collection (FIM encoding, prefix/suffix)
3. Show the GPU cluster with batching and KV cache
4. Add post-processing (license filter, secret detection)
5. Overlay telemetry feedback loop (acceptance rate drives model improvements)

Common Follow-up Questions

"How do you handle latency for inline suggestions?"

- **Debounce** (150ms) avoids sending requests on every keystroke
- **Cancellation** aborts in-flight requests when user keeps typing
- **Fast models** (1-7B params) for inline, quantized to INT8
- **Speculative decoding** uses a tiny draft model verified by the main model
- **Edge routing** to nearest GPU cluster (< 30ms network RTT)

"How do you prevent suggesting copyrighted code?"

- Normalize and fingerprint model output using n-gram hashing
- Compare against a Bloom filter index of public code (~hundreds of millions of files)
- If match found, check license: permissive (allow + cite) vs copyleft (block)
- Enterprise customers can enable/disable this filter
- Also block verbatim matches > N lines regardless of license

"How do you manage GPU costs at scale?"

- **Cancellation** is the biggest lever (70-80% savings)
- **Model tiering**: Cheap small models handle most inline requests
- **Continuous batching**: Pack multiple requests per GPU forward pass
- **Quantization**: INT8/INT4 models use less VRAM, higher throughput
- **Spot/preemptible instances** for batch workloads (fine-tuning, indexing)
- **Auto-scale** GPU node pools based on queue depth, not CPU

"How does the agent mode work (multi-file edits)?"

- Agent receives a high-level task ("add authentication to this app")
- Uses a **ReAct loop**: Reason → Act (call tools) → Observe → Repeat
- Tools include: file read/write, terminal commands, search, web browse
- Each iteration sends updated context to a large model (GPT-4 class)
- User approves/rejects tool calls for safety
- Streams progress via SSE; final result = set of file diffs

"How do you handle multi-tenant isolation?"

- **Network isolation**: Each request authenticated via GitHub token → org membership verified
- **Data isolation**: No cross-tenant data leakage; code context only from user's own repos
- **Model isolation**: Shared model weights (no per-tenant fine-tuning on shared infra)
- **Telemetry isolation**: Org admins see only their org's usage metrics
- **Custom policies**: Orgs can block file patterns, set content exclusions, control data retention

"How would you add support for a new IDE?"

- Implement the IDE's extension/plugin API (IntelliJ Plugin SDK, Neovim Lua API, etc.)
- Reuse the same backend APIs (completions, chat, agents)
- Adapt context collection to each IDE's document model
- IDE-specific: ghost text rendering, inline diff UI, chat panel
- Use Language Server Protocol (LSP) for cross-IDE code intelligence

---

50. Design Azure AI Foundry

Problem Statement

Design an enterprise-grade AI development platform — like Azure AI Foundry (formerly Azure AI Studio) — that enables teams to discover, customize, evaluate, deploy, and govern AI models and intelligent agents at scale. The platform must provide a unified experience across a massive model catalog, prompt engineering, fine-tuning, RAG pipelines, agent orchestration, and fleet-wide governance — all within a multi-tenant cloud environment.

Clarifying Questions to Ask

1. **Scope**: Full MLOps lifecycle or just model serving and agent hosting?
2. **Multi-model**: How many model providers (OpenAI, Meta, Mistral, Hugging Face, custom)?
3. **Multi-tenant**: Per-project isolation or shared compute pools?
4. **Governance**: What compliance frameworks (GDPR, HIPAA, SOC 2, FedRAMP)?
5. **Deployment targets**: Cloud-only or also edge and on-premises?
6. **Agent complexity**: Single-agent tool-use or multi-agent orchestration?
7. **Scale**: How many concurrent model deployments and inference requests per tenant?
8. **Customization**: Fine-tuning on managed compute or bring-your-own training cluster?

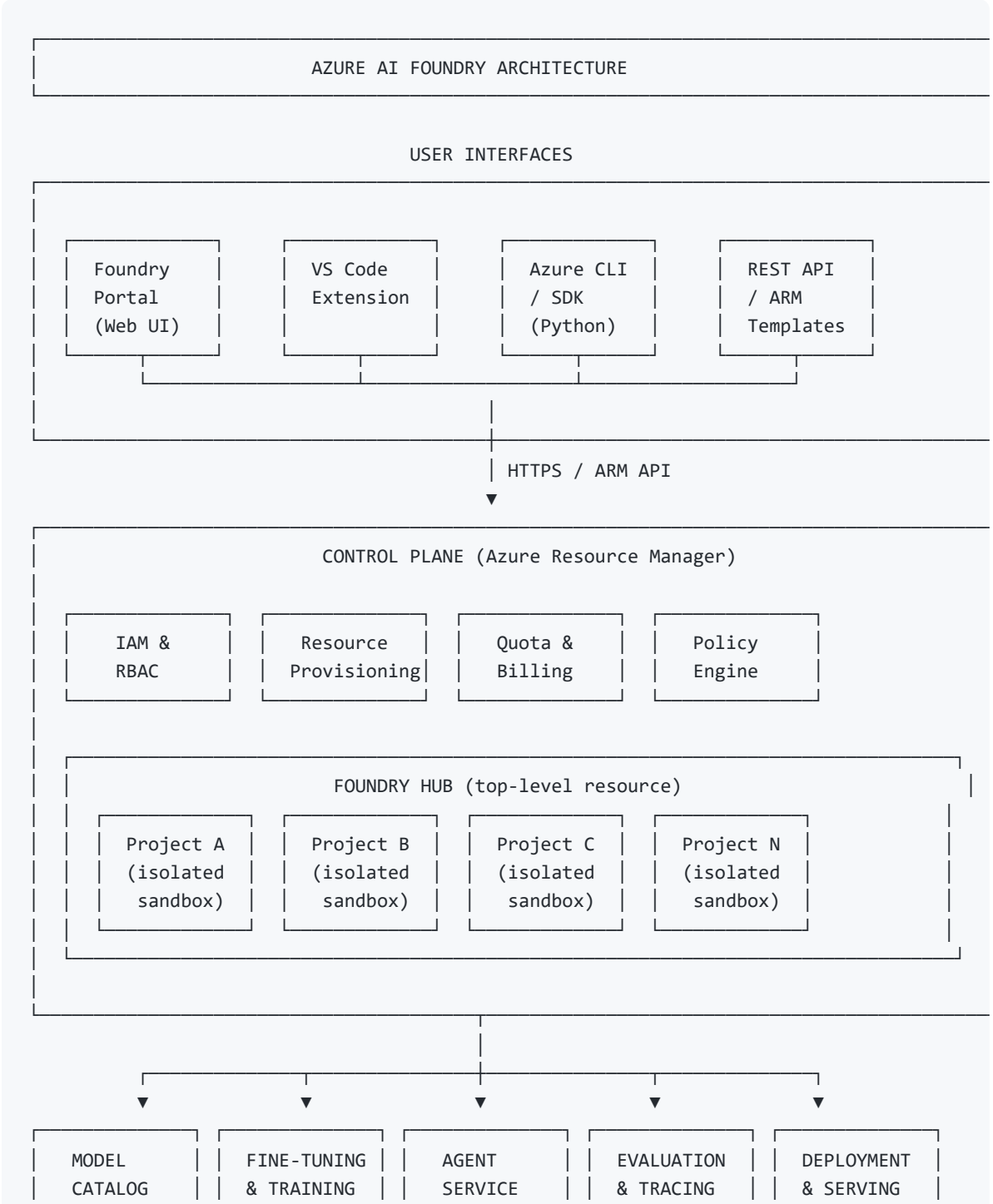
Design Reasoning

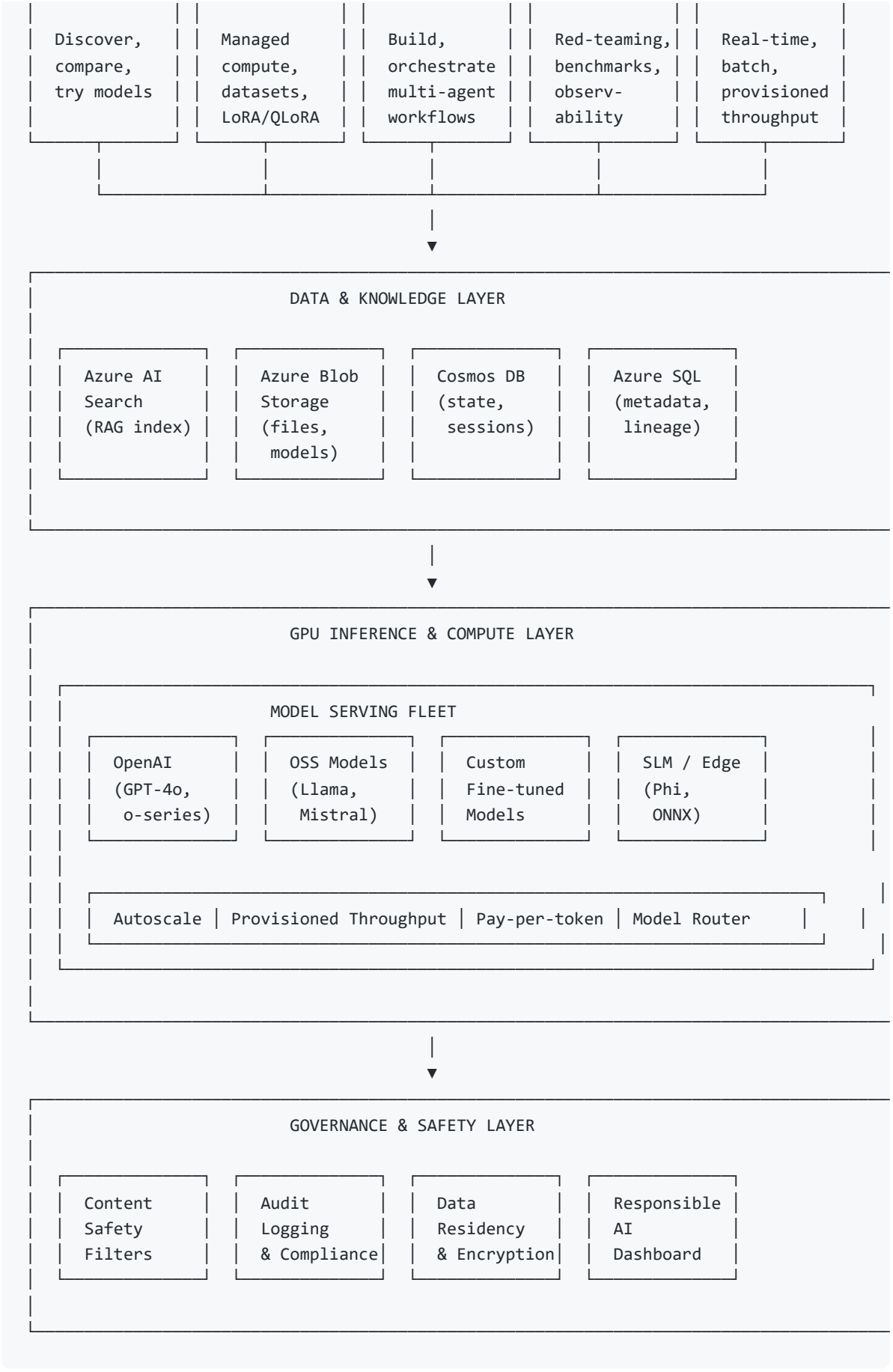
> **Core philosophy**: A unified, compliance-first MLOps platform that balances flexibility and choice (multi-provider models, hybrid deployment modes) with managed simplicity (platform-provisioned compute, built-in governance), enabling organizations to govern and scale AI responsibly while remaining vendor-agnostic.

Decision	Choice	Why	Alternative Considered
Model catalog	Multi-provider (OpenAI, Meta, Mistral, Cohere, Hugging Face, xAI, custom)	Maximizes choice, enables cost optimization through model routing, reduces vendor lock-in	Single provider (OpenAI only) — simpler but limits flexibility, creates vendor lock-in
Deployment model	Hybrid: MaaS (pay-per-token, shared) + Managed Compute (dedicated, autoscaled)	Cost-sensitive projects use shared MaaS; performance-critical use dedicated compute with guarantees	Pure multi-tenant shared — limits performance predictability; pure dedicated — cost-prohibitive
Agent state	Server-side threads in Cosmos DB with TTL cleanup	Simplifies client implementation, reliable persistence, supports multi-turn agent conversations	Stateless client-managed — shifts complexity to users, risks state loss between turns
Fine-tuning compute	Platform-managed GPU with auto-provisioning (LoRA/QLoRA)	Removes friction — no GPU procurement needed; enables reproducibility and audit trails	Customer-managed GPU — requires maintaining clusters, dramatically increases operational burden
RAG / Search	Azure AI Search with hybrid search (vector + BM25) + semantic ranking	Tight platform integration; hybrid search balances recall and precision across structured and unstructured data	Bring-your-own search — adds operational complexity; built-in approach is simpler for most users
Content safety	Always-on by default with customizable severity thresholds	Compliance-first design (GDPR, HIPAA, SOC 2, FedRAMP); protects organizations from harmful content risks	Optional per-deployment — puts risk on customers; compliance teams would mandate it anyway

Model routing	Intelligent router: prompt complexity → model size (simple→Phi-4, reasoning→GPT-4o, complex→o-series)	40-60% cost savings and latency improvement by right-sizing model to task	Static routing per deployment — simpler but misses significant optimization opportunities

High-Level Design




Discover, compare, try models
Managed compute, datasets, LoRA/QLoRA
Build, orchestrate multi-agent workflows
Red-teaming, benchmarks, observ-ability
Real-time, batch, provisioned throughput
DATA & KNOWLEDGE LAYER
Azure AI Search (RAG index)
Azure Blob Storage (files, models)
Cosmos DB (state, sessions)
Azure SQL (metadata, lineage)
GPU INFERENCE & COMPUTE LAYER
MODEL SERVING FLEET
OpenAI (GPT-4o, o-series)
OSS Models (Llama, Mistral)
Custom Fine-tuned Models
SLM / Edge (Phi, ONNX)
Autoscale | Provisioned Throughput | Pay-per-token | Model Router
GOVERNANCE & SAFETY LAYER
Content Safety Filters
Audit Logging & Compliance
Data Residency & Encryption
Responsible AI Dashboard

API Design

Project & Resource Management APIs

Method	Endpoint
`PUT`	`/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.AIFoundry/hubs/`
`PUT`	`.../hubs/{hub}/projects/{project}`
`GET`	`.../hubs/{hub}/projects`
`DELETE`	`.../hubs/{hub}/projects/{project}`

Model Catalog APIs

Method	Endpoint	Description
`GET`	`/v1/catalog/models`	List available models (filter by task, provider, license)
`GET`	`/v1/catalog/models/{modelId}`	Get model details (benchmarks, pricing, capabilities)
`POST`	`/v1/catalog/models/{modelId}/try`	Interactive playground — try a model with sample prompts
`POST`	`/v1/deployments`	Deploy a model (real-time, batch, or provisioned)

Inference APIs (Unified)

Method	Endpoint	Description
`POST`	`/v1/chat/completions`	Chat completion (OpenAI-compatible)
`POST`	`/v1/completions`	Text completion
`POST`	`/v1/embeddings`	Generate embeddings

`POST`	`/v1/images/generations`	Image generation
`POST`	`/v1/audio/transcriptions`	Speech-to-text

Agent APIs

Method	Endpoint	Description
`POST`	`/v1/agents`	Create an agent (with tools, instructions, model)
`POST`	`/v1/agents/{agentId}/threads`	Create a conversation thread
`POST`	`/v1/agents/{agentId}/threads/{threadId}/runs`	Run agent on a thread
`GET`	`/v1/agents/{agentId}/threads/{threadId}/runs/{runId}`	Get run status and output

Fine-Tuning APIs

Method	Endpoint	Description
`POST`	`/v1/fine-tuning/jobs`	Start a fine-tuning job
`GET`	`/v1/fine-tuning/jobs/{jobId}`	Get job status
`POST`	`/v1/fine-tuning/jobs/{jobId}/cancel`	Cancel a running job
`GET`	`/v1/fine-tuning/jobs/{jobId}/checkpoints`	List training checkpoints

Evaluation APIs

Method	Endpoint	Description
`POST`	`/v1/evaluations`	Run evaluation (groundedness, relevance, coherence)
`GET`	`/v1/evaluations/{evalId}/results`	Get evaluation results and metrics
`POST`	`/v1/evaluations/red-team`	Run automated red-team attack simulation

Model Deployment Request

```
POST /v1/deployments
{
  "name": "gpt-4o-production",
  "model": {
    "source": "catalog",
    "id": "openai/gpt-4o",
    "version": "2025-08-06"
  },
  "sku": {
    "name": "ProvisionedManaged",
    "capacity": 100
  },
  "scaling": {
    "type": "autoscale",
    "min_replicas": 2,
    "max_replicas": 10,
    "target_utilization": 0.7
  },
  "content_filter": {
    "policy": "default-v2",
    "custom_blocklists": ["company-terms"]
  },
  "networking": {
    "type": "private_endpoint",
    "vnet_id": "/subscriptions/.../virtualNetworks/ai-vnet"
  },
  "region": "eastus2"
}
```

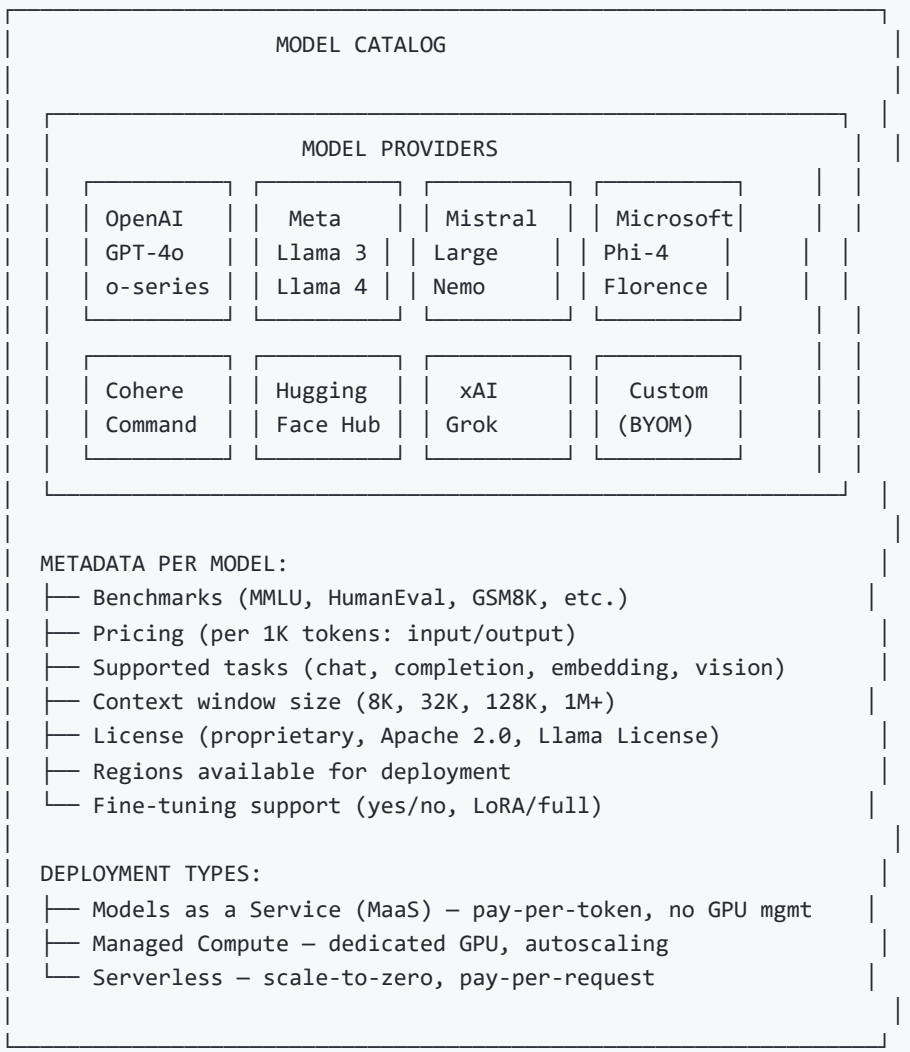
Agent Creation Request

```
POST /v1/agents
{
  "name": "customer-support-agent",
  "model": "gpt-4o-production",
  "instructions": "You are a helpful customer support agent for Contoso...",
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "search_knowledge_base",
        "description": "Search internal support articles",
        "parameters": { "type": "object", "properties": { "query": { "type": "string
      }
    },
    {
      "type": "azure_ai_search",
      "config": {
```

```
          "index_name": "support-articles",
          "endpoint": "https://contoso-search.search.windows.net"
        }
      },
      { "type": "code_interpreter" }
    ],
    "temperature": 0.3,
    "max_tokens": 4096
  }
```

Model Catalog & Routing

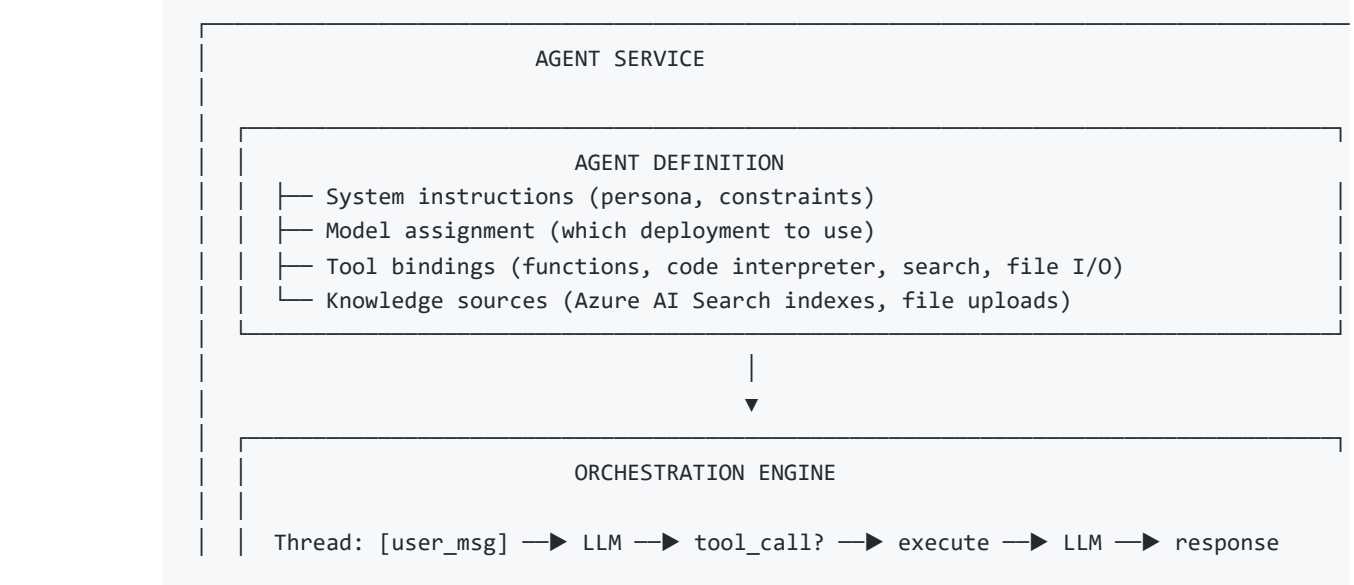
Model Catalog Architecture

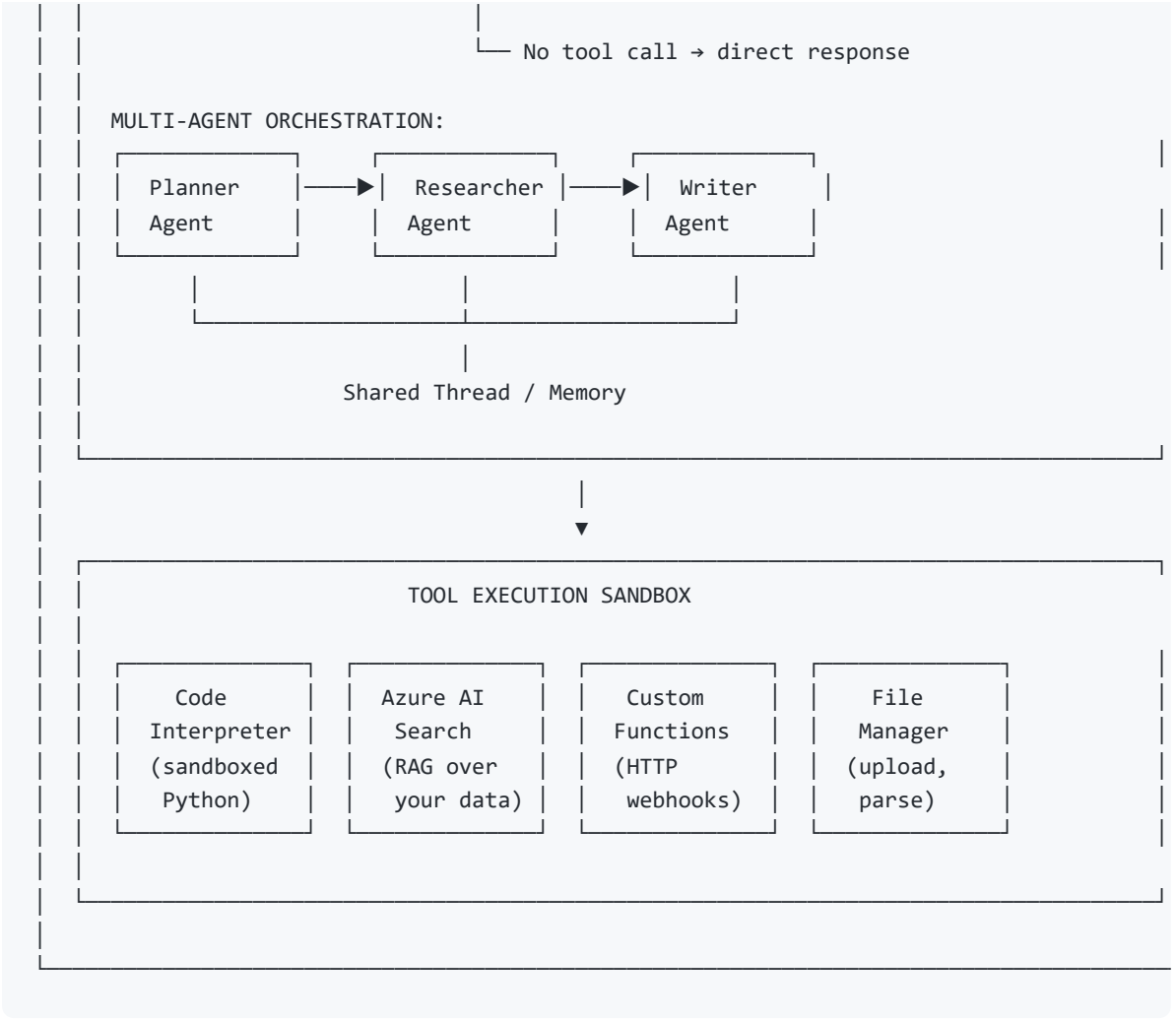


Intelligent Model Router



Agent Service Architecture



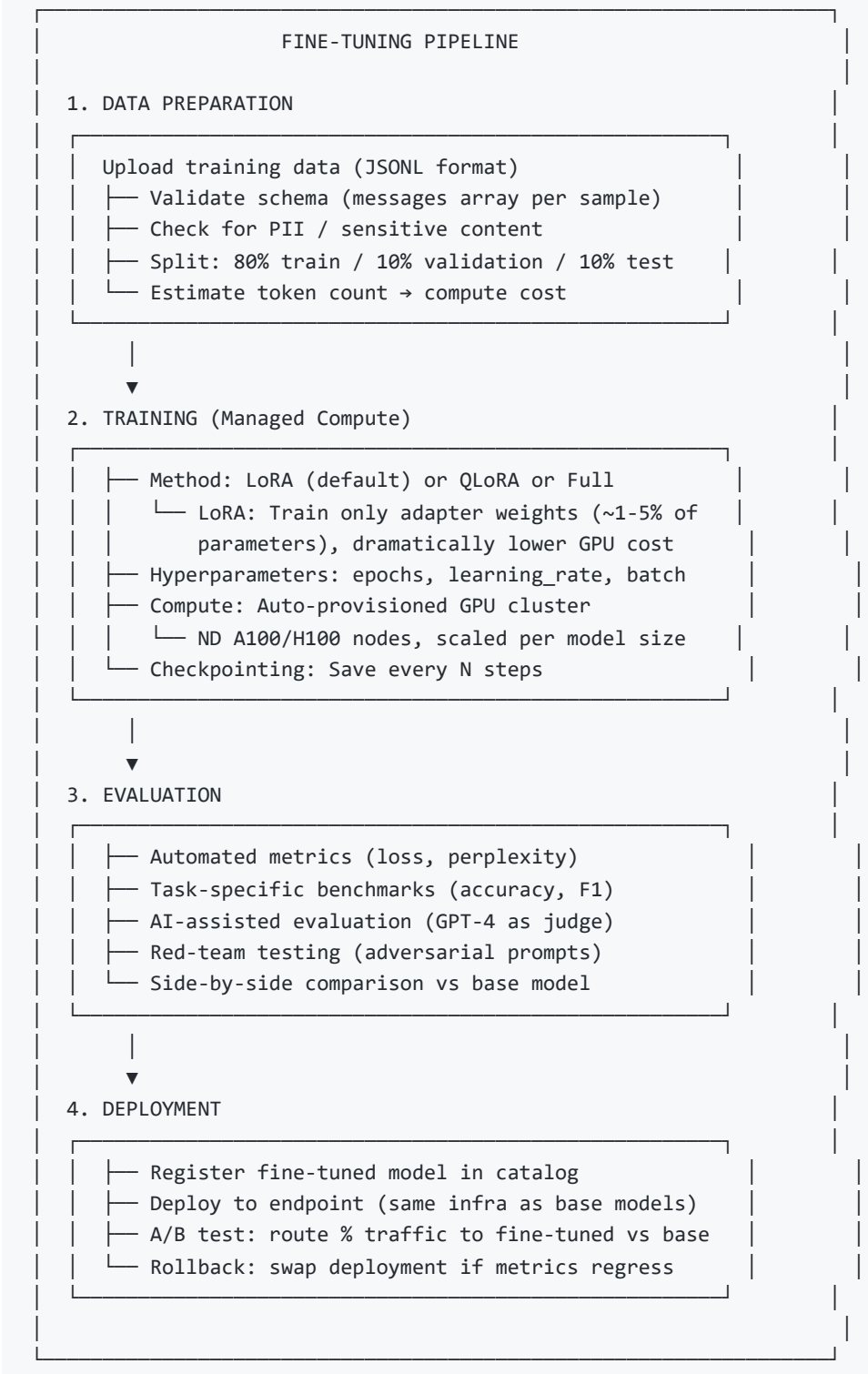


Agent Thread State Management

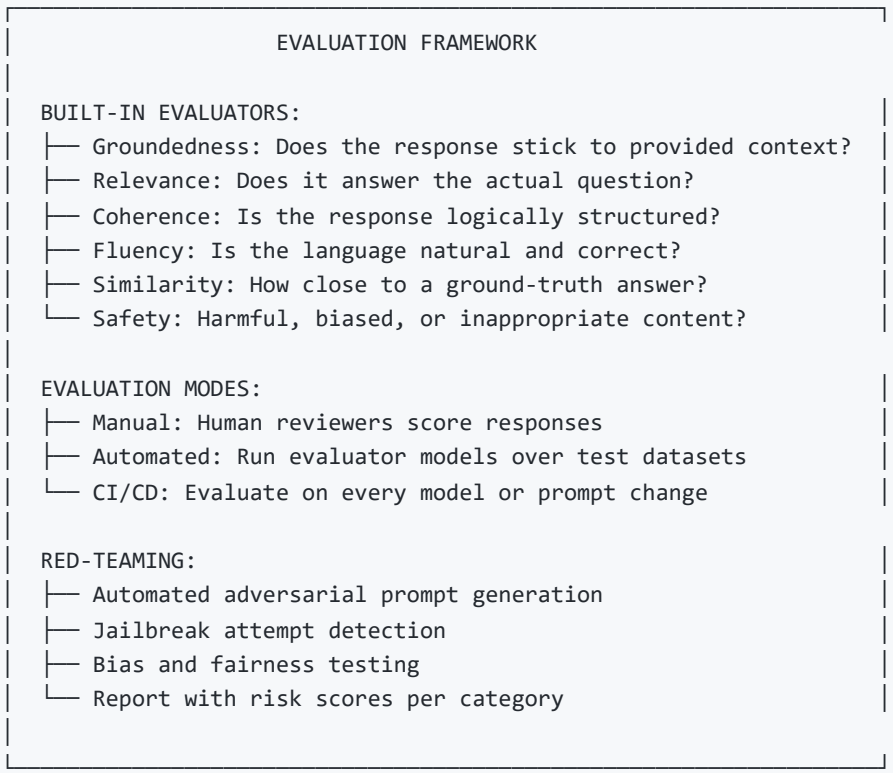
```
Thread (Cosmos DB document):
{
  "thread_id": "thread_abc123",
  "agent_id": "agent_support",
  "messages": [
    { "role": "user", "content": "What's my order status?" },
    { "role": "assistant", "content": null,
      "tool_calls": [{ "function": "lookup_order", "args": {"email": "..."} }] },
    { "role": "tool", "content": "Order #12345: Shipped, ETA Feb 25" },
    { "role": "assistant", "content": "Your order #12345 has shipped..." }
  ],
  "metadata": { "customer_id": "cust_789", "created_at": "..." },
  "ttl": 86400
}

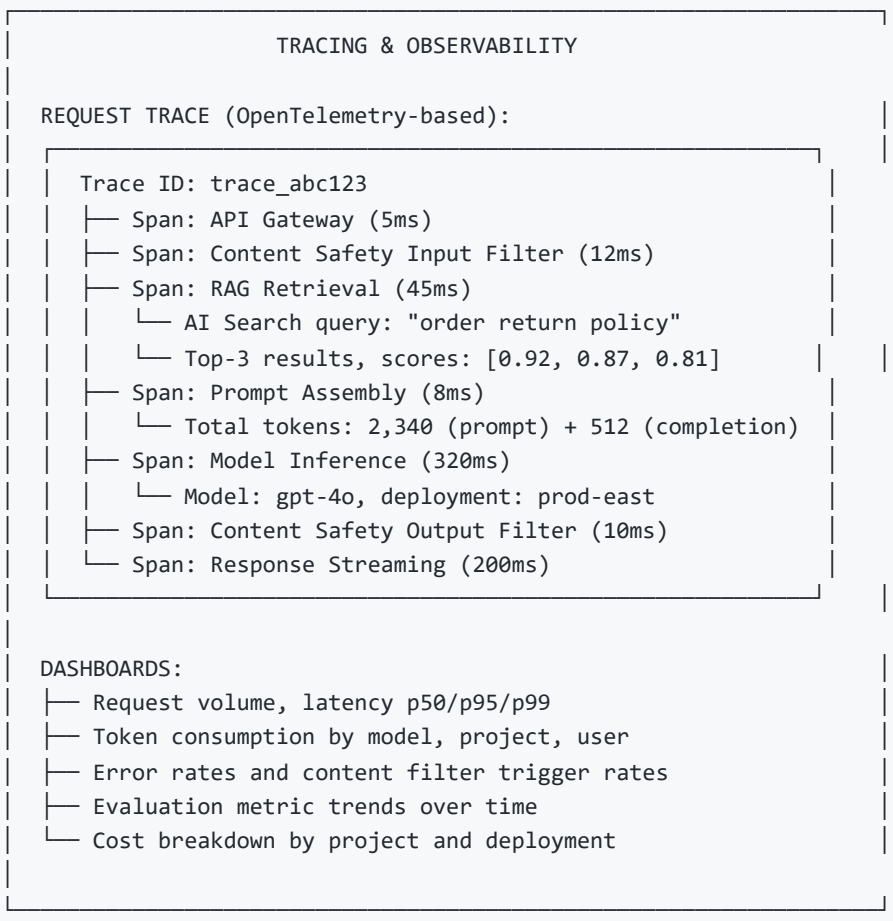
State management strategy:
├── Short-term: Full message history in thread (Cosmos DB)
├── Medium-term: Summarize older messages to stay within context window
└── Long-term: Archive to blob storage, index key facts in AI Search
```

Fine-Tuning Pipeline

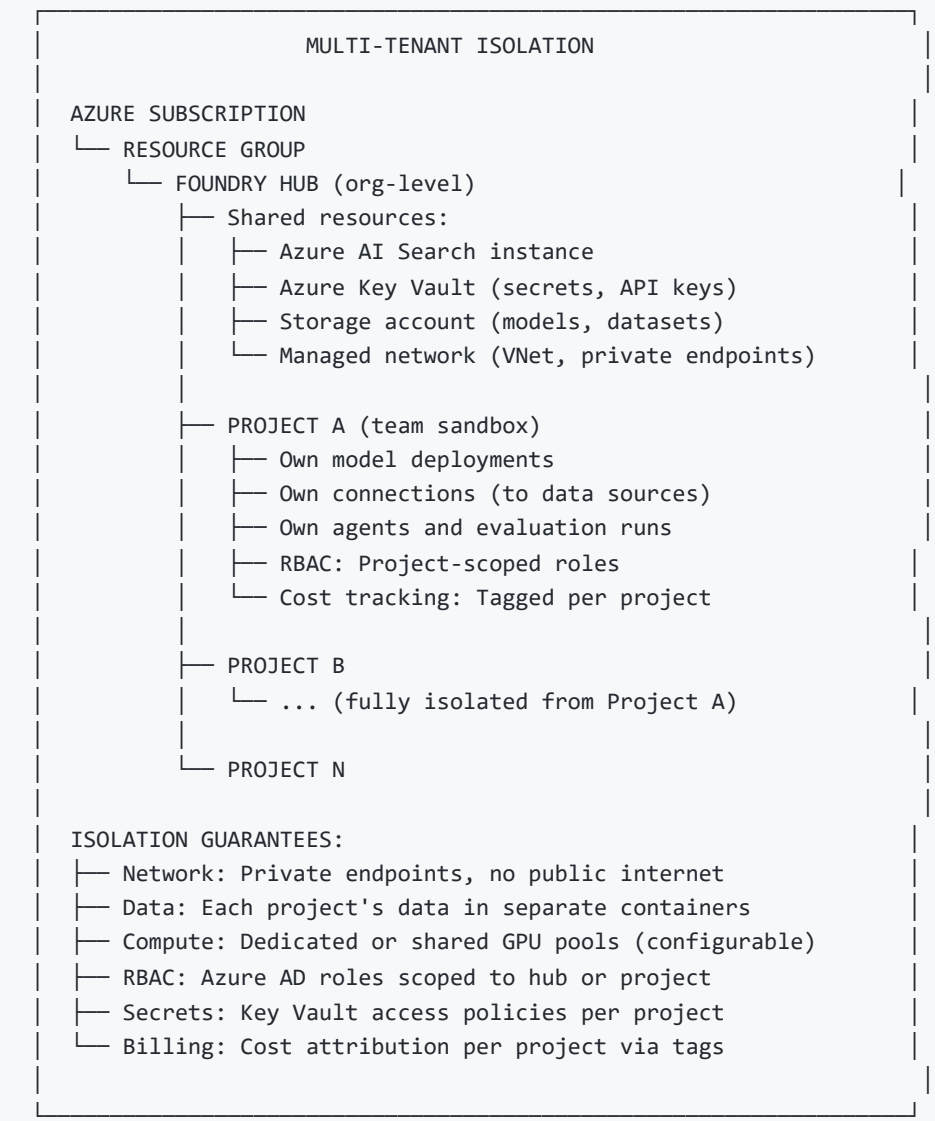


Evaluation & Observability


EVALUATION FRAMEWORK
BUILT-IN EVALUATORS:
├── Groundedness: Does the response stick to provided context?
├── Relevance: Does it answer the actual question?
├── Coherence: Is the response logically structured?
├── Fluency: Is the language natural and correct?
├── Similarity: How close to a ground-truth answer?
└── Safety: Harmful, biased, or inappropriate content?
EVALUATION MODES:
├── Manual: Human reviewers score responses
├── Automated: Run evaluator models over test datasets
└── CI/CD: Evaluate on every model or prompt change
RED-TEAMING:
├── Automated adversarial prompt generation
├── Jailbreak attempt detection
├── Bias and fairness testing
└── Report with risk scores per category


TRACING & OBSERVABILITY
REQUEST TRACE (OpenTelemetry-based):
Trace ID: trace_abc123
├── Span: API Gateway (5ms)
├── Span: Content Safety Input Filter (12ms)
├── Span: RAG Retrieval (45ms)
│ └── AI Search query: "order return policy"
│ └── Top-3 results, scores: [0.92, 0.87, 0.81]
├── Span: Prompt Assembly (8ms)
│ └── Total tokens: 2,340 (prompt) + 512 (completion)
├── Span: Model Inference (320ms)
│ └── Model: gpt-4o, deployment: prod-east
├── Span: Content Safety Output Filter (10ms)
└── Span: Response Streaming (200ms)
DASHBOARDS:
├── Request volume, latency p50/p95/p99
├── Token consumption by model, project, user
├── Error rates and content filter trigger rates
├── Evaluation metric trends over time
└── Cost breakdown by project and deployment

Hub & Project Isolation Model



Key Components

Component	Technology	Purpose
Control Plane	Azure Resource Manager (ARM)	Hub/project provisioning, RBAC, policy enforcement
Model Catalog	Azure SQL + Blob Storage	Model registry, metadata, benchmarks, versioning
Model Serving	Azure ML Managed Endpoints / MaaS	Real-time inference with autoscaling

Model Router	Custom routing service	Intelligent model selection per request
Agent Service	Cosmos DB + AKS	Agent orchestration, thread management, tool execution
Code Interpreter	Hyper-V sandboxed containers	Secure Python execution for agents
RAG / Search	Azure AI Search	Vector + keyword hybrid search over user data
Fine-Tuning	Azure ML Compute (GPU VMs)	LoRA/QLoRA training with managed infra
Evaluation	Azure ML Pipelines	Automated quality, safety, and red-team evaluation
Tracing	Azure Monitor + OpenTelemetry	End-to-end request tracing, latency, token usage
Content Safety	Azure AI Content Safety	Input/output filtering for harmful content
Key Management	Azure Key Vault	API keys, connection strings, certificates
Networking	Azure VNet + Private Link	Network isolation, no data exfiltration
CI/CD	GitHub Actions + Azure DevOps	Prompt/model deployment pipelines

---

Scalability Considerations

1. Inference Scaling

```
Challenge: Bursty, unpredictable inference traffic across thousands of tenants

Solutions:
├── Provisioned Throughput Units (PTU)
│   └── Pre-purchase fixed capacity for predictable workloads
│       (e.g., 100 PTU = ~600 requests/min for GPT-4o)
├── Pay-per-token (serverless)
│   └── Scale to zero, pay only for usage (higher per-token cost)
├── Autoscaling managed endpoints
│   └── Scale replicas 1→N based on request queue depth
├── Model Router
│   └── Shift overflow traffic to cheaper/smaller models
```

```
└── Global deployment
    └── Deploy same model across regions, route to nearest
```

2. Multi-Region Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    GLOBAL DEPLOYMENT                         │
│                                                              │
│   US East ◄──────────────────────────────► US West           │
│   Hub + GPU cluster     Azure Front Door    Hub + GPU cluster│
│      │                                          │            │
│      └──────────── EU West ◄──── APAC ──────────┘            │
│                   Hub + GPU          Hub + GPU               │
│                                                              │
│   Data residency: Models and data stay in declared region    │
│   Failover: Cross-region for availability, not data migration│
│   Catalog: Global read replicas, regional write primary      │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

3. Cost Management

```
Per-project cost tracking:
├── Token consumption metered per deployment per project
├── Fine-tuning compute hours billed per job
├── Storage (datasets, model artifacts) billed per GB
├── Search index queries billed per transaction
└── Quota system prevents runaway spend

Cost optimization levers:
├── Model Router: Auto-select cheapest model that meets quality bar
├── Caching: Semantic cache for repeated queries (~10-15% savings)
├── Batch API: 50% discount for non-real-time workloads
├── Spot/low-priority compute for fine-tuning jobs
└── Scale-to-zero serverless for dev/test deployments
```

Trade-offs

Decision	Option A	Option B	Foundry's Choice
Model hosting	Single-provider (OpenAI only)	Multi-provider catalog	**Multi-provider** — choice, flexibility, cost optimization

Deployment model	Shared multi-tenant GPUs	Dedicated per-customer	**Both** — MaaS (shared) and Managed Compute (dedicated)
Agent state	Stateless (client manages)	Server-side threads	**Server-side threads** — simpler client, reliable state
Fine-tuning compute	Customer-managed GPU	Platform-managed	**Platform-managed** — lower friction, auto-provisioned
RAG approach	Built-in vector DB	Bring-your-own search	**Azure AI Search** — integrated, but supports external connectors
Content safety	Optional per deployment	Always-on	**Always-on by default** — can customize severity thresholds
Evaluation	Manual only	Automated + manual	**Automated + manual** — CI/CD integrated evaluation pipelines
Pricing	Flat subscription	Pay-per-token	**Pay-per-token + PTU** — flexibility for different workloads

---

Technologies Deep Dive

AI & Model Serving

Technology	Purpose	Why
Azure OpenAI Service	Hosted OpenAI models	Enterprise SLA, content safety, private networking
Azure ML Managed Endpoints	OSS model serving	vLLM-backed, autoscaling, GPU node pools
ONNX Runtime	Optimized inference	Cross-platform, quantization, edge deployment
Triton Inference Server	Multi-framework serving	Batching, model ensembles, GPU scheduling
LoRA / QLoRA	Parameter-efficient fine-tuning	Fine-tune large models on single GPU node

Platform Infrastructure

Technology	Purpose	Why
Azure Resource Manager	Control plane	Consistent API, RBAC, policy, ARM templates
Azure Kubernetes Service	Agent + API hosting	Scalable microservices, GPU node pools
Cosmos DB	Agent threads, session state	Global distribution, low-latency, TTL
Azure AI Search	RAG retrieval	Hybrid search (vector + BM25), semantic ranker
Azure Key Vault	Secrets management	API keys, certificates, managed identities
Azure Monitor + App Insights	Observability	Distributed tracing, metrics, alerts
Azure Event Hubs	Telemetry streaming	Usage events, audit logs, billing events

Developer Tooling

Technology	Purpose	Why
Foundry SDK (Python)	Programmatic access	Unified client for all Foundry APIs
Prompt Flow	Prompt orchestration	Visual DAG for chaining prompts, tools, evaluators
VS Code Extension	IDE integration	Inline prompt testing, deployment, debugging
GitHub Actions	CI/CD for AI apps	Automated evaluation + deployment pipelines

---

How to Explain It Well

30-Second Pitch

> "Azure AI Foundry is a unified platform for building AI applications and agents at enterprise scale. It provides a model catalog with thousands of models from OpenAI, Meta, Mistral, and others — deployed as managed endpoints with autoscaling. On top of model serving, it offers agent orchestration with tool-calling, RAG via Azure AI Search, fine-tuning with LoRA, automated evaluation including red-teaming, and fleet-wide governance. The architecture is built on a Hub-and-Project isolation model where each project gets its own sandbox with scoped RBAC, networking, and cost tracking — all managed through Azure Resource Manager."

Key Talking Points

1. **Model Catalog + Router**: Not just hosting one model — curating and intelligently routing across hundreds of models for cost and quality
2. **Hub/Project isolation**: Enterprise multi-tenancy with project-level sandboxing (RBAC, network, data, billing)
3. **Agent orchestration**: Server-side thread management with tool execution in sandboxed environments
4. **Evaluation-as-a-service**: Automated quality + safety metrics integrated into CI/CD, not an afterthought
5. **Responsible AI by default**: Content safety filters always-on, red-team testing built in, audit logging for compliance

Whiteboard Sequence

1. Start with Users → Control Plane → Hub/Projects hierarchy
2. Show the Model Catalog and deployment types (MaaS, Managed, Serverless)
3. Draw the Agent Service with tool execution and thread state
4. Add RAG pipeline (data → AI Search → prompt → model → response)
5. Overlay Governance layer (content safety, audit, RBAC)

Common Follow-up Questions

"How do you handle model versioning and rollback?"

- Each model deployment references a specific model version (e.g., `gpt-4o-2025-08-06`)
- **Blue/green deployments**: Deploy new version alongside old, shift traffic gradually
- **Evaluation gates**: Automated eval runs must pass before traffic shift
- **Instant rollback**: Swap deployment pointer back to previous version (< 30s)
- Fine-tuned models are registered with version IDs and linked to training data lineage

"How does the model router decide which model to use?"

- **Prompt classifier**: Lightweight ML model analyzes prompt complexity (simple QA vs. complex reasoning)
- **Cost policy**: Org-level rules like "prefer models under $X per 1K tokens"
- **Quality threshold**: Route to smallest model that meets the quality bar (measured via evaluations)
- **Fallback chain**: If primary model is overloaded or unavailable, cascade to alternatives
- **Custom rules**: Organizations define routing logic (e.g., "use GPT-4o for legal, Phi for internal FAQ")

"How do you ensure data privacy in a multi-tenant platform?"

- **Network isolation**: Private endpoints — model traffic never traverses public internet
- **Data residency**: Models and data stay in the declared Azure region
- **No cross-tenant training**: Customer data is never used to train shared models
- **Encryption**: At-rest (AES-256) and in-transit (TLS 1.3), customer-managed keys supported
- **RBAC + Managed Identity**: No shared API keys — Azure AD authentication throughout
- **Audit logging**: Every API call, model invocation, and data access logged to Azure Monitor

"How do you handle RAG at scale?"

- **Azure AI Search** provides hybrid retrieval (vector embeddings + BM25 keyword search)
- **Semantic ranker** re-ranks results using a cross-encoder model for higher relevance
- **Chunking strategies**: Configurable document chunking (fixed-size, sentence-level, semantic)
- **Index per project**: Each project has isolated search indexes (no cross-project data leakage)
- **Integrated ingestion**: Upload docs → automatic chunking → embedding → indexing pipeline
- **Grounding evaluation**: Automated checks that responses cite retrieved context accurately

"How does fine-tuning work without managing GPUs?"

- User uploads training data (JSONL) and selects base model
- Platform auto-provisions GPU compute (ND A100/H100 VMs) based on model size
- **LoRA** (default): Only trains small adapter layers (~1-5% of parameters) — faster, cheaper
- Training runs on managed AML compute with automatic checkpointing
- After training, fine-tuned model is registered and deployable like any catalog model
- GPU compute is released immediately after job completes — no idle cost

---

51. Design AI-Powered SOC (Security Operations Center)

Problem Statement

Design an AI-driven Security Operations Center platform that ingests massive volumes of security alerts, event logs, and telemetry from across an enterprise — stored in Kusto (Azure Data Explorer) tables and streaming via Event Hubs — and uses LLMs and ML models to autonomously triage, investigate, correlate, and respond to security incidents. The system should reduce analyst workload by 80%+ through automated investigation, natural language querying ("show me all lateral movement in the last 24h"), and AI-generated incident reports.

Clarifying Questions to Ask

1. **Data scale**: How many events/second ingested? (100K-1M+ EPS typical for large enterprise)
2. **Alert volume**: How many raw alerts/day? (tens of thousands, most are noise)
3. **Data sources**: Which log types? (endpoint, network, identity, cloud, email, firewall)
4. **Kusto schema**: Are the tables normalized (ASIM) or vendor-specific schemas?
5. **Response scope**: Recommend actions only, or auto-execute containment (isolate host, disable account)?
6. **Analyst interaction**: Chat-based investigation or automated playbooks or both?
7. **Multi-tenant**: Single enterprise SOC or MSSP serving multiple customers?
8. **Compliance**: Do we need full audit trail of every AI decision for regulatory review?
9. **Existing tooling**: Integrating into Microsoft Sentinel, or building from scratch?
10. **SLA**: What's the target MTTR (Mean Time to Respond) for critical incidents?

Design Reasoning

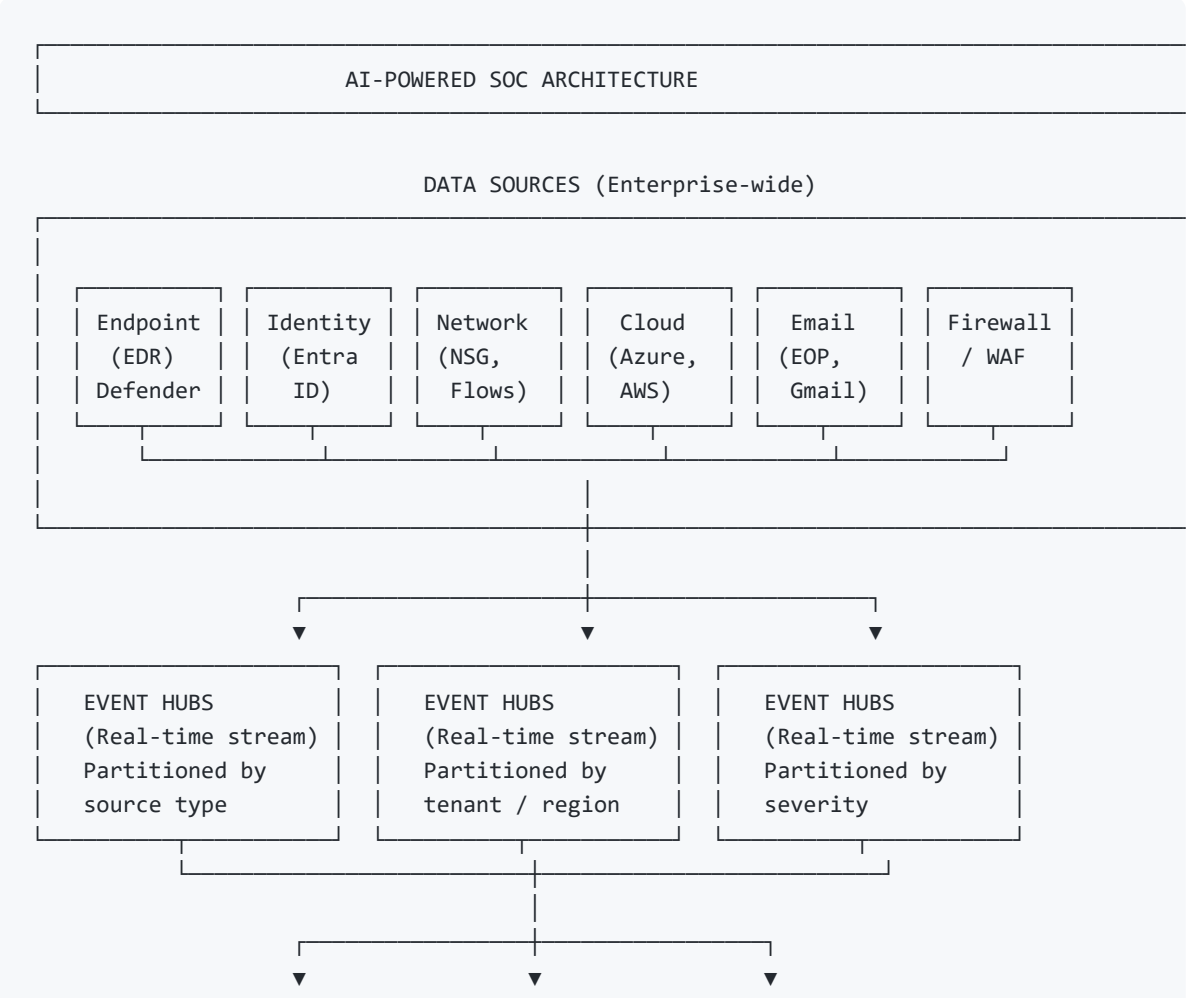
> Core philosophy: Shift analyst effort from noise-filtering to high-value judgment calls — massive alert volumes are automatically triaged via ML, autonomously investigated by an LLM reasoning engine, but human analysts remain the final decision-maker for high-risk actions, with full audit trails ensuring transparency.

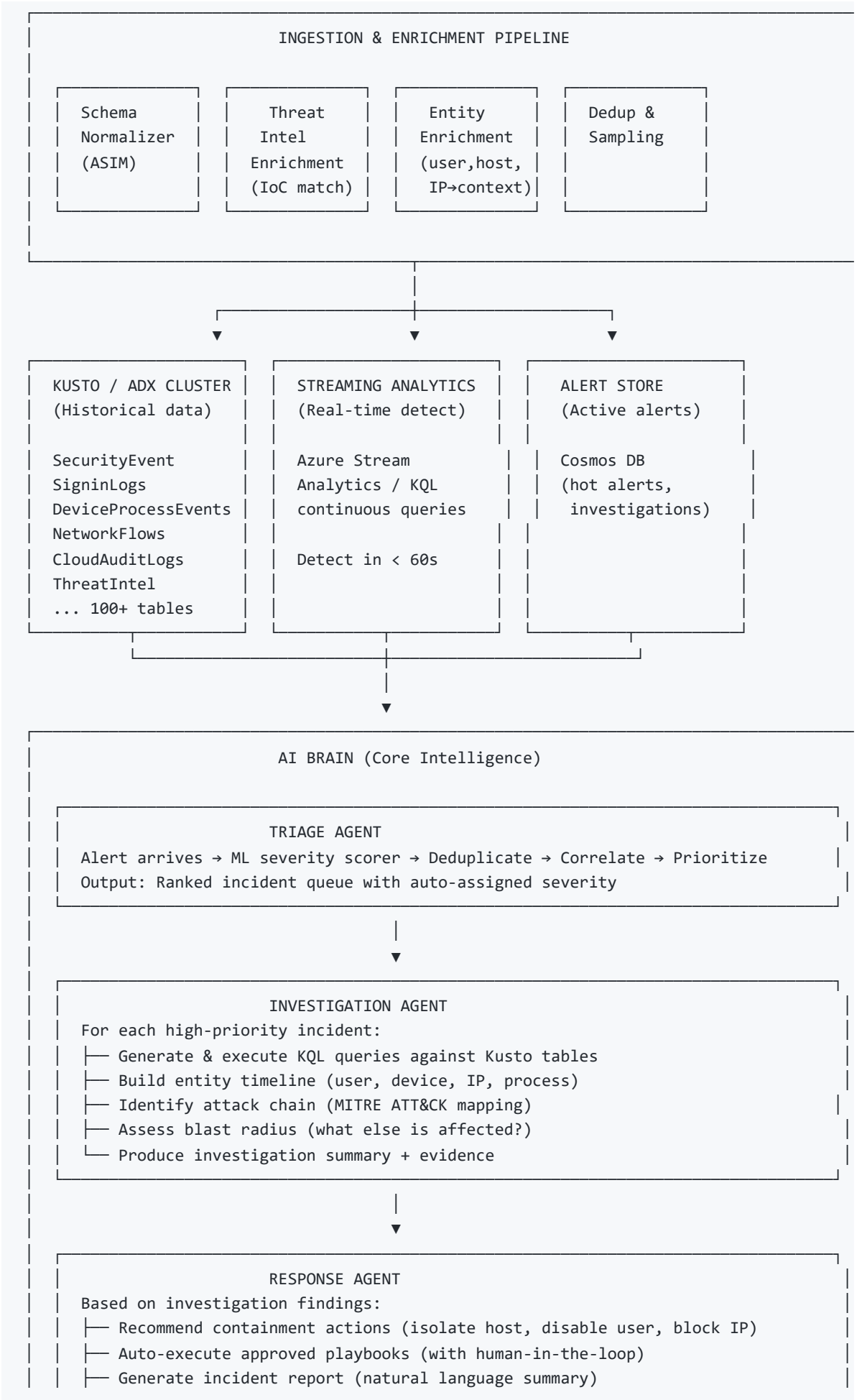
Decision	Choice	Why	Alternative Considered
Event ingestion	Azure Event Hubs (Kafka-compatible,	Handles 500K-1M events/sec; partitioning	Direct DB writes — creates bottlenecks; self-managed Kafka —

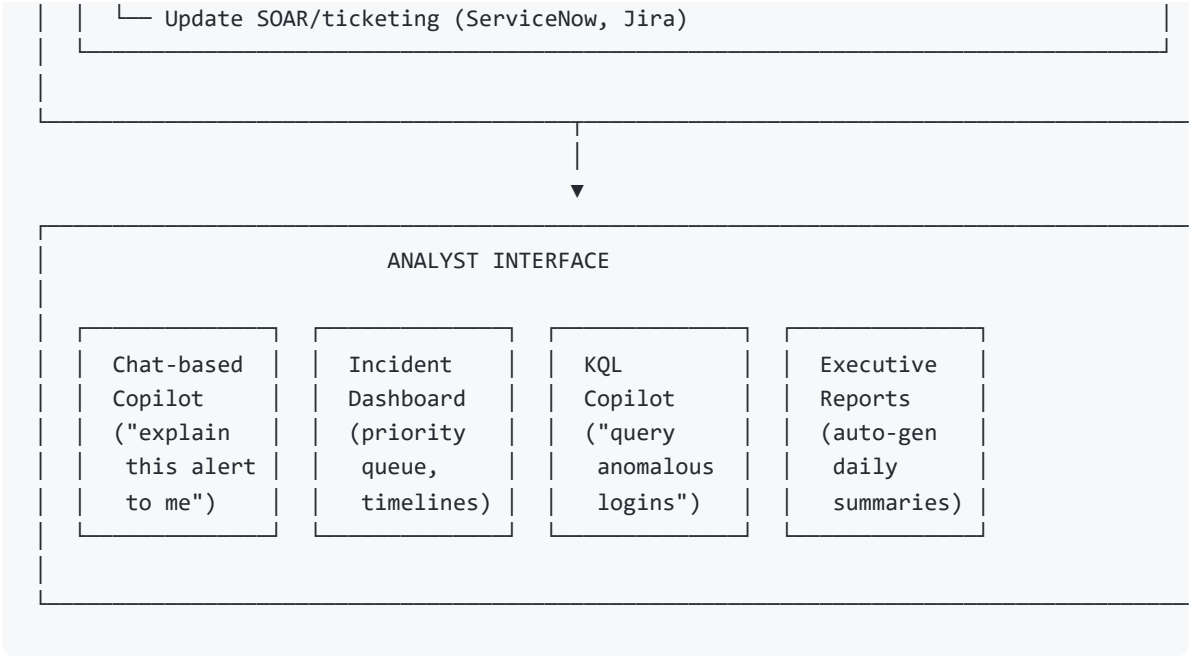
	partitioned by source/severity)	enables parallel agent consumption; 7-day replay window	higher operational overhead
Log analytics store	Azure Data Explorer (Kusto) with hot/warm/cold tiering	Purpose-built for time-series security analytics; KQL is the standard security query language; sub-second queries at petabyte scale	Elasticsearch/Splunk — higher cost at scale; Kusto is optimized for security data patterns
Active incident state	Cosmos DB with TTL-based cleanup	Low-latency CRUD for active investigations; TTL auto-removes stale incidents; globally distributable	Kusto for everything — optimized for append-only analytics, not rapid transactional updates
Entity risk scoring	Redis Cache with Kusto batch baselines + Stream Analytics real-time updates	Sub-millisecond risk lookups during triage (<30s SLA); baselines computed offline to avoid runtime cost	On-the-fly Kusto computation — too slow for real-time triage decisions
Alert triage ML	XGBoost/LightGBM for severity scoring (<5ms inference)	Fast enough for real-time scoring; interpretable feature importance; reduces 10K alerts/day to ~500 incidents	LLM-based triage — API calls too slow and expensive for hundreds-per-second scoring
Investigation engine	GPT-4o with ReAct loop + KQL tool-calling (read-only,	LLM reasons about attack chains and generates contextual KQL;	Pre-built playbook templates — can't adapt to novel attacks;

	max 20 queries, 5-min timeout)	guardrails prevent runaway costs	pure autonomous — safety/cost concerns
Response actions	Tiered approval: auto-approve reversible (block IP), require approval for high-impact (isolate device)	Balances speed for critical containment with safety for high-risk actions; full audit trail	Fully autonomous — organizational risk; approval-for-everything — creates bottleneck
Query interface	KQL as universal language (analysts + LLM agents)	Standard for Microsoft security; LLM-generated queries are auditable; safety-checkable via AST parsing	GraphQL API — adds indirection, harder audit trails; forces analysts to learn new syntax

High-Level Design




INGESTION & ENRICHMENT PIPELINE
Schema Normalizer (ASIM)
Threat Intel Enrichment (IoC match)
Entity Enrichment (user,host, IP→context)
Dedup & Sampling
KUSTO / ADX CLUSTER
(Historical data)
SecurityEvent
SigninLogs
DeviceProcessEvents
NetworkFlows
CloudAuditLogs
ThreatIntel
... 100+ tables
STREAMING ANALYTICS
(Real-time detect)
Azure Stream Analytics / KQL continuous queries
Detect in < 60s
ALERT STORE
(Active alerts)
Cosmos DB (hot alerts, investigations)
AI BRAIN (Core Intelligence)
TRIAGE AGENT
Alert arrives → ML severity scorer → Deduplicate → Correlate → Prioritize
Output: Ranked incident queue with auto-assigned severity
INVESTIGATION AGENT
For each high-priority incident:
├── Generate & execute KQL queries against Kusto tables
├── Build entity timeline (user, device, IP, process)
├── Identify attack chain (MITRE ATT&CK mapping)
├── Assess blast radius (what else is affected?)
└── Produce investigation summary + evidence
RESPONSE AGENT
Based on investigation findings:
├── Recommend containment actions (isolate host, disable user, block IP)
├── Auto-execute approved playbooks (with human-in-the-loop)
├── Generate incident report (natural language summary)



API Design

Alert & Incident APIs

Method	Endpoint	Description
`POST`	`/v1/alerts/ingest`	Ingest a batch of raw alerts
`GET`	`/v1/incidents`	List incidents (filter by severity, status, time)
`GET`	`/v1/incidents/{id}`	Get incident details + AI investigation summary
`POST`	`/v1/incidents/{id}/investigate`	Trigger AI investigation on an incident
`POST`	`/v1/incidents/{id}/respond`	Execute a response action (with approval)
`PATCH`	`/v1/incidents/{id}`	Update incident status (analyst override)

AI Chat / Copilot APIs

Method	Endpoint	Description
`POST`	`/v1/copilot/chat`	Natural language security query (streaming SSE)
`POST`	`/v1/copilot/kql`	Generate KQL from natural language

`POST`	`/v1/copilot/explain`	Explain an alert, script, or command in plain English
`POST`	`/v1/copilot/summarize`	Summarize an incident for executive report

Query & Data APIs

Method	Endpoint	Description
`POST`	`/v1/kusto/query`	Execute KQL query against ADX cluster
`GET`	`/v1/entities/{type}/{id}/timeline`	Get entity timeline (user, device, IP)
`GET`	`/v1/entities/{type}/{id}/risk`	Get entity risk score and contributing factors
`GET`	`/v1/threat-intel/ioc/{value}`	Look up indicator of compromise

Playbook / Response APIs

Method	Endpoint	Description
`GET`	`/v1/playbooks`	List available response playbooks
`POST`	`/v1/playbooks/{id}/execute`	Execute playbook (requires approval for destructive actions)
`POST`	`/v1/actions/isolate-host`	Isolate a device from the network
`POST`	`/v1/actions/disable-user`	Disable an Entra ID account
`POST`	`/v1/actions/block-ip`	Add IP to firewall blocklist

Natural Language Chat Request

```
POST /v1/copilot/chat
{
  "session_id": "sess_soc_001",
  "message": "Show me all lateral movement activity for user jdoe@contoso.com in the
  "context": {
    "current_incident": "INC-2026-4521",
    "analyst_role": "Tier2",
    "tenant": "contoso"
  },
```

```
  "stream": true
}
```

Chat Response (Streamed)


```
{
  "session_id": "sess_soc_001",
  "steps": [
    {
      "type": "reasoning",
      "content": "I'll check RDP, SMB, PsExec, and WMI activity for this user across
    },
    {
      "type": "kql_query",
      "content": "DeviceNetworkEvents\n| where Timestamp > ago(48h)\n| where Initiat
      "query_id": "qry_abc123"
    },
    {
      "type": "kql_results",
      "content": "| RemoteIP | RemotePort | ConnectionCount | DistinctHosts |\n|----
    },
    {
      "type": "analysis",
      "content": "🔴 **Suspicious lateral movement detected.** User jdoe connected t
    },
    {
      "type": "recommendation",
      "content": "**Recommended actions:**\n1. Isolate device DESKTOP-JD01 (source o
    }
  ],
  "audit_trail_id": "audit_xyz789"
}
```

Data Layer: Kusto Tables & Event Hubs

Kusto Table Schema (ASIM-Normalized)

```
┌──────────────────────────────────────────────────────────────────┐
│                    KUSTO / ADX CLUSTER                           │
│                                                                  │
│  HOT DATA (< 30 days): SSD, instant queries                      │
│  WARM DATA (30-90 days): HDD, slightly slower                    │
│  COLD DATA (90-365 days): Blob-backed, cheapest                  │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  CORE SECURITY TABLES                                    │    │
```

```
│  │                                                              │  │
│  │  SecurityAlert          — Detection alerts from all sources │  │
│  │  ├── AlertId, Severity, Title, Description                  │  │
│  │  ├── Tactics, Techniques (MITRE ATT&CK)                     │  │
│  │  ├── Entities (users, hosts, IPs, files involved)           │  │
│  │  ├── SourceSystem (Defender, Sentinel, custom)              │  │
│  │  └── TimeGenerated, ProviderName                            │  │
│  │                                                              │  │
│  │  SigninLogs             — Entra ID authentications          │  │
│  │  ├── UserPrincipalName, IPAddress, Location                 │  │
│  │  ├── ResultType (success/failure/MFA challenge)             │  │
│  │  ├── ConditionalAccessStatus, RiskLevel                     │  │
│  │  └── AppDisplayName, DeviceDetail                           │  │
│  │                                                              │  │
│  │  DeviceProcessEvents   — Process execution on endpoints     │  │
│  │  ├── DeviceName, FileName, ProcessCommandLine               │  │
│  │  ├── InitiatingProcessAccountUpn                            │  │
│  │  ├── SHA256, ParentProcessName                              │  │
│  │  └── FolderPath, ProcessCreationTime                        │  │
│  │                                                              │  │
│  │  DeviceNetworkEvents   — Network connections from endpoints │  │
│  │  ├── DeviceName, LocalIP, RemoteIP, RemotePort              │  │
│  │  ├── Protocol, ActionType (ConnectionSuccess/Blocked)       │  │
│  │  └── InitiatingProcessFileName                              │  │
│  │                                                              │  │
│  │  DeviceFileEvents      — File creation, modification        │  │
│  │  DeviceRegistryEvents  — Registry changes                   │  │
│  │  EmailEvents           — Email metadata (sender, subject)   │  │
│  │  CloudAppEvents        — SaaS app activity (SharePoint,     │  │
│  │                           OneDrive, Teams)                  │  │
│  │  AuditLogs             — Azure/Entra admin activities       │  │
│  │  ThreatIntelIndicators — Known IoCs (hashes, IPs, domains)  │  │
│  │  UserBehaviorAnalytics — UEBA risk scores and anomalies     │  │
│  │                                                              │  │
│  └──────────────────────────────────────────────────────────────┘  │
│                                                                    │
│  SCALE:                                                            │
│  ├── 50-200 TB of hot data                                         │
│  ├── 500K-1M events/second ingestion                               │
│  ├── 100+ tables across all data sources                           │
│  └── Sub-second query response for typical analyst queries         │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

Event Hubs Streaming Architecture

```
┌────────────────────────────────────────────────────────────────────┐
│                    EVENT HUBS TOPOLOGY                             │
│                                                                    │
│  INGESTION HUBS (partitioned by source):                           │
│  ├── eh-endpoint-events     (32 partitions, 100K EPS)              │
│  ├── eh-identity-events     (16 partitions, 50K EPS)               │
│  ├── eh-network-events      (64 partitions, 500K EPS)              │
│  ├── eh-cloud-audit         (8 partitions, 10K EPS)                │
```

```
│  ├── eh-email-events        (8 partitions, 20K EPS)              │
│  └── eh-threat-intel        (4 partitions, 1K EPS)               │
│                                                                   │
│  CONSUMERS:                                                       │
│  ├── Consumer Group: "kusto-ingestion"                            │
│  │   └── ADX streaming ingestion (all events → Kusto tables)     │
│  ├── Consumer Group: "real-time-detection"                        │
│  │   └── Stream Analytics (continuous KQL detection rules)       │
│  ├── Consumer Group: "ai-triage"                                  │
│  │   └── AI Triage Agent (ML scoring + alert correlation)        │
│  └── Consumer Group: "enrichment"                                 │
│      └── Enrichment pipeline (TI lookup, entity resolution)      │
│                                                                   │
│  ALERT OUTPUT HUB:                                                │
│  └── eh-alerts-enriched (triage agent writes scored alerts)       │
│      └── Consumed by: Investigation Agent, Dashboard, SOAR       │
│                                                                   │
│  RETENTION:                                                       │
│  ├── Event Hubs: 7 days (replay window for reprocessing)         │
│  └── Kusto: Hot 30d / Warm 90d / Cold 365d                       │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

AI Brain: Three-Agent Architecture

Agent 1: Triage Agent (Real-Time, < 30 seconds)

```
┌──────────────────────────────────────────────────────────────────┐
│                    TRIAGE AGENT                                   │
│                                                                   │
│  INPUT: Raw alert from Event Hub                                  │
│                                                                   │
│  STEP 1: ML SEVERITY SCORING                                      │
│  ┌──────────────────────────────────────────────────┐             │
│  │  Features:                                       │             │
│  │  ├── Alert source reputation (Defender = high signal)          │
│  │  ├── Entity risk score (user/device UEBA score)  │             │
│  │  ├── Threat intel match (known IoC? = critical)  │             │
│  │  ├── Time of day (3 AM login = more suspicious)  │             │
│  │  ├── Historical false-positive rate for this rule│             │
│  │  └── Asset criticality (domain controller > laptop)            │
│  │                                                  │             │
│  │  Output: true_positive_probability: 0.0 → 1.0    │             │
│  │  Model: XGBoost (fast, <5ms inference) or fine-tuned           │
│  │         classifier, retrained weekly on analyst  │             │
│  │         feedback (true positive / false positive)│             │
│  └──────────────────────────────────────────────────┘             │
│       │                                                           │
│       ▼                                                           │
│  STEP 2: ALERT CORRELATION & DEDUPLICATION                        │
│  ┌──────────────────────────────────────────────────┐             │
```

```
│  │  Group alerts by:                                          │        │
│  │  ├── Same entity (user, device, IP) within time            │        │
│  │  │   window (15 min)                                       │        │
│  │  ├── Same MITRE technique across entities                  │        │
│  │  │   (multiple hosts → campaign)                           │        │
│  │  └── Kill chain progression                                │        │
│  │      (recon → initial access → lateral movement)           │        │
│  │                                                            │        │
│  │  10,000 raw alerts/day → ~500 correlated incidents         │        │
│  └────────────────────────────────────────────────────────────┘        │
│       │                                                                │
│       ▼                                                                │
│  STEP 3: PRIORITY QUEUE                                                │
│  ┌────────────────────────────────────────────────────────────┐        │
│  │  Priority = severity_score × asset_criticality             │        │
│  │             × (1 + entity_risk) × recency_boost            │        │
│  │                                                            │        │
│  │  Queue: [Critical: 5] [High: 23] [Medium: 142]             │        │
│  │                                                            │        │
│  │  Auto-escalation: Critical incidents → immediate           │        │
│  │  Investigation Agent + page on-call analyst                │        │
│  └────────────────────────────────────────────────────────────┘        │
│                                                                        │
│  OUTPUT: Correlated incident with severity, entities,                  │
│          MITRE mapping, and priority score → Cosmos DB                 │
└────────────────────────────────────────────────────────────────────────┘
```

Agent 2: Investigation Agent (Deep Analysis, 30s-5min)

```
┌────────────────────────────────────────────────────────────────────────┐
│                    INVESTIGATION AGENT                                 │
│                                                                        │
│  INPUT: High-priority incident from Triage Agent                       │
│                                                                        │
│  AUTONOMOUS INVESTIGATION LOOP (ReAct pattern):                        │
│                                                                        │
│  ┌──────────────────────────────────────────────────────────────┐      │
│  │  ITERATION 1: Understand the alert                           │      │
│  │  ├── THINK: "Suspicious PowerShell on DESKTOP-JD01.          │      │
│  │  │   I need to check what commands were run and by whom."    │      │
│  │  ├── ACT: Generate & execute KQL:                            │      │
│  │  │   DeviceProcessEvents                                     │      │
│  │  │   | where DeviceName == "DESKTOP-JD01"                    │      │
│  │  │   | where Timestamp > ago(1h)                             │      │
│  │  │   | where FileName == "powershell.exe"                    │      │
│  │  │   | project Timestamp, ProcessCommandLine, AccountName    │      │
│  │  └── OBSERVE: Found encoded PowerShell downloading           │      │
│  │      mimikatz from external IP 185.x.x.x                     │      │
│  └──────────────────────────────────────────────────────────────┘      │
│       │                                                                │
│       ▼                                                                │
│  ┌──────────────────────────────────────────────────────────────┐      │
│  │  ITERATION 2: Check threat intel                             │      │
```

```
  │  ├── THINK: "Need to check if 185.x.x.x is known bad"     │  │
  │  ├── ACT: Query ThreatIntelIndicators table +              │  │
  │  │   external feeds (VirusTotal, AbuseIPDB)                │  │
  │  └── OBSERVE: IP linked to APT29, flagged in 3 feeds       │  │
  └──────────────────────────────────────────────────────────────┘  │
        │                                                          │
        ▼                                                          │
  ┌──────────────────────────────────────────────────────────────┐  │
  │  ITERATION 3: Assess lateral movement                        │  │
  │  ├── THINK: "Mimikatz = credential theft. Check if           │  │
  │  │   stolen creds were used to access other hosts"           │  │
  │  ├── ACT: Query SigninLogs + DeviceNetworkEvents for         │  │
  │  │   jdoe@contoso.com → other hosts via RDP/SMB/WinRM      │  │
  │  └── OBSERVE: jdoe connected to 3 additional servers         │  │
  │      including domain controller DC01                        │  │
  └──────────────────────────────────────────────────────────────┘  │
        │                                                          │
        ▼                                                          │
  ┌──────────────────────────────────────────────────────────────┐  │
  │  ITERATION 4: Build attack timeline                          │  │
  │  ├── THINK: "Map the full kill chain"                        │  │
  │  ├── ACT: Correlate all findings into timeline               │  │
  │  └── OBSERVE: Full chain:                                    │  │
  │      14:32 - Phishing email received                         │  │
  │      14:35 - Malicious link clicked                          │  │
  │      14:36 - PowerShell downloads mimikatz                   │  │
  │      14:38 - Credential dumping (LSASS access)               │  │
  │      14:42 - Lateral movement to 3 hosts                     │  │
  │      14:45 - Access to domain controller DC01                │  │
  │      MITRE: T1566→T1059.001→T1003→T1021→T1078                │  │
  └──────────────────────────────────────────────────────────────┘  │
        │                                                          │
        ▼                                                          │
  OUTPUT:                                                          │
  ├── Investigation summary (natural language)                     │
  ├── Attack timeline with evidence links                          │
  ├── MITRE ATT&CK mapping                                        │
  ├── Blast radius (affected entities)                             │
  ├── Confidence score (how sure is the AI?)                       │
  ├── Recommended response actions                                 │
  └── All KQL queries executed (for analyst review)                │
                                                                   │
  TOOLS AVAILABLE TO AGENT:                                        │
  ├── execute_kql(query, table) — run KQL against Kusto           │
  ├── lookup_threat_intel(ioc) — check IoC databases               │
  ├── get_entity_risk(entity) — UEBA risk score                    │
  ├── get_asset_info(host) — asset criticality, owner, OS          │
  ├── get_user_baseline(upn) — normal behavior profile             │
  ├── search_similar_incidents(description) — past incidents       │
  └── decode_payload(encoded_string) — decode base64/PS            │
                                                                   │
  GUARDRAILS:                                                      │
  ├── Max 20 KQL queries per investigation (cost control)          │
  ├── Max 5-minute wall clock per investigation                    │
  ├── Read-only access to Kusto (cannot modify data)               │
  ├── All queries logged to audit trail                            │
```

```
│   └── Escalate to human if confidence < 0.6                          │
│                                                                       │
└──────────────────────────────────────────────────────────────────────┘
```

Agent 3: Response Agent (Action Execution)

```
┌──────────────────────────────────────────────────────────────────────┐
│                    RESPONSE AGENT                                     │
│                                                                       │
│  INPUT: Investigation findings + recommended actions                  │
│                                                                       │
│  APPROVAL MODES:                                                      │
│  ┌──────────────────────────────────────────────────────┐            │
│  │  AUTO-APPROVE (low-risk, reversible actions):        │            │
│  │  ├── Block external IP at firewall                   │            │
│  │  ├── Quarantine email (move to junk)                 │            │
│  │  ├── Add IoC to threat intel blocklist               │            │
│  │  └── Create ServiceNow ticket                        │            │
│  │                                                      │            │
│  │  REQUIRE APPROVAL (high-impact actions):             │            │
│  │  ├── Isolate device from network                     │            │
│  │  ├── Disable user account                            │            │
│  │  ├── Force password reset                            │            │
│  │  ├── Wipe device                                     │            │
│  │  └── Revoke all active sessions                      │            │
│  │                                                      │            │
│  │  NEVER AUTO-EXECUTE:                                 │            │
│  │  ├── Delete data                                     │            │
│  │  ├── Modify firewall rules broadly                   │            │
│  │  └── Shut down production services                   │            │
│  └──────────────────────────────────────────────────────┘            │
│       │                                                               │
│       ▼                                                               │
│  EXECUTION:                                                           │
│  ├── Call Microsoft Graph API (disable user, revoke sessions)        │
│  ├── Call Defender API (isolate device, run AV scan)                 │
│  ├── Call Firewall API (block IP/domain)                             │
│  ├── Call SOAR webhook (trigger existing playbook)                   │
│  └── Update incident status in Cosmos DB                             │
│       │                                                               │
│       ▼                                                               │
│  POST-ACTION:                                                         │
│  ├── Verify action succeeded (poll API for confirmation)             │
│  ├── Log action to audit trail (who approved, when, why)             │
│  ├── Generate incident report (PDF/Markdown)                         │
│  └── Notify analyst via Teams/email/PagerDuty                        │
│                                                                       │
└──────────────────────────────────────────────────────────────────────┘
```

KQL Copilot: Natural Language → KQL



Entity Risk Scoring (UEBA)

```
┌──────────────────────────────────────────────────────────────────┐
│                    ENTITY RISK ENGINE                            │
│                                                                  │
│  For each entity (user, device, IP), maintain a risk score:      │
│                                                                  │
│  RISK SIGNALS:                                                   │
│  ├── Anomalous login (new geo, impossible travel)        +15     │
│  ├── First-time access to sensitive resource             +10     │
│  ├── Unusual process execution on device                 +20     │
│  ├── Matching threat intelligence indicator              +40     │
│  ├── Multiple failed authentications                     +10     │
│  ├── Off-hours activity (outside normal pattern)         +5      │
│  ├── Privilege escalation attempt                        +25     │
│  └── Data exfiltration pattern (large upload)            +30     │
│                                                                  │
│  RISK DECAY: Score decays 10% per day with no new signals        │
│                                                                  │
│  BASELINE MODEL (per-entity):                                    │
│  ├── Normal login hours: 8 AM - 7 PM EST                         │
│  ├── Normal geolocations: [US-East, US-West]                     │
│  ├── Normal applications: [Outlook, Teams, SharePoint]           │
│  ├── Normal network patterns: Internal only                      │
│  └── Computed from 30-day rolling window via Kusto batch job     │
│                                                                  │
│  STORAGE:                                                        │
│  ├── Risk scores → Redis (real-time lookups, <1ms)               │
│  ├── Baselines → Kusto table (UserBehaviorAnalytics)             │
│  └── Risk history → Kusto (for trend analysis)                   │
│                                                                  │
│  EXAMPLE:                                                        │
│  Entity: jdoe@contoso.com                                        │
│  Risk Score: 87/100 (Critical)                                   │
│  Contributing factors:                                           │
│  ├── Login from Russia (never before)            +15             │
│  ├── Accessed domain controller (first time)     +10             │
│  ├── PowerShell encoded command execution        +20             │
│  ├── Mimikatz hash detected in process           +40             │
│  └── Baseline violation: 3 AM activity           +5              │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Technology	Purpose
Event Ingestion	Azure Event Hubs (Kafka-compatible)	High-throughput streaming ingestion (1M+ EPS)

Data Store	Azure Data Explorer (Kusto)	Petabyte-scale log analytics, sub-second KQL queries
Stream Processing	Azure Stream Analytics / KQL streaming	Real-time detection rules (<60s latency)
Alert Store	Cosmos DB	Hot incident data, investigation state, agent threads
Risk Scores	Redis Cache	Real-time entity risk scores (<1ms lookup)
Triage Agent	XGBoost + correlation engine	ML severity scoring, alert dedup and grouping
Investigation Agent	LLM (GPT-4o) + KQL tools	Autonomous investigation via ReAct loop
Response Agent	LLM + Graph/Defender APIs	Recommend and execute containment actions
KQL Copilot	LLM + schema-aware prompt	Natural language → KQL translation
Threat Intelligence	Kusto table + external APIs	IoC matching (hashes, IPs, domains, URLs)
UEBA Engine	Kusto batch jobs + Redis	Per-entity behavioral baselines and anomaly detection
Audit Trail	Azure Immutable Blob Storage	Every AI decision, query, and action logged immutably
Notification	Azure Logic Apps / Event Grid	PagerDuty, Teams, email alerts for critical incidents
SOAR Integration	REST webhooks	Trigger existing playbooks in Sentinel/XSOAR/Splunk
Dashboard	React + Kusto Direct Query	Real-time incident queue, entity timelines, metrics

---

Scalability Considerations

1. Kusto Cluster Scaling

```
Challenge: 1M events/second, 100+ tables, analysts running ad-hoc queries simultaneously

Solutions:
├── Cluster auto-scale: Add/remove nodes based on CPU and ingestion lag
├── Hot/warm/cold tiering: SSD for recent data, HDD for older, Blob for archive
├── Materialized views: Pre-aggregate common queries (hourly alert counts, etc.)
├── Query limits: Per-user concurrent query cap, automatic timeout after 10 min
├── Follower databases: Read-only replicas for analyst queries (isolate from ingestion)
└── Partitioning: Partition large tables by TimeGenerated (most queries filter by time)

Sizing example (large enterprise):
├── 12-node cluster (D14v2: 16 cores, 112 GB RAM each)
├── Hot cache: 50 TB (30 days)
├── Warm storage: 200 TB (90 days)
├── Cold: 500 TB (365 days)
└── Ingestion: ~800K EPS sustained, 1.5M EPS burst
```

2. Event Hubs Scaling

```
Challenge: Bursty ingestion during incidents (2-5x normal volume)

Solutions:
├── Throughput Units: Auto-inflate from 20 → 100 TU during spikes
├── Partitioning: Partition by source type for parallel consumption
├── Consumer groups: Isolate Kusto ingestion from AI agents from dashboards
├── Capture: Auto-archive all events to Blob Storage (disaster recovery)
└── Kafka compatibility: Use Kafka consumer APIs for flexible processing

Partition strategy:
├── eh-endpoint-events: Partition by DeviceId (locality for per-device analysis)
├── eh-identity-events: Partition by TenantId (multi-tenant isolation)
└── eh-network-events: Partition by hash(SourceIP) (even distribution)
```

3. AI Agent Scaling

```
Challenge: Investigation agent uses expensive LLM calls + multiple KQL queries

Solutions:
├── Batch investigations: Group related incidents, investigate together
├── Caching: Cache KQL results for shared entities (same user, same host)
├── Model tiering: Use small model for triage, large model for investigation
├── Concurrency limit: Max 50 parallel investigations per cluster
├── Priority queue: Critical incidents get GPU priority over medium/low
└── Circuit breaker: If LLM API latency > 30s, queue and retry with backoff

Cost breakdown (per investigation):
├── KQL queries (5-10 queries): ~$0.01
├── LLM tokens (~8K input + 2K output): ~$0.10
├── Threat intel lookups: ~$0.001
```

```
└── Total: ~$0.12 per investigation
└── At 500 incidents/day: ~$60/day = ~$1,800/month
```

Trade-offs

Decision	Option A	Option B	Our Choice
Detection engine	Custom ML models only	LLM-based detection	**Hybrid** — ML for real-time triage, LLM for deep investigation
Alert storage	All in Kusto	Hot in Cosmos DB, cold in Kusto	**Hot in Cosmos DB** — faster CRUD for active incidents, cheaper at alert-level scale
KQL generation	Pre-built query templates	LLM-generated dynamic KQL	**LLM with safety checks** — more flexible, but parse AST before execution
Response automation	Fully autonomous	Human-in-the-loop only	**Tiered** — auto-execute low-risk, require approval for high-impact
Entity risk model	Rule-based scoring	ML anomaly detection	**Hybrid** — rules for known patterns + ML for novel anomalies
Investigation depth	Fixed playbook	Agent explores freely	**Agent with guardrails** — ReAct loop but max 20 queries, 5 min timeout
Multi-tenant data	Shared Kusto cluster	Cluster per tenant	**Shared with row-level security** — cost-efficient, enforce tenant isolation via query policies
Audit trail	Standard logging	Immutable ledger	**Immutable storage** — regulatory requirement for security decisions

Technologies Deep Dive

Data Platform

Technology	Purpose	Why
Azure Data Explorer (Kusto)	Log analytics & detection	Purpose-built for time-series security data, KQL is the query language of security
Azure Event Hubs	Streaming ingestion	Kafka-compatible, millions of events/sec, consumer groups for fan-out
Azure Stream Analytics	Real-time detection	Continuous KQL queries over streaming data, <60s detection latency
Cosmos DB	Active incident store	Low-latency CRUD for investigation state, TTL for auto-cleanup
Redis	Entity risk cache	Sub-millisecond risk score lookups during triage

AI & ML

Technology	Purpose	Why
GPT-4o (Azure OpenAI)	Investigation agent, KQL generation	Strong reasoning, tool-calling, KQL understanding
XGBoost / LightGBM	Triage severity scoring	Fast inference (<5ms), interpretable feature importance
Sentence Transformers	Alert similarity / dedup	Embed alert descriptions, cluster similar alerts
ONNX Runtime	ML model serving	Low-latency inference for triage scoring on CPU

Security & Compliance

Technology	Purpose	Why
Azure Immutable Blob Storage	Audit trail	Write-once, tamper-proof logs of every AI decision
Azure Key Vault	Secrets management	API keys for Defender, Graph, SOAR integrations
Managed Identity	Auth to Azure services	No stored credentials, automatic token rotation

Azure Private Link	Network isolation	All traffic stays on Microsoft backbone

---

How to Explain It Well

30-Second Pitch

> "We're building an AI SOC analyst that processes the massive stream of security events — 1M events/second flowing through Event Hubs into Kusto tables — and uses a three-agent architecture to autonomously handle incidents. The Triage Agent uses ML to score and correlate raw alerts, reducing 10,000 alerts/day down to ~500 actionable incidents. The Investigation Agent uses an LLM in a ReAct loop, autonomously generating and executing KQL queries to build attack timelines and map to MITRE ATT&CK. The Response Agent recommends and executes containment actions with tiered approval — auto-execute for low-risk, human approval for high-impact. Analysts interact via a KQL Copilot that translates natural language to KQL and a chat-based investigator."

Key Talking Points

1. **Data gravity in Kusto**: All security data lives in ADX — the AI agents query it directly via KQL, no data movement needed
2. **Three-agent separation**: Triage (fast ML, <30s) → Investigation (deep LLM, 30s-5min) → Response (action execution) — each with different latency and cost profiles
3. **KQL as the universal language**: Both human analysts and AI agents speak KQL — the LLM generates the same queries analysts would write
4. **Human-in-the-loop by design**: AI recommends, humans approve high-impact actions; full audit trail for every decision
5. **Alert fatigue is THE problem**: 10,000 alerts → 500 incidents → 50 requiring human attention. The AI handles the noise.

Whiteboard Sequence

1. Start with data sources → Event Hubs → Kusto (the data foundation)
2. Draw the three agents: Triage → Investigation → Response pipeline
3. Show the Investigation Agent's ReAct loop with KQL tool-calling
4. Add the KQL Copilot for analyst interaction
5. Overlay the governance layer (audit trail, approval gates)

Common Follow-up Questions

"How do you prevent the LLM from running dangerous KQL queries?"

- **Parse the KQL AST** before execution — reject any mutations (`.set`, `.drop`, `.delete`)
- **Read-only connection**: The Kusto principal used by the agent has `Viewer` role only
- **Row limits**: Automatically append `| take 10000` if no limit is present
- **Cost estimation**: Estimate data scan size from table stats, reject queries scanning >1TB
- **Query timeout**: Hard 2-minute timeout per KQL query
- **Audit logging**: Every generated query is logged with the LLM reasoning that produced it

"How do you handle false positives from the AI?"

- **Analyst feedback loop**: Every AI decision has "Confirm" / "False Positive" buttons
- **Weekly retraining**: Triage ML model retrained on analyst labels (true positive vs false positive)
- **Confidence scores**: Investigation Agent reports confidence; low confidence → escalate to human
- **Tunable thresholds**: Each organization adjusts auto-triage sensitivity per alert source
- **False positive suppression**: Repeated false positives auto-create suppression rules

"How does this scale for an MSSP (multi-tenant)?"

- **Kusto row-level security**: Each query automatically filtered by `TenantId` — no cross-tenant leakage
- **Event Hubs partitioning**: Partition by TenantId for isolation and parallel processing
- **Per-tenant AI config**: Different severity thresholds, approved response actions, and LLM personas
- **Cost attribution**: Token usage and KQL compute metered per tenant
- **Shared infrastructure**: Single Kusto cluster with tenant isolation — far cheaper than cluster-per-tenant

"How do you maintain the entity risk baselines at scale?"

- **Kusto scheduled jobs** (batch): Run daily aggregation queries over 30-day windows per entity
- **Materialized views**: Pre-compute `login_hour_distribution`, `geo_distribution`, `app_usage` per user
- **Redis snapshots**: Push updated risk scores to Redis every 15 minutes
- **Incremental updates**: Stream Analytics updates risk scores in near-real-time as events arrive

- **Cold start**: New entities start at baseline risk = 0, elevate based on first-seen signals

"What happens when the LLM API is down?"

- **Graceful degradation**:
  i. Triage Agent continues (ML model runs locally, no LLM dependency)
  ii. Investigation falls back to pre-built KQL playbook templates (no LLM reasoning)
  iii. Response Agent queues recommended actions for when LLM recovers
  iv. KQL Copilot shows "AI unavailable" — analyst writes KQL manually
- **Multi-region LLM**: Azure OpenAI deployed in 2+ regions with automatic failover
- **Queue-based architecture**: Incidents queue in Cosmos DB, processed when capacity returns

---